

PROVABLY SAFE LEARNING-BASED
ROBOT CONTROL VIA ANOMALY DETECTION
AND GENERALIZATION THEORY

ALEC JACOB FARID

A DISSERTATION
PRESENTED TO THE FACULTY
OF PRINCETON UNIVERSITY
IN CANDIDACY FOR THE DEGREE
OF DOCTOR OF PHILOSOPHY

RECOMMENDED FOR ACCEPTANCE BY
THE DEPARTMENT OF
MECHANICAL AND AEROSPACE ENGINEERING
ADVISER: ANIRUDHA MAJUMDAR

MARCH 2023

© Copyright by Alec Jacob Farid, 2023.

All Rights Reserved

Abstract

How can we ensure the safety of a control policy for a robot that operates using high-dimensional sensor observations such as vision (e.g., an autonomous drone or a home robot)? Machine learning techniques provide an appealing option because of their ability to deal with rich sensory inputs. However, policies that have learning-based components as part of the perception and control pipeline typically struggle to generalize to novel environments. Consider a home robot which is trained to find, grasp, and move objects using thousands of different environments, objects, and goal locations. It is inevitable that such a complex robot will encounter new settings for which it is unprepared. State-of-the-art approaches for synthesizing policies for this robotic system (e.g., based on deep reinforcement learning or imitation learning) generally do not provide assurances of safety, and can lead to policies that fail catastrophically in new environments.

In this dissertation, we will present techniques we have developed to analyze the safety of robotic systems when they are deployed in new and potentially unsafe settings. We use and develop tools from generalization theory in order to leverage statistical guarantees for learning in robotics to make progress in two key research directions. (i) We provide methods for guaranteed detection of when a robotic system is unprepared for the setting it is operating in and could lead to a failure. With this knowledge, an emergency maneuver or backup safety policy can be deployed to keep the robot safe. (ii) We also develop performance bounds for settings which are out-of-distribution with respect to the training dataset. In both cases, we apply the techniques on challenging problems including vision-based drone navigation and autonomous vehicle planning and demonstrate the ability to provide strong safety guarantees for robotic systems.

Acknowledgements

I first want to thank my advisor Ani Majumdar. Ani has not only helped significantly in all aspects of my research, but has helped me develop as a researcher and professional. He has given me just as much flexibility to learn and explore areas I was excited about as guidance for approaching particular research questions. Ani encouraged me to take time off after I was burned out from large projects, advocated for me to receive awards and attend additional conferences to help my career, and accommodated my continuously changing approach to work and meet as I learned what was effective for me. Ani's intuition and drive as a researcher has been a source of inspiration throughout my PhD and it has been a privilege to work with and learn from him.

This work would not be possible without my collaborators and their mentorship. Sushant Veer helped me work through the details of the many detection techniques we have developed. Thank you for the time you have spent mentoring me and for help in determining that, in fact, there is no free lunch. Marco Pavone mentored me during my internship at NVIDIA and always provided incredibly insightful comments on the many ideas we discussed. It has been so fruitful and fulfilling to work with my many collaborators: David Snyder, Michelle Ho, Allen Ren, Karen Leung, Boris Ivanovic, Anoopkumar Sonar, and Divyanshu Pachisia.

I want to thank Tom Griffiths and Naomi Leonard for their continued advice throughout my graduate career, for serving on my generals and PhD committees, and for helping me develop and broaden my ideas. I am also grateful for the invaluable feedback from Ani, Naomi, and Jaime Fernández Fisac on this dissertation. Their comments and suggestions have improved the clarity and accessibility of many concepts throughout this dissertation.

I am fortunate to have been part of the IRoM-Lab group. Alex, May, Sasha, Nate, David, Allen, Meghan, and Vince have all made our office an exciting place to work. Thank you for the reliable daily walks to Small World, and especially to Vince and Meghan for the virtual coffee time talks during the long remote work period (which was vital to my sanity when I was working a few feet from my bed). The environment created by Ani and our group have made my experiences as a graduate student so fulfilling. Thank you David,

for answering my silly questions candidly and enduring the absolute pleasure of being my neighbor in the office. Allen, for efficiently adding hardware results last minute to our papers, and for inspiring us to work harder and more effectively. Sasha, for trying to do everything and help everyone at once, and for your generosity. Nate, for being an incredible travel buddy, caring about lab safety and organization, and being one of the best pilots in our group. Thank you all for accepting that my origami creations must be displayed on everyone's desk.

I would not be here without the love and support of my friends and family. Thank you for listening to me ramble about research projects and for helping me through difficult classes, research challenges, and the years of ups and downs. Susan, for your incredible patience, for being my climbing buddy, and for letting me distract you from work in your office. Meghan, for tackling advanced algorithms with me and for being with me and there for me throughout the entire PhD. Jered, for discussing the subtleties of out-there research ideas and considering them thoroughly as if they were perfectly reasonable. Ash, for the many late nights of remote video games. Claire, for being an incredible teammate, for your persistent support, and for getting me addicted to coffee. Lastly, I want to thank my parents for enabling the incredible opportunities I am privileged to have, and Sammy, for being my role model. Thank you for believing in me and showing me what it looks like to love what you do.

This work was supported by the Office of Naval Research [N00014-21-1-2803, N00014-18-1-2873], the NSF CAREER award [2044149], the NSF Graduate Research Fellowship Program [DGE2039656], the Amazon Research Award, the Google Research Award, the School of Engineering and Applied Science (SEAS) at Princeton University through the generosity of William Addy '82., and the Toyota Research Institute (TRI). This dissertation solely reflects the opinions and conclusions of its author and not NSF, ONR, Amazon, Google, Princeton SEAS, TRI or any other Toyota entity. This dissertation carries T#3439 in the records of the Department of Mechanical and Aerospace Engineering.

To the members of the avocado family group chat.

Contents

Abstract	iii
Acknowledgements	iv
I Out-of-distribution detection and generalization	1
1 Introduction	2
1.1 Motivation	2
1.2 Overview and list of contributions	4
1.3 Related Works	7
2 Background	10
2.1 Problem Formulation	10
2.1.1 Notation	10
2.1.2 Robotic task	10
2.1.3 Sample problem	13
2.2 Policy Synthesis with PAC-Bayes bounds	13
2.2.1 PAC-Bayes bound	14
2.2.2 Policy synthesis	16
3 Out-of-distribution detection	18
3.1 Motivation	18
3.1.1 Problem setup	19
3.2 Approach	20
3.2.1 Policy training via derandomized PAC-Bayes bounds	20

3.2.2	Task-driven OOD detection with statistical guarantees	22
3.3	Simulation and hardware experiments	24
4	Failure prediction	28
4.1	Motivation	28
4.1.1	Problem setup	29
4.2	Guaranteed prediction of failures	31
4.2.1	Bound on Misclassification Error	31
4.2.2	Bound on Class-Conditional Misclassification Error	32
4.2.3	Algorithmic Implementation	34
4.3	Experiments	34
5	Failure detection for trajectory predictors	39
5.1	Motivation	39
5.1.1	Problem setup	41
5.2	Guaranteed detection of prediction failures	42
5.2.1	Bounds on False-positive and False-negative Rates	43
5.3	Experiments	44
5.3.1	Task-relevant Detection	45
5.3.2	Filtering expert driving logs	49
6	Out-of-distribution generalization	52
6.1	Motivation	52
6.2	Generalization to distribution shifts	53
6.2.1	f -divergence	53
6.2.2	Distributionally robust PAC-Bayes	55
6.3	Meta-learning generalization	56
6.3.1	Related work	57
6.3.2	Problem setup	58
6.3.3	Preliminaries: two generalization frameworks	59
6.3.4	Meta-learning bound	61

6.3.5	Experiments	65
7	Towards broader generalization guarantees	71
7.1	Motivation	71
7.1.1	Problem setup	72
7.2	Task reduction	73
7.2.1	Background	74
7.2.2	Task Reduction: Definition and Properties	74
7.3	Relative complexity between tasks	78
7.4	Generalization from one task to another	80
8	Conclusions and future work	83
8.1	Summary of contributions	83
8.2	Limitations and Future work	85
II	Papers	88
9	Overview	89
9.1	Outline	89
9.2	Author Contributions	90
10	Task-Driven Out-of-Distribution Detection with Statistical Guarantees for Robot Learning	93
10.1	Introduction	94
10.2	Related work	96
10.3	Problem formulation	97
10.4	Approach	99
10.4.1	Policy training via derandomized PAC-Bayes bounds	99
10.4.2	Task-driven OOD detection with statistical guarantees	101
10.5	Examples	103
10.5.1	Robotic grasping	104

10.5.2	Vision-based obstacle avoidance with a drone	105
10.6	Conclusion	109
10.7	Appendix	109
10.7.1	Proof of Theorem 10.1	109
10.7.2	Proof of Theorem 10.2	111
10.7.3	Proof of Theorem 10.3	114
10.7.4	Training with Backpropagation	116
10.7.5	Training with Evolutionary Strategies	118
10.7.6	Additional Experimental Details and Results	120

11 Failure Prediction with Statistical Guarantees for Vision-Based Robot

Control		124
11.1	Introduction	125
11.2	Related Work	127
11.3	Problem Formulation	129
11.3.1	Notation	129
11.3.2	Problem Setting	129
11.4	Failure Prediction with Guaranteed Error Bounds	131
11.4.1	PAC-Bayes Theory for Supervised Learning	131
11.4.2	Bound on Misclassification Error	132
11.4.3	Motivating Class-Conditional Misclassification Error	134
11.4.4	Bound on Class-Conditional Misclassification Error	135
11.4.5	Algorithmic Implementation	138
11.5	Experimental Results	138
11.5.1	Obstacle Avoidance with a Drone	139
11.5.2	Grasping Mugs with a Robot Arm	142
11.6	Conclusion	145
11.7	Appendix	146
11.7.1	Class-Conditional Bound Proof	146
11.7.2	Toy Example Calculations	149

11.7.3 Additional Implementation Details	152
12 Task-Relevant Failure Detection for Trajectory Predictors in Autonomous Vehicles	154
12.1 Introduction	155
12.2 Related Work	156
12.3 Problem Definition	158
12.4 p -Quantile Anomaly Detector	160
12.4.1 Detection Algorithm	160
12.4.2 Bounds on False-positive and False-negative Rates	161
12.4.3 Data-free Detector Calibration	162
12.5 Experimental Results	163
12.5.1 Task-relevant Detection	164
12.5.2 Filtering expert driving logs	167
12.6 Limitations, Future Work, and Conclusion	169
12.7 Appendix	170
12.7.1 Proof of Theorem 12.1	170
12.7.2 Additional Experimental Details	171
13 Generalization Bounds for Meta-Learning via PAC-Bayes and Uniform Stability	178
13.1 Introduction	179
13.2 Problem formulation	181
13.3 Related work	182
13.4 Generalization bound on meta-learning	185
13.4.1 Preliminaries: two generalization frameworks	185
13.4.2 Meta-learning bound	187
13.4.3 PAC-BUS algorithm	189
13.5 Examples	191
13.5.1 Example: classification on the unit ball	191
13.5.2 Example: Mini-Wiki	192

13.5.3	Example: memorizable Omniglot	193
13.6	Conclusion and discussion	195
13.7	Appendix	197
13.7.1	Proof of Theorem 13.1	197
13.7.2	Definition of Marginal Distribution P_S	198
13.7.3	Specializing the Bound	199
13.7.4	Bounds on the Uniform Stability Constant	201
13.7.5	Algorithms	202
13.7.6	Sample Convergence Bound	204
13.7.7	Constraining Parameters and Scaling Losses	205
13.7.8	Uniform Stability Considerations	207
13.7.9	Lipschitz and Smoothness Constant Calculation	209
13.7.10	Study on Base-learning Learning Rate and Number of Update Steps	213
13.7.11	Additional Experimental Details	214
14	Towards a Framework for Comparing the Complexity of Robotic Tasks	218
14.1	Introduction	219
14.2	Related Work	220
14.3	Problem Formulation	222
14.4	Task Reduction	223
14.4.1	Background	223
14.4.2	Task Reduction: Definition and Properties	224
14.5	Examples of Reductions	227
14.5.1	Navigation to a Goal with Map Rotations	227
14.5.2	Grasping Objects with Differing Camera Viewpoints	228
14.6	Relative Complexity	229
14.7	Algorithmic Approach	231
14.8	Examples	232
14.8.1	Cartpole Balancing Task	232
14.8.2	Mujoco 2D Walker at varied speeds	235

14.9 Conclusion	237
14.10Appendix	238
14.10.1 Proof of Properties	238
14.10.2 Additional Experimental Details	241

Part I

Out-of-distribution detection and generalization

Chapter 1

Introduction

1.1 Motivation

Consider an autonomous vehicle tasked with driving in a city with other humans, or a home robot which must clean up a complex living space in order to help a family. The future of robotics has huge potential to improve the world, but we do not see such robots today. The challenge of developing robotic systems which have complex and potentially hybrid dynamics using high-dimensional sensory input (such as cameras or LiDAR) to perform a task in a changing and interactive environment is unsolved. Machine learning techniques provide an appealing option to this challenge because of their ability to deal with rich sensory inputs. However, robotic systems which have learning-based components will be susceptible to a key challenge in machine learning: learned systems struggle to generalize to novel settings.

An autonomous driving vehicle which effectively generalizes to novel settings may be able to be trained in one city, and then deployed safely in a new city. Similarly, a home robot which has been developed to clean up living spaces could clean up warehouses and shopping centers. We would like these robotic systems to operate effectively in such *out-of-distribution* (OOD) settings. However, existing state-of-the-art methods for learning-based robotic systems are typically developed for specific settings and can fail catastrophically in even mildly different settings (e.g., if the lighting conditions change or the setting switches from indoor to outdoor). Importantly, even a robotic system which has been trained using

billions of training samples (as is the case for learned autonomous driving agents) will still encounter settings which are unfamiliar and can risk failure if not developed specifically for effective generalization. Thus, we see generalization as a key challenge which must be overcome before robotic systems become more prevalent in the complex, dynamic, and interactive real-world.

Existing methods for providing safety certificates for robotic systems, such as H-infinity control [75], Hamilton-Jacobi reachability analysis [146, 9], or control barrier functions [15] have been shown to be extraordinarily effective in certain settings. However, the settings on which they provide certificates of safety require assumptions about the types of environments the robot will encounter and struggle to deal with high-dimensional sensor input. As such, we see learning as a key component required for robotic systems which must operate in the real world. In this dissertation, we will develop techniques with the goal of learning robotic control policies which carry certificates of safety and generalization when deployed in new environments and settings. Specifically, we develop approaches which contribute to two closely related research directions: (i) *OOD detection*, where the robotic system detects when the setting it is operating in has changed and it may be unprepared, (ii) *OOD generalization*, where a robotic system is trained explicitly to operate effectively in novel settings. In both settings, we develop techniques to provide *statistical* certificates of safety and performance (this notion is formalized in Chapter 2).

OOD detection has strong potential to improve the safety of robotic systems. Detecting when a robot which is unprepared for the setting it is operating in or explicitly when a setting could lead failure for a robotic system can be used to deploy an emergency maneuver (such as pulling over to the side of the road for an autonomous car) and improve the safety of the system. If the OOD detection method carries a performance certificate, then the resulting system including the detector also carries a certificate of safety. There are two key desiderata that detection approaches for safety-critical robotic systems should ideally satisfy: the detection technique should have a guaranteed confidence of detection, and it should be *task-driven* and only sensitive to changes in the robot’s environment that affect the robot’s performance. On the other hand, with a method for OOD generalization, we can directly improve the safety of a system when operating in an OOD setting. However

the settings on which we will be able to provide strong statistical guarantees for OOD generalization are limited. The goal with the work in this dissertation is to develop such techniques in order to meaningfully improve the safety of robotic systems when operating in OOD settings. Throughout the dissertation, we focus heavily on evaluating the proposed techniques on hardware and challenging simulation setups in order to demonstrate the power of the techniques we develop.

1.2 Overview and list of contributions

In Chapter 2, we introduce the notation used throughout the dissertation and make the setting for providing statistical guarantees concrete. In Chapters 3, 4, and 5 we will tackle the more specific problem of trying to predict or detect when the robot is operating in a setting that it is unprepared for. With this knowledge, we can stop the operation of the robot, change the policy the robot uses, or intervene in order to keep the robot safe. In Chapters 6 and 7, we will approach the more general problem of direct generalization to novel settings. In Chapter 8, we conclude as well as discuss limitations and future work of the approaches discussed throughout Part I. Additional details and experiments for most content in Part I can be found in Part II of this dissertation, where we reproduce most of the published work which is summarized and extended here. The specific contributions of Part I are as follows:

- **Chapter 3.** We develop task-driven OOD detection techniques with statistical guarantees on rates of false positive and false negative detection. Specifically, we develop a pipeline for training control policies with a guaranteed bound on performance using recent *derandomized Probably Approximately Correct (PAC)-Bayes* bounds that are well-suited to enable OOD detection. We then develop two OOD detection techniques based on the following intuition: if the costs incurred when the robot is deployed in new environments violate the bound on the policy’s performance, then this indicates that the robot is operating OOD. Since our OOD detection scheme leverages the costs incurred in new environments, it is *only* triggered by task-relevant changes. In particular, environment changes which negatively affect the cost of the policy (we make the

notion of task-relevance precise in Chapter 3). We then demonstrate our approach on a drone navigating a new set of environments. Comparisons with baselines demonstrate the advantages of our approach in terms of providing statistical guarantees and being insensitive to task-irrelevant shifts. We also present a thorough set of hardware experiments for vision-based drone navigation with varying wind conditions and clutter. Our experiments demonstrate the ability of our approach to perform task-driven OOD detection within just a handful of trials for systems with complex dynamics and rich sensing modalities.

- **Chapter 4.** The goal in this chapter is to train a *failure predictor* for a black-box control policy. This failure predictor acts as a *safety layer* that is responsible for predicting (online) if the given policy will lead to a failure (e.g. a crash) as the robot operates in a novel environment. We utilize techniques from PAC-Bayes generalization theory in order to obtain bounds on error rates of the failure predictor. In addition, we develop novel *class-conditional* bounds that allow us to trade-off the relative rates of false negative vs. false positive errors. Our algorithmic approach is then to train a failure predictor (e.g., in the form of a neural network that takes as input a history of sensor observations) by minimizing the theoretical error bounds. We demonstrate our approach in simulation and hardware using vision-based navigation with a drone. The experiments illustrate the ability of our approach to (1) provide strong bounds on failure prediction error rates, and (2) improve the safety of a robotic system by applying our failure prediction scheme.
- **Chapter 5.** In this chapter, we formalize a notion of anomaly that underpins the design of a task-relevant prediction failure. We then develop a fast and efficient task-relevant failure detector for the prediction module used in autonomous vehicle stacks which is simultaneously accompanied by guarantees on the false-positive and false-negative detection rates. Based on the guarantees, we provide a data-free calibration method for our detector, which most methods lack; this allows our method to be widely applicable without the need for difficult-to-obtain or specific data. We show in our experiments that our data-free calibrated detector’s performance is close to

the optimally calibrated detector using the Receiver Operator Characteristic (ROC) curve. We demonstrate the efficacy of our approach on nuScenes [38] and nuPlan [39] datasets and provide comparisons with other state-of-the-art detection methods in the literature.

- **Chapter 6.** We provide OOD generalization guarantees for two distinct settings in this chapter. In the first setting, we present an extension from [136] which provides guarantees for robotic systems where test environments and training environments are from the same distribution to the *distributionally robust* setting where the training and test environment distributions are distinct. In the second setting, we provide a generalization bound for gradient-based meta-learning using uniform stability and PAC-Bayes theory. We develop a regularization scheme for model-agnostic meta-learning (MAML) [72] that explicitly minimizes the derived bound. We then demonstrate our approach on two meta-learning problems: (i) a toy non-convex classification problem on the unit-ball, and (ii) the *Mini-Wiki* benchmark introduced in [109]. Even in these relatively small-scale settings, we demonstrate that recently-developed generalization frameworks for meta-learning provide either near-vacuous or loose bounds, while our method provides significantly stronger bounds. We also demonstrate our approach in larger-scale settings where it remains challenging to obtain non-vacuous bounds (for our approach as well as others). Here, we propose a practical regularization scheme which re-weights the terms in the upper bound from the generalization guarantee and is able to prevent over-fitting in larger-scale settings.
- **Chapter 7.** In this chapter, we take a step towards developing a broader class of meaningful generalization bounds for two different robotic tasks. We introduce a framework for establishing the *relative complexity* of robotic tasks. One key insight is to define a notion of *reduction* between robotic tasks. This definition formalizes the following intuition: Task 1 reduces to Task 2 if we can transform *any* policy that solves Task 2 into a policy that solves Task 1. We also propose a *quantitative measure* for comparing the complexity of one robotic task relative to another. We prove basic properties of our notion of reduction between tasks (e.g., reflexivity, transitivity).

ity, and antisymmetry) and the relative complexity measure (e.g., non-negativity and monotonicity). Lastly, we use the introduced notions of task reduction and relative complexity to develop a novel generalization bound which accounts for task complexity and has the potential to significantly broaden the tasks which can have concrete OOD generalization bounds.

1.3 Related Works

Safety in Robotics. A common approach for creating control policies that have guaranteed performance is to leverage robust control techniques (e.g., H-infinity control [75] or chance-constrained programming [48, 29, 213, 155]). However, this typically requires domain-specific knowledge in the form of an explicit description of the uncertainty affecting the system. Some approaches assume that the environment satisfies certain geometric conditions that allow for safe navigation, such as the assumption of large separation between obstacles [138]. Another such assumption is that the environment satisfies conditions that allow a robot to always succeed. One approach to this is to specify emergency maneuvers that are always guaranteed to succeed [183, 74, 12]. These kinds of conditions are rarely satisfied in real-world environments. Additionally, such conditions are not usually general and can require a significant amount of hand-specification.

Another approach is to model the problem as a Partially Observable Markov Decision Process (POMDP) [107]. However, this approach requires direct knowledge of the distribution over environments the robot might encounter, which is infeasible for real-world environments. Thus, many approaches (including ours) assume only indirect access to the true underlying distribution over environments in the form of examples. For instance, [168, 167] propose an approximation to the POMDP framework for navigation that learns to predict future collision probabilities from training environments. In general, it is challenging to guarantee that using these approaches will lead to good performance on environments that are not part of the training data.

More recent work with deep learning based approach for control provides another set of techniques where datasets of interactions with example environments can be used to learn

control policies, e.g., [122, 124, 7, 134, 203, 84, 228, 198, 119, 223, 111, 112]. Although empirically successful, the methods do not provide strong guarantees that the methods described will perform well on environments that are not part of the training dataset. As with many deep learning approaches, the methods can be sensitive to differences in settings at test time. In this dissertation, we address this challenge by using and expanding on tools from generalization theory.

Generalization Theory. Multiple frameworks have been developed for providing generalization guarantees in the classical supervised learning setting. Early breakthroughs include Vapnik-Chervonenkis (VC) theory [206, 17], Rademacher complexity [188], and the minimum description length principle [31, 169, 114]. More recent frameworks include algorithmic stability bounds [35, 45, 88, 171, 3] and PAC-Bayes theory [192, 142, 186]. The connection between stability and learnability has been established in [189, 211, 88], and suggests that algorithmic stability bounds are a strong choice of generalization framework. PAC-Bayes theory in particular provides some of the tightest known generalization bounds for classical supervised learning approaches such as support vector machines [186, 116, 78, 171, 14, 156]. Since its development, researchers have continued to tighten [116, 141, 162] and generalize the framework [43, 44, 170]. Exciting recent results [63, 149, 150, 23, 20, 162] have demonstrated the promise of PAC-Bayes to provide strong generalization bounds for neural networks on supervised learning problems (see [105] for a recent review of generalization bounds for neural networks). It is also possible to combine frameworks such as PAC-Bayes and uniform stability to derive bounds for supervised learning [130].

In this work, we will primarily use and expand upon PAC-Bayes generalization theory [192, 142, 186] to make strong guarantees about the safety and performance of a robotic system operating in a novel environment. However, a key assumption is that the robot is tested in environments that are similar to the training environments (in that they must come from the same underlying distribution over environments – in Chapter 2 we make this notion concrete). However, frequently robotic systems encounter environments they are unprepared for. The kinds of guarantees mentioned above would not hold and the robotic system may perform poorly. In such cases, anomaly detection or failure prediction can still be used to maintain the safety of a learned robotic system.

Anomaly Detection. There is a long line of work on *anomaly detection* in the signal processing literature (see [24] for a review). The goal is to identify events or inputs (e.g., sensor observations) that deviate significantly from nominal inputs (and may thus cause failures). In recent years, there have been efforts to develop anomaly detection techniques for supervised learning problems with high-dimensional inputs such as images [92, 127, 59, 121, 28, 49] (see [175] for a recent review). This line of work is also closely related to the literature on uncertainty estimation in deep learning [2]. Recent techniques also allow for anomaly detection in reinforcement learning settings using streams of sensor inputs [185, 40, 222, 83]. Within the robotics literature, there has been work on performing uncertainty estimation [108, 132] or anomaly detection [167] specifically for the problem of collision avoidance (which is an example we consider in this paper). In contrast to our goal of failure prediction, the methods highlighted above are generally aimed at detecting anomalies/outliers in environments that the robot is operating in. While anomalies may result in failures, this is not necessarily the case (e.g., for an extremely unlikely yet benign environment). In addition, the methods above do not generally provide guarantees on false positive or negative rates of detection, or the improvement in the robot’s performance that results from applying the anomaly detector.

Failure Prediction. A different line of work foresees failure by explicitly forward-propagating the robot dynamics in the environment. For instance, if the environment map is available (either given or estimated online), and the robot dynamics are known (e.g., car), the robot can check if it falls into a failure state in the next step before executing the action. Typical methods include Hamilton-Jacobi reachability analysis [146, 9], control barrier functions [15], and formal methods [11]. However, these approaches typically assume an explicit description of the uncertainty affecting the system (e.g., bound on actuation noise) and/or the environment (e.g., minimum distance between obstacles), which are often unrealistic to describe with real-world environments. They also work with low-dimensional systems, and are generally unable to provide guarantees for policies trained with rich sensing like vision. Work in safe reinforcement learning [57, 199] approximates the outcome of robot dynamics from data and allows high-dimensional image input; however, they do not provide guarantees on the robot’s safety.

Chapter 2

Background

2.1 Problem Formulation

2.1.1 Notation

Throughout Part I, lowercase variables x, y, u will generally refer to particular instances of a variable in a set $\mathcal{X}, \mathcal{Y}, \mathcal{U}$ of possible values, denoted with calligraphic letters. We will also frequently make use of distributions on the variables P_x, P_y, P_u over the set of possible values using such notation. Explicitly, P_x is a distribution over $\mathcal{X}$ and we can sample independently and identically distributed (i.i.d.) from P_x to get a sample x . To shorten the notation, we will let $x \sim P_x$ represent i.i.d. sampling. We may generate a discrete set S_x, S_y, S_z of samples by sampling n times from some distribution: $S_x = \{x_1, x_2, \dots, x_n\} = \{x_i\}_{i=1}^n \sim P_x^n$ where the superscript n represents n Cartesian products. In future chapters, we will usually drop the subscript x, y, z notation from dataset S as the variable will be clear from context.

2.1.2 Robotic task

Consider a robotic task such as a drone navigating to a goal position while avoiding obstacles, or a robotic arm grasping a mug and lifting it off a table. A particular task τ describes the robot, its goal, the setting, and all other problem parameters of interest. Let e be the environment that a robot is operating in. For example, e could refer to an obstacle arrangement that must be avoided, along with the goal position. The environment will

capture sources of noise such as wind gusts or error in sensor measurement. Additionally, the environment could be dynamic and reactive. We will only require that the environment is drawn from some underlying distribution P_e over environments. We will not assume anything specific about the distribution P_e such as direct access or any specific form of the distribution, however, we must be able to sample i.i.d. from it: $e \sim P_e$. We assume the robot’s dynamics are as follows:

$$x_{t+1} = f_e(x_{t+1}|x_t, u_t), \quad (2.1)$$

where a potentially stochastic function f , which is specific to the environment e , takes the state $x_t \in \mathcal{X}$ and control input $u_t \in \mathcal{U}$ at time-step t and outputs the state (or a distribution over states) at the following time step. We assume that the task τ induces a distribution P_e over environments from which e is drawn. The methods we develop will only require (a finite number of) samples $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$.

Next, let the robot have a (potentially stochastic) sensor $g_e : \mathcal{X} \rightarrow \mathcal{Y}$ which takes a state x_t to an observation y_t in an environment. There are no assumptions about the observability of the environment from the sensor. The robot’s *policy* $\pi : \mathcal{Y} \rightarrow \mathcal{U}$ takes the observation to a control input. To keep the setting general, we will let $\mathcal{Y}$ denote the space of histories of sensor measurements so that the policy π may use historical sensor data. To specify policies π , we will parameterize them with parameters θ . For example, θ could refer to gains in a feedback control policy or weights of a neural network. Let π_θ be the policy parameterized by parameters θ .

As part of the task, there is an objective such as picking up a mug, or reaching a goal position. Let cost function $C_e(\pi_\theta) \in \mathbb{R}$ denote the performance of the robotic policy π when executed within environment e . The cost function may be binary such as 0 if a robot succeeds in picking up a mug and 1 otherwise, or it may be continuous such as the proportion of the distance left to travel to a goal position. Note that since we assume the environment e captures all sources of noise and disturbances, a fixed policy π will always result in the same outcome in a particular environment e . If we allow the policy to run in an environment using a simulator or hardware experiments, the result is a trajectory of states and action

pairs for the time horizon H of the problem: $\{(x_t, u_t)\}_{t=1}^H$. Let $r_e(\pi_\theta)$ be the function which “rolls out” the robotic policy in the environment to generate the trajectory of states. We use C_e to assign a cost with this trajectory according to the particular goal of the task. The *training cost*, or average cost on the training dataset $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$, is given by $\frac{1}{n} \sum_{i=1}^n C_{e_i}(\pi_\theta)$. Later in the dissertation (Chapter 5), we will also make use of an instantaneous cost $c_e(x_t, u_t) \in \mathbb{R}$ associated with the particular state and control input. We will assume that it is possible to compute C_e for the policy or c_e for the state and control action depending on the particular problem.

Note that throughout this setup of a robotic task, we have been very general in the settings that are captured. This is intentional as the tools we use throughout this dissertation are just as general. We will show that we can provide strong safety guarantees for potentially non-linear or hybrid robotic systems that are equipped with rich sensors, in dynamic environments, and performing complex tasks.

Generalization Goal. The objective of generalization is to specify a robotic policy which performs well on task τ using a dataset of sampled environments $S \sim P_e^n$ for training. Explicitly, the goal is to minimize the expected value of C_e over the draw of $e \sim P_e$:

$$\min_{\theta} C_{P_e}(\pi_\theta) := \min_{\theta} \mathbb{E}_{e \sim P_e} C_e(\pi_\theta) \quad (2.2)$$

using only the training dataset $S \sim P_e^n$ to find θ . However, in order to use the PAC-Bayes generalization framework, we will need need to find a *distribution* P_θ over policy parameters instead of particular set of parameters θ . The change from a particular set of policy parameters θ to a distribution P_θ over policy parameters allows us to compare a data-independent *prior* distribution $P_{\theta,0}$ over policy parameters to some optimized distribution P_θ . The use of a data-independent prior distribution is a necessary assumption to apply PAC-Bayes generalization bounds. Thus, the change will be allow us to achieve tight generalization bounds. As such, we will frequently consider the following optimization problem:

$$\min_{P_\theta} C_{P_e}(P_\theta) := \min_{P_\theta} \mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} C_e(\pi_\theta). \quad (2.3)$$

2.1.3 Sample problem

Since we introduce a lot of notation here that is used throughout the dissertation, we will make the notions above explicit in an example to help exposition. Consider the following task τ : in a bounded area, a quadrotor drone must avoid randomly placed and fixed obstacles using first-person depth images. The robot is a lightweight Parrot Swing drone: a hybrid quadrotor fixed-wing drone which takes off vertically and performs forward flight with the help of the fixed wings (the Parrot Swing drone is depicted in Figure 2.1(a)). Sampling the environment $e \sim P_e$ is equivalent to sampling the randomly placed obstacles; the obstacles in Figure 2.1(a) are placed randomly. All other environment factors such as the size bounds of the area and robot starting position remain fixed. The robot’s dynamics are given by f_e and first-person depth sensor is given by g_e . We will use a neural network policy π_θ with weights θ to take depth sensor measurements and map them to a control input. The policy chooses from a series of motion primitives (i.e. avoid left, avoid right, move forward) as the control input, see Figure 2.1(b) for representative examples of motion primitives. If the drone successfully avoids obstacles and reaches the opposite end of the obstacle field, then $C_e(\pi_\theta) = 0$ (see the roll-out in Figure 2.1(a)), otherwise (i.e. the drone crashes or does not reach the end) $C_e(\pi_\theta) = 1$.

2.2 Policy Synthesis with PAC-Bayes bounds

Recall the goal specified in Equation (2.3) of finding a distribution P_θ which minimizes $C_{P_e}(P_\theta)$. We cannot compute $C_{P_e}(P_\theta)$ explicitly because of the expectation taken over the unknown distribution P_e over environments. However, we can upper bound $C_{P_e}(P_\theta)$ in terms of known quantities using generalization theory. If we find a P_θ which minimizes the upper bound, we indirectly minimize $C_{P_e}(P_\theta)$. This section describes the generalization framework we will frequently use in this dissertation as well as a method to find P_θ . Note that in order to write the PAC-Bayes bound, we will make use of a data-independent *prior* distribution $P_{\theta,0}$ over policy parameters θ . We refer to this as the prior, and it may be randomly initialized or trained on held-out data. Note that a good prior will result in a better bound, and we discuss the particular method used for developing a prior throughout

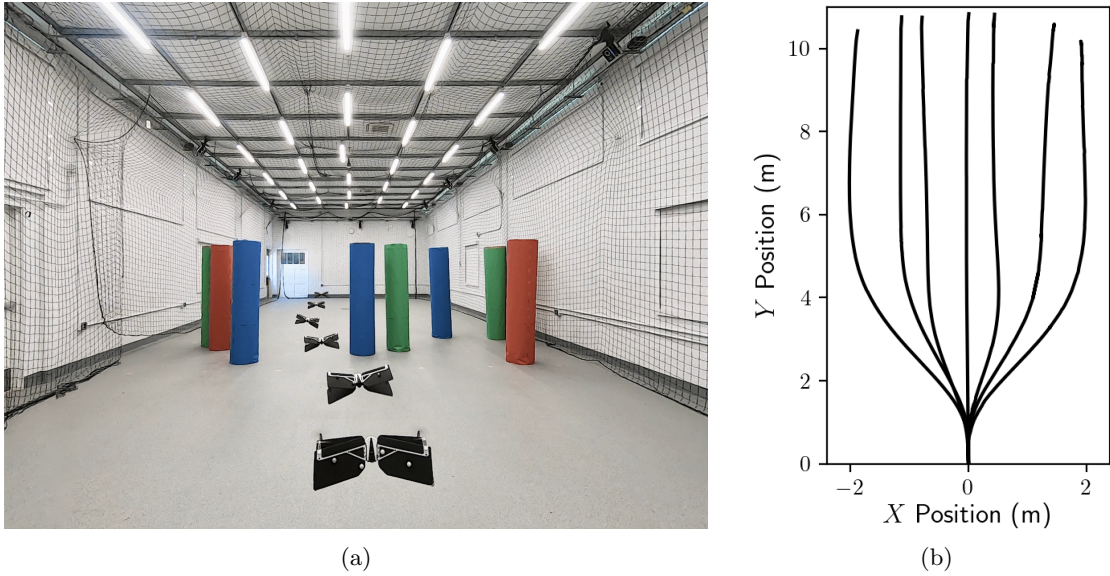


Figure 2.1: **(a)** A Parrot Swing drone is tasked with avoiding randomly placed obstacles in an enclosed space. The figure depicts the trajectory of the drone during the hardware experiment and the result is that the drone successfully avoids the obstacles. **(b)** Representative motion primitives which the neural network policy selects based on the depth-sensor input. If the robot avoids obstacles when rolling out the selected motion primitive in the real-world environment, then the cost $C_e(\pi_\theta) = 0$.

the dissertation. After we have found a P_θ which minimizes the upper bound we refer to it as the *posterior*.

2.2.1 PAC-Bayes bound

We make significant use of PAC-Bayes bounds [142] throughout this dissertation and use this section to provide an introduction to the use of PAC-Bayes bounds in the appropriate context. The PAC-Bayes bounds presented in [142] and other related works are for supervised learning. However, the examples we use in this dissertation are generally for robotic systems operating in a real or simulated environment over multiple time steps (such as the example described above). One of the primary contributions of [136] is the reduction of PAC-Bayes generalization bounds for supervised learning to the policy learning setting. Here we will present the PAC-Bayes bounds for the policy learning setting, but a more thorough introduction to PAC-Bayes bounds for control can be found in [136]. Additionally, we extended the reduction from [136] in [68]. More information on the more general reduction

can be found in Chapter 11. We begin with a relevant assumption

Assumption 2.1 (Bounded function). *The function F is $[0, 1]$ bounded: $0 \leq F(\cdot) \leq 1$ for all inputs for the given problem.*

Note that even though we will generally require the cost function to satisfy Assumption 2.1, any bounded cost can be re-scaled to $[0, 1]$ to apply the methods which rely on this assumption.

The following bound is a tightened version of the bound presented in [142] for when $n \geq 8$ in the policy learning setting [136]:

Theorem 2.1 (PAC-Bayes Generalization Bound [142, 139, 136]). *For any data-independent prior distribution $P_{\theta,0}$ over θ , some cost function C which satisfies Assumption 2.1, $n \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$, the following holds simultaneously for all distributions P_θ over θ :*

$$C_{P_e}(P_\theta) := \mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} C_e(\pi_\theta) \leq \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\theta \sim P_\theta} C_{e_i}(\pi_\theta) + \sqrt{\frac{D_{\text{KL}}(P_\theta \| P_{\theta,0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}, \quad (2.4)$$

where D_{KL} is the Kullback-Leibler (KL) divergence.

It is important to note that this theorem provides a bound on the expected cost on unseen environments $C_{P_e}(P_\theta)$ which *only* depends on seen quantities (i.e. the training dataset S). This was exactly the objective presented Section 2.1.2. This theorem also provides the statistical certificate of safety mentioned in Chapter 1. Specifically, if we calculate the upper bound of Theorem 2.1, we can say that *with high probability* $(1 - \delta)$, the *expected cost* on a new environment $e \sim P_e$ is bounded above by Inequality (2.4). As such we have a *statistical* notion of confidence about the cost. This is the style of guarantee we make throughout this dissertation.

We call the last term on the RHS of Inequality (2.4) the *regularizer*. Note that other PAC-Bayes bounds such as the quadratic variant [172] and PAC-Bayes- λ variant [200] can generally be used and substituted in the analysis throughout the dissertation. Practically, these other PAC-Bayes bounds would change the form of the regularizer term and the weighting between the cost term and the regularizer on the RHS of Inequality (2.4).

One of the key assumptions in Theorem 2.1 is the requirement that the testing data ($e \sim P_e$) is drawn from the same distribution over environments as the training data ($S \sim P_e^n$). In practice, this is difficult to ensure for realistic problems. As such, we will often consider the problem where test data is drawn from a different distribution than the training data (i.e. $e \sim P'_e$ if the training data is drawn from P_e). We discuss the applications of Theorem 2.1 to such *out-of-distribution* (OOD) settings in the following chapters.

2.2.2 Policy synthesis

When we minimize the upper bound of Theorem 2.1, we can, by extension, minimize $C_{P_e}(P_\theta)$. The bound provided in Theorem 2.1 specifies the guarantee on performance for the posterior P_θ without describing how to find a good P_θ . One approach for minimizing the upper bound in Theorem 2.1 is using backpropagation.

In implementation, we will generally make use of multivariate Gaussian distributions to parameterize P_θ . Let multivariate Gaussian distributions $\mathcal{N}(\mu, \Sigma_s)$ have diagonal covariance. I.e. for a vector s , $\Sigma_s := \text{diag}(s)$ where $\text{diag}(s)$ outputs a diagonal matrix with the components of s . Let $\psi := (\mu, \log s)$ (for component-wise log) and we will use shorthand $\mathcal{N}_\psi$. Note that we use $\log s$ in ψ instead of s directly since s must be non-negative, which is not easy to ensure with standard gradient descent. When training the posterior distribution P_θ , we would like to take gradient steps directly with respect to ψ . However, this would require backpropagation through $\mathbb{E}_{w \sim \mathcal{N}_\psi} C_e(\pi_w)$. We follow a similar procedure as in [63] and achieve the desired result of minimizing the upper bound in Inequality (2.4) using an unbiased estimate of $\mathbb{E}_{w \sim \mathcal{N}_\psi} C_e(\pi_w)$:

$$\frac{1}{k} \sum_{i=1}^k C_e(\pi_{w_i}), \quad w_i \sim \mathcal{N}_\psi \quad \forall i \in \{1, 2, \dots, k\} . \quad (2.5)$$

The resulting approach is presented in Algorithm 2.1. Additionally, note that the backpropagation requires a gradient taken through $D_{\text{KL}}(\mathcal{N}_\psi \| \mathcal{N}_{\psi_0})$. We make use of the analytical form for the KL divergence between two multivariate Gaussian distributions p and q :

$$D_{\text{KL}}(p \| q) = \frac{1}{2} \left[\log \frac{|\Sigma_q|}{|\Sigma_p|} + \text{tr}(\Sigma_q^{-1} \Sigma_p) + (\mu_p - \mu_q)^T \Sigma_q^{-1} (\mu_p - \mu_q) - d \right], \quad (2.6)$$

Algorithm 2.1 PAC-Bayes Bound Minimization via Backpropagation

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies

Input: Training dataset S , learning rate γ

Output: Optimized ψ^*

while not converged **do**

 Sample $w_i \sim \mathcal{N}_{\psi} \forall i \in \{1, 2, \dots, k\}$

$$B \leftarrow \frac{1}{nk} \sum_{e \in S} \sum_{i=1}^k C_e(\pi_{w_i}) + \sqrt{\frac{D_{\text{KL}}(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}$$

$\psi \leftarrow \psi - \gamma \nabla_{\psi} B$

end while

where d is the dimension of the multivariate Gaussian.

After training is complete, we will frequently compute the upper bound. However, this would require evaluating an expectation $\theta \sim P_{\theta}$, which may be intractable. Providing a valid PAC guarantee without needing to evaluate the expectation taken over $\theta \sim P_{\theta}$ requires the use of the sample convergence bound [115]. We have the following guarantee with probability $1 - \delta'$ over a random draw of $\{\theta_1, \theta_2, \dots, \theta_m\} \sim P_{\theta}^m$ for any dataset S [115],

$$D_{\text{KL}} \left(\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m C_{e_i}(\pi_{\theta_j}) \left\| \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\theta \sim P_{\theta}} C_{e_i}(\pi_{\theta}) \right. \right) \leq \frac{\log(\frac{2}{\delta'})}{m}. \quad (2.7)$$

We can invert this KL-style bound (i.e. a bound of the form $D_{\text{KL}}(p \parallel q^*) \leq c$) by solving the optimization problem, $q^* \leq D_{\text{KL}}^{-1}(q \parallel c) := \sup\{q \in [0, 1] : D_{\text{KL}}(p \parallel q) \leq c\}$, as described in [63]. After the inversion is performed on Inequality (2.7), we use a union bound to combine the result with Inequality (2.4) and retain a guarantee with probability $1 - \delta - \delta'$ as in [63],

$$C_{P_e}(P_{\theta}) \leq D_{\text{KL}}^{-1} \left(\frac{1}{nm} \sum_{i=1}^n \sum_{j=1}^m C_{e_i}(\pi_{\theta_j}) \left\| \frac{\log(\frac{2}{\delta'})}{m} \right. \right) + \sqrt{\frac{D_{\text{KL}}(P_{\theta} \parallel P_{\theta,0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}. \quad (2.8)$$

Thus, after training, we obtain ψ^* which specifies a posterior $P_{\theta} := \mathcal{N}_{\psi^*}$. We then evaluate Inequality (2.8) to provide the guarantee. Note that use of the sample convergence bound is a loosening step. However, in our experiments, the upper bound in Inequality (2.8) is generally less than 5% looser than unbiased estimates of Inequality (2.4). This can be reduced further at the expense of computation time (if we utilize a larger number of samples in the sample convergence bound).

Chapter 3

Out-of-distribution detection

3.1 Motivation

Imagine a drone trained to perform vision-based navigation using a dataset of indoor environments (see Figure 2.1(a)). It is then deployed in environments with varying wind conditions, obstacle densities, or lighting which were not seen in training. As mentioned in Chapter 1, it is not usually possible to directly generalize to a new setting. In the absence of strong generalization guarantees, what can we do in order to still maintain safety for our robotic system? In this chapter, we will focus on the problem of autonomously detecting when a robot is operating in environments drawn from a different distribution than the one used to train the robot. If successful, the anomaly signal could be used to decide when to deploy a highly conservative policy or cease operation altogether and improve the safety of the robotic system. There are two important characteristics that OOD detection approaches for safety-critical robotic systems should satisfy.

1. The OOD detection technique should have guaranteed confidence of detection.
2. The OOD detector should be task-driven and only sensitive to *task-relevant* changes in the robot’s environment.

Consider the example problem in Section 2.1.3. Suppose the robot’s policy is insensitive to changes in color and lighting. The robot’s OOD detector should not trigger even if it is operating in environments with different color or lighting. However, suppose the policy

is not effective at dealing with wind gusts in the environment since they cannot be sensed directly. The OOD detector should trigger in this case. Current approaches (e.g. [92, 127, 91, 194, 129, 25]) do not typically satisfy both task-relevance and guaranteed confidence of detection; they are often based on heuristics and not task-driven in general.

The key idea for OOD detection in this chapter relies on the following intuition: violation of the performance bound (e.g. the generalization bound provided in Theorem 2.1) on test environments provides evidence that the robot is operating OOD. We formalize this via statistical techniques based on p -values and concentration inequalities. The resulting approach (i) provides guaranteed confidence bounds on OOD detection, and (ii) is *task-driven* and sensitive only to changes that impact the robot’s performance. We demonstrate our approach in simulation and hardware experiments of a drone performing vision-based obstacle avoidance in unfamiliar environments (including wind disturbances and different obstacle densities). Our experiments demonstrate that we can perform task-driven OOD detection within just a handful of trials. Comparisons with baselines also demonstrate the advantages of our approach in terms of providing statistical guarantees and being insensitive to task-irrelevant distribution shifts.

3.1.1 Problem setup

Refer to Section 2.1.2 for the general context used throughout the dissertation. We assume that the robot has access to a training dataset $S = \{e_1, \dots, e_n\}$ of n environments drawn i.i.d. from a training distribution P_e , i.e. $S \sim P_e^n$. After training, the robot is deployed on environments in $S' = \{e'_1, \dots, e'_m\}$ drawn from a test distribution P'_e : $S' \sim P_e'^m$. Importantly, we *do not* assume any explicit knowledge of P_e, P'_e or the space $\mathcal{E}$ of environments. We only have indirect access to P_e, P'_e in the form of the finite datasets S, S' .

After a policy is trained using dataset S , it is deployed in (a typically small number of) environments in S' . The goal is to detect if the environments in S' were drawn from a different distribution than the training distribution (i.e., if P'_e is different from P_e). Moreover, our goal is to perform *task-driven* OOD detection. In particular, our OOD detector should only trigger if:

$$C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta), \tag{3.1}$$

for some π_θ . Thus, our OOD detector should be *insensitive* to changes in the environment distribution that do not adversely impact the robot’s performance. Note that we extend this notion to detect if the expected cost on P'_e is *smaller* than the expected cost on P_e in [71]. This is a challenging task since we only assume access to a finite number of environments from P_e and P'_e . Moreover, our goal is to develop an OOD detection framework that is broadly applicable in challenging settings involving nonlinear/hybrid dynamics, rich sensing modalities (e.g., RGB-D), and neural network-based policies.

3.2 Approach

First, we train a policy with an associated *guarantee* on the expected cost on the training distribution P_e (Section 3.2.1). We then apply our OOD detection scheme which formalizes the following intuition: violation of the bound during deployment implies (with high confidence) that the test distribution P'_e is OOD in a task-relevant manner (Section 3.2.2).

3.2.1 Policy training via derandomized PAC-Bayes bounds

Since our OOD detection scheme will rely on violations of the bound, it is important to obtain bounds that are as tight as possible. We have already established that PAC-Bayes bounds provide an appealing option for tight generalization guarantees. However, the traditional PAC-Bayes upper bound presented in Theorem 2.1 includes an expectation over $\theta \sim P_\theta$. Thus, given a test dataset S' of test environments, many policies from the distribution P must be sampled in order to evaluate/bound the expected cost on S' (using the sample convergence bound as described in 2.2). This is not feasible in an OOD detection setting, where there is single execution on the test environments. As such, we will extend the methods presented in Section 2.2 to leverage *derandomized* PAC-Bayes bounds [209] (which are well-suited to the OOD detection setting). The following is the derandomized PAC-Bayes bounds in the policy learning setting:

Theorem 3.1 (Derandomized PAC-Bayes Generalization Bound [209, 139, 136]). *For any data-independent prior distribution $P_{\theta,0}$ over θ , some cost function C which satisfies Assumption 2.1, $n \geq 8$, and $\delta \in (0,1)$, with probability at least $1 - \delta$ over a sampling of*

$S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$ and a sampling of $\theta \sim P_\theta$, the following holds simultaneously for all distributions P_θ over θ :

$$C_{P_e}(P_\theta) \leq \frac{1}{n} \sum_{i=1}^n C_{e_i}(\pi_\theta) + \sqrt{\frac{D_2(P_\theta \| P_{\theta,0}) + \ln \frac{2\sqrt{n}}{(\delta/2)^3}}{2n}}, \quad (3.2)$$

where D_2 is the Rényi Divergence for $\alpha = 2$.

Proof. The proof is in Section 10.7.1. We use [209, Theorem 2], a general pointwise PAC-Bayes bound. We perform the reduction from supervised learning to policy learning presented in [136]. $\square$

Note that the posterior must be generated by a deterministic algorithm (i.e., for a fixed dataset S , the algorithm always generates the same posterior distribution P_θ). In practice, this is not difficult to achieve by fixing a seed for generating random numbers in the program which finds P_θ . As in Section 2.2, we can still use the above theorem to find a posterior P_θ by minimizing the sum of the training cost and regularizer term via backpropagation.

We can directly apply Algorithm 2.1 by replacing the regularizer with the one in Inequality 3.2. We also fix a random seed. As in Section 2.2, we parameterize the prior $P_{\theta,0}$ and posterior P_θ with multivariate Gaussian distributions $\mathcal{N}(\mu, \text{diag}(s))$ and let $\psi := (\mu, \log s)$. We use the shorthand $\mathcal{N}_\psi$ for $\mathcal{N}(\mu, \text{diag}(s))$. The backpropagation requires a gradient taken through $D_2(\mathcal{N}_\psi \| \mathcal{N}_{\psi_0})$. We make use of the analytical form for the Rényi divergence between two multivariate Gaussian distributions, presented in [79], in order to tractably compute the gradients.

$$D_2(q \| p) = (\mu_q - \mu_p)^T \Sigma_2 (\mu_q - \mu_p) - \frac{1}{2} \ln \frac{|\Sigma_2| |\Sigma_q|}{|\Sigma_p|^2}, \quad (3.3)$$

where $\Sigma_2 = 2\Sigma_p - \Sigma_q$. We also note that there is a restriction on how far the posterior's variance can drift from the prior. The following expression must be satisfied for $D_2(\mathcal{N}_\psi \| \mathcal{N}_{\psi_0})$ to be finite [79]:

$$2\Sigma_s^{-1} - \Sigma_{s_0}^{-1} \succ 0. \quad (3.4)$$

In practice, we project any problematic variances into the range of allowable variances. The resulting algorithm is presented in Algorithm 3.1.

Algorithm 3.1 Derandomized PAC-Bayes Bound Minimization via Backpropagation

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies

Input: Fixed seed for random number generation

Input: Training dataset S , learning rate γ

Output: Optimized ψ^*

while not converged **do**

 Sample $w_i \sim \mathcal{N}_{\psi} \forall i \in \{1, 2, \dots, k\}$

$B \leftarrow \frac{1}{nk} \sum_{e \in S} \sum_{i=1}^k C_e(\pi_{w_i}) + \sqrt{\frac{D_2(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0}) + \ln \frac{2\sqrt{n}}{(\delta/2)^3}}{2n}}$

$\psi \leftarrow \psi - \gamma \nabla_{\psi} B$

end while

After training, since we have used the derandomized PAC-Bayes bound in Theorem 3.1, we compute the upper bound with a single $w \sim \mathcal{N}_{\psi}$ in contrast to traditional PAC-Bayes bounds. Thus, the resulting policy π_w is deterministic and applicable in a broad range of settings, including ones which require a pre-trained network. The resulting policy carries a PAC-Bayes guarantee.

3.2.2 Task-driven OOD detection with statistical guarantees

We now tackle the problem of task-relevant OOD detection: determining if $C_{P'_e}(P_{\theta}) > C_{P_e}(P_{\theta})$. To shorten notation, let

$$\overline{C}(\pi_{\theta}, S) := \frac{1}{n} \sum_{i=1}^n C_{e_i}(\pi_{\theta}) + \sqrt{\frac{D_2(P_{\theta} \parallel P_{\theta,0}) + \ln \frac{2\sqrt{n}}{(\delta/2)^3}}{2n}}, \quad (3.5)$$

which is upper bound on $C_{P_{\theta}}(P_{\pi})$ in Theorem 3.1. Note that since a deterministic algorithm was used to generate P_{θ} , $\overline{C}(\pi_{\theta}, S)$ only depends on the sampling of $\theta \sim P_{\theta}$ and S . From the PAC-Bayes training pipeline outlined in Section 3.2.1, we have a policy π_{θ} and the associated upper bound $\overline{C}(\pi, S)$ which holds with probability $1 - \delta$ over the sampling of the training dataset $S \sim P_e^n$ and the policy parameters $\theta \sim P_{\theta}$.

The key idea we use in this chapter for OOD detection is that if our PAC bound $\overline{C}(\pi_{\theta}, S)$ is violated by π_{θ} in the test environments S' (drawn from the test distribution P'_e), then this indicates that the test environments are OOD. We formalize this intuition below using hypothesis testing via p -value and confidence interval overlap.

Hypothesis testing

Let H be the null-hypothesis which claims that $C_{P'_e}(\pi_\theta) \leq C_{P_e}(\pi_\theta)$ and let $\neg H$ be the negation of H : $C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta)$ for a given policy π_θ . The p -value for null-hypothesis H bounds the probability of drawing test environments with average cost larger than the observed test cost $\frac{1}{|S'|} \sum_{e \in S'} C_e(\pi_\theta)$ assuming that H holds. Thus, we choose a threshold $\alpha \in (0, 1)$ before seeing data, and perform a hypothesis test using the p -value.

If the p -value drops below the significance level α , we can reject the null-hypothesis by concluding that the observed test dataset S' had a small probability of being drawn. Explicitly, we have the following definition of the p -value for null-hypothesis H

Definition 3.1 (p -value for null-hypothesis H [96]). *Fix a control policy π_θ . Let P'_e be the test distribution and $S' \sim P_e'^m$ be an observed testing dataset. Then, the p -value is defined as:*

$$p(S') := \mathbb{P}_{\hat{S} \sim P_e'^m} \left[\frac{1}{m} \sum_{e \in \hat{S}} C_e(\pi_\theta) \geq \frac{1}{m} \sum_{e \in S'} C_e(\pi_\theta) \mid H \right]. \quad (3.6)$$

Note that $\hat{S}$ is *any* dataset of cardinality m drawn i.i.d. from the test distribution $P_e'^m$, while S' is the observed test dataset which we must use for inference. Using Theorem 3.1, we can upper bound the p -value of null-hypothesis H .

Theorem 3.2. *Let P_e be the training distribution and P_θ be the posterior distribution on the space of policies obtained through the training procedure described in Section 3.2.1. Let $S' \sim P_e'^m$ be a test dataset, $p(S')$ be the p -value for S' assuming null-hypothesis H , and $\delta \in (0, 1)$. Then, the following holds with at least probability $1 - \delta$ over a sampling of $S \sim P_e^n$ and $\theta \sim P_\theta$,*

$$p(S') \leq \exp(-2m\tilde{c}^2), \quad (3.7)$$

where $\tilde{c} := \max[\frac{1}{m} \sum_{e \in S'} C_e(\pi_\theta) - \overline{C}(\pi_\theta, S), 0]$.

Proof. The proof relies on Hoeffding's inequality and is provided in Section 10.7.2. $\square$

Theorem 3.2 provides an upper bound on the p -value which holds with probability $1 - \delta$. If the upper bound is below the significance level α , then with high probability, we can say that the p -value is below α . If we reject the null-hypothesis H , then we have high confidence that $C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta)$ and there is a detection of a task-relevant distribution shift.

Confidence interval on the difference in expected train and test costs

Next, we present another method for task-driven OOD detection by lower bounding $C_{P'_e}(\pi_\theta) - C_{P_e}(\pi_\theta)$ with high probability. Let the lower bound be ΔC . If ΔC is positive, then we can conclude that $C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta)$ and the robot is operating in a task-relevant OOD environment. Additionally a larger ΔC indicates the test distribution is “further” OOD in the context of the task.

Theorem 3.3. *Let P_e be the training distribution, P'_e be the test distribution, and P_θ be the posterior distribution on the space of policies obtained through the training procedure described in Section 3.2.1. For $\delta, \delta' \in (0, 1)$ such that $\delta + \delta' < 1$. Then the following holds with at least probability $1 - \delta - \delta'$ over a sampling of $S \sim P_e^n$, $\theta \sim P_\theta$, and $S' \sim P_e'^m$*

$$\Delta C := \frac{1}{m} \sum_{e \in S'} C_e(\pi_\theta) - \sqrt{\frac{\ln(1/\delta')}{2m}} - \bar{C}(\pi_\theta, S) \leq C_{P'_e}(\pi_\theta) - C_{P_e}(\pi_\theta). \quad (3.8)$$

Proof. A detailed proof of this theorem is provided in App. 10.7.3. □

To use Theorem 3.3 for detecting if the test dataset S' is OOD, we compute ΔC with a desired confidence level $1 - \delta - \delta'$. If $\Delta C > 0$, then with confidence at least $1 - \delta - \delta'$ we can claim that $C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta)$, and therefore the robot is operating OOD. Importantly, this also provides a guaranteed upper bound of $\delta + \delta'$ on the number of false positives (or false detections) for the detector.

3.3 Simulation and hardware experiments

Overview. In both the simulation and hardware portions of this example, we aim to avoid an obstacle field with the Parrot Swing drone; this is an agile quadrotor/fixed-wing hybrid drone shown in Figure 2.1(a). We train a DNN control policy in a simulation setup based

on the hardware system shown in Figure 2.1(a). The policy takes in a 50×50 depth image and outputs a softmax corresponding to a set of pre-computed motion primitives with the goal of avoiding obstacles by the largest distance. Since we designed the simulation portion of this example with application to hardware in mind, we have created motion primitives by capturing (with a Vicon motion tracking system) the trajectories of open-loop control inputs, which result in different maneuvers (see Figure 2.1(b)). The use of motion primitives allows us to perform accurate sim-to-real transfer (since the motion primitives are recorded directly from the hardware system).

Training. Environments consist of a set of randomly placed cylindrical obstacles. We record the minimum distance $d_{\min}$ from the obstacles (as recorded by the robot’s 120° field of view depth sensor) and assign a cost of $\max(0, 1 - \frac{d_{\min}}{30\text{cm}})$. Using 10,000 training environments S , we train a prior $P_{\theta,0}$ to assign larger values to motion primitives which achieve a larger distance from obstacles. See Section 10.7.6 for further details on the training procedure for the prior. We then use another 10,000 environments to train the posterior distribution using Algorithm 3.1. Note that training the prior and posterior only take around 3 minutes total. We sample a policy π_θ from the trained posterior and compute the PAC-Bayes bound $\overline{C}(\pi_\theta, S) = 0.222$ for $\delta = 0.01$.

Simulation Results

We compare our task-driven OOD detection approach with two baselines: (i) maximum softmax probability (MSP) [92] (an effective and popular baseline for OOD detection), and (ii) MaxLogit [91] (a recent state-of-the-art OOD detection baseline). We note that these baselines are specifically designed for networks which output categorical distributions, and thus provide strong benchmarks. We determine the threshold for detection for the baselines by choosing a false positive rate of 5% for the training environments. However, note that the baselines do not provide any guarantees; they may violate the false positive rate even on new environments drawn from the training distribution (as we observe in our experiments). We select a p -value threshold of 0.05 as the indication that an environment is OOD, and a guaranteed false-positive rate of $\delta + \delta' = 5\%$ for the confidence interval method. Note that we do not get a similar a guaranteed for the false-negative rate. Each method receives

a dataset S' of 10 environments. We use the average outputs of the baselines on the 10 environments for OOD detection. We generate OOD environments of varying difficulty by changing the number of obstacles and the maximum or minimum gap-size between obstacles.

Varied environment difficulty. We sample 2000 datasets S' (of size 10) for each setting and plot the proportion of these detected as OOD for all methods in Figure 3.1(a). At 0 on the x-axis of this plot, we draw datasets from the training distribution. All other environments are OOD. We note that the p -value and confidence interval-based methods perform similarly. The plot demonstrates our ability to perform task-driven OOD detection. In particular, we only detect the “harder” distributions (i.e., environments with higher increased cost) as OOD, while the baselines are triggered even for the easier distributions. In addition, neither baseline provides any guarantees and both violate the 5% false-positive rate on the training distribution. Our methods have a low false positive rate, albeit at the cost of conservatism for moderate distribution shifts.

Task-irrelevant shift. We also compare the baselines with our methods on a distribution where environments consist of 4 (uniformly) randomly located obstacles. In this setting, the control policy achieves a near-identical expected cost $C_{P'_e}(\pi_\theta)$ (as estimated by exhaustive sampling of environments) to the expected training cost $C_{P_e}(\pi_\theta)$ (in particular, $C_{P'_e}(\pi_\theta) - C_{P_e}(\pi_\theta) = -0.02$). For this setting, MSP [92] classified 95.7% of test datasets as OOD and MaxLogit [91] classified 50.9% as OOD. Thus, the baselines are triggered by a task-irrelevant shift in the distribution. In contrast, our methods had an OOD detection rate of 0.1% in this setting.

Hardware Results

We use a Parrot Swing drone for the hardware experiments (Figure 2.1(a)). We simulate a depth sensor for the drone (as if the sensor was mounted on the drone) by generating a synthetic depth image using the positions of objects from the Vicon motion capture system. We do not provide any other information to the policy (e.g., obstacle positions). We generate each environment the same way as in simulation, and then place the real-world obstacles in the generated locations.

Varied environment difficulty and wind disturbances. We deploy the policy

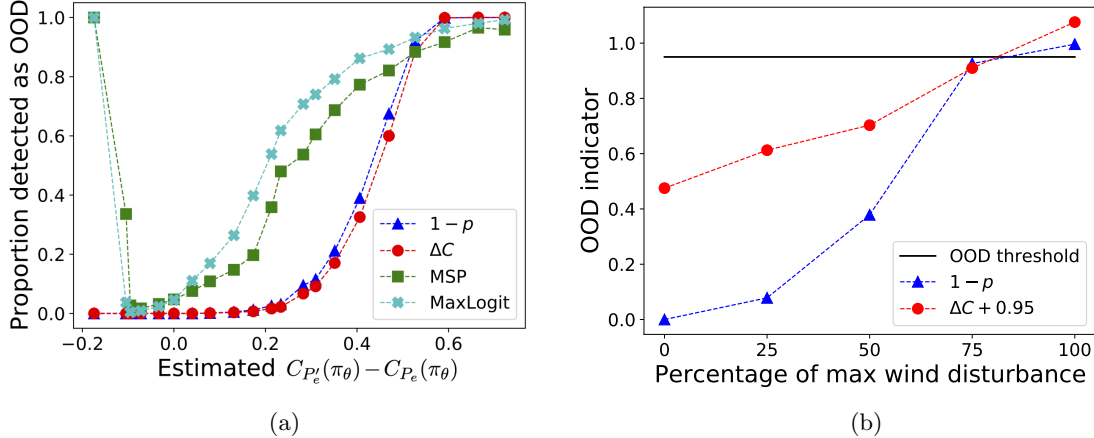


Figure 3.1: OOD Detection on settings of variable difficulty. **(a)** Comparison of the performance of our OOD detectors with baselines MSP [92] and MaxLogit [91]. Both our methods do not detect OOD when $C_{P'_e}(\pi_\theta) < C_{P_e}(\pi_\theta)$ whereas the baselines detect these task-irrelevant shifts in the environment. For $C_{P'_e}(\pi_\theta) > C_{P_e}(\pi_\theta)$ the baselines detect a higher proportion of datasets as OOD for smaller values of $C_{P'_e}(\pi_\theta) - C_{P_e}(\pi_\theta)$ compared to our methods. **(b)** Comparison of our OOD detectors on the Parrot Swing hardware for increasing wind disturbance. Both OOD indicators increase monotonically with $C_{P'_e}(\pi_\theta) - C_{P_e}(\pi_\theta)$ and are able to detect OOD at 100% wind.

trained in simulation on three kinds of OOD environments in hardware: (i) environments with a smaller number of obstacles (“easier” environments), (ii) environments with smaller gaps between obstacles (i.e., “harder” environments), and (iii) environments with wind generated using a fan with the same obstacle distribution as training. Videos from each setting are available at <https://youtu.be/VxKCAjaih8M>. For each setting, we run 10 trials on the hardware and use this for OOD detection. As expected, our OOD detectors are not triggered by the easier environments. For the harder environments, we compute $1-p \geq 0.81$ and $\Delta C = -0.11$. Results from the windy environments are shown in Figure 3.1(b) for increasing values of wind (up to about 5 m/s). We note that the sim-to-real distribution shift (corresponding to the zero wind case) is not viewed as being OOD in a task-relevant manner by our approaches. Both our approaches assign increasingly high confidence as the wind speed is increased. We note that disturbances such as wind cannot be detected via the depth image given to the robot’s policy. Thus any OOD detection technique which relies solely on the output of the policy, such as MSP [92] and MaxLogit [91], would be unable to detect these environments as OOD.

Chapter 4

Failure prediction

4.1 Motivation

In the previous chapter, we developed an OOD detector with the following key characteristics: the OOD detector (i) had guaranteed confidence of detection, and (ii) was task-driven and only sensitive to task-relevant changes in the robot’s environment. A key challenge with this method is that detection occurs *after* multiple rollouts of the policy in the test environments. In other words, the robot must perform poorly in order to detect the OOD environments. However, if poor performance or a failure is costly, then the method is not as applicable because we may need to see multiple failures in order to determine when to stop operation. Additionally, the methods in Chapter 3 could only provide guaranteed bounds on the false positive rate of the OOD detector. In this chapter, we address some of the key challenges from the previous chapter. We focus specifically on failures and aim to not just detect them but predict them ahead of time so that the robot can be stopped or an emergency maneuver can be performed. Thus, we have the following desired characteristics for our detector:

1. The failure predictor should have guaranteed bounds on false positive and false negative error rates.
2. The predictor should determine (in real-time) if the robot will fail *prior* to the failure.

Additionally, in the previous chapter, we developed a specific training pipeline for the robotic

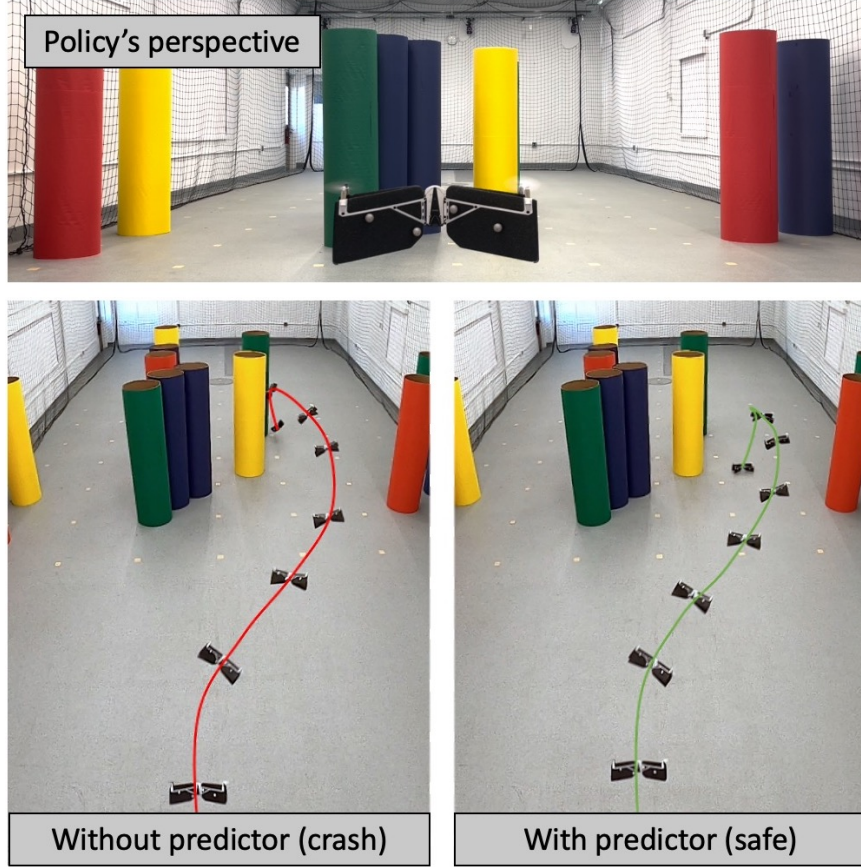


Figure 4.1: We train a failure predictor which guarantees (with high probability) detection of a failure ahead of time. A blackbox policy is tasked with avoiding obstacles in a novel environment using a first-person depth image (top). The bottom images show the entire environment, including multiple obstacles which are occluded from view. Since it is operating in a novel setting, the policy alone fails to avoid occluded obstacles and crashes (left). When the failure predictor is deployed, the policy stops safely before a crash (right).

policy in order to create the detectors. In this chapter and the next, we will consider the more general setting of a black-box robotic policy π (i.e., we do not have access to parameters θ nor the ability to change the policy). We will develop a failure predictor for this policy which acts as a safety layer. Such a predictor could enable the robot to deploy a backup policy in order to ensure safety, e.g. see Figure 4.1.

4.1.1 Problem setup

As mentioned, we assume the robotic policy π is a black-box. However as in Section 2.1.2, we still assume the policy takes a sensor input $y \in \mathcal{Y}$ and outputs a control action such as a motion primitive (Figure 2.1(b)). The failure predictor f takes in a history (of up to H

past sensor inputs) and maps them to a $\{0, 1\}$ value indicating the robot’s predicted failure (e.g., 0 if it reaches a target and 1 if not).

As in previous chapters, we are given a dataset $S \sim P_e^n$ of environments. We assume that we can run the blackbox policy in each of the n environments to determine if the policy failed or not. Thus, each environment e has a true label $z \in \{0, 1\}$ associated with success or failure based on the result of running π in environment e . The predictor maps a fixed horizon H of past observations to a predicted class $\hat{z} \in \{0, 1\}$. There are four possible outcomes:

1. **True positive:** ($\hat{z} = 1 \cap z = 1$), predicting 1 at least once before failure
2. **True negative:** ($\hat{z} = 0 \cap z = 0$), never predicting 1 throughout entire successful rollout
3. **False positive:** ($\hat{z} = 1 \cap z = 0$), incorrectly predicting 1 at least once during a successful rollout
4. **False negative:** ($\hat{z} = 0 \cap z = 1$), incorrectly predicting 0 at all steps of a failed rollout

The errors are the false positives and false negatives. As such, we define *false positive rate* (FPR) as the predicted positive rate given a negative true label, $\text{FPR} := \mathbb{P}(\hat{z} = 1 | z = 0)$; similarly, *false negative rate* (FNR) is the predicted negative rate given a positive true label, $\text{FNR} := \mathbb{P}(\hat{z} = 0 | z = 1)$.

To evaluate the performance of the failure predictor, we introduce the error of applying the predictor f in an environment e where the policy π is running:

$$C_e(f, \pi) := \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{z}_t) \neq z], \quad (4.1)$$

where T_{fail} is the step when failure occurs. If the whole rollout is successful, T_{fail} is larger than the time horizon T . This misclassification error describes if the prediction is misclassified (false positive or false negative). However, consider the case where false negatives are more “costly” than false positives (e.g. a safety critical system where a failure can break the

robotic system). It will also be useful to consider a weighting of the above cost depending on the type of error (false positive vs false negative):

$$\tilde{C}_e(f, \pi) := \lambda_0 \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{z}_t) = 1 \cap z = 0] + \lambda_1 \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{z}_t) = 0 \cap z = 1] \quad (4.2)$$

where λ_0, λ_1 weighs the relative importance of false positives and false negatives (e.g., when $\lambda_0 = 0$ and $\lambda_1 = 1$, the cost function only considers false negatives).

Objective. The cost functions in Equation 4.1 and 4.2 are similar in form to $C_e(\pi_\theta)$ from Section 2.1.2 and we analogously will try to find a *distribution* P_f over failure predictors f in order to minimize the costs over the unknown distribution P_e over environments for a fixed policy π .

$$\min_{P_f} C_{P_e}(P_f, \pi) := \min_{P_f} \mathbb{E}_{e \sim P_e} \mathbb{E}_{f \sim P_f} C_e(f, \pi) \quad (4.3)$$

$$\min_{P_f} \tilde{C}_{P_e}(P_f, \pi) := \min_{P_f} \mathbb{E}_{e \sim P_e} \mathbb{E}_{f \sim P_f} \tilde{C}_e(f, \pi) \quad (4.4)$$

Based on the problem setup, we will be able to use many of the tools we have introduced in Chapter 2 in order to provide guarantees on the performance of the failure predictor for the objective in Equation (4.3). We will also extend the analysis for the objective in Equation (4.4).

4.2 Guaranteed prediction of failures

4.2.1 Bound on Misclassification Error

Our approach for learning failure predictors with guaranteed error bounds relies on a reduction to results from the PAC-Bayes generalization theory from supervised learning which is detailed in Chapter 11. We first present the associated PAC-Bayes generalization guarantee for failure prediction, which is very similar in form to Theorem 2.1.

Theorem 4.1 (PAC-Bayes Generalization Bound on Failure Predictor Misclassification Error). *For any data-independent prior distribution $P_{f,0}$ over f , some cost function C which satisfies Assumption 2.1, $n \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling*

of $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$, the following holds simultaneously for all distributions P_f over f :

$$C_{P_e}(P_f, \pi) \leq \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{f \sim P_f} C_{e_i}(f, \pi) + \sqrt{\frac{D_{\text{KL}}(P_f \| P_{f,0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}, \quad (4.5)$$

where D_{KL} is the Kullback-Leibler (KL) divergence.

PAC-Bayes bounds such as the one presented in Theorem 4.1 have the potential to provide strong upper bounds (see e.g. [63, 136, 162]). However, even a strong upper bound on the misclassification error can be insufficient for certain practical cases. Consider the case where the policy π succeeds on a high proportion (i.e. 99%) of the dataset S . A strong bias in the size of the classes (i.e. policy success vs policy failure) can lead to a strong bias in the failure predictor’s performance on each class. In particular, if we optimize the upper bound in Inequality (4.5) directly on a such a biased dataset, the resulting failure predictor will be more likely to predict policy successes correctly (i.e. low FPR) and less likely to predict policy failures correctly (i.e. large FNR). One could bias the training by re-weighting the data or cost function in order to achieve better performance, but the upper bound in the Theorem 4.1 does not capture this intuition. In the next section, we will provide the natural solution of using a weighted (class-conditional) misclassification error and bound FNR and FPR instead.

4.2.2 Bound on Class-Conditional Misclassification Error

In practice, our approach is to generalize the method above and utilize a weighted misclassification error. We can interpret the total misclassification as the class-weighted sum of conditional error probabilities. The following shows the desired generalization to an arbitrary weight $\lambda \in [0, 1]$:

$$\begin{aligned}
\mathbb{P}(\text{error}) &= \mathbb{P}(\hat{z} = 0 \cap z = 1) + \mathbb{P}(\hat{z} = 1 \cap z = 0) \\
&= \mathbb{P}(\hat{z} = 0|z = 1)\mathbb{P}(z = 1) + \mathbb{P}(\hat{z} = 1|z = 0)\mathbb{P}(z = 0) \\
&= \mathbb{P}(\hat{z} = 0|z = 1)(1 - \lambda^*) + \mathbb{P}(\hat{z} = 1|z = 0)(\lambda^*) \\
\rightarrow \text{ generalize to } &= \mathbb{P}(\hat{z} = 0|z = 1)(1 - \lambda) + \mathbb{P}(\hat{z} = 1|z = 0)(\lambda),
\end{aligned} \tag{4.6}$$

where $\mathbb{P}z = 0$ and $\mathbb{P}z = 1$ are true proportions of successes and failures in environments from P_e . Since the FNR and FPR, are unknown, we can use the empirical FNR and FPR, $\mathbb{P}_S(\hat{z} = 0|z = 1)$ and $\mathbb{P}_S(\hat{z} = 1|z = 0)$, present in the dataset S instead and propose the following error function as the empirical mean over environments in S :

$$\frac{1}{n} \sum_{e \in S} \mathbb{E}_{f \sim P_f} \hat{C}_e(f, \pi) := (1 - \lambda)\mathbb{P}_S(\hat{z} = 0|z = 1) + \lambda\mathbb{P}_S(\hat{z} = 1|z = 0). \tag{4.7}$$

However, these empirical proportions are random variables over the draw of S , and thus $\frac{1}{n} \sum_{e \in S} \hat{C}_e(f, \pi)$ will not admit a PAC-Bayes bound. Instead, we need an alternate error function that holds *with high probability* over dataset S . We provide one which is a tight over-approximation of the cost in Equation (4.7). Let $\underline{p}_i$ be a lower bound on $\mathbb{P}(z = i)$, which we find using Bernstein's Inequality. The error function then becomes:

$$\tilde{C}_e(f, \pi) = \frac{\lambda}{c_\lambda \underline{p}_0} \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{z}_t) = 1 \cap z = 0] + \frac{1 - \lambda}{c_\lambda \underline{p}_1} \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{z}_t) = 0 \cap z = 1], \tag{4.8}$$

$$c_\lambda := \frac{\lambda}{\underline{p}_0} + \frac{1 - \lambda}{\underline{p}_1}. \tag{4.9}$$

The cost function is bounded in $[0, 1]$ and does not rely on quantities that are random variables over the draw of S (unlike $\hat{C}$). Next, we present the corresponding PAC-Bayes bound for $\tilde{C}_e(f, \pi)$:

Theorem 4.2 (Class-Conditioned PAC-Bayes Bound). *For any data-independent prior distribution $P_{f,0}$ over f , cost function $\tilde{C}$ which satisfies Assumption 2.1, $n \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$, the*

following holds simultaneously for all distributions P_f over f :

$$\mathbb{E}_{e \sim P_e} \mathbb{E}_{f \sim P_f} \tilde{C}_e(f, \pi) \leq \frac{1}{n} \sum_{e \in S} \mathbb{E}_{f \sim P_f} \hat{C}_e(f, \pi) + R_\lambda, \quad (4.10)$$

$$R_\lambda = \frac{5}{3} \sqrt{\frac{(1-p) \log \frac{2}{\delta}}{n\underline{p}}} + C_\lambda \sqrt{\frac{D_{\text{KL}}(P_f \| P_{f,0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}. \quad (4.11)$$

Proof. A detailed proof is provided in Section 11.7.1. We utilize a union bound to ensure that all necessary events hold with high probability, and then combine the new error functions into the PAC-Bayes framework to obtain an adjusted regularizer term R_λ . $\square$

This theorem achieves two principal benefits: (i) The bounds meaningfully hold for settings where failures and successes are not balanced in the sample dataset. (ii) For a given confidence level, a constant number of instances of the less-common class is necessary to achieve a bound. Thus, the fundamental information constraint is the number of instances of the class that have been seen.

4.2.3 Algorithmic Implementation

Analogous to Section 2.2, we parameterize the failure predictors with neural networks and specify distributions over failure predictors using multivariate Gaussian distributions over the weights of the networks. We present the specific training algorithm along with further implementation details in Section 11.7.3. After training is complete, we can use the posterior P_f to compute PAC-Bayes bounds on error rates. The PAC-Bayes bound on the misclassification error can be computed directly with Inequality (4.5). To compute PAC-Bayes bounds on the FNR and FPR, we can use $\lambda = 0$ and $\lambda = 1$ respectively in Inequality (4.10).

4.3 Experiments

Overview. In this example, we train a failure predictor for a drone executing an obstacle-avoidance policy as in Section 2.1.3. The failure predictor uses depth images taken from the perspective of the drone. We train the failure predictor in simulation and apply it on a

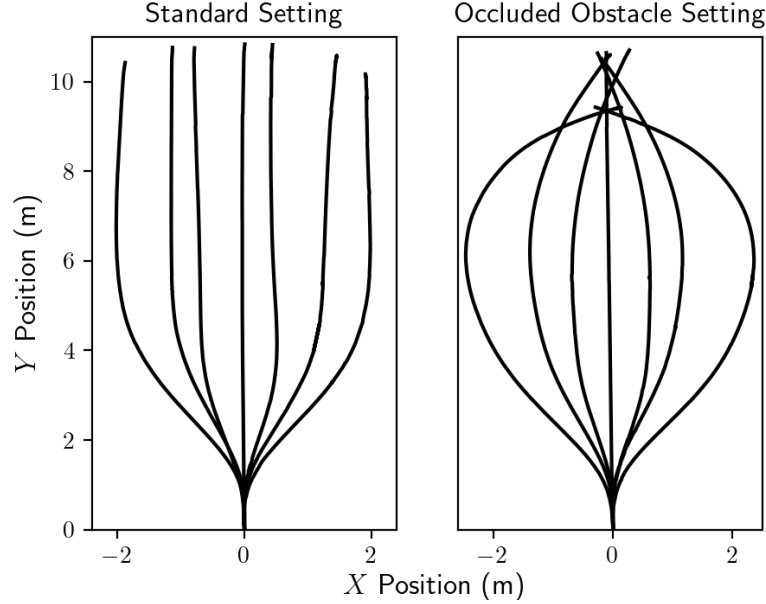


Figure 4.2: (Left) Representative motion primitives for the policy used in the standard setting in the navigation task. (Right) Representative motion primitives from the policy used in the occluded obstacle settings.

hardware platform with a Parrot Swing drone (see Figure 4.1). We consider a failure to be any instance in which the drone collides with an obstacle. If the failure predictor indicates that a crash will occur, we trigger an emergency policy that causes the drone to stop and land safely. In this section, we evaluate our bounds and failure prediction performance in simulation and also present extensive hardware experiments.

Policies and Environment Distributions. As in Section 2.1.3, the drone’s obstacle avoidance policy consists of a deep neural network (DNN) classifier which uses a depth image to select one out of a set of open-loop motion primitives for the drone to execute. We use a Vicon motion capture system to locate all obstacles in the environment as well as the drone’s position to generate artificial depth images from the perspective of the drone. It is important to note that neither the policy nor the failure predictor has access to any other information about the environment besides the first-person depth images.

We use two distinct policies and environment distributions in order to demonstrate our approach. In the first (“standard”) setting, obstacles are placed (uniformly) randomly in the environment; the drone’s policy then selects from a set of motion primitives which

cause the drone to travel in mostly straight paths (see Figure 4.2 (left)). In the second (“occluded”) setting, the environment is generated in two stages. First, a number of obstacles are (uniformly) randomly placed. Second, we generate additional obstacles that are placed exclusively in locations that are occluded by obstacles generated in the first stage. The drone’s policy then uses the depth image captured from its starting location to select a motion primitive from the set shown in Figure 4.2 (right). These primitives cause the drone to travel in curved paths (which may intersect initially-occluded obstacles). Since obstacles in this setting are specifically placed in portions of the environment that are occluded, this setting is adversarial in nature and leads to a large number of failures. The policy for the standard setting results in a failure rate of 25.3%, while the policy for the occluded setting results in a failure rate of 51.4%. Our approach allows for the use of a black-box policy which is not necessarily deployed in settings where it was trained nor where it would perform well. We aim to examine the improvement in safety of the policy with the addition of the failure predictor; thus, we test in settings that are challenging and even adversarial to the policy.

Failure Predictor and Training. The failure predictor consists of a DNN which takes as input depth images from the perspective of the drone. The network receives a new image at a frequency of 20Hz and stacks the four most recent images as input to predict failure or no failure at some point in the future (i.e. 2-classes), with a Softmax layer at the end. If at any time step, the failure class is largest, we say it is a prediction of failure; otherwise, it is a prediction of success. We use 10,000 training environments to train the prior distribution $P_{f,0}$ over failure predictors and an additional 10,000 training environments to optimize the PAC-Bayes upper bound in Inequality (4.5). We use Algorithm 11.1 to obtain a posterior distribution P_f and the associated PAC-Bayes generalization guarantee. We note that the physical drone takes approximately 0.5s to slow to a stop; thus, the failure predictor needs to predict collisions 10 time-steps ahead (since the frequency of the input is 20Hz).

After training, we compute the bound on the misclassification error with Inequality (2.4), in addition to the bounds on the FNR and FPR with Inequality (4.10). At test time, we sample a failure predictor $f \sim P_f$ and fix it for a particular test environment.

Simulation Results. We first verify our failure predictors in simulation before testing

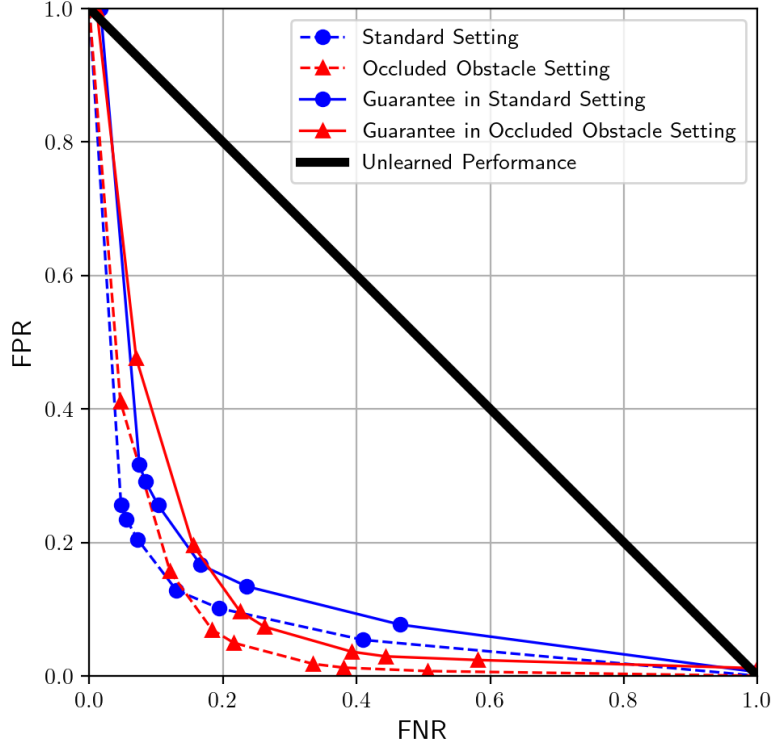


Figure 4.3: The dashed lines show the resulting failure predictor’s FNR and FPR when the importance of a false negative is varied. The solid lines and markers show the associated PAC-Bayes guarantee on the FNR and FPR using the class-condition guarantees introduced in Sec. 4.2.2.

them on the hardware setup. There is an intrinsic trade-off between the false-positive and false-negative rates in general. To validate our theoretical framework, we plot the bounds on the FNR and FPR given by Inequality (4.10) (solid lines in Figure 4.3). We compare these bounds with the empirical FNR and FPR (estimated with 20000 held out environments) of the predictor with varied false-negative importance during training; these are plotted using dashed lines in Figure 4.3. We obtain strong bounds on the failure predictors’ errors, with closely-matching empirical performance along all points along the curve.

Hardware Results. We select a single predictor for each of the settings (standard and occluded) to use in the hardware experiments. We choose the failure predictors with the tightest guaranteed total error rate (i.e. tightest upper bound from Inequality (4.5)). When the failure predictor stops the rollout due to a prediction of failure, we re-run the trial without the failure predictor to determine the true label. See Figure 4.1 for an example

Table 4.1: Results for Failure Prediction on Navigation Task

Setting	Standard	Occluded Obstacle
True Expected Failure (Sim)	0.253	0.514
Misclassification Bound	0.128	0.154
True Expected Misclassification (Sim)	0.101	0.125
True Expected Misclassification (Real)	0.067	0.133

of the obstacle placement for the occluded obstacle setting. Additionally, a video with representative trials from both settings is available at <https://youtu.be/z4UwQzTjhqo>. We run 15 trials in each of the settings and show the misclassification error of the failure predictors along with the guarantee on the misclassification in Table 4.1. In the standard setting, $\mathbb{P}(\hat{z} = 1 \cap z = 1) = \frac{2}{15}$, $\mathbb{P}(\hat{z} = 0 \cap z = 0) = \frac{12}{15}$, $\mathbb{P}(\hat{z} = 1 \cap z = 0) = \frac{1}{15}$, $\mathbb{P}(\hat{z} = 0 \cap z = 1) = 0$, and in the occluded obstacle setting $\mathbb{P}(\hat{z} = 1 \cap z = 1) = \frac{9}{15}$, $\mathbb{P}(\hat{z} = 0 \cap z = 0) = \frac{4}{15}$, $\mathbb{P}(\hat{z} = 1 \cap z = 0) = \frac{2}{15}$, $\mathbb{P}(\hat{z} = 0 \cap 1) = 0$. The misclassification bounds are validated by results from both the standard and occluded obstacle settings. Additionally, in all trials where the failure predictor was used, the drone never crashed due to a missed failure (i.e., there were no false negatives). These experiments demonstrate the power of the failure predictor to maintain the robot’s safety in novel and even adversarial settings.

Chapter 5

Failure detection for trajectory predictors

5.1 Motivation

Throughout this chapter, we will use an autonomous robotic driver, or autonomous vehicle (AV), as the example as it demonstrates one of the key challenges of larger scale robotic systems: robotic systems with high complexity and many learning modules are susceptible to failure when they encounter novel scenarios and edge cases. In general, AV have four key modules which make up the *control stack*: (i) perception, where objects in the environment are recognized and classified, (ii) prediction, where future trajectories are predicted for each object in the environment, (iii) planning, where a motion plan is computed based on the output of the prediction module, and (iv) control, where the plan is converted to lower-level control of the vehicle and operation is run in the environment.

The accuracy of the predictions is crucial for the motion planner to ensure that the AV not enter an unsafe state. However, trajectory prediction has a high degree of uncertainty due to the stochasticity of human driving, biking, and walking behavior. This is made worse from the challenge that the predictions should be conditioned on the motion plan, and collecting such data which meaningfully connects potential plans with appropriate predictions for other agents is difficult, if possible at all. Thus, in a very uncertain

driving environment, how can we ensure the safety of the AV without compromising its functionality? We approach this challenge by developing a *task-relevant* failure detector for the prediction module that only triggers if the estimate of the realized cost at a given time-instant is significantly worse than the cost predictions based on trajectory forecasting.

In the previous chapter, we developed a real-time failure predictor for a blackbox policy operating in novel environments. The key intuition was to formulate a PAC-Bayes bound on the misclassification error of the failure predictor as well as a class-conditional misclassification error in order to independently guarantee detection of false-positives and false-negatives. The key characteristics of the failure predictor were (i) guaranteed confidence of detection and (ii) real-time failure prediction. However, an important challenge with this method was the availability of a dataset S of environments which we could run the blackbox policy in order to learn to predict failures. In practice, it may be difficult or costly to obtain such a dataset. Additionally, in the previous chapter, we focused on detecting failures. However, for larger scale robotic systems such as autonomous vehicles (AV), detection of even a slight degradation in performance may be crucial in maintaining the safety of the vehicle and other human drivers.

In this chapter, we will avoid the requirement of a training dataset S and aim to detect any anomalous setting which is causing degradation in performance. However, we still aim to operate in the more general setting of a black-box control policy and thereby avoid the requirement of a specific training pipeline for the control policy. Thus, the key desired characteristics for the failure detector we will develop in this chapter are as follows:

1. The detector should have a guaranteed confidence of detection.
2. The failure detector should operate in real-time in order to inform the policy of potentially unsafe settings.
3. The failure detector should be task-relevant and only signal a detection if the cost is negatively affected.
4. The detector should operate without the need to see examples of failures.

Importantly, we will not require PAC-Bayes bounds in order to provide the strong guaran-

teed confidence on detection.

5.1.1 Problem setup

As in Section 2.1.2, let $\mathcal{X}$ be the state space, $\mathcal{Y}$ be the space of observations that the ego agent receives from its sensors, $\mathcal{U}$ be the space of the control inputs for the ego agent, and $f_e(x_{t+1}|x_t, u_t)$ be the stochastic state-transition function depending on a particular environment e . We let a state $x \in \mathcal{X}$ be composed of the state of the ego agent x^e , non-ego agents x^{ne} , and other variables such as the environment map x^{m} . To simplify notation, we will drop the subscript notation for an environment e in this chapter as it will not be necessary for the exposition.

Let $c_t : \mathcal{X} \times \mathcal{U} \rightarrow \mathbb{R}$ be an instantaneous cost function at a time instant t . Our framework is agnostic to the choice of the cost function; we only assume that the cost function is formulated such that higher values imply less favorable outcomes for the AV. An example cost function is the closest distance of an AV to other non-ego agents and obstacles in the map. Given the predicted state $\hat{x}_{t+\tilde{t}}$ and the control input $u_{t+\tilde{t}}$ at $\tilde{t}$ time steps in the future, the predicted cost is $\hat{c}_{t+\tilde{t}} := c_{t+\tilde{t}}(\hat{x}_{t+\tilde{t}}, u_{t+\tilde{t}})$.

With a slight abuse of notation, let $x_t \in \mathcal{X}$ be an *estimate* of the state (instead of the true state) at time t provided by the perception module by processing the sensor input y_t . The prediction module takes in a history of state estimates $x_{0:t}$ and provides a distribution $\psi(\hat{x}_{t+1:t+T}^{\text{ne}}|x_{0:t})$ on the predicted future state trajectories $\hat{x}_{t+1:t+T}^{\text{ne}}$ of non-ego agents over the horizon T . The distribution produced by the prediction module induces a sequence of distributions $\{\phi_{t+\tilde{t}}(\hat{c}_{t+\tilde{t}}|x_{0:t})\}_{\tilde{t}=1}^T$ on the predicted cost $\hat{c}_{t+\tilde{t}}$ for each time-step in the prediction horizon T . The observed cost $c_{t+\tilde{t}}^* := c_{t+\tilde{t}}(x_{t+\tilde{t}}, u_{t+\tilde{t}})$ at time-step $t + \tilde{t}$ is the cost estimated or “observed” by the AV given state-estimates of the ego and non-ego agents $x_{t+\tilde{t}}$ and the control input $u_{t+\tilde{t}}$. We define the notion of prediction anomalies explicitly which will underpin our approach to failure detection:

Definition 5.1 (*p*-quantile anomaly). *For any time-step $t + \tilde{t}$, let $\phi_{t+\tilde{t}}(\hat{c}_{t+\tilde{t}}|x_{0:t})$ be the distribution on the predicted cost as described above. If the observed cost $c_{t+\tilde{t}}^*$ lies in the top p -quantile of $\phi_{t+\tilde{t}}(\hat{c}_{t+\tilde{t}}|x_{0:t})$, then the prediction module suffers from a p -quantile anomaly.*

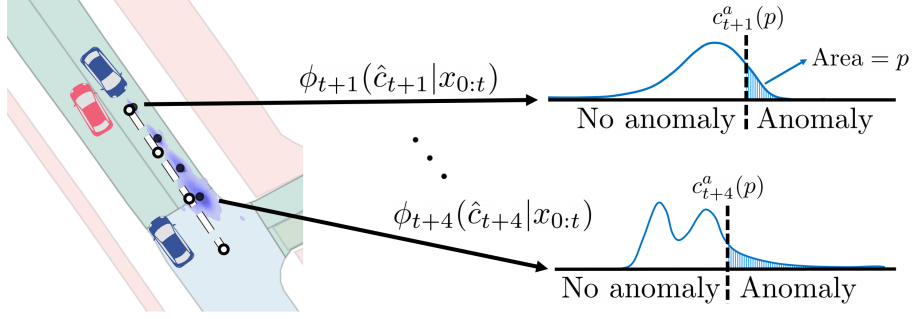


Figure 5.1: Illustration of the concept of p -quantile anomalies. The ego vehicle is red, non-ego vehicles are blue, ground truth trajectories are white, and trajectory predictions are the purple contour plots. At each time step, the cost function induces a distribution $\phi_{t+\tilde{t}}$ on predicted costs depicted by the blue plots on the right. If the observed cost $c_{t+\tilde{t}}^*$ is within the top- p quantile of $\phi_{t+\tilde{t}}$, it is a p -quantile anomaly.

Note that the observed cost $c_{t+\tilde{t}}^*$ is not necessarily drawn from the predicted cost distribution $\phi_{t+\tilde{t}}$. However, if there are a relatively small number of task-relevant prediction errors (which is a reasonable assumption for a larger scale AV system which has millions of training data points), we can expect the observed cost to be similar to the sample predicted costs. This motivates Definition 5.1 and a prediction failure detector based on the concept of a p -quantile anomaly. See Figure 5.1 for an illustration of the concept of a p -quantile anomaly. Note that since the p -quantile anomaly is based on the planning cost c , it captures the notion of task-relevance. As such, an observed cost in the p lower quantile would correspond to *task-irrelevant* prediction error.

5.2 Guaranteed detection of prediction failures

In this section, we develop a method for detecting p -quantile anomalies which carries guaranteed false-positive and false-negative rates. Let $c_{t+\tilde{t}}^a(p)$ be the cost which separates the top p proportion of $\phi_{t+\tilde{t}}(\hat{c}_{t+\tilde{t}}|x_{0:t})$ from the bottom $1 - p$. If $c_{t+\tilde{t}}^* \geq c_{t+\tilde{t}}^a(p)$, then we have a p -quantile anomaly. However, it is not usually possible to compute $c_{t+\tilde{t}}^a(p)$ explicitly since we do not have an analytical expression for ϕ . We will use a sampling-based approach for detecting if the observed cost lies within the top p -quantile of the distribution on the predicted cost.

At time $t + \tilde{t}$, we will sample M predicted costs $S_{t+\tilde{t}} := \{\hat{c}_{t+\tilde{t}}^1, \dots, \hat{c}_{t+\tilde{t}}^M\} \sim \phi_{t+\tilde{t}}^M(\hat{c}_{t+\tilde{t}}|x_{0:t})$.

We then arrange them in increasing order such that $\hat{c}_{t+\tilde{t}}^1 \leq \hat{c}_{t+\tilde{t}}^2 \leq \dots \leq \hat{c}_{t+\tilde{t}}^M$. We define the following detector D :

$$D(n, S_{t+\tilde{t}}) := c_{t+\tilde{t}}^* \geq \hat{c}_{t+\tilde{t}}^{M-n}, \quad (5.1)$$

which returns **true** if the observed cost is greater than at least $M - n$ predicted costs in $S_{t+\tilde{t}}$ and **false** otherwise. Using this detector, an intuitive algorithm follows for determining if there has been a p -quantile anomaly.

First, we sample M trajectories of T time steps each for all agents in the scene using the trajectory prediction module. At each time step $\tilde{t} \in \{1, \dots, T\}$, we compute $D(n, S_{t+\tilde{t}})$. If the detector ever returns **true**, then we label the prediction as anomalous. Note that by separating costs by agent (i.e. each agent in the scene has its own predicted and observed cost), we can detect which agent is causing the anomalous predictions. We call the algorithm *quantile anomaly detection (QAD)* and provide more discussion in Section 12.4.1.

Consider an AV system using the detector we have described above. If there are many extra detections when no anomaly is occurring, the ability of the system to finish a particular goal will be diminished. Analogously, missed detections could cause safety critical failures. As such, it is important to limit the number of errors for detectors operating on safety critical systems. Based on the detection approach we have described, we will prove bounds on the false-positive and false-negative rates for detection.

5.2.1 Bounds on False-positive and False-negative Rates

Intuitively, a false-positive occurs when the detector outputs **true** when the observed cost $c_{t+\tilde{t}}^*$ is below the p -quantile anomaly threshold $c_{t+\tilde{t}}^a(p)$. Analogously, a true-negative occurs when the detector outputs **false** when the observed cost is above the threshold for a p -quantile anomaly. Formally, we have the follow definitions for the FPR and FNR

$$\text{FPR} := \mathbb{P}_{S_{t+\tilde{t}} \sim \phi_{t+\tilde{t}}^M} [c_{t+\tilde{t}}^* \geq \hat{c}_{t+\tilde{t}}^{M-n} \wedge c_{t+\tilde{t}}^* < c_{t+\tilde{t}}^a(p)] \quad (5.2)$$

$$\text{FNR} := \mathbb{P}_{S_{t+\tilde{t}} \sim \phi_{t+\tilde{t}}^M} [c_{t+\tilde{t}}^* < \hat{c}_{t+\tilde{t}}^{M-n} \wedge c_{t+\tilde{t}}^* \geq c_{t+\tilde{t}}^a(p)] . \quad (5.3)$$

Based on these definitions, we provide the following bound on the FPR and FNR rates:

Theorem 5.1 (FPR and FNR Bound). *Let $D(n, S_{t+\tilde{t}})$ be the detector for the occurrence of a p -quantile anomaly for a fixed n and sorted dataset $S_{t+\tilde{t}} \sim \phi_{t+\tilde{t}}^M(\hat{c}_{t+\tilde{t}}|x_{0:t})$. Then, the FPR and the FNR for the detector are bounded as follows:*

$$FPR \leq \overline{FPR} := \sum_{i=0}^n \binom{M}{i} p^i (1-p)^{M-i}, \quad (5.4)$$

$$FNR \leq \overline{FNR} := \sum_{i=n+1}^M \binom{M}{i} p^i (1-p)^{M-i}. \quad (5.5)$$

Proof. The proof of this theorem is presented in Section 12.7.1. □

This ensures that if the detector identifies an anomaly (i.e., D holds true), then with probability at least $1 - \overline{FPR}$, the observed cost does lie in the top- p quantile of $\phi_{t+\tilde{t}}$, while if our detector does not detect an anomaly (i.e., D is not true), then with probability at least $1 - \overline{FNR}$, the observed cost does not lie in the top- p quantile of $\phi_{t+\tilde{t}}$.

The upper bounds for the FPR and the FNR in Theorem 5.1 allows data-free calibration of D by letting us choose M and n so as allow a user to specify FPR and FNR values. This was one of the key desired characteristics of our detector mentioned in Section 5.1 as it prevents the need for a potentially difficult-to-obtain dataset of anomalies. However, we cannot simultaneously provide a strong bound on the FPR and FNR (as we could in Chapter 4). This follows from the observation that the upper bounds in Theorem 5.1 satisfy $\overline{FPR} + \overline{FNR} = 1$. Note that not a fundamental limitation of our detector. We show in the examples in next section that our detector achieves low FPR and low FNR simultaneously in practice.

5.3 Experiments

In this section, we benchmark the performance of our detector on nuScenes [38] and nuPlan teaser [39] datasets against various other methods. We study the detector in two applications: (i) run-time monitoring of task-relevant prediction network failures and (ii) filtering interesting scenarios from unlabelled expert driving logs. Through these experiments we demonstrate the ability of our detector to provide low FPR and low FNR for

detecting task-relevant mispredictions. All experiments were conducted on a desktop computer with an Intel i9-10980XE CPU (18 cores) and 64 GB of RAM. Our code is available at: <https://github.com/NVlabs/pred-fail-detector>.

Trajectory prediction module. While our method is agnostic to the particular prediction method used, in our experiments we employ Trajectron++ [177], a state-of-the-art multi-agent trajectory forecasting model, to predict the motion of non-ego agents. It is a graph-structured recurrent neural network that predicts an agent’s future position distribution given its past trajectory history and the past trajectories of its neighboring agents. At its core, Trajectron++ uses a conditional variational autoencoder (CVAE) to model the potential for multiple future trajectories.

Detection and comparison methods. We compare our detector— p -quantile anomaly detection (QAD)—against four other approaches: (i) *likelihood detection* baseline which triggers when the likelihood of the achieved states of an agent in the predicted GMM is below a threshold; (ii) *uniform and partial cost degradation* tests [83] which uses p -values to detect shifts in the cost distribution; (iii) *time-to-collision (TTC)* detector which triggers when the TTC drops below a threshold; and (iv) detection based on *Hamilton-Jacobi (HJ) reachability* analysis [123] which triggers if a collision is possible assuming an adversarial behavior by the non-ego agent. See Section 12.7.2 for more information about the comparison methods.

5.3.1 Task-relevant Detection

Overview. We study the use of our detector and other baseline detectors mentioned above as run-time monitors for task-relevant prediction failure. The experiments are conducted in the recently released nuPlan teaser dataset [39] which allows the inclusion of reactive agents in the driving logs.

Planner. We assume a behavior generator provides us with a reference trajectory to follow. Our planner is then tasked with following this trajectory while avoiding collision with other agents on the road, ensuring ego comfort and satisfying the ego’s dynamical constraints. The planner achieves this by generating a motion primitive tree [180] and choosing the trajectory that minimizes the weighted sum of a series of cost functions; further

details on the planner are supplied in Section 12.7.2. We use multiple standard AV planning costs: time-to-collision cost c_{ttc} (computed assuming that agents continue moving at a constant speed in their current heading), momentum-shaped distance to other agents cost c_{d2a} , distance to goal cost c_{d2g} , distance to the reference trajectory cost c_{d2r} , speed limit cost c_{velocity} , ego comfort cost c_{comfort} , and reversing cost c_{reverse} which penalizes the car for going against the heading direction of the lane:

$$c_{t+\tilde{t}} = w_1 c_{\text{ttc}} + w_2 c_{\text{d2a}} + w_3 c_{\text{d2g}} + w_4 c_{\text{d2r}} + w_5 c_{\text{velocity}} + w_6 c_{\text{comfort}} + w_7 c_{\text{reverse}} . \quad (5.6)$$

The exact expressions for the cost terms and the weights $w_1, \dots, w_7$ used in the experiments are provided in Section 12.7.2. Note that the performance of the planner is sensitive to the weights and cost function design. Ultimately, we do not aim to design a strong planner but to use the planning cost for quantile anomaly detection so that anomalies are *task-relevant* prediction errors. The motion planner uses trajectory forecasts of other non-ego agents drawn from Trajectron++ to compute the momentum-shaped distance and the time-to-collision cost. The cost functions take about 1 ± 0.5 ms to evaluate. Note that these cost evaluations for planning are reused by our anomaly detector, thereby minimizing any excessive computational overhead by our detector. The anomaly detection computation takes less than 0.1 ms.

Dataset and Labeling. We test on 260 nuPlan [39] driving scenarios which comprises of a total of 2568 prediction horizons. We hand-label each prediction horizon for task-relevant prediction errors by visually inspecting plots of true trajectories of agents along with their predicted trajectories. We emphasize that the labeling is done merely by visual inspection and does not use the planning costs or prediction likelihoods. This is to ensure that the labeling procedure is independent of the detection methods to promote fair comparisons; see Section 12.7.2 for more details on the labeling procedure.

It is important to note that despite the use of reactive agents along with our custom planner in nuPlan, there is a strong bias in the dataset towards benign scenarios. Only 3.5% of this dataset is labeled as positive. Due to the imbalance, there is a bias toward detectors to exhibit lower FPR and higher FNR, since false-negative errors are amplified

while the false-positive errors are mitigated.

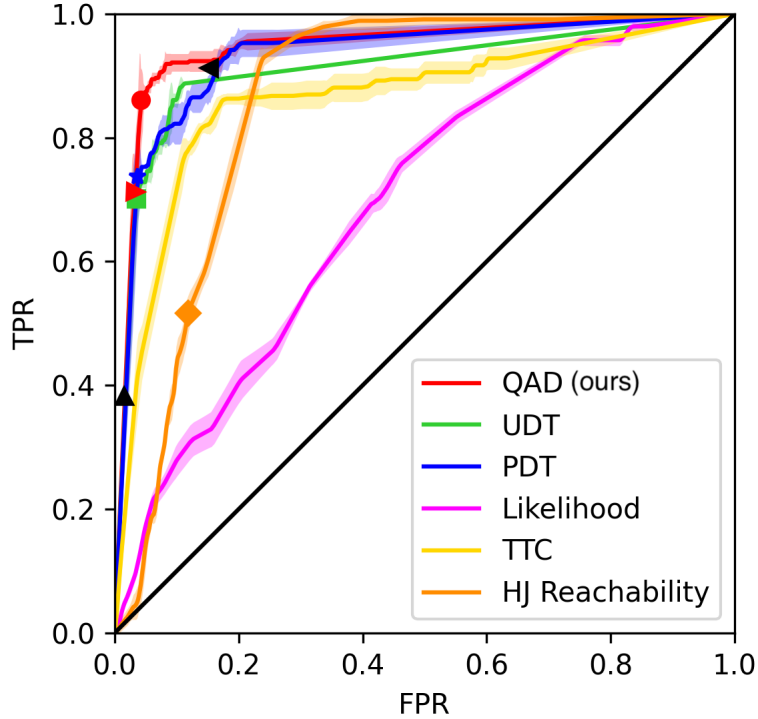


Figure 5.2: ROC curve for detectors. We report the mean of 5 trials, the shaded region is 2 times the standard deviation. The colored shapes correspond to the data-free calibration detectors in Table 5.2.

Table 5.1: AUROC and best point from ROC ($\uparrow$ AUROC)

Detection Method	AUROC	FPR %	FNR %
QAD (ours)	0.946 ± 0.002	8.4 ± 0.4	7.9 ± 0.7
PDT [83]	0.932 ± 0.005	18.5 ± 0.2	5.8 ± 1.1
UDT [83]	0.912 ± 0.002	10.6 ± 0.3	11.5 ± 0.4
TTC	0.864 ± 0.004	17.3 ± 0.5	13.7 ± 0.4
HJ Reachability [123]	0.862 ± 0.003	23.7 ± 0.3	7.2 ± 0.6
Likelihood	0.693 ± 0.006	45.9 ± 0.4	24.5 ± 0.4

Results. We plot the Receiver Operator Characteristic (ROC) curve in Figure 5.2 for each detector by varying: the detection rank $M - n$ in Equation (5.1) for our detector, the p -value for UDT and PDT, the likelihood threshold for the likelihood-based detector, the TTC threshold for TTC detector, and the sub-level set of the value function for the HJ reachability based detector. The ROC curve for our detector is the closest to an ideal detector and has the highest Area Under the ROC (AUROC) curve. We obtain the best

Table 5.2: Data-free calibration ($\uparrow$ distance from $y = x$)

Detection Method		FPR %	FNR %	Dist.
QAD (FNR) (ours)	●	4.2±0.3	13.9±1.1	0.58±0.01
FNR Ensemble (ours)	◀	15.2±0.3	8.7±1.2	0.54±0.01
PDT (FPR) [83]	★	3.7±0.3	26.3±0.6	0.50±0.01
QAD (FPR) (ours)	▶	3.4±0.3	28.8±0.9	0.48±0.01
UDT (FPR) [83]	■	3.5±0.3	29.9±0.9	0.47±0.01
HJ Reachability [123]	◆	11.8±0.2	48.6±1.7	0.28±0.01
FPR Ensemble (ours)	▲	1.5±0.3	61.6±1.8	0.26±0.01

calibration for each detector by choosing the point that is farthest from the $y = x$ line in Figure 5.2 and report the empirical FPR and FNR for them in Table 5.1. Our detector has the lowest FPR and the third-lowest FNR. PDT has a slightly better FNR than our method, but the FPR is much higher, while for all other methods both FPR and FNR are higher than our method. As one would expect, the naive likelihood baseline does not perform very well while the HJ reachability detector is overly conservative exhibiting a lot of false-positives.

We also compare the performance of our detector with data-free calibration using Theorem 5.1 with the data-free calibration of UDT, PDT, and HJ reachability. For our detector, we consider an anomaly to have occurred when the true cost lies in the top-5% of the predicted cost distribution (see Definition 5.1); we chose the 5% quantile because it is a commonly used threshold for detection in statistics. We include two versions of our detector in this study: (i) calibrated to bound the FPR below 5% and (ii) calibrated to bound the FNR below 5% using Theorem 5.1. As discussed in Section 5.2, we cannot simultaneously obtain a strong bound on both FPR and FNR. We use a p -value of 0.05 for UDT and PDT, and a 0-sublevel set of the value function for HJ reachability. Additionally, we construct an FPR ensemble detector, which only detects positive if *all* of QAD (FPR), UDT, PDT, and HJ reachability are positive, and an FNR ensemble detector, which detects positive if *any* of QAD (FNR), UDT, PDT, and HJ reachability are positive. For the ensemble detectors, we use the data-free calibration for each method described above.

These detectors are labeled using markers in the ROC curves in Figure 5.2. Their empirical FPR and FNR is reported in Table 5.2 and we sort the table by distance from the

line $y = x$. We observe that our FPR bounded detector satisfies the 5% bound provided by Theorem 5.1; however, our FNR bounded detector violates the bound. This disagreement between the theory and the empirical performance for FNR occurs because of the mismatch between our anomaly criteria of top-5% quantile and what the human data labeller considers to be an anomaly. A higher empirical FNR suggests that the anomalous quantile should be expanded beyond 5%. This is also illustrated by the fact that the optimal p -quantile anomaly detector obtained using the ROC curve in Table 5.1 with 7% FNR corresponds to approximately the top-35% quantile as an anomaly. Furthermore, the imbalance in the dataset also contributes to this problem, as discussed above. More data as well as a more balanced dataset can perhaps remedy this mismatch and lower the FNR. Despite this mismatch, we see in Table 5.2 that our FNR detector performs the best out of all detectors in terms of distance to the $y = x$ line in the ROC plot. The FNR ensemble and PDT also perform well, favoring false-negatives and false-positive respectively. Since false-negatives can be safety critical, a designer may choose our FNR ensemble detector at the cost of more false-positives. Ultimately, our FNR detector is similar to the optimal detector obtained from the ROC curve, i.e., with *no data at all* we are able to calibrate our detector using Theorem 5.1 to perform very well.

5.3.2 Filtering expert driving logs

Overview. Given driving data logs, we consider the task of filtering scenarios with anomalies that may be of interest for diagnostic purposes or for qualitatively evaluating the prediction module. We use the nuScenes and nuPlan datasets as the driving logs. For the likelihood detector we choose the threshold 0.05, and for TTC we choose 1 second, while for all other detectors, we use the data-free calibration specified in Section 5.3.1.

Dataset. The dataset consists of 375 nuPlan and 152 nuScenes scenarios with a total of 3890 prediction horizons, in all of which the ego-vehicle is driven by a human driver and there are no collisions.

Cost functions. Since the nuScenes and nuPlan driving logs are collected by a human driver, we do not have access to the true planning “cost” used by the driver. Instead, we use a weighted sum of cost terms contributed by the momentum-shaped distance and the

time-to-collision for each non-ego agent as a proxy cost: $c_{t+\bar{t}} = w_1 c_{\text{ttc}} + w_2 c_{\text{d2a}}$. The exact expressions of c_{ttc} and c_{d2a} are provided in Section 12.7.2. Our results suggest that this cost function is well-aligned with human intuition as it allows us to filter out interesting scenarios while ignoring the uninteresting ones.

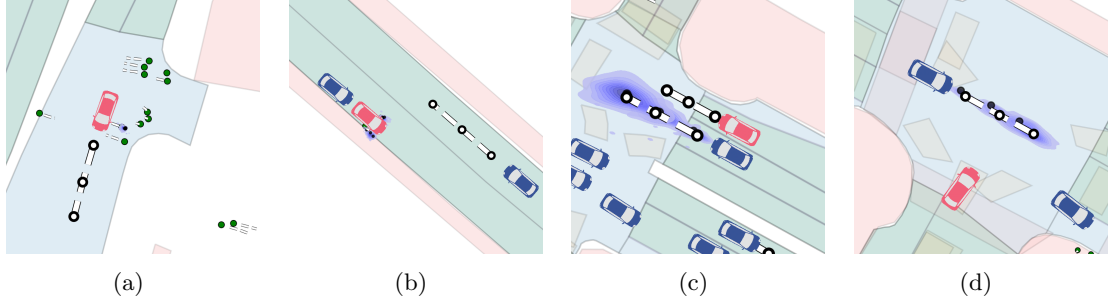


Figure 5.3: Selected scenarios from nuScenes. The ego vehicle is red, non-ego vehicles are blue, pedestrians are green circles, ground truth trajectories are white, and Trajectron++ predictions are the purple contour plots.

Table 5.3: Detections from expert driving logs

Detection Method	Percentage Detected
UDT [83]	5.4 ± 0.1
QAD (FPR) (ours)	5.4 ± 0.1
PDT [83]	5.8 ± 0.1
QAD (FNR) (ours)	6.4 ± 0.1
HJ Reachability [123]	8.8 ± 0.0
TTC	17.4 ± 0.0
Likelihood	58.5 ± 0.0

Results. We compare our detector to all baseline methods described earlier. See Table 5.3 for a comparison of the methods and Figure 5.3 for representative examples of positive detections. We observe that the likelihood detection and the TTC-based detection both have many more positives than the other methods, followed by HJ reachability. Other methods have a similar number of positives. Ultimately, no driving scenario from this dataset is unsafe, so we inspect the positive detections from each method to find interesting scenarios. All methods detect a handful of interesting scenarios, such as Fig 5.3(b) where pedestrians are right beside the ego (possibly entering or exiting). We also see additional interesting scenarios detected by our method but not HJ reachability such as in Figure 5.3(a) where a close-by pedestrian is predicted to stand still, but moves quickly towards the ego at

a crosswalk. Qualitatively, it seems that our detector, UDT, and PDT all do a good job of identifying interesting scenarios such as those shown in Figure 5.3(a) and (b). We see that HJ reachability, and likelihood detection methods more frequently labels driving scenarios where the predictions are accurate and there is nothing inherently unsafe as anomalous such as in Figure 5.3(c) and (d).

Chapter 6

Out-of-distribution generalization

6.1 Motivation

In the past three chapters, we have developed anomaly or failure detectors with guaranteed confidence of detection in each case. In Chapter 5, we were able to perform detection without a dataset of failures, with guarantees, in real time, and such that only task-relevant errors were detected. Although these methods are powerful, ultimately they are all limited. The methods presented in Chapter 3 operate after the fact and cannot be used to improve the safety of a robotic system online. The methods presented in Chapter 4 require a dataset of examples from the testing distribution, which may not be possible to obtain in practice. The methods presented in Chapter 5 rely on strong prediction module performance and cannot detect errors with other modules of a complex autonomous robotic control stack. An approach to deal with all of the challenges is to provide direct out-of-distribution (OOD) generalization. Can we develop a training procedure which provides strong (PAC style) guarantees on a robotic system when deployed in an OOD setting? If we can learn policies with *a priori* guarantees on performance and safety when deployed in OOD environments, we would not need OOD detectors in order to provide strong performance and safety when deployed in OOD environments.

The approach of providing direct generalization to new settings is more powerful. However, the settings on which we can provide out-of-distribution generalization guarantees is more limited. In this chapter we will present two settings on which out-of-distribution gen-

eralization is tractable and we are able to provide tight generalization guarantees. In the first, we derive a generalization guarantee that holds over a distribution shift of bounded KL divergence. In the second, we derive a generalization guarantee on a meta-learning setting for environment types of different but related contexts.

6.2 Generalization to distribution shifts

In this section, we will extend the methods in Section 2.2 to provide a guaranteed on a test distribution of environments which have been shifted by bounded shift from the training distribution. We first analyze the problem using a general f -divergence (Definition 6.1), and then specialize to the KL divergence.

6.2.1 f -divergence

Following the same setup as in Section 2.2, we are given a dataset of training data $S \sim P_e^n$ in order to train our robotic system. We will be tested on a shifted distribution P'_e , but do not receive any samples from this distribution. We will assume that the distribution P'_e from which test environments are drawn is bounded in terms of an f -divergence from the training distribution P_e and formulate a robust version of the PAC-Bayes bound we presented in Theorem 2.1.

Definition 6.1 (f -divergence between P'_e and P_e [151]). *For any convex $f(x)$ such that $f(1) = 0$, let*

$$D_f(P'_e \| P_e) := \mathbb{E}_{e \sim P_e} \left[f \left(\frac{P'_e}{P_e} \right) \right]. \quad (6.1)$$

The f -divergences encapsulate a broad class of divergences between distributions and include the KL divergence as a special case (with $f(x) = x \log x$). We will assume that the test distribution P'_e is bounded in terms of an f -divergence: $D_f(P'_e \| P_e) \leq \mathcal{B}$ (but no further assumption on P'_e will be made). The control policy we learn will have an associated guarantee on *any* test distribution that satisfies this assumption.

Theorem 6.1 (f -divergence between P'_e and P_e in terms of f and f^* [151]). *For a given $f(x)$ and its convex conjugate $f^*(y) := \sup_{x \in \mathbb{R}} [xy - f(x)]$, we can write the f -divergence*

$D_f(P'_e \| P_e)$ in terms of only f and its conjugate:

$$D_f(P'_e \| P_e) = \sup_C \left(\mathbb{E}_{e \sim P'_e} \mathbb{E}_{\theta \sim P_\theta} C_e(\pi_\theta) - \mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} f^*(C_e(\pi_\theta)) \right). \quad (6.2)$$

The supremum above is taken over all functions C that result in the expectations in the RHS being finite. Thus, for any particular cost function C , we can obtain a lower bound on the supremum term. This allows us to obtain the following useful corollary.

Corollary 6.1 (*f*-divergence variational inequality). *If $D_f(P'_e \| P_e) \leq \mathcal{B}$, then*

$$C_{P'_e}(P_\theta) := \mathbb{E}_{e \sim P'_e} \mathbb{E}_{\theta \sim P_\theta} C_e(\pi_\theta) \leq \mathcal{B} + \mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} f^*(C_e(\pi_\theta)). \quad (6.3)$$

Note that this corollary is valid for any f -divergence. In particular, it holds when $f(x) = x \log x$, making $f^*(y) = \exp(y - 1)$. With this choice of f , we obtain an upper bound on $C_{P'_e}(P_\theta)$ in terms of the bound $\mathcal{B}$ on the KL divergence.

Corollary 6.2 (KL divergence variational inequality). *For f -divergence with $f(x) = x \log x$ and $D_f(P'_e \| P_e) \leq \mathcal{B}$, we have $D_f(P'_e \| P_e) = D_{\text{KL}}(P'_e \| P_e) \leq \mathcal{B}$ and*

$$C_{P'_e}(P_\theta) \leq \mathcal{B} + \mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} \exp(C_e(\pi_\theta)) - 1. \quad (6.4)$$

While Corollary 6.2 provides a valid inequality in the special case of the KL divergence, a tighter bound can be obtained using the *Donsker-Varadhan (DV)* inequality.

Theorem 6.2 (Donsker-Varadhan variational inequality [62]; Theorem 3.2 in [82]). *If $D_{\text{KL}}(P'_e \| P_e) \leq \mathcal{B}$, then*

$$C_{P'_e}(P_\theta) \leq \mathcal{B} + \log \left(\mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} \exp(C_e(\pi_\theta)) \right). \quad (6.5)$$

The DV inequality provides a tighter bound than inequality (6.4) since $x - 1 \geq \log(x)$ for all $x > 0$. For the rest of this analysis, we will specialize our discussion to the KL divergence and use the DV inequality. However, we note that our approach generalizes to any f -divergence by leveraging Corollary 6.1.

6.2.2 Distributionally robust PAC-Bayes

As written, Inequality (6.5) cannot be used to directly upper bound $C_{P'_e}(P_\theta)$ since $\mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} \exp(C_e(\pi_\theta))$ is not an observable quantity. However, we can directly leverage the Inequality (2.4) to obtain an upper bound on $C_{P'_e}(P_\theta)$ in terms of observable quantities. Since Theorem 2.1 holds for any cost function between 0 and 1, we will be able to apply Inequality (2.4) if we replace the cost with an exponentiated one, as long as we rescale to stay between 0 and 1. Thus, if we make the substitution

$$C_e(\pi_\theta) \leftarrow \frac{\exp(C_e(\pi_\theta)) - 1}{a},$$

then $C_e(\pi_\theta)$ is bounded in $[0, 1]$ when $a = \text{Euler's constant} - 1$. We use constant a to avoid notation conflict with environments e . Thus, we obtain the following bound using Inequality (2.4):

$$\mathbb{E}_{e \sim P_e} \mathbb{E}_{\theta \sim P_\theta} \exp(C_e(\pi_\theta)) \leq \frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\theta \sim P_\theta} \exp(C_{e_i}(\pi_\theta)) + a \sqrt{\frac{D_{\text{KL}}(P_\theta \| P_{\theta,0}) + \log \frac{2\sqrt{N}}{\delta}}{2n}}. \quad (6.6)$$

This inequality holds because the transformation keeps the cost between $[0, 1]$. Now, since we assumed that $D_{\text{KL}}(P'_e \| P_e) \leq \mathcal{B}$, we can apply Theorem 6.2 to bound $C_{P'_e}(P_\theta)$.

Corollary 6.3 (Distributionally-robust PAC-Bayes bound). *For any data-independent prior distribution $P_{\theta,0}$ over θ , some cost function C that satisfies Assumption 2.1, $n \geq 8$, and $\delta \in (0, 1)$, any P'_e such that $D_{\text{KL}}(P'_e \| P_e) \leq \mathcal{B}$, with probability at least $1 - \delta$ over a sampling of $S = \{e_1, e_2, \dots, e_n\} \sim P_e^n$, the following holds simultaneously for all distributions P_θ over θ :*

$$C_{P'_e}(P_\theta) \leq \mathcal{B} + \log \left(\frac{1}{n} \sum_{i=1}^n \mathbb{E}_{\theta \sim P_\theta} \exp(C_{e_i}(\pi_\theta)) + a \sqrt{\frac{D_{\text{KL}}(P_\theta \| P_{\theta,0}) + \log \frac{2\sqrt{n}}{\delta}}{2n}} \right). \quad (6.7)$$

Note that since $\mathcal{B}$ appears as an additive factor in Inequality (6.7), when $\mathcal{B} \geq 1$, the robust PAC-Bound becomes vacuous. Specifically, $C_{P'_e}(P_\theta) \geq 1$ means that nothing can be said about the performance of the controller on the test environments.

The RHS of Inequality (6.7) gives us an upper bound on $C_{P'_e}(P_\theta)$ instead of $C_{P_e}(P_\theta)$

Algorithm 6.1 Robust PAC-Bayes Bound Minimization via Backpropagation

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies

Input: Training dataset S , learning rate γ

Output: Optimized ψ^*

while not converged **do**

 Sample $w_i \sim \mathcal{N}_{\psi} \forall i \in \{1, 2, \dots, k\}$

$$B \leftarrow \mathcal{B} + \left(\frac{1}{nk} \sum_{e \in S} \sum_{i=1}^k C_e(\pi_{w_i}) + \sqrt{\frac{D_{\text{KL}}(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}} \right)$$

$\psi \leftarrow \psi - \gamma \nabla_{\psi} B$

end while

as in Inequality (2.4). We can apply an analogous procedure to Algorithm 2.1 to obtain a distributionally-robust stochastic policy by minimizing this upper bound in Inequality (6.7), the result is presented in Algorithm 6.1. As in Section 2.2, we parameterize the distribution over policy parameters P_{θ} with a diagonal-covariance multivariate Gaussian distribution $\mathcal{N}(\mu, \text{diag}(s))$. We use shorthand $\mathcal{N}_{\psi}$ where $\psi := (\mu, \log s)$ and optimize to find ψ^* in the algorithm. Note that since $\mathcal{B}$ depends on P_e and P'_e , (and not the distribution over parameters P_{θ}) we cannot optimize with respect to it, and it does not show up in the gradients of B .

Experimental results based on the distributionally robust PAC-Bayes bound (Corollary 6.3) can be found in [135]. Although we are able to achieve strong generalization guarantees using the methods presented here, there is an important assumption which must be made: there is a bounded KL divergence between the training and test environment distributions: $D_{\text{KL}}(P'_e \parallel P_e) \leq \mathcal{B}$. In practice it is possible to construct such an example, but it requires explicit knowledge of the training *and* test distribution. So far, we have tried to avoid assuming knowledge of the underlying training/test distributions as it is rarely possible to obtain this knowledge in the real-world. In the next section, we will provide another concrete method for learning policies which are robust to distribution shifts. However, we will not require specific information about the training and test distributions.

6.3 Meta-learning generalization

By switching to the meta-learning setting, and assuming there is some underlying distribution P_{τ} over tasks (instead of environments based on a particular tasks), we will be able to

provide strong generalization guarantees on the performance in a new task sampled from P_τ which may be unseen. We switch to the supervised learning setting instead of working in the policy learning setting as we have done in the previous chapters. There are a number of key technical challenges that arise in this section that limit the applicability of the methods present to robotic systems. However, the resulting tight guarantees in the meta-learning setting provide strong motivation for future work for similar bounds which are more applicable to robotic settings. This motivates Chapter 7 where we discuss techniques for broader generalization between robotic tasks.

In the remainder of this chapter, we will first discuss related work in meta-learning and generalization for meta-learning, since we did not previously discuss it in Section 1.3. We will then introduce the generalization frameworks we use to provide the meta-learning bound in the appropriate context. We prove the proposed bound, and then demonstrate the performance and tightness of the bound on two examples. As mentioned, the technical challenges of this approach limit scale of problems on which it applies. We also present a heuristic based on the rigorously derived bound which provides an effective regularization technique even on larger scale problems. Finally, we demonstrate the heuristic on a meta-learning problem which causes strong over-fitting for other methods.

6.3.1 Related work

Meta-learning. Meta-learning is a well-studied technique for exploiting similarities between learning tasks [181, 201]. Often used to reduce the need for large amounts of training data, a number of approaches for meta-learning have been explored over decades [26, 34, 42, 94, 210, 178, 164, 99]. Recently, methods based on model-agnostic meta-learning (MAML) [72] have demonstrated strong performance across different application domains and benchmarks such as *Omniglot* [113] and *Mini-ImageNet* [212]. These methods operate by optimizing a set of initial parameters that can be quickly fine-tuned via gradient descent on a new task. The approaches mentioned above typically do not provide any generalization guarantees, and none of them compute explicit numerical bounds on generalization performance. Our approach has the structure of gradient-based meta-learning while providing guarantees on generalization.

Generalization bounds for meta-learning. Meta-learning bounds must account for two “levels” of generalization (base level and meta level). The approach presented in [140] utilizes algorithmic stability bounds at both levels. However, this requires both meta and base learners to be uniformly stable. This is a strong requirement that is challenging to ensure at the meta level. Another recent method, known as follow-the-meta-regularized-leader (FMRL) [109], provides guarantees for a regularized meta-learning version of the follow-the-leader (FTL) method for online learning, see e.g. [90]. The generalization bounds provided are derived from the application of online-to-batch techniques [10, 58]. A regret bound for meta-learning using an aggregation technique at the meta-level and an algorithm with a uniform generalization bound at the base level is provided in [10]. The techniques mentioned do not present an algorithm which makes use of validation data (in contrast to our approach). Using validation data (i.e., held-out data) is a common technique for improving performance in meta-learning and is particularly important for the few-shot learning case.

6.3.2 Problem setup

The setup is based on the one presented in 2.1.2, but is presented for supervised learning.

Samples, tasks, and datasets. Formally, consider the setting where we have an unknown meta distribution P_τ over tasks (roughly, “tasks” correspond to different, but potentially related, learning problems). A sampled task $\tau \sim P_\tau$ induces an (unknown) distribution $P_{z|\tau}$ over sample space $\mathcal{Z}$. We assume that all sampling is independent and identically distributed (i.i.d.). Note that the sample space $\mathcal{Z}$ is shared between tasks, but the distribution $P_{z|\tau}$ may be different. We then sample within-task samples $z \sim P_{z|\tau}$ and within-task datasets $S = \{z_1, z_2, \dots, z_m\} \sim P_{z|\tau}^m$. We assume that each sample z has a single corresponding label $o(z)$, where the function o is an oracle which outputs the correct label of z . At meta-training time, we assume access to l datasets, which we call $\mathbf{S} = \{S_1, S_2, \dots, S_l\}$. Each dataset S_i in $\mathbf{S}$ is drawn by first selecting a task τ_i from P_τ , and then drawing $S_i \sim P_{z|\tau_i}^m$.

Hypotheses and losses. Let h denote a hypothesis and $L(h, z)$ be the loss incurred by hypothesis h on sample z . The loss is computed by comparing $h(z)$ with the true label $o(z)$. For simplicity, we assume that there is no noise on the labels; we can thus

assume that all loss functions have access to the label oracle function o and thus the loss depends only on hypothesis h and sample z . We note that this assumption is not required for our analysis and is made for the ease of exposition. Overloading the notation, we let $L(h, P_{z|\tau}) := \mathbb{E}_{z \sim P_{z|\tau}} L(h, z)$ and $\widehat{L}(h, S) := \frac{1}{|S|} \sum_{i=1}^{|S|} L(h, z_i)$.

Meta-learning. As with model-agnostic meta-learning (MAML) [72], we let meta parameters $\theta \in \mathbb{R}^{n_\theta}$ correspond to an initialization of the base learner’s hypothesis. Let h_θ be the θ -initialized hypothesis. Generally, the initialization θ is learned from the multiple datasets we have access to at meta-training time. In this work, we will learn a *distribution* P_θ over initializations so that we can use bounds from the PAC-Bayes framework. At test time, a new task $\tau \sim P_\tau$ is sampled and we are provided with a new dataset $S \sim P_{z|\tau}^m$. The base learner uses an algorithm A (e.g., gradient descent), the dataset S , and the initialization $\theta \sim P_\theta$ in order to fine-tune the hypothesis and perform well on future samples drawn from $P_{z|\tau}$. We denote the base learner’s updated hypothesis by $h_{A(\theta, S)}$. More formally, our goal is to learn a distribution P_θ with the following objective:

$$\min_{P_\theta} \mathcal{L}(P_\theta, P_\tau) := \min_{P_\theta} \mathbb{E}_{\tau \sim P_\tau} \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|\tau}). \quad (6.8)$$

We are particularly interested in the the few-shot learning case, where the number of samples which the base learner can use to adapt is small. A common technique to improve test performance in the few-shot learning case is to allow for validation data at meta-training time. Thus, in addition to a generalization guarantee on meta-learning without validation data, we will derive a bound when allowing for the use of validation data $S_{\text{va}} \sim P_{z|\tau}^n$ during meta-training.

6.3.3 Preliminaries: two generalization frameworks

We use two different frameworks for the two levels of generalization required in a meta-learning bound. We utilize the PAC-Bayes framework to bound the expected training loss on future tasks, and uniform stability bounds to argue that if we have a low training loss when using a uniformly stable algorithm, then we achieve a low test loss. The following section will introduce these frameworks independently. We then present the overall meta-

learning bound and associated algorithm to find a distribution over initialization parameters (i.e., meta parameters) that minimizes the upper bound.

Uniform stability

Let $S = \{z_1, z_2, \dots, z_m\} \in \mathcal{Z}^m$ be a set of m elements of $\mathcal{Z}$. Let $S^i = \{z_1, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_m\}$ be identical to dataset S except that the i^{th} sample z_i is replaced by some $z'_i \in \mathcal{Z}$. Note that our analysis can be extended to allow for losses bounded by some finite M , but we work with losses bounded within $[0, 1]$ for the sake of simplicity. With these precursors, we define an analogous notion of *uniform stability* to [88, Definition 2.1] for deterministic algorithms A and distributions P_θ over initializations. Note that we use deterministic algorithms to avoid excess computation when calculating the provided meta-learning upper bounds. See Section 13.7.4 for further details.

Definition 6.2 (Uniform Stability). *A deterministic algorithm A has $\beta > 0$ uniform stability with respect to loss L if $\forall z \in \mathcal{Z}, \forall S \in \mathcal{Z}^m, \forall i \in \{1, \dots, m\}$, and all distributions P_θ over initializations, the following holds:*

$$\mathbb{E}_{\theta \sim P_\theta} |L(h_{A(\theta, S)}, z) - L(h_{A(\theta, S^i)}, z)| \leq \beta. \quad (6.9)$$

We define β_{US} as the minimal such β .

In this work, we will bound β_{US} as a function of the algorithm, form of the loss, and number of samples that the algorithm uses (See Section 13.7.4 for further details on the bounds on β_{US} for our setup). We then establish a relationship between uniform stability and generalization in expectation. The following is adapted from [88, Theorem 2.2] for the notion of uniform stability presented in Definition 6.2:

Theorem 6.3 (Algorithmic Stability Generalization in Expectation). *Fix a task $\tau \sim P_\tau$. The following inequality holds for hypothesis $h_{A(\theta, S)}$ learned using β_{US} uniformly stable algorithm A with respect to loss L :*

$$\mathbb{E}_{S \sim P_\tau^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|\tau}) \leq \mathbb{E}_{S \sim P_\tau^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}. \quad (6.10)$$

Proof. The proof is similar to the one presented for [88, Theorem 2.2] and is presented in Section 13.7.1. $\square$

PAC-Bayes theory

For the meta-level bound, we make use of the PAC-Bayes generalization bound introduced in [142]. Note that other PAC-Bayes bounds such as the quadratic variant [172] and PAC-Bayes- λ variant [200] may be used and substituted in the following analysis. We first present a general version of the PAC-Bayes bound and then specialize it to our meta-learning setting in Section 6.3.4. Let $f(\theta, s)$ be an arbitrary loss function which only depends on parameters θ and the sample s which has been drawn from an arbitrary distribution P_s . The following bound is a tightened version of the bound presented in [142] for when $l \geq 8$.

Theorem 6.4 (PAC-Bayes Generalization Bound [139]). *For any data-independent prior distribution $P_{\theta,0}$ over θ , some loss function f which satisfies Assumption 2.1, $l \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of $\{s_1, s_2, \dots, s_l\} \sim P_s^l$, the following holds simultaneously for all distributions P_θ over θ :*

$$\mathbb{E}_{s \sim P_s} \mathbb{E}_{\theta \sim P_\theta} f(\theta, s) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} f(\theta, s_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l), \quad (6.11)$$

where the PAC-Bayes “regularizer” term is defined as follows

$$R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l) := \sqrt{\frac{D_{\text{KL}}(P_\theta \| P_{\theta,0}) + \ln \frac{2\sqrt{l}}{\delta}}{2l}}, \quad (6.12)$$

and D_{KL} is the Kullback-Leibler (KL) divergence.

6.3.4 Meta-learning bound

In order to obtain a generalization guarantee for meta-learning, we utilize the two frameworks above. We first specialize the PAC-Bayes bound in Theorem 6.4 to bound the expected training loss on future tasks. We then utilize Theorem 6.3 to demonstrate that if we have a low expected training loss when using a uniformly stable algorithm, then we achieve a low expected test loss. These two steps allow us to combine the generalization frameworks

above to derive an upper bound on Inequality (6.8) which can be computed with known quantities. The resulting generalization bound is presented in Theorem 6.5.

Theorem 6.5 (Meta-Learning Generalization Guarantee). *For hypotheses $h_{A(\theta, S)}$ learned with β_{US} uniformly stable algorithm A , data-independent prior $P_{\theta, 0}$ over initializations θ , loss L which satisfies Assumption 2.1, $l \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of the meta-training dataset $\mathbf{S} \sim P_S^l$, the following holds simultaneously for all distributions P_θ over θ :*

$$\mathcal{L}(P_\theta, P_\tau) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \beta_{\text{US}}. \quad (6.13)$$

Proof. The proof can be split into three steps:

Step 1.

Let P_S in Theorem 6.4 be the marginal distribution P_S over datasets of size m (see Section 13.7.2 for details) and note that sampling $S \sim P_S$ is equivalent to first sampling $\tau \sim P_\tau$ and then sampling $S \sim P_{z|\tau}^m$. Additionally let $f(\theta, S) := \widehat{L}(h_{A(\theta, S)}, S)$ where $A(\theta, S)$ is any deterministic algorithm. Plugging in these definitions into Inequality (6.11) results in the following inequality which holds under the same assumptions as Theorem 6.4, and with probability at least $1 - \delta$ over the sampling of $\mathbf{S} \sim P_S^l$:

$$\begin{aligned} \mathbb{E}_{S \sim P_S} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) &= \mathbb{E}_{\tau \sim P_\tau} \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) \\ &\leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l). \end{aligned} \quad (6.14)$$

Step 2.

Now assume that algorithm A is β_{US} uniformly stable. For a fixed task $\tau \sim P_\tau$ we have the following by Theorem 6.3:

$$\mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|\tau}) \leq \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}.$$

Take the expectation over $\tau \sim P_\tau$. We then have:

$$\mathbb{E}_{\tau \sim P_\tau} \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|\tau}) \leq \mathbb{E}_{\tau \sim P_\tau} \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}, \quad (6.15)$$

since $\mathbb{E}_{\tau \sim P_\tau} \beta_{\text{US}} = \beta_{\text{US}}$. This establishes a bound on the true expected loss for a new task after running algorithm A on a training dataset corresponding to the new task.

Step 3.

Note that Inequality (6.14) provides an upper bound on the first term of the RHS of Inequality (6.15) when algorithm A is β_{US} uniformly stable. Thus we have the following by plugging Inequality (6.14) in the RHS of Inequality (6.15):

Under the same assumptions as both Theorems 6.3 and 6.4, and with probability at least $1 - \delta$ over the sampling of $\mathbf{S} \sim P_S^l$:

$$\mathbb{E}_{\tau \sim P_\tau} \mathbb{E}_{S \sim P_{z|\tau}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|\tau}) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \beta_{\text{US}},$$

completing the proof. $\square$

Theorem 6.5 is presented for any distributions P_θ and $P_{\theta, 0}$ over initializations. However, in practice we will use multivariate Gaussian distributions for both. The specialization of Theorem 6.5 to Gaussian distributions is provided in Section 13.7.3. Next, we allow for validation data $S_{\text{va}} \sim P_{z|\tau}^n$ at meta-training time so that the bound is more suited to the few-shot learning case. We compute the upper bound using the evaluation data $S_{\text{ev}} = \{S, S_{\text{va}}\}$ sampled from the marginal distribution $P_{S_{\text{ev}}}$ over datasets of size $m + n$. However, we still only require m samples at meta-test time; see Section 13.7.3 for the derivation. Note that the training data S is often excluded from the data used to update the meta-learner. However, this is necessary for our approach to obtain a guarantee on few-shot learning performance. The result is a guarantee with high probability over a sampling of $\mathbf{S}_{\text{ev}} \sim P_{S_{\text{ev}}}^l$:

$$\mathcal{L}(P_\theta, P_\tau) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_{\text{ev}, i}) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \frac{m\beta_{\text{US}}}{m+n}. \quad (6.16)$$

PAC-BUS algorithm

Recall that we aim to find a distribution P_θ over initializations that minimizes $\mathcal{L}(P_\theta, P_\tau)$ as stated in Equation (6.8). We cannot minimize $\mathcal{L}(P_\theta, P_\tau)$ directly due to the expectations taken over unknown distributions P_τ and $P_{z|\tau}$ for sampled task τ , but we may indirectly minimize it by minimizing the upper bounds in Inequalities (6.13) or (6.16).

Computing the upper bound requires evaluating an expectation taken over $\theta \sim P_\theta$. In general, this is intractable. However, we aim to minimize this upper bound to provide the tightest guarantee possible. Similar to the method in [63], we use an unbiased estimator of $\mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, \cdot)$. Let P_θ be a multivariate Gaussian distribution over initializations θ with mean μ and covariance $\text{diag}(s)$; thus $P_\theta = \mathcal{N}(\mu, \text{diag}(s))$ and $P_{\theta,0} = \mathcal{N}(\mu_0, \text{diag}(s_0))$. Further, let $\psi := (\mu, \log(s))$, and use the shorthand $\mathcal{N}_{\psi_0}$ for the prior and $\mathcal{N}_\psi$ for the posterior distribution over initializations. We use the following estimator of $\mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, \cdot)$:

$$L(h_{A(\theta, S)}, \cdot), \quad \theta \sim \mathcal{N}_\psi. \quad (6.17)$$

We present the resulting training technique in Algorithm 6.2. This algorithm can be used to learn a distribution over initializations that minimizes the upper bound presented in Theorem 6.5 and its specializations. This is presented for the case when A is β_{US} uniformly stable for some β_{US} . For gradient-based algorithms, the learning rate α often appears directly in the bound for β_{US} [88]. Thus it is potentially beneficial to update α as well. We present Algorithm 6.2 without learning the learning rate. To meta-learn the learning rate, we can augment ψ_0 to include a parameterization of a prior distribution over learning rates and update it using the same gradient step presented in 6.2 for ψ .

Determining the gradient of $B(\psi, \theta'_1, \theta'_2, \dots, \theta'_l)$ with respect to ψ requires computing the Hessian of the loss function if algorithm $A(\theta, S)$ uses a gradient update to compute θ'_i . First order approximations often perform similarly to the second-order meta-learning techniques [81, 72, 152], and can be used to speed up the training. Additionally, Algorithm 6.2 can be modified to use mini-batches of tasks instead of all tasks in the meta update to improve training times; we present an algorithm which uses mini-batches of tasks in Section 13.7.5.

In practice, we are interested in algorithms such as stochastic gradient descent (SGD)

Algorithm 6.2 PAC-BUS: meta-learning via PAC-Bayes and Uniform Stability

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over initializations
Input: β_{US} uniformly stable Algorithm A
Input: Meta-training dataset $\mathbf{S}$, learning rate γ
Initialize: $\psi \leftarrow \psi_0$
Output: Optimized ψ^*
 $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l) := \frac{1}{l} \sum_{i=1}^l \widehat{L}(h_{\theta'_i}, S_i) + R_{\text{PAC-B}}(\mathcal{N}_{\psi}, \mathcal{N}_{\psi_0}, \delta, l) + \beta_{\text{US}}$
while not converged **do**
 Sample $\theta \sim \mathcal{N}_{\psi}$
 for $i = 1$ **to** l **do**
 $\theta'_i \leftarrow A(\theta, S_i)$
 end for
 $\psi \leftarrow \psi - \gamma \nabla_{\psi} B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l)$
end while

and gradient descent (GD) for the base learner. We can obtain bounds on the uniform stability constant β_{US} when using gradient methods with the results from [88]. See Section 13.7.4 for details on the β_{US} bounds we use in this work. With a bound on β_{US} , we can calculate all the terms in $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l)$ and use Algorithm 6.2 to minimize the meta-learning upper bound. When evaluating the upper bound, we use the sample convergence bound [115, 63] to upper bound the expectation taken over $\theta \sim P_{\theta}$. See Section 13.7.6 for details.

6.3.5 Experiments

We demonstrate our approach on three examples below. All examples we provide are few-shot meta-learning problems. To adapt at the base level, m examples from each class are given for an “ m -shot” learning problem. If applicable, n samples can be given as validation data for each task during the meta-training step. In the first two examples, our primary goal is to demonstrate the tightness of our generalization bounds compared to other meta-learning bounds. We also present empirical test performance on held-out data; however, we emphasize that the focus of our work is to obtain improved generalization guarantees (and not necessarily to improve empirical test performance). In the third example, we present an algorithm that is motivated by our theoretical framework and demonstrate its ability to improve empirical performance on a challenging task. All the code required to run the following examples is available at <https://github.com/irom-lab/PAC-BUS>.

Table 6.1: We present the generalization bounds (for $\delta = 0.01$) provided by each method if applicable, and use the sample convergence bound [115] for MR-MAML, and PAC-BUS, but not MLAP-M.² Note that for these methods, we specifically minimize their respective meta-learning bounds. We also report the meta-test loss (the softmax activated cross-entropy loss – CEL_s) for all methods. We present the mean and standard deviation after 5 trials. We highlight that our approach provides the strongest generalization guarantee.

Classification on Ball	MAML [72]	MLAP-M [16]	MR-MAML [225]	PAC-BUS (ours)
Bound $\downarrow$	None	1.0538 $\pm$ 0.0012	0.3422 $\pm$ 0.0006	0.2213$\pm$0.0012
Test Loss $\downarrow$	0.1701 $\pm$ 0.0070	0.1645 $\pm$ 0.0045	0.1584$\pm$0.0012	0.1657 $\pm$ 0.0014

Example: classification on the unit ball

We evaluate the tightness of the generalization bound in Equation (6.16) on a toy two-class classification problem where the sample space $\mathcal{Z}$ is the unit ball $B^2(0, 1)$ in two dimensions with radius 1 and centered at the origin. Data points for each task are sampled from $P_{z|\tau}$, where a task corresponds to a particular concept which labels the data as (+) if within $B^2(c_t, r_t)$ and (−) otherwise. Center c_t is sampled uniformly from the $y \geq 0$ semi-ball $B_{y \geq 0}^2(0, 0.4)$ of radius 0.4. The radius r_t is then sampled uniformly from $[0.1, 1 - \|c_t\|]$. Notably, the decision boundary between classes is nonlinear. Thus, generalization bounds which rely on convex losses (such as [109]) will have difficulty with providing guarantees for networks that perform well. We choose the softmax-activated cross-entropy loss, CEL_s , as the loss function. Before running Algorithm 6.2, we address a few technical challenges that arise from Assumption 2.1 as well as computing c_L and c_S . We address these in Section 13.7.5. Also, note that due to high computation times associated with estimating the MLAP upper bound, the value for the MLAP-M bound in Table 6.1 is not computed with the sample convergence bound as the other upper bounds are. Thus, the value presented does not carry a guarantee, but would be similar if computed with the sample convergence bound. The value is shown to give a qualitative sense of the guarantee.

We then apply Algorithm 6.2 using the few-shot learning bound in Inequality (6.16). We present the guarantee on the meta-test loss associated with each training method in Table 6.1. In addition, we present the average meta-test loss after training with 10 samples. We compare our bounds and empirical performance with the meta-learning by adjusting pri-

ors (MLAP) technique [16] and the meta-regularized MAML (MR-MAML) technique [225]. All methods are given held-out data to learn a prior before minimizing their respective upper bounds (see Section 13.7.11 for further details on the prior training step). Additionally, since all bounds require the loss to be within $[0, 1]$, networks N are constrained such that the Frobenius norm of the output is bounded by r , i.e., $\|N(z)\|_F \leq r$. We compare the aforementioned methods' meta-test loss to MAML with weights constrained in the same manner (note that MAML does not provide a guarantee). Upper bounds which use the PAC-Bayes framework are computed with many evaluations from the posterior distribution. This allows us to apply the sample convergence bound [115] (as in Equation (13.35) for our bound) unless otherwise noted.

We find that PAC-BUS provides a significantly stronger guarantee compared with the other methods. Note that the guarantee provided by MLAP-M [16] is vacuous because the meta-test loss is bounded between 0 and 1, while the guarantee is above 1.

Example: Mini-Wiki

Next, we present results on the *Mini-Wiki* benchmark introduced in [109]. This is derived from the Wiki3029 dataset presented in [21]. The dataset is comprised of 4-class, m -shot learning tasks with sample space $\mathcal{Z} = \{z \in \mathbb{R}^d \mid \|z\|_2 = 1\}$. Sentences from various Wikipedia articles are passed through the continuous-bag-of-words GloVe embedding [159] into dimension $d = 50$ to generate samples. For this learning task, we use a k -class version of CEL_s and logistic regression. Since this example is convex, we can use GD and bound β_{US} with Theorem 13.4 [88]. We keep the loss bounded by constraining the network $\|N(z)\|_F \leq r$ and scale the loss as in the previous example. The tightness of the bounds on c_L and c_S affected the upper bound in Inequality (6.16) more than in the previous example, so we bound them as tightly as possible. See Section 13.7.9 for the calculations.

We apply Algorithm 6.2 using the bound which allows for validation data, Inequality (6.16), to learn on 4-way *Mini-Wiki* $m = \{1, 3, 5\}$ -shot. The results are presented in Table 6.2. We compare our results with the FMRL variant which provides a guarantee [109], follow-the-last-iterate (FLI)-Batch, and with MR-MAML [225]. FLI-Batch does not require bounded losses explicitly, but requires that the parameters of the network lie within a ball of

Table 6.2: We compare the generalization bounds (for $\delta = 0.01$) provided by each method where applicable and use the sample convergence bound for MR-MAML and PAC-BUS. Since we specifically minimize these methods’ upper bounds, we can fairly compare the relative tightness of each bound. We also report the meta-test loss (CEL_s) for each method for exposition. We report the mean and standard deviation after 5 trials. We highlight that our approach provides the strongest guarantee.

4-Way <i>Mini-Wiki</i>	1-shot ↓	3-shot ↓	5-shot ↓
FLI-Batch Bound [109]	0.6638 ± 0.0011	0.6366 ± 0.0006	0.6343 ± 0.0014
MR-MAML Bound [225]	0.7400 ± 0.0003	0.7312 ± 0.0003	0.7283 ± 0.0005
PAC-BUS Bound (ours)	0.4999 ± 0.0003	0.5058 ± 0.0002	0.5101 ± 0.0002
MAML [72]	0.3916 ± 0.0009	0.3868 ± 0.0005	0.3883 ± 0.0005
FLI-Batch [109]	0.4091 ± 0.0008	0.4078 ± 0.0005	0.4097 ± 0.0012
MR-MAML [225]	0.3922 ± 0.0009	0.3869 ± 0.0003	0.3884 ± 0.0005
PAC-BUS (ours)	0.3922 ± 0.0009	0.3878 ± 0.0003	0.3895 ± 0.0005

radius r . For the logistic regression used in the example, this is equivalent to $\|N(z)\|_F \leq r$. Thus, we scale the loss and use the same r for each method to provide a fair comparison. We also show the results of training with MAML constrained in the same way for reference. Each method is given the same amount of held-out data for training a prior (see Section 13.7.11 for further details on training the prior).

As in the previous example, PAC-BUS provides a significantly tighter guarantee than the other methods (Table 6.2). We see similar empirical meta-test loss for MAML [72], MR-MAML [225], and PAC-BUS with slightly higher loss for FLI-Batch [109]. In addition, we computed the meta-test accuracy as the percentage of correctly classified sentences. See Table 13.4 in Section 13.7.11 for these results along with other experimental details.

Example: memorizable Omniglot

We have demonstrated the ability of our approach to provide strong generalization guarantees for meta-learning in the settings above. We now consider a more complex setting where we are unable to obtain strong guarantees. In this example, we employ a learning heuristic based on the PAC-BUS upper bound, $\text{PAC-BUS}(H)$; see Section 13.7.5 for the details and the Algorithm. We relax Assumption 2.1 and no longer constrain the network as in previous sections. Instead, we maintain and update estimates of the Lipschitz and smooth-

Table 6.3: We present the meta-test accuracy as a percentage on non-mutually-exclusive *Omniglot* [225]. In contrast to the previous examples, here we aim to achieve the best empirical performance for each method. In particular, this task compares each methods’ ability to prevent memorization. We report the mean and standard deviation after 5 trials.

20-WAY <i>Omniglot</i>	NME 1-SHOT $\uparrow$	NME 5-SHOT $\uparrow$
MAML [72]	23.4 ± 2.2	75.1 ± 4.8
FLI-ONLINE [109]	22.4 ± 0.5	39.1 ± 0.5
MR-MAML(W) [225]	84.2 ± 2.2	94.3 ± 0.3
PAC-BUS(H) (OURS)	87.9 ± 0.5	95.0 ± 0.9

ness constants of the network, using [195], and incorporate them into the uniform stability regularizer term, β_{US} . We then scale each regularizer term (i.e., $R_{\text{PAC-B}}(P_{\theta}, P_{\theta,0}, \delta, l)$ and β_{US}) by hyper-parameters λ_1 and λ_2 respectively. Analogous to the technique described in [225], we aim to incorporate the form of the theoretically-derived regularizer into the loss, without requiring it to be as restrictive during learning. The result is a regularizer that punishes large deviation from the prior $P_{\theta,0}$ and too much adaptation at the base-learning level.

We test our method on *Omniglot* [113] for 20-way, $m = \{1, 5\}$ -shot classification in the non-mutually exclusive (NME) case [225]. In [225], the problem of memorization in meta-learning is explored and demonstrated with non-mutually exclusive learning problems. *NME Omniglot* corresponds to randomization of class labels for a task at test time only. This worsens the performance of any network that memorized class labels; see [225] for more details. We compare our method to an analogous heuristic presented in [225], which also has a $D_{\text{KL}}(P_{\theta} \| P_{\theta,0})$ term in the loss. Thus, this heuristic (referred to as MR-MAML(W) [225]) regularizes the change in weights of the network. Additionally, we compare to the heuristic described in [109] (FLI-Online) which performs better in practice than the FLI-Batch method. We do not provide data for training a prior in this case since we do not aim to compute a bound in this example. We use standard MAML as a reference. See Table 6.3 for the results.

We see that MAML [72] and FLI-Online [109] do not prevent memorization on *NME Omniglot* [225]. This is especially apparent in the 1-shot learning case, where their per-

formance suffers significantly due to this memorization. Both MR-MAML(W) [225] and PAC-BUS(H) prevent memorization, with PAC-BUS(H) outperforming MR-MAML(W). Note that PAC-BUS(H) outperforms MR-MAML(W) by a wider margin in the 1-shot case as compared with the 5-shot case. We believe this is due to the effectiveness of the uniform stability regularizer at the base level. MR-MAML(W) suffers more in the 1-shot case because over-adaptation is more likely with fewer within-task examples.

Chapter 7

Towards broader generalization guarantees

7.1 Motivation

In the previous chapter, we presented two frameworks for out-of-distribution generalization based on PAC-Bayes theory. The first relied on a bounded KL divergence shift between the training and test distribution. However, this assumes knowledge about these distributions which may be difficult to obtain in practice. The second framework relied on the assumption that the tasks we will see in testing come from an underlying distribution over tasks. Additionally, since the bound depended on restrictive Lipschitz and smoothness constants, the applicability to larger-scale problems was limited as mentioned in Section 6.3. Despite the power of the methods presented in Chapter 6, both are limited in their applicability to realistic robotic systems. How can we approach the problem of *more broad* generalization guarantees for OOD settings?

Consider the following pair of robotic tasks: (i) avoiding a set of obstacles where all obstacles are red, (ii) avoiding a set of obstacles where all obstacles are blue. Assume everything else about the tasks are the same. Intuitively the two tasks are just as hard as each other. Specifically, if we can construct a robotic policy which can successfully avoid red obstacles, we should also be able to construct a robotic policy which can avoid the blue

obstacles. How can we *formally* compare the *complexity* of this pair of tasks? In robotics, we lack a precise mathematical notions for answering such questions. However, consider *computational complexity theory* [19], where there is a unifying framework and set of tools for comparing the difficulty of all computation problems. For example, a problem that is as hard as 3-SAT will not admit a polynomial-time solution (unless $P = NP$). Motivated by this, we will take a step towards developing a precise framework for establishing the *relative complexity* of robotic tasks. Using the introduced notions, we will derive a novel PAC-Bayes bound which provide a generalization guarantee across tasks.

The notions we propose in this chapter draw inspiration from computation complexity theory [19]. Complexity theoretic questions in robotics have been studied extensively such as for motion planning problems [165, 41, 148, 32, 33, 89, 87]. In contrast to these approaches, we are specifically interested in the computational resources required by the robot as it is performing a given task. There has been additional work on comparing robotic systems themselves such as in [61, 154], which propose an intrinsic complexity of robotic tasks. Other methods compare the power of different sensors [118, 65]. Our goal is to compare different robotic *tasks* as it encompasses a more general framework than ones which only compare robots or sensors. We begin with a formal problem setup and definition of a robotic task to clarify the setting we consider.

7.1.1 Problem setup

We will use the same formulation of robotic task introduced in Section 2.1.2 with a few additional notions. We have (potentially stochastic) function f describe the robot's dynamics, a sensor g and a policy π which takes sensor observations y to control inputs u . We consider robotic tasks that are prescribed using reward functions; let $\sum_t r(x_t, u_t) \in \mathbb{R}$ denote the cumulative reward over a given (finite or infinite) time horizon. Note that we will use reward instead of cost throughout this chapter as it makes many of the notions presented more intuitive. However, cost can be used as well. We will denote the set of all policies (i.e., all mappings from $\mathcal{Y}$ to $\mathcal{U}$) by Π . We will define the reward achieved by a

policy as:

$$R(\pi) := \min \left(\mathbb{E}_{f_0, f, g} \sum_t r(x_t, \pi(y_t)), R^* \right), \quad (7.1)$$

where R^* is a chosen success threshold and forms an upper bound on $R(\pi)$.

In this chapter, we let a *task* be defined by partially observable Markov decision process (POMDP) using the tuple $\tau := (\mathcal{X}, \mathcal{U}, \mathcal{Y}, f, g, r, f_0, R^*)$ which describes the state space, action space, observation space, dynamics, sensor, reward function, the distribution over initial states f_0 , and the threshold for success. For example, a task τ could correspond to a drone navigating through random obstacle environments (where the randomness over obstacle placements is defined using f_0 and the state x_t encapsulates the combined state of the robot and its environment). We will index tasks using ξ and let $\mathcal{T} = \{\tau_\xi\}_\xi$ denote a set of tasks. A particular task in $\mathcal{T}$ is thus denoted as $\tau_\xi := (\mathcal{X}_\xi, \mathcal{U}_\xi, \mathcal{Y}_\xi, f_\xi, g_\xi, r_\xi, f_{0,\xi}, R_\xi^*)$. We will say that a policy π_ξ is *admissible* on task τ_ξ if $R_\xi(\pi_\xi) = R_\xi^*$. We use shorthand π_ξ^* to denote an admissible policy on task τ_ξ . Let $\Pi_\xi^* \subseteq \Pi_\xi$ be the set of all admissible policies on task τ_ξ .

In the next two sections, we will introduce notions which meaningfully and quantitatively compare the complexity of robotic tasks. First, we aim to develop a *binary relation* (denoted by “ $\preceq$ ”) between two tasks which will provide a notion of reduction, i.e., if task τ_1 reduces to task τ_2 ($\tau_1 \preceq \tau_2$), then task τ_2 is at least as complex as task τ_1 . Second, we aim to formulate a measure $\mathcal{T} \times \mathcal{T} \rightarrow [0, 1]$ that compares the relative complexity of one task with respect to another. The goal is to not only establish if one task is more complex than another, but the *degree* to which it is more complex.

7.2 Task reduction

We propose a definition of *reduction* for robotic tasks in order to formalize what it means for one task to be as hard as another. The idea of reductions comes from the theory of computational complexity; we review basic definitions in Section 7.2.1 before describing reductions between robotic tasks in Section 7.2.2.

7.2.1 Background

We begin with the definition of a *Turing* reduction between two computational problems. Let A and B be *decision problems*, i.e., problems where each instance has a yes/no answer (e.g., 3-SAT). Let O_B be an oracle for decision problem B . This oracle is a blackbox which outputs the solution to any instance $b \in B$.

Definition 7.1 (Turing Reduction [196]). *Decision problem A reduces to B (written $A \preceq_T B$) if we can compute the solution to all instances of A using an oracle O_B for B .*

Intuitively, A reduces to B if one can use an oracle for B as a sub-routine for solving instances of A , and thus B is at least as complex as A . Note that this definition does not include reference to the complexity of the resulting function for solving A . We are specifically interested in *efficient* reductions. A particular notion of efficiency is formalized by *polynomial-time* reductions, as defined below.

Definition 7.2 (Cook Reduction [196]). *Decision problem A is polynomial-time reducible to B (written $A \preceq_T^P B$) if $A \preceq_T B$ and the solution to any instance of A makes only a polynomial number of calls to O_B .*

Importantly, if O_B is efficient (i.e., runs in polynomial time) and we have $A \preceq_T^P B$, then the resulting function for solving A is also efficient. We aim to define a notion of reduction for robotic tasks that is inspired by the notions above for computational problems. In particular, we will leverage the core concept of using an oracle for one decision problem in order to solve another.

7.2.2 Task Reduction: Definition and Properties

In this section, we propose a notion of reduction between two robotic tasks and demonstrate that this defines a partial ordering on a given space $\mathcal{T}$ of tasks. Our notion of reduction captures the following intuition: task τ_1 reduces to task τ_2 if we can utilize a policy (“oracle”) for τ_2 in order to solve τ_1 . In order to formalize this, we first introduce *encoders* and *decoders*.

Definition 7.3 (Encoder and Decoder). *Let $\mathcal{T}$ be a space of tasks and let $\tau_1, \tau_2 \in \mathcal{T}$ be two tasks (as defined in Section 7.1.1). Let $\mathcal{Y}_1, \mathcal{Y}_2$ and $\mathcal{U}_1, \mathcal{U}_2$ be the corresponding observation*

and control action spaces. Let $H_{1,2}$ denote a space of functions from $\mathcal{Y}_1$ to $\mathcal{Y}_2$, and let $D_{2,1}$ denote a space of functions from $\mathcal{U}_2$ to $\mathcal{U}_1$. We will refer to a function $h \in H_{1,2}$ as an encoder and a function $d \in D_{2,1}$ as a decoder.

Intuitively, a task τ_1 reduces to τ_2 if we can utilize *any* admissible policy for τ_2 and transform it via an encoder and decoder into a policy that solves τ_1 ; see Fig. 7.1 for a visual representation of this transformation. Importantly, none of the elements which define a task ($\tau := (\mathcal{X}, \mathcal{U}, \mathcal{Y}, f, g, r, f_0, R^*)$) need to be shared between the two tasks. We formalize the notion of *task reduction* below.

Definition 7.4 (Task Reduction). *Task τ_1 reduces to task τ_2 (written $\tau_1 \preceq \tau_2$) if for all admissible policies $\pi_2^* \in \Pi_2^*$, there exists an encoder $h \in H_{1,2}$ and a decoder $d \in D_{2,1}$ such that:*

$$d \circ \pi_2^* \circ h \in \Pi_1^* \quad (7.2)$$

This definition is illustrated in Fig. 7.1. This notion of reduction captures the relative complexity of tasks in terms of the complexity of online task execution. Thus, we compare the complexity of solutions rather than the complexity of computing solutions. It is important to note that the definition of reduction calls for the ability to transform *any* admissible policy $\pi_2^* \in \Pi_2^*$ into an admissible policy for τ_1 . We also note that task reductions are conditioned on the selection of $H_{1,2}$ and $D_{2,1}$. Intuitively, this corresponds to the “complexity” of the reduction. For example, if $H_{1,2}$ and $D_{2,1}$ only include functions which can be evaluated in polynomial time, then the reduction $\tau_1 \preceq \tau_2$ is *efficient* (analogous to Cook reductions). Additionally, note that a robot executing the policy $d \circ \pi_2^* \circ h \in \Pi_1^*$ for task τ_1 over a time horizon of T requires T evaluations of π_2^* . Thus, if $H_{1,2}$ and $D_{2,1}$ only include efficiently-computable functions and π_2^* is efficiently computable, then the *online* execution of τ_1 is efficient. One can specify different classes of functions for $H_{1,2}$ and $D_{2,1}$ in order to capture different notions of efficiency (e.g., neural networks with bounded size). Next, we define *equivalence* between two tasks.

Definition 7.5 (Task Equivalence). *Tasks τ_1 and τ_2 are equivalent ($\tau_1 \equiv \tau_2$) if $\tau_1 \preceq \tau_2$ and $\tau_2 \preceq \tau_1$.*

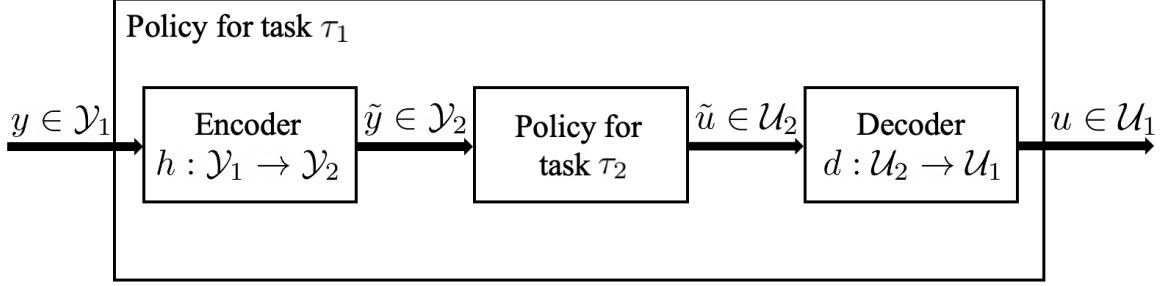


Figure 7.1: The transformation of a policy for one task to another using *encoders* and *decoders*. An observation $y \in \mathcal{Y}_1$ from task τ_1 is encoded to an observation $\tilde{y} \in \mathcal{Y}_2$ for τ_2 . The policy for τ_2 outputs an action $\tilde{u} \in \mathcal{U}_2$ which is decoded to an action $u \in \mathcal{U}_1$ for task τ_1 . Together, the encoder, policy for τ_2 , and decoder are a policy for τ_1 .

Note that task reduction and equivalence are both binary relations (over $\mathcal{T} \times \mathcal{T}$). As we show below, these relations satisfy properties that are intuitively desirable for any definition of reduction (or equivalence) between robotic tasks. In order to state these properties, we first introduce a notion of function closure.

Definition 7.6 (Closure under Composition on $\mathcal{T}$). *Let $d_1 \in D_{2,1}$, $d_2 \in D_{3,2}$, $h_1 \in H_{1,2}$, and $h_2 \in H_{2,3}$. We say that $D_{3,1}$ and $H_{1,3}$ are closed under composition if for any $\tau_1, \tau_2, \tau_3 \in \mathcal{T}$, the following are true:*

$$d_1 \circ d_2 \in D_{3,1}, \forall d_1 \in D_{2,1} \forall d_2 \in D_{3,2}, \quad (7.3)$$

$$h_2 \circ h_1 \in H_{1,3}, \forall h_1 \in H_{1,2} \forall h_2 \in H_{2,3}. \quad (7.4)$$

When $D_{\zeta,\xi}$ and $H_{\xi,\zeta}$ are the same for all tuples of tasks $(\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, this definition simplifies to closure under function composition for $D_{\zeta,\xi}$ and $H_{\xi,\zeta}$.

Proposition 7.1 (Task Reduction is a Non-Strict Partial Ordering Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $D_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task reductions satisfy the following properties and thus define a non-strict partial ordering relation.*

Property 7.1.a. *Reflexivity: $\tau_1 \preceq \tau_1$.*

Property 7.1.b. *Antisymmetry:* $\tau_1 \prec \tau_2 \implies \neg(\tau_2 \preceq \tau_1)$, where $\tau_1 \prec \tau_2$ is defined as $(\tau_1 \preceq \tau_2) \wedge \neg(\tau_1 \equiv \tau_2)$.

Property 7.1.c. *Transitivity:* $(\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \implies \tau_1 \preceq \tau_3$.

Proof. **Property 7.1.a:** $\tau_1 \preceq \tau_1 \implies \exists d \in D_{1,1}, h \in H_{1,1}$ such that

$$d \circ \pi_1^* \circ h \in \Pi_1^*. \quad (7.5)$$

If d and h are the identity function, then $d \circ \pi_1^* \circ h = \pi_1^* \forall \pi_1^* \in \Pi_1^*$. Thus, $\tau_1 \preceq \tau_1$ when $D_{1,1}$ and $H_{1,1}$ include their respective identity functions.

Property 7.1.b: Suppose $\tau_1 \prec \tau_2$ and thus $(\tau_1 \preceq \tau_2) \wedge \neg(\tau_1 \equiv \tau_2)$. Note that $\neg(\tau_1 \equiv \tau_2) \implies \neg((\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)) \implies \neg(\tau_1 \preceq \tau_2) \vee \neg(\tau_2 \preceq \tau_1)$. We assumed $(\tau_1 \preceq \tau_2)$, so we must have that $\neg(\tau_2 \preceq \tau_1)$.

Property 7.1.c: Suppose $\tau_1 \preceq \tau_2$ and $\tau_2 \preceq \tau_3$. By Definition 7.4, $\exists d_1 \in D_{2,1}, d_2 \in D_{3,2}, h_1 \in H_{1,2}$, and $h_2 \in H_{2,3}$ such that $d_1 \circ \pi_2^* \circ h_1 \in \Pi_1^* \forall \pi_2^* \in \Pi_2^*$ and $d_2 \circ \pi_3^* \circ h_2 \in \Pi_2^* \forall \pi_3^* \in \Pi_3^*$. Consider

$$\underbrace{d_1 \circ \overbrace{d_2 \circ \pi_3^* \circ h_2}^{\in \Pi_2^*} \circ h_1}_{\in \Pi_1^*} \quad (7.6)$$

for all $\pi_3^* \in \Pi_3^*$. Let $d_3 := d_1 \circ d_2$ and $h_3 := h_2 \circ h_1$ so that $d_3 \circ \pi_3^* \circ h_3 \in \Pi_1^*$ for all $\pi_3^* \in \Pi_3^*$. If $D_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$, then $d_3 \in D_{3,1}$ and $h_3 \in H_{1,3}$ and $\tau_1 \preceq \tau_3$. Thus, task reductions are transitive if $D_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. $\square$

Additionally, note that *strict* task reduction $\tau_1 \prec \tau_2$ is a strict partial ordering relation. We provide a complete proof of this in Appendix 14.10.1. These properties formalize a series of intuitions about task reductions. Task reduction consists of transforming an admissible policy from one task to another. In the case of reflexivity, intuition suggest that if the tasks are the same, no transformation will be required for the policy to work. Thus, D and H must include the identity to allow for this. Antisymmetry establishes a clear sense of “dominance” between tasks in terms of complexity — if a task τ_1 strictly reduces to another task τ_2 , then τ_2 should not reduce to τ_1 . Building on antisymmetry, we also require

transitivity to make a clear chain of increasingly complex tasks. A series of tasks which reduce to each other cyclically (i.e. $\tau_1 \preceq \tau_2 \wedge \tau_2 \preceq \tau_3 \wedge \tau_3 \preceq \tau_1$) should only be possible if all the tasks are equivalent. The following proposition proves useful properties of task equivalence.

Proposition 7.2 (Task Equivalence is an Equivalence Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $D_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task equivalence satisfies the following properties and thus defines an equivalence relation.*

Property 7.2.a. *Reflexivity:* $\tau_1 \equiv \tau_1$.

Property 7.2.b. *Symmetry:* $\tau_1 \equiv \tau_2 \implies \tau_2 \equiv \tau_1$.

Property 7.2.c. *Transitivity:* $\tau_1 \equiv \tau_2 \wedge \tau_2 \equiv \tau_3 \implies \tau_1 \equiv \tau_3$.

Proof. Property 7.2.a: $\tau_1 \equiv \tau_1 \implies \tau_1 \preceq \tau_1$ by Property 14.1.a when $D_{1,1}$ and $H_{1,1}$ include the identity. Thus, task equivalence is reflexive if $D_{1,1}$ and $H_{1,1}$ include the identity.

Property 7.2.b: $\tau_1 \equiv \tau_2 \implies (\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)$ by Definition 14.5 $\implies (\tau_2 \preceq \tau_1) \wedge (\tau_1 \preceq \tau_2) \implies \tau_2 \equiv \tau_1$.

Property 7.2.c: $(\tau_1 \equiv \tau_2) \wedge (\tau_2 \equiv \tau_3) \implies (\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \wedge (\tau_3 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)$ by Definition 7.5. $(\tau_3 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1) \implies (\tau_3 \preceq \tau_1)$ by Property 7.1.c when $D_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. Similarly, $(\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \implies (\tau_1 \preceq \tau_3)$. Thus $(\tau_1 \preceq \tau_3) \wedge (\tau_3 \preceq \tau_1) \implies \tau_1 \equiv \tau_3$. Thus task equivalence is transitive if $D_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. $\square$

7.3 Relative complexity between tasks

In general, it may be difficult to establish a reduction between two completely distinct tasks such as grasping an object and avoiding obstacles; indeed an arbitrary pair of tasks is unlikely to satisfy the notion of reduction introduced in Section 7.2.2. Additionally, our definition of reduction is a binary one and does not capture *how* complex a particular task is relative to another. Motivated by these observations, we propose a definition of *relative complexity* that captures the degree to which one task is more complex than another. This

quantity can be thought of as a “continuous” or “smoothed” version of task reductions (in a precise sense, which we elucidate below). Note that we will omit the subscripts on D and H when it is clear which tasks they transform between.

Definition 7.7 (Relative Complexity). *The relative complexity of task τ_1 with respect to task τ_2 is*

$$C_{\tau_1/\tau_2} := \sup_{\pi_2^* \in \Pi_2^*} \inf_{h \in H, d \in D} \left[1 - \frac{R_1(d \circ \pi_2^* \circ h)}{R_1^*} \right], \quad (7.7)$$

where we assume that rewards are non-negative. We use the notation $C_{\tau_1/\tau_2}(H, D)$ when we want to highlight dependence on H and D .

Intuitively, if $\tau_1 \preceq \tau_2$, then this implies that $C_{\tau_1/\tau_2} = 0$ (as we show formally below). If $C_{\tau_1/\tau_2} > 0$, then an admissible policy $\pi_2^* \in \Pi_2^*$ may not be transformed into an admissible policy for τ_1 using encoders and decoders in H and D . A key advantage of this definition is that we can compare *any* two tasks and quantify the relative complexity of one with respect to another. As with our previous definitions, we prove a set of useful properties below. Importantly, Properties 7.3.c and 7.3.d will establish a link between the relative complexity and task reduction.

Proposition 7.3 (Properties of the Relative Complexity). *Relative Complexity satisfies the following properties:*

Property 7.3.a. *Nonnegativity and boundedness: $C_{\tau_1/\tau_2} \in [0, 1]$.*

Property 7.3.b. *Monotonicity with respect to H and D : If $H \subseteq H'$ and $D \subseteq D'$, then $C_{\tau_1/\tau_2}(H', D') \leq C_{\tau_1/\tau_2}(H, D)$.*

Assume that the supremum and infimum in Definition 7.7 are attained by functions in Π_2^, H, D . Then:*

Property 7.3.c. *Equivalence between reduction and 0 relative complexity: $C_{\tau_1/\tau_2} = 0 \iff \tau_1 \preceq \tau_2$.*

Property 7.3.d. *Equivalence between no reduction and positive relative complexity: $C_{\tau_1/\tau_2} \in (0, 1] \iff \neg(\tau_1 \preceq \tau_2)$.*

Proof. Property 7.3.a: $R_1(d \circ \pi_2^* \circ h) \in [0, R_1^*]$. Therefore, $R_1(d \circ \pi_2^* \circ h)/R_1^* \in [0, 1] \implies C_{\tau_1/\tau_2} \in [0, 1]$ for any H, D .

Property 7.3.b: Consider H, H' such that $H \subseteq H'$ and D, D' such that $D \subseteq D'$. For any function f , the following is true $\forall \pi_2^*$:

$$\inf_{h \in H', d \in D'} f(h, d, \pi_2^*) \leq \inf_{h \in H, d \in D} f(h, d, \pi_2^*). \quad (7.8)$$

This implies the following:

$$\sup_{\pi_2^* \in \Pi_2^*} \inf_{h \in H', d \in D'} \left[1 - \frac{R_1(d \circ \pi_2^* \circ h)}{R_1^*} \right] \leq \sup_{\pi_2^* \in \Pi_2^*} \inf_{h \in H, d \in D} \left[1 - \frac{R_1(d \circ \pi_2^* \circ h)}{R_1^*} \right]. \quad (7.9)$$

Property 7.3.c: Assume $C_{\tau_1/\tau_2} = 0$ for some H and $D \iff$ for any $\pi_2^* \in \Pi_2^* \exists d \in D$ and $h \in H$ such that $R_1(d \circ \pi_2^* \circ h) = R_1^*$. $R_1(\pi_1) = R_1^* \iff \pi_1 \in \Pi_1^*$. Thus, for all $\pi_2^* \in \Pi_2^* \exists d \in D$ and $h \in H$ such that $d \circ \pi_2^* \circ h \in \Pi_1^* \iff \tau_1 \preceq \tau_2$.

Property 7.3.d: The contrapositive of Property 7.3.c is $\neg(\tau_1 \preceq \tau_2) \iff C_{\tau_1/\tau_2} \neq 0$. By Property 7.3.a, the complexity measure is $C_{\tau_1/\tau_2} \in [0, 1]$, therefore, $C_{\tau_1/\tau_2} \in (0, 1] \iff C_{\tau_1/\tau_2} \neq 0$. Thus $C_{\tau_1/\tau_2} \in (0, 1] \iff \neg(\tau_1 \preceq \tau_2)$. $\square$

7.4 Generalization from one task to another

As computational complexity theory has done for decision problems, establishing concrete notions for task complexity and difficulty can be a powerful tool for comparing and classifying robotic tasks. In this section, we will illustrate this point with a PAC-Bayes generalization guarantee between tasks. Consider two arbitrary tasks τ_1 and τ_2 . Specifically the goal will be to use a policy which we have developed task τ_2 to generalize directly to τ_1 without explicitly needing samples or knowledge about the underlying distribution over environments. To begin, we introduce a stronger notion of task reduction.

Definition 7.8 (Strong Task Reduction). *Task τ_1 strongly reduces to task τ_2 (written $\tau_1 \leq \tau_2$) if for all admissible policies $\pi_2^* \in \Pi_2^*$*

$$d \circ \pi_2^* \circ h \in \Pi_1^* \quad (7.10)$$

for some encoder $h \in H_{1,2}$ and decoder $d \in D_{2,1}$.

The key difference in strong task reduction is that a single encoder h and decoder d must transform all policies π_2^* to admissible possibilities in Π_1^* . This is in contrast to task reduction, where the encoder and decoder can be selected *after* $\pi_2^* \in \Pi_2^*$ is picked.

Consider the problem setup provided in Section 2.1.2. We will also use the same notation as in Section 6.3 for an induced distribution $P_{e|\tau}$ over environments. Task τ_1 induces $P_{e|\tau_1}$ and τ_2 induces $P_{e|\tau_2}$. We then aim to find a distribution over parameters P_{θ_2} for control policy π_{θ_2} which solve τ_2 . However, with the addition of encoder h and decoder d , we aim to perform well on task τ_1 . Explicitly, the goal for performing well on τ_1 is as follows:

$$\min_{P_{\theta_2}} \mathbb{E}_{e \sim P_{e|\tau_1}} \mathbb{E}_{\theta_2 \sim P_{\theta_2}} C_e(d \circ \pi_{\theta_2} \circ h). \quad (7.11)$$

for a particular encoder $h \in H$ and decoder $d \in D$. Note that we are testing the policy given by $d \circ \pi_{\theta_2} \circ h$ on environments drawn from $P_{e|\tau_1}$ instead of $P_{e|\tau_2}$.

Now suppose τ_1 strongly reduces to τ_2 : $\tau_1 \leq \tau_2$. We are also given a dataset of training environments from τ_2 : $S_2 = \{e_1, e_2, \dots, e_n\} \sim P_{e|\tau_2}$ and train a policy which performs well on τ_2 using e.g. Algorithm 2.1. We will call a posterior distribution over policy parameters *admissible* if the following is satisfied:

$$\mathbb{P}_{e \sim P_{e|\tau_2}} \mathbb{P}_{\theta_2 \sim P_{\theta_2}} [C_e(\pi_{\theta_2}) \leq C_2^*] \geq 1 - \delta' \quad (7.12)$$

Where C_2^* is a chosen cost success threshold on τ_2 . In other words, with high probability, a policy created with the parameters sampled from the policy distribution is admissible on τ_2 . With these preliminaries, we can directly write a PAC-Bayes bound on the performance of $P_{e|\tau_2}$ on τ_1 :

Theorem 7.1 (Strong Task Reduction PAC-Bayes Generalization Bound). *If $\tau_1 \leq \tau_2$ with encoder h and decoder d , then for any data-independent prior distribution $P_{\theta_{2,0}}$ over θ_2 , some cost function C which satisfies Assumption 2.1, $C_1^* = C_2^*$, $n \geq 8$, and $\delta, \delta' \in (0, 1)$, with probability at least $1 - \delta - \delta'$ over a sampling of $S_2 = \{e_1, e_2, \dots, e_n\} \sim P_{e|\tau_2}^n$, the*

following holds simultaneously for all admissible posterior distributions P_{θ_2} on τ_2 :

$$\mathbb{E}_{e \sim P_{e|\tau_1}} \mathbb{E}_{\theta_2 \sim P_{\theta_2}} C_e(d \circ \pi_{\theta_2} \circ h) \leq \frac{1}{n} \sum_{e \in S_2} \mathbb{E}_{\theta \sim P_{\theta_2}} C_{e_i}(\pi_{\theta_2}) + \sqrt{\frac{D_{\text{KL}}(P_{\theta_2} \| P_{\theta_2,0}) + \ln \frac{2\sqrt{n}}{\delta}}{2n}}, \quad (7.13)$$

and D_{KL} is the Kullback-Leibler (KL) divergence.

Proof. Equation 7.13 follows from using a union bound on Equations 2.4 and 7.12 applied to the objective in Equation 7.11. $\square$

It is important to note that we introduce Theorem 7.1 as an illustration of the potential power of notions of task reduction and relative complexity between robotic tasks and not as a tool for developing distributions over policies as we have used it in Chapters 2 and 6. There are a few key challenges with applying Theorem 7.1 directly: (i) The requirement that distribution P_{θ_2} be admissible on τ_2 may be difficult to achieve on an arbitrary problem. (ii) It may be impossible in general to find encoder h and decoder d for a pair of tasks as strong task reduction is more challenging to prove than task reduction. (iii) Knowledge that τ_1 strongly reduces to τ_2 would likely imply specific knowledge of the distribution over environments $P_{e|\tau_2}$ and $P_{e|\tau_1}$ which we have tried to avoid in this dissertation as it is difficult ensure in practice.

Chapter 8

Conclusions and future work

8.1 Summary of contributions

Throughout this dissertation, we have presented and evaluated novel methods for out-of-distribution (OOD) detection and OOD generalization. We are motivated by learning control policies for complex robotic systems so they can be deployed in the complex and interactive real-world. We presented a PAC-Bayes bound in the policy learning setting (Theorem 2.1 [136]) and showed an approach for learning control policies that provably generalize well to novel environments given a dataset of example environments. The approach is powerful for developing effective control policies, but falls short of dealing with out-of-distribution settings in testing.

We extend the framework from Section 2.2 in Chapter 3, where we present a derandomized PAC-Bayes bound in the policy learning setting. Using the derandomized PAC-Bayes bound, we developed methods for performing task-driven OOD detection with statistical guarantees. We trained a policy with a bound on the expected cost on the training distribution and performed OOD detection on test environments by checking for violations of the bound (using p-values and concentration inequalities). Our simulated and hardware experiments demonstrated the ability of our approach to perform OOD detection within a handful of trials. Comparisons with baselines also demonstrate two key advantages: our OOD detectors (i) are only sensitive to task-relevant distribution shifts, and (ii) provide statistical guarantees on detection.

The methods in Chapter 3 provided strong results for offline analysis of distribution shifts; however, we aimed to develop a method for online failure prediction. In Chapter 4, we propose an approach for training a real-time failure predictor with rigorous guarantees. We develop a novel class-conditional bounds that allow us to trade-off the relative rates of false-positive and false-negative errors. We present an algorithmic approach that parameterizes the predictor using a neural network and minimizes the theoretical error bounds. Through experiments on a vision-based drone navigation, we demonstrate improved safety when deploying the failure prediction and strong guarantees on false-positive and false-negative rates.

In Chapter 5 we presented a task-relevant detector for mispredictions from trajectory forecasting networks used in AVs. In contrast to Chapter 4, the proposed method does not require examples of failure in order to provide strong guarantees on detection. Our detector builds a distribution of the predicted future costs and identifies an anomaly when the true cost of the planner lies in the top- p quantile of that distribution. We provided false positive and false negative rate guarantees for our detector that facilitate data-free calibration. In experiments, we demonstrated that our detector out-performs all other baseline approaches we compared against and is compute-efficient. Additionally, we showed that our data-free calibrated detector’s performance is close to the optimal detector chosen from the ROC curve.

The methods in Chapters 3, 4, and 5 are all detectors for distribution shifts or anomalies. Although the methods are capable of improving the safety of robotic systems as we show, this also requires identifying what to do when a failure or anomaly is detected. Consider an AV operating in a busy city center, an incorrect emergency maneuver could significantly worsen the outcomes for the ego vehicle and other agents. With OOD generalization, we can directly improve the safety of robotic systems operating in an OOD setting. In Chapter 6 we present two such OOD generalization bounds. We first presented a *distributionally robust* PAC-Bayes generalization bound for distribution shifts of bounded KL divergence between the training and testing environment distribution. We then presented a generalization bound for gradient-based meta-learning which uses different generalization frameworks to tackle the distinct challenges of generalization at the two levels of meta-learning. We employ

uniform stability bounds and PAC-Bayes bounds at the base- and meta-learning levels respectively. In experiments, we provide significantly tighter generalization guarantees as compared to state-of-the-art meta-learning bounds while maintaining comparable empirical performance. However, in larger-scale settings, tight generalization guarantees could not be achieved.

Motivated by this, we aimed to develop a framework for comparing the complexity of tasks and eventually generalizing between tasks. In Chapter 7, we defined a notion of reduction between two tasks that captures the ability of a robot to solve one task given a policy for another. We also presented a measure of relative complexity that quantifies how complex one task is relative to another. Our theoretical results establish basic properties satisfied by these notions and also establish the relationship between reductions and relative complexity. Based on these notions, we present a PAC-Bayes generalization bound which can be used to prove strong performance on an OOD task if it reduces to the task on which the robot was trained.

8.2 Limitations and Future work

There are a number of limitations with the work discussed in this dissertation which motivate exciting directions for future work. For the OOD detection methods in Chapters 3, 4, and 5, we did not provide a generative or guaranteed method for determining what to do once an anomaly or potential failure was signalled. In Chapter 4, we deployed an emergency maneuver to stop the drone system mid-flight to avoid a crash, but this would not always work for an AV operating in traffic. We may have a set of emergency maneuvers for such a system (e.g. pull over to the side of the road, brake quickly in a straight line, swerve to avoid another agent), but picking the incorrect maneuver could be detrimental. Future work can explore how to modify the robotic policy once a detection occurs. One promising approach is to use causality [163] to determine *why* a detection occurred. With a strong causal understanding of the setting, the appropriate way to change the policy may be more clear.

We make frequent use of PAC-Bayes generalization bounds which we have shown can

provide a strong guarantee on performance. However, safety critical robotic systems may need significantly strong bounds in order to be effective in the real-world. In both the OOD detection and OOD generalization lines of research, stronger bounds are an important direction for future work. For OOD detection, this would mean high confidence of detection and better guarantees on the maximum rates of false-positive and false-negative errors. For OOD generalization, this corresponds to tightening the performance bound for a robot operating in an OOD setting. One of the key technical challenges with achieving strong bounds is the $(\tilde{O}\sqrt{1/N})$ sample complexity of the regularizer term in PAC-Bayes bound. One approach would be to use more data in training and for computing the bounds, but that may not always be practical or possible. Another approach is to provide generalization guarantees based on conformal prediction. Recent techniques have been proposed for learning failure predictors with error bounds [131] using conformal prediction. Importantly, the framework allows for guarantees with better sample efficiency ($\tilde{O}(1/N)$, compared to $\tilde{O}(\sqrt{1/N})$ for PAC-Bayes bounds). However, the guarantees hold in expectation over both the test sample and training datasets, which is even more limited than PAC-Bayes. Future work can explore how to combine favorable aspects of such generalization frameworks in order to tighten guarantees.

A key advantage of methods such as Hamilton-Jacobi reachability [146] (which provides safety certificates for *worse case* outcomes) over PAC-Bayes guarantees (which provide safety certificates in expectation) is that they are more applicable for safety critical system. However, the worse case guarantee is also limited as it may be overly restrictive for systems which must operate in a complex and changing world. In particular, a driver must take risks by driving to work, and not all unfavorable outcomes are preventable. An exciting direction future work is to develop guarantees for the *risk* when deploying a robotic system in a novel environment. Risk metrics such as conditional value-at-risk are appealing options for robotic systems which operate in uncertainty [137] and could provide a powerful middle-ground between bounded expected performance and bounded worse-case performance.

Another exciting direction, not just for improving and broadening generalization guarantees, but for robotics as a field, is to develop methods for comparing the complexity of robotic tasks. We present notions of task reduction and relative complexity in Chapter 7,

but it would be interesting to devise more general notions of reductions by, e.g., allowing the policy from one task to be used as an oracle to solve another task in a more general manner (instead of the specific encoder-policy-decoder architecture we utilize). Another research direction is to use our notion of reduction to establish equivalences between broad classes of tasks (similar to complexity classes P and NP in computational complexity). It may also be of practical interest to define relative complexity in terms of different metrics such as data efficiency or difficulty of finding a policy instead of the notion of complexity we use, which is based on the complexity of online execution of policies. With more developed robotics complexity theory, there is potential for more general OOD generalization guarantees. Indeed, because of the impact of computation complexity theory [19], determining if a new decision problem reduces to any existing decision problems is just as important, if not more important, than solving it. We have many powerful algorithms and methods for solving challenging robotic problem and providing strong guarantees in certain settings, but can use such methods more widely with a better understanding of how differing robotic tasks relate.

Lastly, we discuss the high energy cost of training and developing methods based on machine learning. Machine learning methods can require a lot of data and training time. We presented methods which required expensive simulations to generate data and meta-learning methods which took hundreds of hours to train. The results in this dissertation (both computing the results presented but also all the development for each method) represent months of continuous computation time on powerful computers or Amazon Web Servers (AWS). This creates challenges with energy usage. One promising paper [88] shows that machine learning models which have trained faster also generalize better. Indeed, we show in Chapter 6 that this intuition can be used to develop a tight generalization guarantee in the meta-learning setting. It is important future work to explore more efficient learning methods to both provide stronger generalization, and reduce energy usage from machine learning.

Part II

Papers

Chapter 9

Overview

9.1 Outline

In this part of the dissertation, we reproduce the five published papers which constitute a majority of the content in Part I. Additional details provided in these papers are cited throughout Part I. Note that * indicates equal contribution.

- **Chapter 10:** A. Farid,* S. Veer,* and A. Majumdar. Task-driven out-of-distribution detection with statistical guarantees for robot learning. *Proceedings of the Conference on Robot Learning*, 2021. [70]
- **Chapter 11:** A. Farid,* D. Snyder,* A. Z. Ren, and A. Majumdar. Failure prediction with statistical guarantees for vision-based robot control. *Proceedings of Robotics: Science and Systems*, 2022. [68]
- **Chapter 12:** A. Farid,* S. Veer,* B. Ivanovic, K. Leung, and M. Pavone. Task-relevant failure detection for trajectory predictors in autonomous vehicles. *Proceedings of the Conference on Robot Learning*, 2022. [69]
- **Chapter 13:** A. Farid and A. Majumdar. Generalization bounds for meta-learning via PAC-Bayes and uniform stability. *Advances in Neural Information Processing Systems*, 34:2173–2186, 2021. [67]
- **Chapter 14:** M. Ho,* A. Farid,* and A. Majumdar. Towards a framework for

comparing the complexity of robotic tasks. *Proceedings of the Workshop on the Algorithmic Foundations of Robotics*, 2022. [95]

Other pre-prints or published papers on which I am an author are listed explicitly here. These papers have also influenced the work described in Part I and are cited where necessary.

- A. Majumdar, A. Farid, and A. Sonar. PAC-Bayes control: learning policies that provably generalize to novel environments. *The International Journal of Robotics Research*, 40(2-3):574–593, 2021. [135]
- A. Farid,* S. Veer,* D. Pachisia, and A. Majumdar. Task-driven detection of distribution shifts with statistical guarantees for robot learning. *arXiv preprint arXiv:2106.13703*, 2021. [71]
- S. Veer, A. E. Gurgen, A. Farid, and A. Majumdar. Probably approximately correct vision-based planning using motion primitives. *arXiv preprint arXiv:2002.12852*, 2022. [207]

9.2 Author Contributions

I am a lead author on the five papers reproduced in this part. Here, we list the explicit contributions of all authors:

- **Chapter 10:** A. Majumdar served as a close advisor throughout this project. A. Majumdar and A. Farid developed the idea for detecting out-of-distribution environments by checking violations of a performance bound and all authors iterated on various approaches for providing detection with confidence. A. Farid proved Theorem 10.1. S. Veer proved the methods for detection with confidence which are presented in the paper based on Theorems 10.2 and 10.3. S. Veer also performed the experiments in Section 10.5.1 and produced the figures in this section. A. Farid performed the experiments in Section 10.5.2, set up the hardware platform used in Section 10.5.2, and generated the figures in this section. All authors contributed to writing and edited the paper extensively.

- **Chapter 11:** A. Majumdar served as a close advisor throughout this project. A. Majumdar and A. Farid developed the idea for online failure prediction in order to improve the safety of a robotic system. A. Farid created Figure 11.1. A. Farid and D. Snyder worked on the reduction presented in Table 11.1. D. Snyder produced Figures 11.2 and 11.3, and developed and proved Theorem 11.3. A. Farid developed the training technique for improving safety of a robotic system using Theorems 11.2 and 11.3. A. Farid also performed all experiments in Section 11.5.1, set up the hardware platform used in Section 11.5.1, and generated the figures/tables in this section. A. Z. Ren performed all experiments in Section 11.5.2, set up the hardware platform used in Section 11.5.2, and generated the figures/tables in this section. All authors contributed to writing and edited the paper extensively.
- **Chapter 12:** M. Pavone and S. Veer served as close advisors throughout this project. S. Veer had the idea to use samples from the predicted distribution over future non-ego agent trajectories in order to perform task-relevant failure detection. S. Veer created Figure 12.1. A. Farid created Figure 12.2. S. Veer and A. Farid developed Algorithm 12.1. A. Farid proved Theorem 12.1. A. Farid developed the experimental setups in Section 12.5. B. Ivanovic and K. Leung assisted in overcoming technical challenges throughout the project: B. Ivanovic especially with Trajectron++ and K. Leung especially with the Hamilton-Jacobi reachability analysis comparison method. A. Farid performed all experiments and created all figures and tables in Section 12.5 and the Appendix. A. Farid and S. Veer wrote a majority of the paper and edited it extensively.
- **Chapter 13:** A. Majumdar had the idea to use generalization guarantees based on uniformly stable algorithms in order to provide a meta-learning bound. A. Farid developed and proved all results with close guidance and extensive advise from A. Majumdar. A. Farid developed the training techniques, set up the experiments, ran all experiments, and generated all tables in the paper and appendix. A. Farid wrote a majority of the paper with extensive edits from A. Majumdar.
- **Chapter 14:** A. Majumdar and A. Farid served as close advisors to M. Ho on this

project. All authors generated ideas for comparing the complexity of robotic tasks and eventually settled on the idea presented. A. Farid created Figure 14.1. A. Farid and M. Ho proved the results in Section 14.4.2. A. Farid proved the results in Sections 14.5 and 14.6. All authors developed the training approach in Section 14.7. M. Ho developed the experiments in Section 14.8 and ran all experiments. A. Farid generated the figures in 14.8. A. Majumdar and A. Farid wrote a majority of the paper and edited it extensively.

Chapter 10

Task-Driven Out-of-Distribution Detection with Statistical Guarantees for Robot Learning

ALEC FARID,* SUSHANT VEER,* AND ANIRUDHA MAJUMDAR

Our goal is to perform *out-of-distribution (OOD) detection*, i.e., to detect when a robot is operating in environments that are drawn from a different distribution than the environments used to train the robot. We leverage Probably Approximately Correct (PAC)-Bayes theory in order to train a policy with a *guaranteed bound* on performance on the training distribution. Our key idea for OOD detection then relies on the following intuition: violation of the performance bound on test environments provides evidence that the robot is operating OOD. We formalize this via statistical techniques based on p-values and concentration inequalities. The resulting approach (i) provides guaranteed confidence bounds on OOD detection, and (ii) is *task-driven* and sensitive only to changes that impact the robot’s performance. We demonstrate our approach on a simulated example of grasping objects with unfamiliar poses or shapes. We also present both simulation and hardware experiments for a drone performing vision-based obstacle avoidance in unfamiliar environments (including wind disturbances and different obstacle densities). Our examples demonstrate that we can perform task-driven OOD detection within just a handful of trials. Comparisons with

baselines also demonstrate the advantages of our approach in terms of providing statistical guarantees and being insensitive to task-irrelevant distribution shifts.

10.1 Introduction

Imagine a drone trained to perform vision-based navigation using a dataset of indoor environments and deployed in environments with varying wind conditions, obstacle densities, or lighting (Fig. 10.1). Similarly, consider a robot arm manipulating a new set of objects or an autonomous vehicle deployed in a new city. State-of-the-art techniques for learning-based control of robots typically struggle to generalize to such *out-of-distribution* (OOD) environments. This lack of OOD generalization is particularly pressing in safety-critical settings, where the price of failure is high. In this work, we focus on the problem of autonomously *detecting* when a robot is operating in environments drawn from a different distribution than the one used to train the robot. This ability to perform *OOD detection* has the potential to improve the safety of robotic systems operating in OOD environments. For example, a drone operating in a new set of environments could either deploy a highly conservative policy or cease its operations altogether. In addition, OOD detection can also allow the robot to improve its policy by re-training using additional data collected from the new environments.

There are two important desiderata that OOD detection approaches for safety-critical robotic systems should ideally satisfy. First, we would like to develop OOD detection techniques with *guaranteed confidence bounds*. Second, we would like our OOD detectors to be *task-driven* and only sensitive to *task-relevant* changes in the robot’s environment. As an example, consider again the drone navigation setting in Fig. 10.1 and suppose that the robot’s policy is insensitive to changes in color and lighting. Here, the robot’s OOD detector should *not* trigger even if the robot is operating in environments with different color/lighting and should only trigger if there are task-relevant variations (e.g., variations in the obstacle density). Unfortunately, current approaches (Sec. 10.2) do not typically satisfy both desiderata; they are often based on heuristics and not task-driven in general.

Statement of Contributions. We develop task-driven OOD detection techniques with

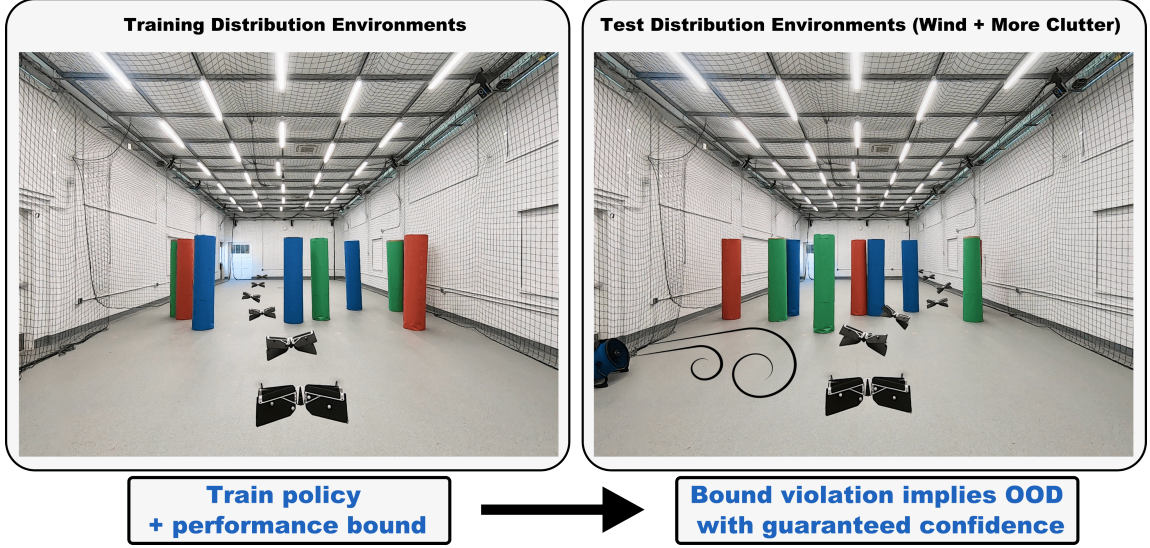


Figure 10.1: A schematic of our overall approach. We learn policies with guaranteed bounds on expected performance on the training distribution. Violation of this bound during deployment implies that the robot is operating OOD (with high confidence). We present simulation experiments in manipulation and navigation settings, and hardware experiments for a drone navigating in new environments with varying wind conditions and clutter.

statistical guarantees on correctness. To this end, we make three specific contributions (see Fig. 10.1 for an overview).

- Given a dataset of environments drawn from an (unknown) training distribution, we develop a pipeline based on *generalization theory* for training control policies with a *guaranteed bound* on performance (a bound on the expected cost of the policy on the unknown training distribution). Specifically, we leverage recently-developed *derandomized Probably Approximately Correct (PAC)-Bayes* bounds that are well-suited to enable OOD detection (Sec. 10.4.1).
- We develop two OOD detection techniques with complementary statistical interpretations based on the following intuition (Sec. 10.4.2): if the costs incurred when the robot is deployed in new environments violate the bound on the policy’s performance, then this indicates that the robot is operating OOD. We formalize this via p-values and concentration inequalities and provide statistical guarantees on the false positive rate of OOD detection. Since our OOD detection scheme leverages the costs incurred in new environments, it is *only* triggered by task-relevant changes.

- We demonstrate our approach on two simulated examples (Sec. 10.5): (i) a robotic manipulator grasping a new set of objects, and (ii) a drone navigating a new set of environments. Comparisons with baselines demonstrate the advantages of our approach in terms of providing statistical guarantees and being insensitive to task-irrelevant shifts. We also present a thorough set of hardware experiments for vision-based drone navigation with varying wind conditions and clutter (Fig. 10.1). Our experiments demonstrate the ability of our approach to perform task-driven OOD detection within just a handful of trials for systems with complex dynamics and rich sensing modalities.

10.2 Related work

Anomaly/OOD detection in supervised learning. Anomaly detection in low-dimensional signals has been well-studied in the signal processing literature (see [24] for a review). Recent work in machine learning has focused on OOD detection for high-dimensional inputs (e.g., images) in supervised learning settings (see [175] for a review). Popular approaches use threshold-based detectors for the output distribution of a given pre-trained neural classifier [92, 127, 91]. Other methods use a specific pipeline for training a neural network in order to improve OOD detection on test samples [59, 120, 93]. However, these methods are often susceptible to adversarial attacks [49]. Thus, approaches for addressing adversarial data have been developed [121, 49, 28]. Some of these approaches are also able to provide theoretical guarantees of performance on adversarial data [49, 145, 144]. Other methods provide PAC-style statistical guarantees [194, 129] or p-values [25]. However, these methods typically focus on supervised learning settings and often require specific network outputs (e.g., softmax) that are incompatible with non-classification tasks. In contrast, we focus on OOD detection for policy learning settings in robotics and do not make assumptions about the specific structure of the policy.

Task-driven OOD detection. The methods above are aimed at detecting *any* distributional shift in the data and can be sensitive even to task-*irrelevant* shifts (i.e., ones that do not impact performance) as we demonstrate in our experiments (Sec. 10.5). A recent approach determines an estimate of input atypicality for pre-trained networks and uses it

as an OOD detector in supervised learning settings [191]. Recent methods have also been developed specifically for reinforcement learning (RL) [184, 185, 40, 222, 83]. In particular, [83] presents a general task-driven approach for OOD detection on sequential rewards, which is optimal in certain settings. However, neither this method nor others in the RL context provide statistical guarantees on detection. We propose an OOD detection framework which is both task-driven and provides statistical guarantees by leveraging generalization theory.

Generalization theory. Generalization theory provides a way to learn hypotheses (in supervised learning) with a bound on the true expected loss on the underlying data-generating distribution given only a finite number of training examples. Original frameworks include Vapnik-Chervonenkis (VC) theory [206] and Rademacher complexity [188]. However, these methods often provide vacuous generalization bounds for high-dimensional hypothesis spaces (e.g., neural networks). Bounds based on PAC-Bayes generalization theory [192, 142, 186] have recently been shown to provide strong guarantees in a variety of settings [63, 116, 78, 23, 105, 162], and have been significantly extended and improved [43, 44, 141, 172, 200, 64]. PAC-Bayes has also recently been extended to learn policies for robots with guarantees on generalization to novel environments [136, 208, 166, 135]. In the present work, we leverage recently-proposed *derandomized* PAC-Bayes bounds [209]; this framework allows us to train a single deterministic policy with a guaranteed bound on expected performance on the training distribution (in contrast to [136, 208, 166, 135], which train stochastic neural network policies). This forms the basis for our OOD detection framework: by observing violations of the PAC-Bayes bound on test environments, we are able to perform task-driven OOD detection with statistical guarantees.

10.3 Problem formulation

Dynamics and environments. Let $s_{t+1} = f_E(s_t, a_t)$ describe the robot’s dynamics, where $s_t \in \mathcal{S} \subseteq \mathbb{R}^{n_s}$ is the state of the robot at time-step t , $a_t \in \mathcal{A} \subseteq \mathbb{R}^{n_a}$ is the action, and $E \in \mathcal{E}$ is the environment that the robot is operating in. “Environment” here broadly refers to factors that are external to the robot, e.g., a cluttered room that a drone is navigating, disturbances such as wind gusts, or an object that a manipulator is grasping. The dynamics

of the robot may be nonlinear/hybrid. We denote the robot’s sensor observations (e.g., RGB-D images) by $o_t \in \mathcal{O} \subseteq \mathbb{R}^{n_o}$.

Cost functions. The robot’s task is encoded via a cost function and we let $C_E(\pi)$ denote the cost incurred by a (deterministic) policy π when deployed in environment E over a finite time horizon T . The policy $\pi \in \Pi$ is a mapping from (histories of) sensor observations to actions (e.g., parameterized using a neural network). In the context of obstacle avoidance, the cost could capture how close the drone gets to an obstacle; in the context of grasping, the cost could be 0 if the robot successfully lifts the object or 1 otherwise. We assume that the cost is bounded; without further loss of generality, we assume $C_E(\pi) \in [0, 1]$. We also assume that the robot has access to the cost $C_E(\pi)$ after performing a rollout on E (i.e., at the end of an episode of length T). This is a relatively benign assumption in robotics contexts since the cost function often has physical meaning and can be measured by the robot’s sensors. For example, a drone equipped with a depth sensor can measure the smallest reported depth value during its operation in an environment, and a manipulator equipped with a camera and/or force-torque sensor can measure if it successfully grasped an object. We make no further assumptions on the cost function (e.g., we *do not* assume continuity, Lipschitzness, etc.).

Training and testing distribution. We assume that the robot has access to a training dataset $S = \{E_1, \dots, E_m\}$ of m environments drawn i.i.d. from a training distribution $\mathcal{D}$, i.e. $S \sim \mathcal{D}^m$. After training, the robot is deployed on environments in $S' = \{E'_1, \dots, E'_n\}$ drawn from a test distribution $\mathcal{D}'$: $S' \sim \mathcal{D}'^n$. Importantly, we *do not* assume any explicit knowledge of $\mathcal{D}, \mathcal{D}'$ or the space $\mathcal{E}$ of environments. We only have indirect access to $\mathcal{D}, \mathcal{D}'$ in the form of the finite training datasets S, S' .

Goal: task-driven OOD detection. After being deployed in (a typically small number of) environments in S' , the robot’s goal is to detect if these environments were drawn from a different distribution than the training distribution (i.e., if $\mathcal{D}'$ is different from $\mathcal{D}$). Moreover, our goal is to perform *task-driven* OOD detection. In particular, our OOD detector should only trigger if:

$$C_{\mathcal{D}'}(\pi) := \mathbb{E}_{E' \sim \mathcal{D}'} C_{E'}(\pi) > C_{\mathcal{D}}(\pi) := \mathbb{E}_{E \sim \mathcal{D}} C_E(\pi) . \quad (10.1)$$

Thus, our OOD detector should be *insensitive* to changes in the environment distribution that do not adversely¹ impact the robot’s performance. This is a challenging task since we only assume access to a finite number of environments from $\mathcal{D}$ and $\mathcal{D}'$. Moreover, our goal is to develop an OOD detection framework that is broadly applicable in challenging settings involving nonlinear/hybrid dynamics, rich sensing modalities (e.g., RGB-D), and neural network-based policies.

10.4 Approach

Our overall approach is illustrated in Fig. 10.1. First, we train a policy with an associated *guarantee* on the expected cost on the training distribution $\mathcal{D}$ (Sec. 10.4.1). We then apply our OOD detection scheme which formalizes the following intuition: violation of the bound during deployment implies (with high confidence) that the test distribution $\mathcal{D}'$ is OOD in a task-relevant manner (Sec. 10.4.2).

10.4.1 Policy training via derandomized PAC-Bayes bounds

Given a training dataset $S = \{E_1, \dots, E_m\}$ of m environments drawn i.i.d. from the training distribution $\mathcal{D}$, our goal is to learn a policy π with a guaranteed bound on the expected cost $C_{\mathcal{D}}(\pi) := \mathbb{E}_{E \sim \mathcal{D}} C_E(\pi)$. Since our OOD detection scheme will rely on violations of the bound, it is important to obtain bounds that are as tight as possible. In this work, we utilize the *Probably Approximately Correct (PAC)-Bayes* framework [192, 142, 186] to train policies with strong guarantees. More specifically, we leverage recently-developed *derandomized* PAC-Bayes bounds [209], which are well-suited to the OOD detection setting (as we explain further below).

PAC-Bayes applies to settings where one chooses a *distribution* over policies (e.g., a distribution over weights of a neural network), and learning algorithms that have the following structure: (1) choose a “*prior*” distribution P_0 over the policy space Π *before* observing any data (this can be used to encode domain/expert knowledge); (2) obtain a training dataset

¹The OOD detection scheme we present can also be modified to detect if the expected cost on $\mathcal{D}'$ is *smaller* than the expected cost on $\mathcal{D}$; however, we focus on the other case since that is of greater practical interest.

S and choose a *posterior* distribution P over the policy space Π . Let P be the output of an algorithm A which takes P_0 and S as input. Denote the cost incurred by a policy π on the training environments in S as $C_S(\pi) := \frac{1}{m} \sum_{E \in S} C_E(\pi)$. The following result is our primary theoretical tool for training policies with bounds on performance.

Theorem 10.1. *For any distribution $\mathcal{D}$, prior distribution P_0 , $\delta \in (0, 1)$, cost bounded in $[0, 1]$, $m \geq 8$, and deterministic algorithm A which outputs the posterior distribution P , we have the following:*

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)} \left[C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S) \right] \geq 1 - \delta, \quad (10.2)$$

where $\overline{C}_{\delta}(\pi, S) := C_S(\pi) + \sqrt{R}$, $R := (D_2(P||P_0) + \ln \frac{2\sqrt{m}}{(\delta/2)^3})/(2m)$, and D_2 is the Rényi Divergence for $\alpha = 2$ defined as: $D_2(P||P_0) = \ln (\mathbb{E}_{\pi \sim P_0} [(\frac{P(\pi)}{P_0(\pi)})^2])$.

Proof. The proof is in App. 10.7.1. We use [209, Theorem 2], a general pointwise PAC-Bayes bound. We perform the reduction from supervised learning to policy learning presented in [136]. $\square$

This result allows us to obtain policies with guaranteed bounds on the expected cost. In particular, we can search for a posterior P in order to minimize the sum of the training cost and the “regularizer” $\sqrt{R}$. We describe such training methods via backpropagation and blackbox optimization below. Sampling from the resulting posterior P provides a policy with a bound on $C_{\mathcal{D}}(\pi)$ that holds with high probability (over the sampling of the training dataset S and the policy π).

Recent work has demonstrated the effectiveness of PAC-Bayes to provide strong bounds for deep neural networks [63, 23, 162] and specifically for policy learning [136, 208, 166, 135]. However, the bounds used by these approaches do not provide a viable approach for performing OOD detection. The approaches are based on traditional PAC-Bayes bounds, where a *distribution* P over policies (e.g., a distribution over neural network weights) is chosen; the resulting bound is on $\mathbb{E}_{\pi \sim P} C_{\mathcal{D}}(\pi)$ instead of $C_{\mathcal{D}}(\pi)$. Thus, given a test dataset S' of environments, many policies from the distribution P must be sampled in order to evaluate/bound the expected cost on S' . This is not feasible in an OOD detection setting,

where there is single execution on the test environments. Our use of the derandomized PAC-Bayes bound in Theorem 10.1 avoids this issue since we can bound $C_{\mathcal{D}}(\pi)$ for a *particular* policy sampled from P .

We provide approaches for optimizing the bound provided in Theorem 10.1 using back-propagation (App. 10.7.4) and Evolutionary Strategies (ES) [220] (App. 10.7.5). Since Theorem 10.1 requires a deterministic training algorithm, we fix the random seed for stochastic training methods. This makes the algorithm deterministic as the same input will always produce the same output. We choose multivariate Gaussian distributions with diagonal covariance $\text{diag}(s)$, i.e., $P = \mathcal{N}(\mu, \text{diag}(s))$, for the posterior P and prior P_0 distributions. Further, let $\psi := (\mu, \log s)$; we use the shorthand $\mathcal{N}_{\psi}$ for $\mathcal{N}(\mu, \text{diag}(s))$. We denote π_w with weights $w \sim \mathcal{N}_{\psi}$ as a parameterization of the robot’s policy (e.g., neural networks with weights w). After training, we sample and fix a w from the trained posterior for deployment on test environments. We then compute the PAC-Bayes upper bound $\overline{C}_{\delta}(\pi, S)$.

10.4.2 Task-driven OOD detection with statistical guarantees

We now tackle the problem of OOD detection as defined in Sec. 10.3. The PAC-Bayes training pipeline from Sec. 10.4.1 produces a policy π and an associated bound $\overline{C}_{\delta}(\pi, S)$ on the expected cost $C_{\mathcal{D}}(\pi)$ that holds with probability $1 - \delta$ over the sampling of the training dataset $S \sim \mathcal{D}^m$ and the policy $\pi \sim P$. Our key idea for OOD detection is that if our PAC bound $\overline{C}_{\delta}(\pi, S)$ is violated by π in the test environments S' (drawn from the test distribution $\mathcal{D}'$), then this indicates that the test environments are OOD. We formalize this intuition below using two popular frequentist statistical inference tools — hypothesis testing via p-value and confidence interval overlap.

Method 1: Hypothesis testing

In accordance with our notion of task-driven OOD detection (10.1) described in Sec. 10.3, let H_0 be the null-hypothesis which claims that $C_{\mathcal{D}'}(\pi) \leq C_{\mathcal{D}}(\pi)$ and let H_1 be the alternate hypothesis which claims that $C_{\mathcal{D}'}(\pi) > C_{\mathcal{D}}(\pi)$ for a given policy π . Then, to perform a hypothesis test, we compute the p-value and check if it drops below a significance level $\alpha \in (0, 1)$, which is chosen before looking at the data. The p-value bounds the probability

of drawing test environments with average cost larger than the observed test cost $C_{S'}(\pi)$, assuming that the null-hypothesis holds. In the event that the p-value is smaller than α , we can conclude that under the null-hypothesis the observed test dataset S' had a very small probability of being drawn; therefore, the null-hypothesis H_0 can be rejected. A mathematically precise definition of the p-value is given as follows.

Definition 10.1 (adapted from [96]). *Let $\mathcal{D}'$ be the test distribution and $S' \sim \mathcal{D}'^n$ be an observed dataset. Let π be the robot's control policy. Then, the p-value is defined as:*

$$p(S') := \mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) \geq C_{S'}(\pi) \mid H_0] . \quad (10.3)$$

Note that $\hat{S}$ is *any* dataset of cardinality n drawn i.i.d. from the test distribution $\mathcal{D}'^n$, while S' is the observed test dataset which we must use for inference. We present an upper bound on the p-value by leveraging the PAC-Bayes generalization bound (Theorem 10.1). This upper bound holds with probability $1 - \delta$ (over the sampling of S and π).

Theorem 10.2. *Let $\mathcal{D}$ be the training distribution and P be the posterior distribution on the space of policies obtained through the training procedure described in Sec. 10.4.1. Let $S' \sim \mathcal{D}'^n$ be a test dataset, $p(S')$ be the p-value for S' defined in Definition 10.1, and $\delta \in (0, 1)$. Then,*

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)} [p(S') \leq \exp(-2n\bar{\tau}(S)^2)] \geq 1 - \delta , \quad (10.4)$$

where $\bar{\tau}(S) := \max\{C_{S'}(\pi) - \bar{C}_\delta(\pi, S), 0\}$.

Proof. The proof relies on Hoeffding's inequality and is provided in App. 10.7.2. □

Theorem 10.2 provides an upper bound on the p-value which holds with high confidence. If the upper bound is below the significance level α , then with high confidence we can say that the p-value is below α ; thereby, Theorem 10.2 facilitates OOD detection through hypothesis testing.

Method 2: Confidence interval on the difference in expected train and test costs

We now present another method for performing task-driven OOD detection (Sec. 10.3 and Eq. (10.1)) by lower bounding $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$ with high probability. If the lower bound is positive, then with high confidence we can conclude that the robot is operating in (task-relevant) OOD environments as the test cost is larger than the train cost. Furthermore, this lower bound serves as a measure of how “far” the two distributions are from the perspective of the task.

Theorem 10.3. *Let $\mathcal{D}$ be the training distribution, $\mathcal{D}'$ be the test distribution, and P be the posterior distribution on the space of policies obtained through the training procedure described in Sec. 10.4.1. Let $\delta, \delta' \in (0, 1)$ such that $\delta + \delta' < 1$, $\gamma := \sqrt{\frac{\ln(1/\delta')}{2n}}$, and $\Delta C := C_{S'}(\pi) - \gamma - \bar{C}_{\delta}(\pi, S)$. Then,*

$$\mathbb{P}_{(S, \pi, S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^m)} [C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi) \geq \Delta C] \geq 1 - \delta - \delta' . \quad (10.5)$$

Proof. A detailed proof of this theorem is provided in App. 10.7.3. $\square$

To use Theorem 10.3 for detecting if a test dataset S' is OOD, we first choose δ and δ' to obtain a desired confidence level $1 - \delta - \delta'$ for our OOD detector. Using these constants and the policy π drawn from P trained using the PAC-Bayes upper bound, we then compute ΔC . If $\Delta C > 0$, then with confidence at least $1 - \delta - \delta'$ we can claim that $C_{\mathcal{D}'} > C_{\mathcal{D}}$, and therefore, with the same level of confidence the test dataset S' must be OOD according to our task-relevant notion in (10.1). Furthermore, we obtain a guaranteed upper bound of $\delta + \delta'$ on the false positive rate of our OOD detector, i.e., the rate at which our detector will misclassify training distribution data as OOD.

10.5 Examples

We demonstrate the ability of our approach to perform task-driven OOD detection with guaranteed confidence bounds on two simulated examples: a manipulator grasping a new set of objects and a drone navigating a new set of environments. For the navigation task,

we compare our methods with popular OOD detection baselines. Our code is available at: https://github.com/irom-lab/Task_Relevant_OOD_Detection

10.5.1 Robotic grasping

Overview. We use the Franka Panda arm (Fig. 10.2(a)) for grasping objects in the PyBullet simulator [53] and build upon the open-source code provided in [166]. The robot employs a vision-based control policy that uses a depth map of the object obtained from an overhead camera and returns an open-loop action $a := (x, y, z, \theta)$ which corresponds to the desired grasp position and yaw orientation of the gripper. We train the manipulator to grasp mugs placed in $SE(2)$ poses drawn from a specific distribution. Then, we demonstrate the efficacy of our OOD detection framework by (i) gradually modifying the distribution on the mug poses and (ii) changing the objects from mugs to bowls.

Control policy. The control policy is a deep neural network (DNN) which inputs a 128×128 depth map of the object and a latent state $z \in \mathbb{R}^{10}$ sampled from a multivariate Gaussian distribution $\mathcal{N}_\psi$ with a diagonal covariance, and outputs an open-loop grasp action a ; see Fig. 10.4 in App. 10.7.6 for the policy. In [166], the distribution $\mathcal{N}_\psi$ on the latent space encodes prior domain/expert knowledge.

Training. Mugs from the ShapeNet dataset [46] are randomly scaled in all dimensions to generate a training dataset S of 500 mugs. If the robot is able to lift the mug by 10 cm and the gripper palm does not contact it, then we consider the rollout successful and assign a cost of 0; otherwise the cost is set to 1. In training, we optimize the distribution $\mathcal{N}_\psi$ on the latent space to minimize the PAC-Bayes upper bound provided in Theorem 10.1 using Alg. 10.2, while the weights of the CNN and MLP networks in Fig. 10.4 in App. 10.7.6 remain fixed. The prior $\mathcal{N}_{\psi_0}$ is chosen as the normal distribution with zero mean and identity covariance. A policy π is sampled from the trained posterior $\mathcal{N}_\psi$ and the PAC-Bayes bound for this policy is computed as $\overline{C}_\delta(\pi, S) = 0.1$ with $\delta = 0.01$.

OOD detection results. We perform OOD detection using the two methods presented in Theorem 10.2 and Theorem 10.3. For detection with p-value, we choose a significance level $\alpha = 95\%$, while, for detection using ΔC we choose a confidence level of 95%, i.e., $\delta + \delta' = 0.05$, which ensures that the false-positive rate of our detector is no greater than

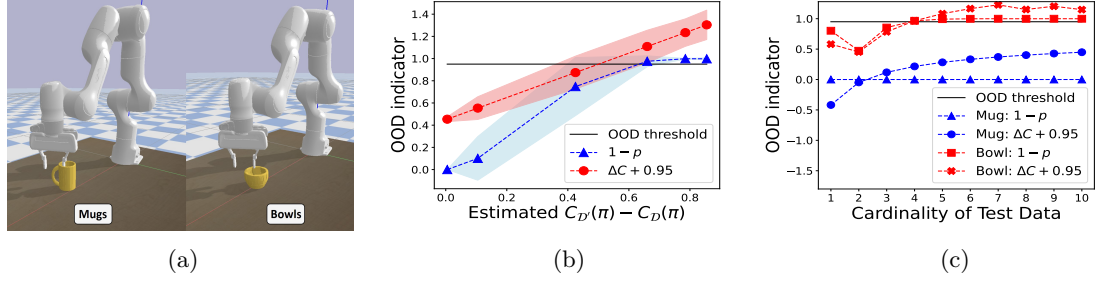


Figure 10.2: OOD detection for grasping. **(a)** Franka Panda arm in PyBullet grasping a mug (left) and a bowl (right). **(b)** Performance of our OOD detectors for different distributions on mug placement. Both our approaches perform similarly and the OOD indicators increase monotonically with $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$. **(c)** Comparison of our OOD detectors for grasping mugs and grasping bowls. Both our approaches detect OOD using a small number of test environments (just 4) for bowls and do not detect OOD for mugs (as expected).

5%. We perform two experiments to demonstrate the efficacy of our approach for OOD detection. First, we make the distribution on the mug’s initial placement progressively more challenging; see App. 10.7.6 for the exact distributions. For each distribution, we sample 20 test datasets of cardinality 10 and compute our OOD indicators: (i) the lower bound on $1 - p$ (where p is the p-value) and (ii) ΔC using Theorem 10.3. Fig. 10.2(b) plots the mean (dashed line) and a one standard deviation spread (shaded region) for the OOD indicators as a function of $C_{\mathcal{D}'} - C_{\mathcal{D}}$ (estimated via exhaustive sampling). Note that we plot $\Delta C + 0.95$ so that the OOD threshold is the same (0.95) for both methods. As the cost of the policy deteriorates on test distributions our OOD indicators reliably increase, capturing the shift of the test distributions away from the training distribution. In the second experiment, we change the objects that the manipulator must grasp from mugs to bowls. Fig. 10.2(c) shows that with a small test dataset S' of cardinality 5, both our approaches detect OOD when bowls are used (red curves). As expected, our OOD detectors are not triggered for mugs (blue curves), which are drawn from the training distribution.

10.5.2 Vision-based obstacle avoidance with a drone

Overview. In both the simulation and hardware portions of this example, we aim to avoid an obstacle field with the Parrot Swing drone; this is an agile quadrotor/fixed-wing hybrid drone shown in Fig. 10.1. We train a DNN control policy in a simulation setup based on

the hardware system shown in Fig. 10.1. The policy takes in a 50×50 depth image and outputs a softmax corresponding to a set of pre-computed motion primitives with the goal of avoiding obstacles by the largest distance. Since we designed the simulation portion of this example with application to hardware in mind, we have created motion primitives by capturing (with a Vicon motion tracking system) the trajectories of open-loop control inputs, which result in different maneuvers; the two images in Fig. 10.1 represent two of these trajectories. The use of motion primitives allows us to perform accurate sim-to-real transfer (since the motion primitives are recorded directly from the hardware system).

Training. Environments consist of a set of randomly placed cylindrical obstacles. We record the minimum distance $d_{\min}$ from the obstacles (as recorded by the robot’s 120° field of view depth sensor) and assign a cost of $\max(0, 1 - \frac{d_{\min}}{30\text{cm}})$. Using 10,000 training environments S , we train a prior to assign larger values to motion primitives which achieve a larger distance from obstacles. See App. 10.7.6 for further details on the training procedure for the prior. We then use another 10,000 environments to train the posterior distribution using Alg. 10.1. We sample a policy π from the trained posterior and compute the PAC-Bayes bound $\overline{C}_\delta(\pi, S) = 0.222$ for $\delta = 0.01$.

Simulation Results

We compare our task-driven OOD detection approach with two baselines: (i) maximum softmax probability (MSP) [92] (an effective and popular baseline for OOD detection), and (ii) MaxLogit [91] (a recent state-of-the-art OOD detection baseline). We note that these baselines are specifically designed for networks which output categorical distributions, and thus provide strong benchmarks. We determine the threshold for detection for the baselines by choosing a false positive rate of 5% for the training environments. However, note that the baselines do not provide any guarantees; they may violate the false positive rate even on new environments drawn from the training distribution (as we observe in our experiments). We select a p-value of 0.05 as the indication that an environment is OOD, and a guaranteed false-positive rate of $\delta + \delta' = 5\%$ for the confidence interval method. Each method receives a dataset S' of 10 environments. We use the average outputs of the baselines on the 10 environments for OOD detection. We generate OOD environments of varying difficulty by

changing the number of obstacles and the maximum or minimum gap-size between obstacles.

Varied environment difficulty. We sample 2000 datasets S' (of size 10) for each setting and plot the proportion of these detected as OOD for all methods in Fig. 10.3(a). At 0 on the x-axis of this plot, we draw datasets from the training distribution. All other environments are OOD. We note that the p-value and confidence interval-based methods perform similarly. The plot demonstrates our ability to perform task-driven OOD detection. In particular, we only detect the “harder” distributions (i.e., environments with higher increased cost) as OOD, while the baselines are triggered even for the easier distributions. In addition, neither baseline provides any guarantees and both violate the 5% false-positive rate on the training distribution. Our methods have a low false positive rate, albeit at the cost of conservatism for moderate distribution shifts.

Task-irrelevant shift. We also compare the baselines with our methods on a distribution where environments consist of 4 (uniformly) randomly located obstacles. In this setting, the control policy achieves a near-identical expected cost $C_{\mathcal{D}'}(\pi)$ (as estimated by exhaustive sampling of environments) to the expected training cost $C_{\mathcal{D}}(\pi)$ (in particular, $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi) = -0.02$). For this setting, MSP [92] classified 95.7% of test datasets as OOD and MaxLogit [91] classified 50.9% as OOD. Thus, the baselines are triggered by a task-irrelevant shift in the distribution. In contrast, our methods had an OOD detection rate of 0.1% in this setting.

Hardware Results

We use a Parrot Swing drone for the hardware experiments (Fig. 10.1). We simulate a depth sensor for the drone (as if the sensor was mounted on the drone) by generating a synthetic depth image using the positions of objects from the Vicon motion capture system. We do not provide any other information to the policy (e.g., obstacle positions). We generate each environment the same way as in simulation, and then place the real-world obstacles in the generated locations.

Varied environment difficulty and wind disturbances. We deploy the policy trained in simulation on three kinds of OOD environments in hardware: (i) environments with a smaller number of obstacles (“easier” environments), (ii) environments with smaller

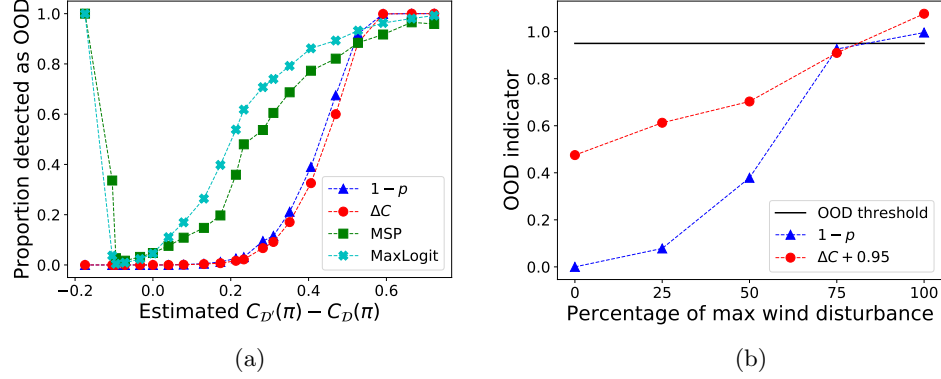


Figure 10.3: OOD Detection on settings of variable difficulty. **(a)** Comparison of the performance of our OOD detectors with baselines MSP [92] and MaxLogit [91]. Both our methods do not detect OOD when $C_{\mathcal{D}'}(\pi) < C_{\mathcal{D}}(\pi)$ whereas the baselines detect these task-irrelevant shifts in the environment. For $C_{\mathcal{D}'}(\pi) > C_{\mathcal{D}}(\pi)$ the baselines detect a higher proportion of datasets as OOD for smaller values of $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$ compared to our methods. **(b)** Comparison of our OOD detectors on the Parrot Swing hardware for increasing wind disturbance. Both OOD indicators increase monotonically with $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$ and are able to detect OOD at 100% wind.

gaps between obstacles (i.e., “harder” environments), and (iii) environments with wind generated using a fan (Fig. 10.1 right) with the same obstacle distribution as training. Videos from each setting are available at <https://youtu.be/VxKCAjaih8M>. For each setting, we run 10 trials on the hardware and use this for OOD detection. As expected, our OOD detectors are not triggered by the easier environments. For the harder environments, we compute $1-p \geq 0.81$ and $\Delta C = -0.11$. Results from the windy environments are shown in Fig. 10.3(b) for increasing values of wind (up to about 5 m/s). We note that the sim-to-real distribution shift (corresponding to the zero wind case) is not viewed as being OOD in a task-relevant manner by our approaches. Both our approaches assign increasingly high confidence as the wind speed is increased. We note that disturbances such as wind cannot be detected via the depth image given to the robot’s policy. Thus any OOD detection technique which relies solely on the output of the policy, such as MSP [92] and MaxLogit [91], would be unable to detect these environments as OOD.

10.6 Conclusion

We have presented a framework for performing task-driven OOD detection with statistical guarantees. Our approach uses PAC-Bayes theory to train a policy with a bound on the expected cost on the training distribution. We then perform OOD detection on test environments by checking for violations of the bound (using p-values and concentration inequalities). Our simulated and hardware experiments demonstrate the ability of our approach to perform OOD detection within a handful of trials. Comparisons with baselines also demonstrate two advantages: our OOD detectors (i) are only sensitive to task-relevant distribution shifts, and (ii) provide statistical guarantees on detection.

Challenges and future work. The approach we present here allows us to bound the false positive rate. An interesting theoretical question is whether we can also bound the false negative rate. It would also be of practical interest to extend our approach to settings where the robot encounters environments in an *online* manner (instead of the batch setting we consider here). Another particularly exciting direction is to develop versions of our approach that are more *proactive*; instead of having to incur costs on the test environments, one could potentially perform OOD detection based on *predicted* costs (thus avoiding the need to potentially fail on the test environments).

10.7 Appendix

10.7.1 Proof of Theorem 10.1

We begin with an introduction to the PAC-Bayes framework, and then provide a complete proof of Theorem 10.1. PAC-Bayes provides an upper bound on the expected cost of deploying a policy distribution P on environments E drawn from an unknown distribution $\mathcal{D}$, i.e., $\mathbb{E}_{E \sim \mathcal{D}} \mathbb{E}_{\pi \sim P} C_E(\pi)$. This upper bound only depends on the cost of deploying P in a finite set of training environments $S \sim \mathcal{D}^m$, i.e., the training cost $\mathbb{E}_{\pi \sim P} C_{E_i}(\pi)$, and a regularizer which depends on the KL-divergence between P and a prior P_0 that is chosen before observing S ; note that P_0 need not be a Bayesian prior. The following is the PAC-Bayes bound that was presented in [142] and tightened in [139]:

Theorem 10.4 (PAC-Bayes Bound [142]). *For any distribution over environments $\mathcal{D}$, data-independent prior distribution P_0 , cost C bounded in $[0, 1]$, $m \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of $S \sim \mathcal{D}^m$, the following holds for all posterior distributions P :*

$$\mathbb{E}_{E \sim \mathcal{D}} \mathbb{E}_{\pi \sim P} C_E(\pi) \leq \sum_{i=1}^n \mathbb{E}_{\pi \sim P} C_{E_i}(\pi) + \sqrt{\frac{D_{\text{KL}}(P \| P_0) + \ln \frac{2\sqrt{m}}{\delta}}{2m}} \quad (10.6)$$

where D_{KL} is the KL-divergence.

The above theorem and the forthcoming PAC-Bayes theorems in this section are presented for policy learning instead of supervised learning using the reduction provided in [136]. Note that this bound provides a guarantee for a *distribution* over policies rather than a specific policy. This allows for a regularizer dependent on the KL-divergence between the prior and posterior distributions rather than one which is a direct expression of the complexity of the policy space (such as the VC-dimension). However, this creates a challenge for calculating the upper bound, which requires computing an expectation over $\pi \sim P$, or using potentially-loosening sample convergence bounds. Thus, we make use of the recent work which provides a framework for derandomized PAC-Bayes bounds (i.e. bounds which hold for a sampling of policy π rather than an expectation over $\pi \sim P$) [209]. The following is a general theorem for formulating the derandomized PAC-Bayes bounds:

Theorem 10.5 (Pointwise PAC-Bayes Bound [209]). *For any positive function ϕ , distribution $\mathcal{D}$, prior distribution P_0 , and $\delta \in (0, 1)$, with probability $1 - \delta$ over a sampling of $S \sim \mathcal{D}^m$ and $\pi \sim P$, the following holds for any posterior distribution P :*

$$\frac{\alpha}{\alpha - 1} \ln(\phi(\pi, S)) \leq D_\alpha(P \| P_0) + \ln \left(\frac{1}{(\delta/2)^{\frac{\alpha}{\alpha-1}+1}} \mathbb{E}_{S' \sim \mathcal{D}^m} \mathbb{E}_{\pi' \sim P_0} \phi(\pi', S')^{\frac{\alpha}{\alpha-1}} \right) \quad (10.7)$$

where P is the output of algorithm A on the training data S , i.e. $P := A(P_0, S)$ and D_α is the Rényi divergence.

Now we can proceed with the statement and proof.

Theorem 1. *For any distribution $\mathcal{D}$, prior distribution P_0 , $\delta \in (0, 1)$, cost bounded in*

$[0, 1]$, and deterministic algorithm A which outputs the posterior distribution P we have the following:

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)} \left[C_{\mathcal{D}}(\pi) \leq C_S(\pi) + \sqrt{\frac{D_2(P \| P_0) + \ln \frac{2\sqrt{m}}{(\delta/2)^3}}{2m}} \right] \leq 1 - \delta \quad (10.8)$$

where D_2 is the Rényi Divergence for $\alpha = 2$.

Proof. We begin with the statement in Theorem 10.5, which is proved in [209]. Let $\alpha = 2$ and $\phi(\pi, S) = \exp[\frac{\alpha-1}{\alpha} m D_{\text{KL}}(C_S(\pi) \| C_{\mathcal{D}}(\pi))]$. Thus, we have the following with at least probability $1 - \delta$ over the random choice $S \sim \mathcal{D}^m$ and $\pi \sim P$:

$$D_{\text{KL}}(C_S(\pi) \| C_{\mathcal{D}}(\pi)) \leq \frac{1}{m} \left[D_2(P \| P_0) + \ln \left(\frac{1}{(\delta/2)^3} \mathbb{E}_{S' \sim \mathcal{D}^m} \mathbb{E}_{\pi' \sim P_0} e^{m D_{\text{KL}}(C_{S'}(\pi') \| C_{\mathcal{D}}(\pi'))} \right) \right] \quad (10.9)$$

From [139], we can upper bound $\mathbb{E}_{S' \sim \mathcal{D}^m} \mathbb{E}_{\pi' \sim P_0} e^{m D_{\text{KL}}(C_{S'}(\pi') \| C_{\mathcal{D}}(\pi'))}$ by $2\sqrt{m}$ when $m \geq 8$.

This gives us the following bound

$$D_{\text{KL}}(C_S(\pi) \| C_{\mathcal{D}}(\pi)) \leq \frac{1}{m} \left[D_2(P \| P_0) + \ln \frac{2\sqrt{m}}{(\delta/2)^3} \right]. \quad (10.10)$$

We then apply the Pinsker's inequality, i.e. $D_{\text{KL}}(p \| q) \leq c \implies q \leq p + \sqrt{c/2}$, which results in Inequality (10.2). Note that we could also use a quadratic version of the upper bound for the KL divergence between two distributions and produce an upper bound analogous to the one presented in [172]. $\square$

10.7.2 Proof of Theorem 10.2

For the readers' convenience, we restate Theorem 10.2 here and provide a detailed proof.

Theorem 2. *Let $\mathcal{D}$ be the training distribution and P be the posterior distribution on the space of policies obtained through the training procedure described in Sec. 10.4.1. Let $S' \sim \mathcal{D}^n$ be a test dataset, $p(S')$ be the p-value for S' defined in Definition 10.1, and*

$\delta \in (0, 1)$. Then,

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2)] \geq 1 - \delta, \quad (10.11)$$

where $\bar{\tau}(S) := \max\{C_{S'}(\pi) - \bar{C}_\delta(\pi, S), 0\}$.

Proof. We prove this theorem by considering two cases: when the PAC-Bayes cost inequality holds, i.e., $C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S)$, and when it does not, i.e., $C_{\mathcal{D}}(\pi) > \bar{C}_\delta(\pi, S)$; the two cases are considered in (10.12)-(10.14). In the latter case, we cannot say anything about the p-value, while in the former case, which holds with probability at least $1 - \delta$, we show in (10.15)-(10.26) that $p(S') \leq \exp(-2n\bar{\tau}(S)^2)$.

Let us begin the proof by conditioning $\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2)]$ as follows:

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2)] \quad (10.12)$$

$$\begin{aligned} &= \mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S)] \underbrace{\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S)]}_{\geq 1 - \delta \text{ (from Theorem 10.1)}} \\ &+ \underbrace{\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) > \bar{C}_\delta(\pi, S)]}_{\geq 0} \underbrace{\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) > \bar{C}_\delta(\pi, S)]}_{\geq 0} \end{aligned} \quad (10.13)$$

$$\geq \mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S)](1 - \delta) \quad (10.14)$$

Now, we claim:

$$\mathbb{P}_{(S, \pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S)] = 1, \quad (10.15)$$

which on using in (10.14) completes the proof of this theorem. The remainder of this proof is dedicated to establishing the claim in (10.15).

We are given

$$C_{\mathcal{D}}(\pi) \leq \bar{C}_\delta(\pi, S). \quad (10.16)$$

From Definition 10.1, we have

$$p(S') = \mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) \geq C_{S'}(\pi) \mid C_{\mathcal{D}'}(\pi) \leq C_{\mathcal{D}}(\pi)] \quad (10.17)$$

$$= \mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) - \overline{C}_{\delta}(\pi, S) \geq C_{S'}(\pi) - \overline{C}_{\delta}(\pi, S) \mid C_{\mathcal{D}'}(\pi) \leq C_{\mathcal{D}}(\pi)] \quad (10.18)$$

$$= \mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) - \overline{C}_{\delta}(\pi, S) \geq \bar{\tau} \mid C_{\mathcal{D}'}(\pi) \leq C_{\mathcal{D}}(\pi)] . \quad (10.19)$$

From (10.16) and the assumption that the null hypothesis holds in (10.19), it follows that $C_{\mathcal{D}'}(\pi) \leq \overline{C}_{\delta}(\pi, S)$, which ensures that the following implication holds for $\bar{\tau}$ defined in the statement of the theorem:

$$C_{\hat{S}}(\pi) - \overline{C}_{\delta}(\pi, S) \geq \bar{\tau} \implies C_{\hat{S}}(\pi) - C_{\mathcal{D}'}(\pi) \geq \bar{\tau} . \quad (10.20)$$

Therefore, if $\bar{\tau} > 0$ we have that

$$\mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) - \overline{C}_{\delta}(\pi, S) \geq \bar{\tau} \mid C_{\mathcal{D}'}(\pi) \leq C_{\mathcal{D}}(\pi)] \leq \mathbb{P}_{\hat{S} \sim \mathcal{D}'^n} [C_{\hat{S}}(\pi) - C_{\mathcal{D}'}(\pi) \geq \bar{\tau}] \leq \exp(-2n\bar{\tau}^2) ,$$

where the last upper bound follows from Hoeffding's inequality. Hence, for $\bar{\tau} > 0$, using the above in (10.19) gives

$$p(S') \leq \exp(-2n\bar{\tau}^2) . \quad (10.21)$$

If $\bar{\tau} = 0$, the vacuous bound holds:

$$p(S') \leq 1 = \exp(-2n0) = \exp(-2n\bar{\tau}^2) . \quad (10.22)$$

Combining the two cases for $\bar{\tau} > 0$ in (10.21) and $\bar{\tau} = 0$ in (10.22) gives us the following implication:

$$C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S) \implies p(S') \leq \exp(-2n\bar{\tau}(S)^2) . \quad (10.23)$$

Now, we expand the left-hand side of (10.15) using the definition of conditional probability:

$$\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)] \quad (10.24)$$

$$= \frac{\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \wedge C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)]}{\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)]} \quad (10.25)$$

From (10.23), we know that

$$\{(S, \pi) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)\} \subseteq \{(S, \pi) \mid p(S') \leq \exp(-2n\bar{\tau}(S)^2)\},$$

therefore,

$$\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \wedge C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)] = \mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)]$$

which on using in (10.25) gives the following:

$$\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[p(S') \leq \exp(-2n\bar{\tau}(S)^2) \mid C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)] = \frac{\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)]}{\mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)}[C_{\mathcal{D}}(\pi) \leq \bar{C}_{\delta}(\pi, S)]} = 1, \quad (10.26)$$

completing the proof of the claim (10.15) as well as the theorem. $\square$

10.7.3 Proof of Theorem 10.3

For the readers' convenience, we restate Theorem 10.3 here and provide a detailed proof.

Theorem 3. *Let $\mathcal{D}$ be the training distribution, $\mathcal{D}'$ be the test distribution, and P be the posterior distribution on the space of policies obtained through the training procedure described in Sec. 10.4.1. Let $\delta, \delta' \in (0, 1)$ such that $\delta + \delta' < 1$, $\gamma := \sqrt{\frac{\ln(1/\delta')}{2n}}$, and $\Delta C := C_{S'}(\pi) - \gamma - \bar{C}_{\delta}(\pi, S)$. Then,*

$$\mathbb{P}_{(S,\pi,S') \sim (\mathcal{D}^m \times P \times \mathcal{D}^n)}[C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi) \geq \Delta C] \geq 1 - \delta - \delta'. \quad (10.27)$$

Proof. To lower bound the difference between $C_{\mathcal{D}'}(\pi)$ and $C_{\mathcal{D}}(\pi)$ with high probability we obtain a lower bound on $C_{\mathcal{D}'}(\pi)$ which holds with probability at least $1 - \delta'$ using Hoeffding's inequality in (10.28)-(10.31). Then we use this bound with the PAC-Bayes bound (10.2) which holds with probability at least $1 - \delta$ to obtain (10.27) by following the steps in (10.32)-(10.37).

Let γ be defined as in the statement of the theorem, then, using the independence of $\mathcal{D}'^n$ from $\mathcal{D}^m \times P$, we can write²

$$\begin{aligned} & \mathbb{P}_{(S, \pi, S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] \\ &= \int_{(S, \pi)} \mathbb{P}_{S' \sim \mathcal{D}'^n} [C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma \mid S, \pi] d(\mathcal{D}^m \times P)(S, \pi) . \end{aligned} \quad (10.28)$$

For *any* given (S, π) , we can apply Hoeffding's inequality to get:

$$\mathbb{P}_{S' \sim \mathcal{D}'^n} [C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] = \mathbb{P}_{S' \sim \mathcal{D}'^n} [C_{S'}(\pi) - C_{\mathcal{D}'}(\pi) \leq \gamma] \geq 1 - \exp(-2n\gamma^2) = 1 - \delta' . \quad (10.29)$$

Using (10.29) in (10.28) we get that:

$$\mathbb{P}_{(S, \pi, S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] \geq \int_{(S, \pi)} (1 - \delta') d(\mathcal{D}^m \times P)(S, \pi) \quad (10.30)$$

$$= (1 - \delta') \int_{(S, \pi)} d(\mathcal{D}^m \times P)(S, \pi) = 1 - \delta' . \quad (10.31)$$

Now, observe that

$$C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S) \wedge C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma \implies C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi) \geq C_{S'}(\pi) - \gamma - \overline{C}_{\delta}(\pi, S) . \quad (10.32)$$

²Note that $C_{\mathcal{D}'}(\pi)$ and $C_{S'}(\pi)$ implicitly depend on S because the posterior distribution P , from which π is sampled, is trained on S .

From the implication (10.32), it follows that

$$\mathbb{P}_{(S,\pi,S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi) \geq C_{S'}(\pi) - \gamma - \overline{C}_{\delta}(\pi, S)] \quad (10.33)$$

$$\geq \mathbb{P}_{(S,\pi,S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S) \wedge C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] \quad (10.34)$$

Now using the Fréchet inequality $\mathbb{P}[E_1 \wedge E_2] \geq \mathbb{P}[E_1] + \mathbb{P}[E_2] - 1$ (where E_1 and E_2 are arbitrary random events) on (10.34) we obtain:

$$\mathbb{P}_{(S,\pi,S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S) \wedge C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] \quad (10.35)$$

$$\geq \mathbb{P}_{(S,\pi) \sim (\mathcal{D}^m \times P)} [C_{\mathcal{D}}(\pi) \leq \overline{C}_{\delta}(\pi, S)] + \mathbb{P}_{(S,\pi,S') \sim (\mathcal{D}^m \times P \times \mathcal{D}'^n)} [C_{\mathcal{D}'}(\pi) \geq C_{S'}(\pi) - \gamma] - 1 \quad (10.36)$$

$$\geq 1 - \delta - \delta' , \quad (10.37)$$

where the last inequality follows by using (10.2) and (10.31) in (10.36). Finally, using (10.37) in (10.34) completes the proof. $\square$

10.7.4 Training with Backpropogation

In this section, we describe a method to minimize the upper bound in Theorem 10.1 using backpropogation. We make use of multivariate Gaussian distributions $\mathcal{N}_{\psi}$ with diagonal covariance $\Sigma_s := \text{diag}(s)$ where $\psi := (\mu, \log s)$. When training the posterior distribution P , we would like to take gradient steps directly with respect to ψ . However, this would require backpropogation through $\mathbb{E}_{w \sim \mathcal{N}_{\psi}} C_E(\pi_w)$. We follow a similar procedure as in [63] and achieve the desired result of minimizing the upper bound in Inequality (10.2) using an unbiased estimate of $\mathbb{E}_{w \sim \mathcal{N}_{\psi}} C_E(\pi_w)$:

$$\frac{1}{k} \sum_{i=1}^k C_E(\pi_{w_i}), \quad w_i \sim \mathcal{N}_{\psi} \ \forall \ i \in \{1, 2, \dots, k\} . \quad (10.38)$$

Algorithm 10.1 PAC-Bayes Bound Minimization via Backpropagation

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies, fixed seed for random number generation
Input: Training dataset S , learning rate γ
Output: Optimized ψ^*
while not converged **do**
 Sample $w_i \sim \mathcal{N}_{\psi} \forall i \in \{1, 2, \dots, k\}$
 $B \leftarrow \frac{1}{mk} \sum_{E \in S} \sum_{i=1}^k C_E(\pi_{w_i}) + \sqrt{R}$
 $\psi \leftarrow \psi - \gamma \nabla_{\psi} B$
end while

The resulting approach is presented in Algorithm 10.1. Note that the algorithm must be deterministic in order to maintain the assumptions of Theorem 10.1. We achieve this by training with a fixed seed for generating random numbers. Additionally, note that the backpropagation requires a gradient taken through $D_2(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0})$. We make use of the analytical form for the Rényi divergence between two multivariate Gaussian distributions, presented in [79], in order to tractably compute the gradients.

$$D_2(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0}) = D_2(\mathcal{N}(\mu, \Sigma_s) \parallel \mathcal{N}(\mu_0, \Sigma_{s_0})) = (\mu - \mu_0)^T \Sigma_2 (\mu - \mu_0) - \frac{1}{2} \ln \frac{|\Sigma_2| |\Sigma_s|}{|\Sigma_{s_0}|^2}, \quad (10.39)$$

where $\Sigma_2 = 2\Sigma_{s_0} - \Sigma_s$. We also note that there is a restriction on how far the posterior's variance can drift from the prior. The following expression must be satisfied for $D_2(\mathcal{N}_{\psi} \parallel \mathcal{N}_{\psi_0})$ to be finite [79]:

$$2\Sigma_s^{-1} - \Sigma_{s_0}^{-1} \succ 0. \quad (10.40)$$

In practice, we project any problematic variances into the range of allowable variances.

After training, since we have used the pointwise PAC-Bayes bound in Theorem 10.1, we compute the upper bound with a single $w \sim \mathcal{N}_{\psi}$ in contrast to traditional PAC-Bayes bounds. Thus, the resulting policy π_w is deterministic and applicable in a broad range of settings, including ones which require a pre-trained network. The resulting policy carries a PAC-Bayes guarantee.

10.7.5 Training with Evolutionary Strategies

To train robot control policies in settings where backpropagation is not feasible (e.g. presence of a “blackbox” in the form of a simulator or robot hardware in the forward pass), we use Evolutionary Strategies (ES) which is a class of blackbox optimizers [220]. ES addresses this challenge by estimating the gradient via a Monte-Carlo estimator:

$$\nabla_{\psi} C_S(\mathcal{N}_{\psi}) := \frac{1}{m} \sum_{E \in S} \nabla_{\psi} \mathbb{E}_{w \sim \mathcal{N}_{\psi}} [C_E(\pi_w)] = \frac{1}{m} \sum_{E \in S} \mathbb{E}_{w \sim \mathcal{N}_{\psi}} [C_E(\pi_w) \nabla_{\psi} \ln \mathcal{N}_{\psi}(w)] . \quad (10.41)$$

Although we can compute the gradient of the regularizer analytically (as mentioned in App. 10.7.4), using different methods to estimate the gradient of the empirical cost (ES) and the gradient of the regularizer (analytically) results in poor convergence. To alleviate this, we estimate the regularizer’s gradient using ES as well by leveraging the expectation form of Rényi divergence in Theorem 10.1. This takes the following form:

$$\begin{aligned} & \nabla_{\psi} (C_S(\mathcal{N}_{\psi}) + \sqrt{R}) \\ &= \frac{1}{m} \sum_{E \in S} \mathbb{E}_{w \sim \mathcal{N}_{\psi}} \left[\underbrace{\left(C_E(\pi_w) + \frac{e^{\ln \left(\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right) - D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0})}}{4m\sqrt{R}}} \right)}_{\tilde{C}_E(w)} \nabla_{\psi} \ln \mathcal{N}_{\psi}(w) \right] . \end{aligned} \quad (10.42)$$

Derivation of (10.42)

To derive (10.42), note that

$$\nabla_{\psi} (C_S(\mathcal{N}_{\psi}) + \sqrt{R}) = \nabla_{\psi} C_S(\mathcal{N}_{\psi}) + \nabla_{\psi} \sqrt{R} = \nabla_{\psi} C_S(\mathcal{N}_{\psi}) + \frac{1}{2\sqrt{R}} \nabla_{\psi} R . \quad (10.43)$$

From (10.41) we know the gradient for $\nabla_{\psi} C_S(\mathcal{N}_{\psi})$. In the rest of this derivation, therefore, we will focus on the computing the gradient of the second term.

Note that the Rényi divergence for multivariate Gaussian distributions can be written as:

$$D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0}) = \ln \left(\mathbb{E}_{w \sim \mathcal{N}_{\psi_0}} \left[\left(\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right)^2 \right] \right) . \quad (10.44)$$

Let

$$\eta := \mathbb{E}_{w \sim \mathcal{N}_{\psi_0}} \left[\left(\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right)^2 \right] , \quad (10.45)$$

then, using (10.44) we have that

$$D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0}) = \ln \eta \quad (10.46)$$

which allows us to express R as

$$R = \frac{\ln \eta + \ln(2\sqrt{m}/(\delta/2)^3)}{2m} . \quad (10.47)$$

Hence,

$$\nabla_{\psi} R = \frac{1}{2m\eta} \nabla_{\psi} \eta = \frac{e^{-D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0})}}{2m} \nabla_{\psi} \eta , \quad (10.48)$$

where the last equality follows from (10.46).

For computing the gradient using ES, we require the cost to be an expectation over the posterior, however, η is an expectation on the prior. To address this we perform a change of measure which gives us the following:

$$\eta = \mathbb{E}_{w \sim \mathcal{N}_{\psi_0}} \left[\left(\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right)^2 \right] = \mathbb{E}_{w \sim \mathcal{N}_{\psi}} \left[\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right] . \quad (10.49)$$

Using (10.49) in (10.48) gives us

$$\nabla_{\psi} R = \frac{e^{-D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0})}}{2m} \nabla_{\psi} \mathbb{E}_{w \sim \mathcal{N}_{\psi}} \left[\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \right] = \frac{e^{-D_2(\mathcal{N}_{\psi} || \mathcal{N}_{\psi_0})}}{2m} \mathbb{E}_{w \sim \mathcal{N}_{\psi}} \left[\frac{\mathcal{N}_{\psi}(w)}{\mathcal{N}_{\psi_0}(w)} \nabla_{\psi} \ln \mathcal{N}_{\psi}(w) \right] . \quad (10.50)$$

Using (10.50) and (10.41) in (10.43) and combining the expectation terms gives

$$\nabla_\psi(C_S(\mathcal{N}_\psi) + \sqrt{R}) = \quad (10.51)$$

$$\frac{1}{m} \sum_{E \in S} \mathbb{E}_{w \sim \mathcal{N}_\psi} \left[\left(C_E(\pi_w) + \frac{e^{-D_2(\mathcal{N}_\psi || \mathcal{N}_{\psi_0})}}{4m\sqrt{R}} \frac{\mathcal{N}_\psi(w)}{\mathcal{N}_{\psi_0}(w)} \right) \nabla_\psi \ln \mathcal{N}_\psi(w) \right]. \quad (10.52)$$

Finally, we note that the dimensionality d of w can be large, in which case the term $\mathcal{N}_\psi(w)/\mathcal{N}_{\psi_0}(w)$ is numerically unstable because it involves the product of d terms. Hence, we express $\mathcal{N}_\psi(w)/\mathcal{N}_{\psi_0}(w)$ as $e^{\ln(\mathcal{N}_\psi(w)/\mathcal{N}_{\psi_0}(w))}$ which gives us (10.42) as the final form of the gradient.

Training algorithm

The gradient of the PAC-Bayes upper bound is estimated from (10.42). Since Theorem 10.1 requires the training algorithm to be deterministic, we train with a fixed seed. The psuedo-code for our training is provided in Algorithm 10.2. After training, a single w is drawn from $\mathcal{N}_{\psi^*}$, which corresponds to a policy π_w , and the derandomized PAC-Bayes bound is computed for this policy.

Algorithm 10.2 PAC-Bayes Bound Minimization via ES

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies, fixed seed for random number generation
Input: Training dataset S , learning rate γ
Output: Optimized ψ^*
while not converged **do**
 Sample $w_i \sim \mathcal{N}_\psi \forall i \in \{1, 2, \dots, k\}$
 grad $\leftarrow \frac{1}{mk} \sum_{E \in S} \sum_{i=1}^k \tilde{C}_E(w_i)$
 $\psi \leftarrow \psi - \gamma \cdot \mathbf{grad}$
end while

10.7.6 Additional Experimental Details and Results

Robotic grasping

Training platform. Training was performed on a Lambda Blade server with 2x Intel Xeon Gold 5220R (96 CPU threads) and 768 GB RAM.

Distributions on the initial position of the mugs. For all datasets, mugs are

placed upright on the table with random yaw orientations sampled from the uniform distribution $\mathcal{U}([-\pi \text{ rad}, \pi \text{ rad}])$. The following distributions on the mug’s placement were used to generate the plot in Fig. 10.2(b):

1. $\mathcal{U}([0.45 \text{ m}, 0.55 \text{ m}] \times [-0.05 \text{ m}, 0.05 \text{ m}])$ (training distribution)
2. $\mathcal{U}([0.40 \text{ m}, 0.60 \text{ m}] \times [-0.10 \text{ m}, 0.10 \text{ m}])$
3. $\mathcal{U}([0.35 \text{ m}, 0.65 \text{ m}] \times [-0.15 \text{ m}, 0.15 \text{ m}])$
4. $\mathcal{U}([0.30 \text{ m}, 0.70 \text{ m}] \times [-0.20 \text{ m}, 0.20 \text{ m}])$
5. $\mathcal{U}([0.25 \text{ m}, 0.75 \text{ m}] \times [-0.25 \text{ m}, 0.25 \text{ m}])$
6. $\mathcal{U}([0.20 \text{ m}, 0.80 \text{ m}] \times [-0.30 \text{ m}, 0.30 \text{ m}])$

Control policy architecture. The manipulator’s control policy architecture is provided in Fig. 10.4. The weights of the DNN are borrowed from [166] and the training is

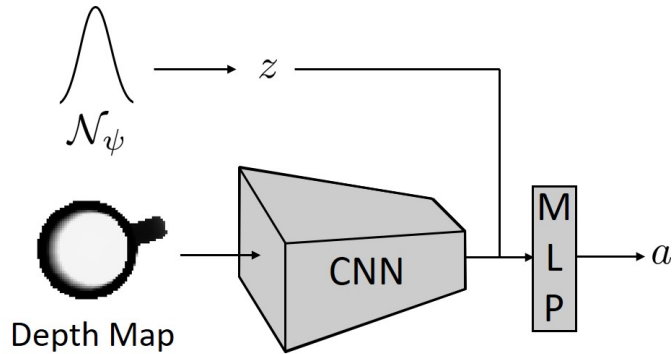


Figure 10.4: Network architecture for the manipulator’s grasping control policy.

warm-started with the posterior provided in [166, App. A5.1].

Vision-based obstacle avoidance with a drone

The approximate $C_{\mathcal{D}}(\pi)$ (estimated with 50,000 held-out environments) is 0.149; PAC-Bayes thus provides a strong bound.

Environment generation. Training environments have 9 obstacles and have at least one gap which is wide enough to navigate through. We generate environments by ran-

domly placing a set of cylindrical obstacles whose locations are sampled from the uniform distribution $\mathcal{U}([4.5 \text{ m}, 7 \text{ m}] \times [-3.5 \text{ m}, 3.5 \text{ m}])$ relative to the drone’s starting point.

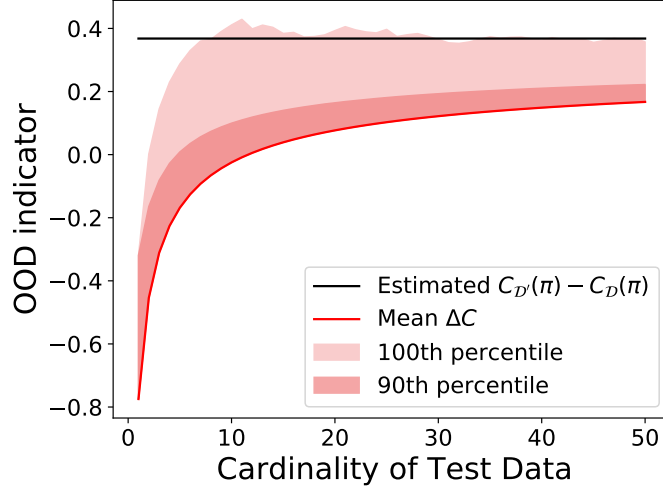


Figure 10.5: Numerical validation of lower bound in Theorem 10.3.

Training the prior. Training takes place completely in simulation. To allow for accurate sim-to-real transfer, the motion primitives are recorded trajectories of open-loop control inputs for the Parrot Swing hardware platform. We record multiple rollouts of each open-loop control policy. In simulation, when the policy selects a motion primitive, we randomly select one of the corresponding recorded trajectories to run. We train the prior $\mathcal{N}_{\psi_0}$ over policies by transforming the problem into a supervised learning setting. For each of 10,000 training environment in S the policy receives a depth map. Leveraging the simulation, we simulate each primitive (sampled uniformly from the set of recorded trajectories for that primitive) through each environment. We generate a label for each depth map by recording the minimum distance to an obstacle achieved by each of the primitives and passing the vector of distances through a softmax transformation. Note that even in simulation, we do not assume knowledge of the exact location of obstacles and record the closest distance as viewed by the robot’s 120° field of view depth sensor. These depth maps and softmax labels can then be used for training the prior over policies in a supervised learning setting. We use the cross-entropy loss to train . The result is a policy trained to assign larger values to motion primitives which achieve a larger distance from

obstacles.

Training platform. Training was performed on a desktop computer with an Intel i7-8700k CPU (12 CPU threads) and an NVIDIA Titan Xp GPU with 32 GB RAM.

Numerical validation of Theorem 10.3. We numerically validate our confidence bound in Fig. 10.5. We plot (i) the difference $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$ (estimated via exhaustive sampling of environments), (ii) the maximum computed lower-bound on $C_{\mathcal{D}'}(\pi) - C_{\mathcal{D}}(\pi)$ (computed using a confidence level of 0.9) over 500,000 datasets S' , and (iii) the 90th percentile value of the bound over the 100,000 datasets. As guaranteed by Theorem 10.3, the bound is valid greater than 90% of the time.

Chapter 11

Failure Prediction with Statistical Guarantees for Vision-Based Robot Control

ALEC FARID,* DAVID SNYDER,* ALLEN Z. REN, AND ANIRUDHA MAJUMDAR

We are motivated by the problem of performing *failure prediction* for safety-critical robotic systems with high-dimensional sensor observations (e.g., vision). Given access to a black-box control policy (e.g., in the form of a neural network) and a dataset of training environments, we present an approach for synthesizing a failure predictor with *guaranteed bounds* on false-positive and false-negative errors. In order to achieve this, we utilize techniques from *Probably Approximately Correct (PAC)-Bayes* generalization theory. In addition, we present novel *class-conditional* bounds that allow us to trade-off the relative rates of false-positive vs. false-negative errors. We propose algorithms that train failure predictors (that take as input the history of sensor observations) by minimizing our theoretical error bounds. We demonstrate the resulting approach using extensive simulation and hardware experiments for vision-based navigation with a drone and grasping objects with a robotic manipulator equipped with a wrist-mounted RGB-D camera. These experiments illustrate the ability of our approach to (1) provide strong bounds on failure prediction error rates (that closely match empirical error rates), and (2) improve safety by predicting

failures.

11.1 Introduction

How can we guarantee the safety of a control policy for a robot that operates using high-dimensional sensor observations (e.g., a vision-based navigation policy for a drone; Fig. 11.1)? This is particularly challenging for policies that have learning-based components such as neural networks as part of the perception and control pipeline; state-of-the-art approaches for synthesizing such policies (e.g., based on deep reinforcement learning) do not provide guarantees on safety, and can lead to policies that fail catastrophically in novel environments.

Motivated by this challenge, we consider the following problem in this paper. Given access to a black-box control policy (e.g., one with neural network components), our goal is to train a *failure predictor* for this policy. This failure predictor acts as a ‘safety layer’ that is responsible for predicting (online) if the given policy will lead to a failure as the robot operates in a novel (i.e., previously unseen) environment. Such a predictor could enable the robot to deploy a backup policy (e.g., transitioning to hover) in order to ensure safety. In certain settings (e.g., factories with robotic manipulators), the triggering of a failure predictor could also allow the robot to seek help from a human supervisor. We envision that the addition of a failure predictor could substantially improve the safety of the overall robotic system.

In order to be deployed in safety-critical settings, a failure predictor should ideally have associated *formal guarantees* on its prediction performance. More precisely, we would like to upper bound the *false negative* and *false positive* rates of the predictor. Here, a false negative corresponds to a case where the policy leads to a failure, but the failure predictor does not predict this in advance. Similarly, a false positive is a case where the failure predictor triggers, but the policy remains safe. In addition to bounding these error rates, one would also ideally like to *trade-off* the relative proportion of false negatives vs. false positives. For example, ensuring a low false negative rate is generally more crucial in safety-critical settings (potentially at the cost of a higher false positive rate).

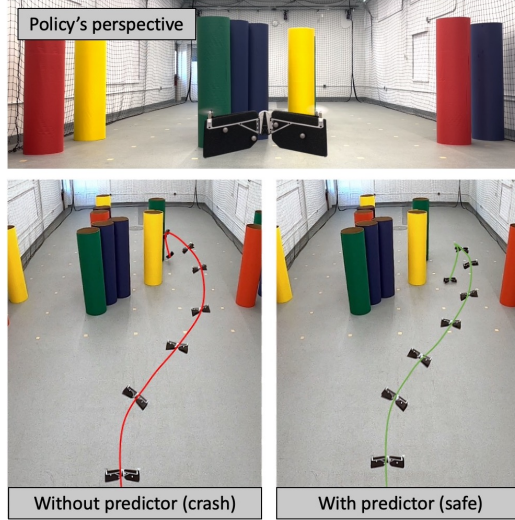


Figure 11.1: We train a failure predictor which guarantees (with high probability) detection of a failure ahead of time. A policy is tasked with avoiding obstacles in a novel environment using a first-person depth image (top). The bottom images show the entire environment, including multiple obstacles which are occluded from view. Since it is operating in a novel setting, the policy alone fails to avoid occluded obstacles and crashes (left). When the failure predictor is deployed, the policy stops safely before a crash (right).

Statement of contributions. Our primary contribution is to develop an approach for learning a failure predictor with guaranteed bounds on error rates, given a black-box control policy that operates on a robotic system with high-dimensional sensor observations (e.g., vision or depth). Assuming access to a dataset of environments on which we can execute the black-box policy for training purposes, we present a *reduction* of the failure prediction problem (which involves non-i.i.d. data in the form of sensor observations) to a standard supervised learning setting based on the reduction presented in [135]. This allows us to utilize techniques from *Probably Approximately Correct (PAC)-Bayes* generalization theory in order to obtain bounds on error rates. In addition, we develop novel *class-conditional* bounds that allow us to trade-off the relative rates of false negative vs. false positive errors. Our algorithmic approach is then to train a failure predictor (e.g., in the form of a neural network that takes as input a history of sensor observations) by minimizing the theoretical error bounds. We demonstrate our approach on two examples in simulation and hardware including vision-based navigation with a drone and grasping objects with a robotic manipulator equipped with a wrist-mounted RGB-D camera. These experiments illustrate the ability of our approach to (1) provide strong bounds on failure prediction error rates,

and (2) improve the safety of a robotic system by applying our failure prediction scheme.

11.2 Related Work

Anomaly Detection. There is a long line of work on *anomaly detection* in the signal processing literature (see [24] for a review). The goal is to identify events or inputs (e.g., sensor observations) that deviate significantly from nominal inputs (and may thus cause failures). In recent years, there have been efforts to develop anomaly detection techniques for supervised learning problems with high-dimensional inputs such as images [92, 127, 59, 121, 28, 49] (see [175] for a recent review). This line of work is also closely related to the literature on uncertainty estimation in deep learning [2]. Recent techniques also allow for anomaly detection in reinforcement learning settings using streams of sensor inputs [185, 40, 222, 83]. Within the robotics literature, there has been work on performing uncertainty estimation [108, 132] or anomaly detection [167] specifically for the problem of collision avoidance (which is an example we consider in this paper). In contrast to our goal of failure prediction, the methods highlighted above are generally aimed at detecting anomalies/outliers in environments that the robot is operating in. While anomalies may result in failures, this is not necessarily the case (e.g., for an extremely unlikely yet benign environment). In addition, the methods above do not generally provide guarantees on false positive or negative rates of detection, or the improvement in the robot’s performance that results from applying the anomaly detector.

Failure Prediction. A different line of work foresees failure by explicitly forward-propagating the robot dynamics in the environment. For instance, if the environment map is available (either given or estimated online), and the robot dynamics are known (e.g., car), the robot can check if it falls into a failure state in the next step before executing the action. Typical methods include Hamilton-Jacobi reachability analysis [146, 9], control barrier functions [15], and formal methods [11]. However, these approaches typically assume an explicit description of the uncertainty affecting the system (e.g., bound on actuation noise) and/or the environment (e.g., minimum distance between obstacles), which are often unrealistic to describe with real-world environments. They also work with low-dimensional

systems, and are generally unable to provide guarantees for policies trained with rich sensing like vision. Work in safe reinforcement learning [57, 199] approximates the outcome of robot dynamics from data and allows high-dimensional image input; however, they do not provide guarantees on the robot’s safety.

Recently, techniques from *conformal prediction* [214, 187] have been used to perform failure prediction with error bounds [131]. This method provides single-episode generalization with the milder assumption of training data exchangeability, as opposed to the i.i.d. assumption on the training data for PAC learning (see e.g. [51] for reference). However, because the resulting guarantee fundamentally operates on a ‘centroidal measure’ of the performance distribution, it does not generally provide high-probability guarantees over the predictors synthesized from the training data. A modification of conformal prediction that does give high-probability guarantees, however, is computationally intractable except for very simple settings [73]. We provide a comparison of our approach with conformal prediction in Sec. 11.4.4.

PAC-Bayes Learning Theory. Generalization bounds based on PAC-Bayes theory [142, 192, 186] have recently been shown to provide strong guarantees for a variety of large-scale supervised learning settings [116, 78, 63, 23, 105, 162, 209]. Since their original development, there has been significant work on strengthening and extending PAC-Bayes bounds [43, 44, 141, 172, 200, 64]. Previous work has also extended PAC-Bayes theory in order to learn control policies for robots with guarantees on generalization to novel environments [135, 208, 166]. This work requires implementing a specific training pipeline for learning a policy (by optimizing a PAC-Bayes generalization bound). In contrast, we seek to perform failure prediction for *any* given (black-box) policy. PAC-Bayes theory allows us to provide guarantees on the false positive and negative rates of failure prediction. Our method can thus be used in a ‘plug-and-play’ manner to improve the safety of an existing policy. One recent work [100] also improves safety of the policies trained using PAC-Bayes theory and detects failure by learning a safety value function using Hamilton-Jacobi reachability-based reinforcement learning, but it does not explicitly provide guarantees on failure prediction.

11.3 Problem Formulation

11.3.1 Notation

In this paper, general spaces are denoted by capitalized Greek letters and calligraphic lettering. As examples, Π represents the space of black-box policies and $\mathcal{F}$ represents the space of failure prediction hypotheses. The only exception, $\mathcal{D}$, instead denotes probability distributions. A subscript on a distribution denotes the (assumed measurable where needed) space over which the distribution acts; e.g. $\mathcal{D}_{\mathcal{E}}$ denotes the distribution over the space of environments. Unless otherwise noted, samples from a distribution are always assumed to be i.i.d.. Let $\cap$ denote the joint event of a predicted class and a true class. For example, $p_{0\cap 1}$ denotes the probability of the joint event ‘predict class 0 and true class is 1,’ which is exactly the probability of false negative. *Class-conditional* prediction probabilities follow the same convention. For example, $p_{0|1}$ is the conditional probability of ‘predicting class 0 given the true class is 1.’ The indicator function $\mathbb{1}(x) = 1$ if x is true and $\mathbb{1}(x) = 0$ if x is false.

11.3.2 Problem Setting

Environment and Task Policy. We consider a setting where environments E are drawn from an underlying distribution $\mathcal{D}_{\mathcal{E}}$ over environments. Here, ‘environment’ refers to factors that are external to the robot (e.g., an obstacle field for drone navigation, or objects for robotic grasping). We do not assume any knowledge of or structure on $\mathcal{D}_{\mathcal{E}}$, and only assume indirect access to the distribution via a dataset $S = \{E_1, E_2, \dots, E_N\}$ of i.i.d. samples from $\mathcal{D}_{\mathcal{E}}$. At test time, we must perform failure prediction in new environments drawn from $\mathcal{D}_{\mathcal{E}}$. In all environments, we assume the robot has a sensor that provides an observation $o \in \mathcal{O}$ (e.g., RGB-D image) at each time step. We consider a deterministic, black-box task policy $\pi : \mathcal{O} \rightarrow \mathcal{A}$ that maps (potentially a history of) observations to a control input. Deploying the policy in an environment induces a label $y \in \{0, 1\}$ indicating the robot’s failure (e.g., 0 if it reaches a target and 1 if not).

Failure Predictor. Given a fixed policy, our goal is to train a failure predictor using the dataset S of environments. The predictor hypothesis $f : \mathcal{O}^H \rightarrow \mathcal{Y}$ maps a fixed horizon

of past observations to a predicted class $\hat{y} \in \{0, 1\}$. We consider that the predictor has two possible outputs: class 0 for predicting success and class 1 for predicting failure. Thus there are four possible outcomes: (1) true positive ($1 \cap 1$), predicting 1 *at least once before failure*; (2) true negative ($0 \cap 0$), *never predicting 1 during the entire successful rollout*; (3) false positive ($1 \cap 0$), incorrectly predicting 1 at least once during a successful rollout; and (4) false negative ($0 \cap 1$), incorrectly predicting 0 *at all steps of a failed rollout*. The rates of the four outcomes sum to 1: $p_{1 \cap 1} + p_{0 \cap 0} + p_{1 \cap 0} + p_{0 \cap 1} = 1$. The *misclassification error* refers to the sum of the rate of false positive and false negative, $p_{1 \cap 0} + p_{0 \cap 1}$. In addition, we define *false positive rate* (FPR) as the ratio between false positive and sum of false positive and true negative, $\text{FPR} := p_{1 \cap 0} / (p_{1 \cap 0} + p_{0 \cap 0}) = p_{1|0}$; similarly, *false negative rate* (FNR) is the ratio between false negative and sum of false negative and true positive, $\text{FNR} := p_{0 \cap 1} / (p_{0 \cap 1} + p_{1 \cap 1}) = p_{0|1}$.

Let $r_f : \mathcal{E} \times \Pi \rightarrow \mathcal{X}^T \times \mathcal{Y}^T$ denote the function that ‘rolls out’ the system with the given policy and the predictor f for T steps, i.e., r_f maps an environment E to the trajectory of states x_t (resulting from applying the policy) and failure predictions $\hat{y}_t$, for $t \in \{1, \dots, T\}$. To evaluate the performance of the failure predictor, we introduce the error of applying the predictor f in an environment E where a policy π is running:

$$C(r_f(E, \pi)) := \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{y}_t) \neq y], \quad (11.1)$$

where T_{fail} is the step when failure occurs; if the whole rollout is successful, $T_{\text{fail}} = T + 1$. This *misclassification error* describes if the prediction is misclassified (false positive or false negative). As shown in Sec. 11.4.3, it is often useful to consider the *class-conditional misclassification error* that allows us to trade-off false positives and false negatives:

$$\tilde{C}(r_f(E, \pi)) := \sum_{y \in \{0, 1\}} \lambda_y \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{y}_t) \neq y], \quad (11.2)$$

where λ_y weighs the relative importance of false positives and false negatives (e.g., $\lambda_0 = 0.3$ and $\lambda_1 = 0.7$ considers false negatives to be more costly).

Goal. Our goal is to use the training environments S to learn failure predictors that

minimize the errors and *provably generalize* to unseen environments drawn from the distribution $\mathcal{D}_{\mathcal{E}}$. We also employ a slightly more general formulation where a *distribution* $\mathcal{D}_{\mathcal{F}}$ over predictors f instead of a single predictor is used. Below we present the two optimization problems corresponding to misclassification error and class-conditional misclassification error:

$$\inf_{\mathcal{D}_{\mathcal{F}}} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [C(r_f(E, \pi))], \quad (11.3)$$

$$\inf_{\mathcal{D}_{\mathcal{F}}} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [\tilde{C}(r_f(E, \pi))]. \quad (11.4)$$

We emphasize that we do not have direct access to the distribution $\mathcal{D}_{\mathcal{E}}$ over environments. We must thus train failure predictors that will *generalize* beyond the finite training dataset S of environments we assume access to.

11.4 Failure Prediction with Guaranteed Error Bounds

Our approach for learning failure predictors with guaranteed error bounds relies on a reduction to results from the PAC-Bayes generalization theory from supervised learning; in order to present this reduction, we first provide a brief background of PAC-Bayes in the standard supervised learning setting [63].

11.4.1 PAC-Bayes Theory for Supervised Learning

Consider $\mathcal{Z}$ the input space, and $\mathcal{D}_{\mathcal{Z}}$ the (unknown) true distribution on $\mathcal{Z}$. We assume access to $S_{\mathcal{Z}} = \{z_1, z_2, \dots, z_N\}$, N i.i.d. samples of data $z_i \in \mathcal{Z}$ from $\mathcal{D}_{\mathcal{Z}}$. To each z_i there corresponds a class label $y_i \in \{0, 1\}$. Let $\mathcal{H}$ be a class of hypotheses consisting of functions $h : \mathcal{Z} \rightarrow \{0, 1\}$, parameterized by some $w \in \mathcal{W} \subseteq \mathbb{R}^d$ (e.g., neural networks parameterized by weights w). Consider a loss function $l : \mathcal{H} \times \mathcal{Z} \rightarrow [0, 1]$. Now let $\mathcal{W}$ denote the space of probability distributions on the parameter space $\mathbb{R}^d$. We assume there is a ‘prior’ distribution $\mathcal{D}_{\mathcal{W},0} \in \mathcal{W}$ before observing the N samples; and afterwards, we

choose a *posterior distribution* $\mathcal{D}_{\mathcal{W}} \in \mathcal{W}$. Denote the training loss of the posterior $\mathcal{D}_{\mathcal{W}}$ as:

$$l_S(\mathcal{D}_{\mathcal{W}}) := \frac{1}{N} \sum_{z \in S} \mathbb{E}_{w \sim \mathcal{D}_{\mathcal{W}}} [l(w; z)]. \quad (11.5)$$

The following theorem allows us to bound the true expected loss (over unknown data from $\mathcal{D}_{\mathcal{Z}}$) achieved by any $\mathcal{D}_{\mathcal{W}}$.

Theorem 11.1 (Supervised Learning PAC-Bayes Bound [142]). *For any $\delta \in (0, 1)$ and ‘prior’ distribution $\mathcal{D}_{\mathcal{W},0}$ over w , the following inequality holds with probability at least $1 - \delta$ over training sets $S \sim \mathcal{D}_{\mathcal{Z}}^N$ for all posterior distributions $\mathcal{D}_{\mathcal{W}}$:*

$$\begin{aligned} \mathbb{E}_{z \sim \mathcal{D}_{\mathcal{Z}}} \mathbb{E}_{w \sim \mathcal{D}_{\mathcal{W}}} [l(w; z)] &\leq l_S(\mathcal{D}_{\mathcal{W}}) + R(\mathcal{D}_{\mathcal{W}}, \mathcal{D}_{\mathcal{W},0}, \delta), \\ R(\mathcal{D}_{\mathcal{W}}, \mathcal{D}_{\mathcal{W},0}, \delta) &= \sqrt{\frac{D_{\text{KL}}(\mathcal{D}_{\mathcal{W}} \parallel \mathcal{D}_{\mathcal{W},0}) + \log(\frac{2\sqrt{N}}{\delta})}{2N}}, \end{aligned} \quad (11.6)$$

where $D_{\text{KL}}(\cdot \parallel \cdot)$ is the *Kullback-Leibler (KL) divergence*.

Note that the prior distribution $\mathcal{D}_{\mathcal{W},0}$ may be chosen arbitrarily, but random or poorly chosen priors may negatively affect the guarantee. We elaborate on how we select the prior (and posterior) in Sec. 11.4.5.

11.4.2 Bound on Misclassification Error

We now present a result that tackles (11.3) for learning failure predictors with bounds on the expected misclassification error. Assume there is a prior distribution $\mathcal{D}_{\mathcal{F},0}$ on the failure predictors, and then denote the training error of the posterior $\mathcal{D}_{\mathcal{F}}$ on the training dataset S of environments as:

$$C_S(\mathcal{D}_{\mathcal{F}}) := \frac{1}{N} \sum_{E \in S} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [C(r_f(E, \pi))]. \quad (11.7)$$

Analogous to results in Sec.11.4.1, we present the following theorem that upper bounds the true expected misclassification error.

Theorem 11.2 (PAC-Bayes Bound on Misclassification Error). *For any $\delta \in (0, 1)$ and any*

prior distribution over failure predictors $\mathcal{D}_{\mathcal{F},0}$, the following inequality holds with probability at least $1 - \delta$ over training sets $S \sim \mathcal{D}_{\mathcal{E}}^N$ for all posterior distributions $\mathcal{D}_{\mathcal{F}}$:

$$\begin{aligned} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [C(r_f(E, \pi))] &\leq C_S(\mathcal{D}_{\mathcal{F}}) + R(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_{\mathcal{F},0}, \delta), \\ R(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_{\mathcal{F},0}, \delta) &= \sqrt{\frac{D_{\text{KL}}(\mathcal{D}_{\mathcal{F}} \parallel \mathcal{D}_{\mathcal{F},0}) + \log(\frac{2\sqrt{N}}{\delta})}{2N}}. \end{aligned} \quad (11.8)$$

Proof. We apply a reduction of failure prediction to the supervised learning setting, from which the results follow immediately. An outline of the reduction is presented in Table 11.1. See [135] for a similar proof construction.

Table 11.1: Reduction of Failure Prediction to Supervised Learning in PAC-Bayes Theory

Supervised Learning	←	Failure Prediction
Input Data $z \in \mathcal{Z}$		Environment $E \in \mathcal{E}$
Hypothesis $h_w : \mathcal{Z} \rightarrow \mathcal{Z}'$		Rollout $r_f : \mathcal{E} \times \Pi \rightarrow \mathcal{X}^T \times \mathcal{Y}^T$
Loss $l(w; z)$		Error $C(r_f(E, \pi))$

Remark. It is worth noting that the use of trajectories does not violate the i.i.d. data assumption required for PAC-Bayes theory. While, for example, the states visited along the multi-step trajectory are not i.i.d., the error and guarantee are defined for each *rollout*, and thus *are i.i.d.* Our guarantee holds over the course of the entire rollout; each trajectory ‘collapses’ into a single ‘datum,’ and the independence of these ‘data’ (each environment/rollout) is preserved. $\square$

In training, we try to minimize the right-hand side of the PAC-Bayes bound in order to find a posterior distribution $\mathcal{D}_{\mathcal{F}}$ that provides a tight bound on the expected error $\mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} C(r_f(E, \pi))$. This corresponds to the probability that the predictor will be incorrect when deployed in a new environment from the unknown distribution $\mathcal{D}_{\mathcal{E}}$. Thus, on a new environment, the resulting $\mathcal{D}_{\mathcal{F}}$ will only make an unsuccessful prediction with probability less than the upper bound; a small upper bound guarantees a small probability of incorrect prediction.

While this approach provides a clean upper bound on the misclassification error of the failure predictor, for certain practical cases, this can be insufficient. In the next subsection we construct a simple example to illustrate that misclassification error can lead to poor

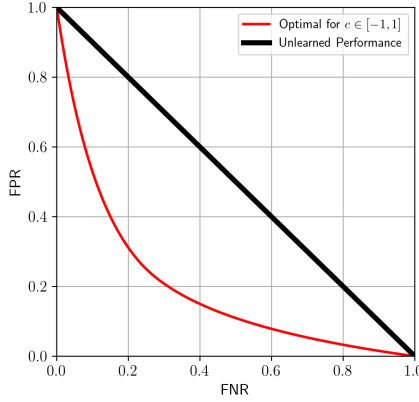


Figure 11.2: Continuum of optimal predictors (red) for varying class population ratios (successes and failures). The failure predictor can perform badly (i.e., on the bottom part of the curve) even with infinite data.

prediction. The natural solution to this problem is to use a weighted (class-conditional) misclassification error and bound FNR and FPR instead (Sec. 11.4.4).

11.4.3 Motivating Class-Conditional Misclassification Error

Consider a single-step setting where the true state x is the sum of the observation o and an unobserved noise ϵ associated with an environment $E \sim \mathcal{D}_{\mathcal{E}}$. The observation and noise are drawn uniformly on the interval $[-1, 1]$ (i.e., $o, \epsilon \sim \mathcal{U}(-1, 1)$). Define the true success-failure label y_i for a policy π and environments E_i as:

$$y_i = \mathbb{1}[[x_i \triangleq o_i + \epsilon_i] \geq c], \quad (11.9)$$

where $c \in [-2, 2]$ is a known failure threshold. This threshold allows us to vary the expected fraction of 1-labels (failures) from the underlying distribution $\mathcal{D}_{\mathcal{E}}$. If the predictor is aware of the uncertainty in observation and noise, then the optimal predictor in expectation under Eq. (11.1) is,

$$\hat{y}^* = f^*(o) = \mathbb{1}[o \geq c].$$

However, this implies that for the case $|c| \geq 1$, the optimal prediction is always the more-likely class, since $o \in [-1, 1]$. But even for $|c| = 1$, the less-likely class can still occur

up to 12.5% of the time (see Appendix 11.7.2 for calculations). Then the predictor always predicts success and misses all failures; the misclassification error is 0.125. This shows the misclassification error biases the failure predictor towards the more common class. If that class is benign, the predictor under-detects failures.

This simple problem motivates us to instead minimize the *conditional probability of misclassification* of the predictor, which is the *weighted* sum of the FNR and FPR. Fig. 11.2 shows conditional failure probabilities of the optimal predictors (red curve) as the parameter c varies in $[-1, 1]$. Although varying c only changes the true proportion of classes 0 and 1 among training samples, the conditional performance of the predictor can degrade considerably depending on c . Even with infinite data, the optimal predictor will *never* predict a failure when at least 87.5% of training samples are successes. For analytical details regarding the policy and predictor, see Appendix 11.7.2.

11.4.4 Bound on Class-Conditional Misclassification Error

The example above shows that minimizing the total misclassification error can fail to perform well when the relative importance of each class does not match its prevalence in the data (i.e., failures are important but rare). Our direct approach to generalize the method therefore utilizes a weighted misclassification error. Interpreting the total misclassification error as the class-weighted sum of conditional error probabilities, the following shows the desired generalization to an arbitrary weight $\lambda \in [0, 1]$:

$$\begin{aligned}
p_{\text{error}} &= p_{0|1} + p_{1|0} \\
&= p_{0|1}p_1 + p_{1|0}p_0 \\
&= p_{0|1}(1 - \lambda^*) + p_{1|0}(\lambda^*) \\
\rightarrow \text{generalize to } &= p_{0|1}(1 - \lambda) + p_{1|0}(\lambda),
\end{aligned} \tag{11.10}$$

where p_0 and p_1 are true proportions of successes and failures in environments from $\mathcal{D}_{\mathcal{E}}$. Since the true FNR and FPR, $p_{0|1}$ and $p_{1|0}$, are unknown, we use the empirical FNR and FPR, $\hat{p}_{0|1}$ and $\hat{p}_{1|0}$, present in the dataset S instead and propose the following error function

as the empirical mean over environments in S :

$$\hat{C}_S(r_f(E, \pi), S) \triangleq (1 - \lambda)\hat{p}_{0|1} + \lambda\hat{p}_{1|0}. \quad (11.11)$$

However, these empirical proportions are random variables over the draw of S , and thus $\hat{C}_S(r_f(E, \pi), S)$ will not admit a PAC-Bayes bound. Instead, we need an alternate error function that holds *with high probability* over dataset S and is a provably tight over-approximation of (11.11). Defining $\underline{p}_i$ to be high-probability lower bounds for p_i (found via Bernstein's Inequality), the necessary error function is (11.12).

$$\begin{aligned} \tilde{C}(r_f(E, \pi)) &= \frac{\lambda}{C_\lambda \underline{p}_0} \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{y}_t) = 1 \cap y = 0] \\ &\quad + \frac{1 - \lambda}{C_\lambda \underline{p}_1} \mathbb{1}[(\max_{t < T_{\text{fail}}} \hat{y}_t) = 0 \cap y = 1], \end{aligned} \quad (11.12)$$

where $C_\lambda \triangleq \frac{\lambda}{\underline{p}_0} + \frac{1 - \lambda}{\underline{p}_1}$.

The cost function is bounded in $[0, 1]$ and does not rely on quantities that are random variables over the draw of S . Now, we can present the class-conditional bound.

Theorem 11.3 (Class-Conditioned PAC-Bayes Bound). *For any $\delta \in (0, 1)$ and any prior distribution $\mathcal{D}_{\mathcal{F}, 0}$ over failure predictors, the following inequality holds with probability at least $1 - \delta$ over training samples $S \sim \mathcal{D}_{\mathcal{E}}^N$ for all posterior distributions $\mathcal{D}_{\mathcal{F}}$:*

$$\begin{aligned} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [\tilde{C}(r_f(E, \pi))] &\leq \hat{C}_S(r_f(E, \pi), S) + R_\lambda, \\ R_\lambda &= \frac{5}{3} \sqrt{\frac{(1 - \underline{p}) \log \frac{2}{\delta}}{N \underline{p}}} + C_\lambda R(\mathcal{D}_{\mathcal{F}}, \mathcal{D}_{\mathcal{F}, 0}, \delta). \end{aligned} \quad (11.13)$$

A key result is that the regularizer R_λ remains $\tilde{O}(\frac{1}{\sqrt{N}})$.

Proof. A detailed proof is deferred to Appendix 11.7.1. In summary, we utilize a union bound to ensure that all necessary events hold with high probability, and then combine the new error functions into the PAC-Bayes framework to obtain the new ‘regularizer’ term R_λ . □

Remark. This framework achieves two principal benefits. First, the bounds meaningfully hold for settings where failures and successes are not balanced in the sample dataset. Additionally, for a given confidence level, only a fixed number of instances of the less-common class is necessary to achieve a bound. This reflects the intuition that the fundamental information constraint is the number of instances of the class that have been seen.

Remark. Without the class-conditional bound, estimating the conditional accuracy of the detector (e.g., ‘what fraction of failures are being predicted?’) can only be done loosely (dividing $p_{1 \cap 1}$ by p_1 , the latter of which is assumed to be small). Further, without the class-conditional objective, the predictor itself will be optimized to predict the more common class; that is, the objective function will implicitly be biased against its intended application (finding failures).

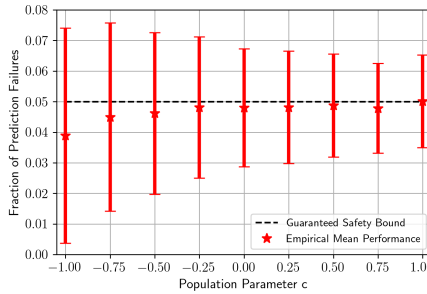


Figure 11.3: Results of conformal prediction on the toy problem in Sec. 11.4.3, which hold in *expectation* over training samples S and a test case (red dots). Empirical performance is better than the safety guarantee of $\epsilon^* = 0.05$ (black dashed line), as expected. However, the variance over predictors illustrated in the $1\text{-}\sigma$ error bars indicates that many predictors fail the guaranteed safety bound.

Comparison to Conformal Prediction. Recently, techniques based on *conformal prediction* have been proposed for learning failure predictors with error bounds [131]. Here, we illustrate the fundamental differences between the guarantees provided by conformal learning and our PAC-Bayes approach. The conformal prediction framework allows for guarantees with better sample efficiency ($\tilde{O}(1/N)$, compared to $\tilde{O}(\sqrt{1/N})$ for PAC-Bayes bounds) that hold in expectation over both the test sample and training datasets. PAC-Bayes-style guarantees, on the other hand, hold in expectation over the test sample but *with high probability* over training datasets.

The expected performance of conformal prediction, shown in the red dots of Fig. 11.3 meets the guaranteed bound (dashed black line) at $\tilde{O}(1/N)$ sample complexity. However, many of the synthesized failure predictors (the decision rules learned by conformal prediction using different training datasets) do not meet the desired safety levels of performance. The $1\text{-}\sigma$ error bars in Fig. 11.3 represent variance over the predictors, indicating that many predictors themselves will violate the guaranteed safety bound. Thus, if the policy designer has access to a single training dataset to learn a failure predictor, conformal prediction does not guarantee that the expected error of the resulting policy will be below the desired threshold. This highlights a key difference between the approaches: the fraction of ‘bad’ decision rules can be made arbitrarily small in the PAC-Bayes approach by reducing the parameter δ ; in conformal prediction, reducing the fraction of ‘bad’ predictors is non-trivial.

11.4.5 Algorithmic Implementation

In the experimental setups, we parameterize failure predictors using neural networks and specify distributions over failure predictors using multivariate Gaussian distributions over the weights of the networks. We set the covariance matrix to be diagonal; thus, we can write the prior as $\mathcal{D}_{\mathcal{F},0} = \mathcal{N}(\mu_0, \text{diag}(s_0))$, and posterior as $\mathcal{D}_{\mathcal{F}} = \mathcal{N}(\mu, \text{diag}(s))$. We train the posterior $\mathcal{D}_{\mathcal{F}}$ by optimizing the bounds on error rates provided by our theory. We present the training algorithm along with further implementation details in Appendix 11.7.3. Before training the posterior, we first train the prior $\mathcal{D}_{\mathcal{F},0}$ on held-out data to improve performance and resulting guarantees. After training is complete, we can use the posterior to compute PAC-Bayes bounds on error rates. The PAC-Bayes bound on the misclassification error can be computed directly with (11.8). To compute PAC-Bayes bounds on the FNR and FPR, we can use $\lambda = 0$ and $\lambda = 1$ respectively in (11.13).

11.5 Experimental Results

Through extensive simulation and hardware experiments, we aim to demonstrate strong guarantees on (class-conditional) misclassification error of trained predictors. We also validate the guarantees by evaluating the predictors on test environments in both simulation

and on hardware.

11.5.1 Obstacle Avoidance with a Drone

Overview. In this example, we train a failure predictor for a drone executing an obstacle-avoidance policy. The failure predictor uses depth images taken from the perspective of the drone. We train the failure predictor in simulation and apply it on a hardware platform with a Parrot Swing drone (Fig. 11.1); this is an agile quadrotor/fixed-wing hybrid drone that is capable of vertical takeoff/landing, and hovering in place. We consider a failure to be any instance in which the drone collides with an obstacle. If the failure predictor indicates that a crash will occur, we trigger an emergency policy that causes the drone to stop and land safely. In this section, we evaluate our bounds and failure prediction performance in simulation and also present extensive hardware experiments.

Policies and Environment Distributions. The drone’s obstacle avoidance policy consists of a deep neural network (DNN) classifier which uses a depth image to select one out of a set of open-loop motion primitives for the drone to execute. We use a Vicon motion capture system to locate all obstacles in the environment as well as the drone’s position to generate artificial depth images from the perspective of the drone. It is important to note that neither the policy nor the failure predictor has access to any other information about the environment besides the first-person depth images.

In order to match our simulation setup to the hardware system, we generate a set of motion primitives for the drone by applying sequences of open-loop control actions on the hardware platform. We then record the resulting trajectories using a Vicon motion capture system and use these trajectories for our simulation experiments (in order to emulate the hardware implementation). Thus these motion primitives capture noise in the hardware dynamics and help to bridge the sim-to-real gap.

We use two distinct policies and environment distributions in order to demonstrate our approach. In the first (‘standard’) setting, obstacles are placed (uniformly) randomly in the environment; the drone’s policy then selects from a set of motion primitives which cause the drone to travel in mostly straight paths (Fig. 11.4 left). In the second (‘occluded’) setting, the environment is generated in two stages. First, a number of obstacles are (uniformly)

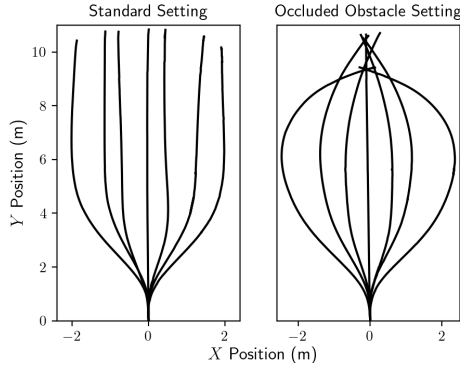


Figure 11.4: (Left) Representative motion primitives for the policy used in the standard setting in the navigation task. (Right) Representative motion primitives from the policy used in the occluded obstacle settings.

randomly placed. Second, we generate additional obstacles that are placed exclusively in locations that are occluded by obstacles generated in the first stage. The drone’s policy then uses the depth image captured from its starting location to select a motion primitive from the set shown in Fig. 11.4 (right). These primitives cause the drone to travel in curved paths (which may intersect initially-occluded obstacles). Since obstacles in this setting are specifically placed in portions of the environment that are occluded, this setting is adversarial in nature and leads to a large number of failures. The policy for the standard setting results in a failure rate of 25.3%, while the policy for the occluded setting results in a failure rate of 51.4%. Our approach allows for the use of a black-box policy which is not necessarily deployed in settings where it was trained nor where it would perform well. We aim to examine the improvement in safety of the policy with the addition of the failure predictor; thus, we test in settings that are challenging and even adversarial to the policy.

Failure Predictor and Training. The failure predictor consists of a DNN which takes as input depth images from the perspective of the drone. The network receives a new image at a frequency of 20Hz and stacks the four most recent images as input to predict failure or no failure at some point in the future (i.e. 2-classes), with a Softmax layer at the end. If at any time step, the failure class is largest, we say it is a prediction of failure; otherwise, it is a prediction of success. We use 10,000 training environments to train the prior distribution $\mathcal{D}_{\mathcal{F}_0}$ over failure predictors and an additional 10,000 training

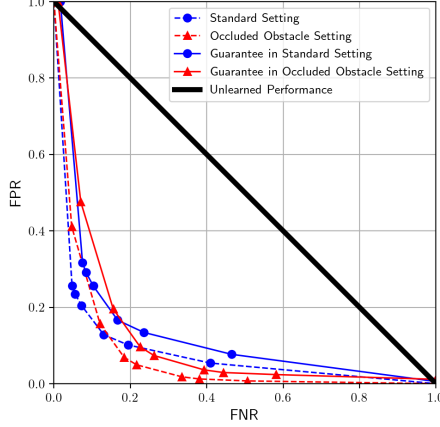


Figure 11.5: The dashed lines show the resulting failure predictor’s FNR and FPR when the importance of a false negative is varied. The solid lines and markers show the associated PAC-Bayes guarantee on the FNR and FPR using the class-condition guarantees introduced in Sec. 11.4.4.

environments to optimize the PAC-Bayes upper bound in (11.8). We use Algorithm 11.1 to obtain a posterior distribution $\mathcal{D}_{\mathcal{F}}$ and the associated PAC-Bayes generalization guarantee. We note that the physical drone takes approximately $0.5s$ to slow to a stop; thus, the failure predictor needs to predict collisions 10 time-steps ahead (since the frequency of the input is 20Hz).

After training, we compute the bound on the misclassification error with (11.8), in addition to the bounds on the FNR and FPR with (11.13). At test time, we sample a failure predictor $f \sim \mathcal{D}_{\mathcal{F}}$ and fix it for a particular test environment.

Simulation Results. We first verify our failure predictors in simulation before testing them on the hardware setup. As shown in Sec. 11.4.3, there is an intrinsic trade-off between the false-positive and false-negative rates in general. To validate our theoretical framework, we plot the bounds on the FNR and FPR given by (11.13) (solid lines in Fig. 11.5). We compare these bounds with the empirical FNR and FPR (estimated with 20000 held out environments) of the predictor with varied false-negative importance during training; these are plotted using dashed lines in Fig. 11.5. We obtain strong bounds on the failure predictors’ errors, with closely-matching empirical performance along all points along the curve.

Hardware Results. We select a single predictor for each of the settings (standard

Table 11.2: Results for Failure Prediction on Navigation Task

Setting	Standard	Occluded Obstacle
True Expected Failure (Sim)	0.253	0.514
Misclassification Bound	0.128	0.154
True Expected Misclassification (Sim)	0.101	0.125
True Expected Misclassification (Real)	0.067	0.133

and occluded) to use in the hardware experiments. We choose the failure predictors with the tightest guaranteed total error rate (i.e. tightest upper bound from (11.8)). When the failure predictor stops the rollout due to a prediction of failure, we re-run the trial without the failure predictor to determine the true label. See Fig. 11.1 for an example of the obstacle placement for the occluded obstacle setting. Additionally, a video with representative trials from both settings is available at <https://youtu.be/z4UwQzTjhqo>. We run 15 trials in each of the settings and show the misclassification error of the failure predictors along with the guarantee on the misclassification in Table 11.2. In the standard setting, $p_{1\cap 1} = \frac{2}{15}$, $p_{0\cap 0} = \frac{12}{15}$, $p_{1\cap 0} = \frac{1}{15}$, $p_{0\cap 1} = 0$, and in the occluded obstacle setting $p_{1\cap 1} = \frac{9}{15}$, $p_{0\cap 0} = \frac{4}{15}$, $p_{1\cap 0} = \frac{2}{15}$, $p_{0\cap 1} = 0$. The misclassification bounds are validated by results from both the standard and occluded obstacle settings. Additionally, in all trials where the failure predictor was used, the drone never crashed due to a missed failure (i.e., there were no false negatives). These experiments demonstrate the power of the failure predictor to maintain the robot’s safety in novel and even adversarial settings.

11.5.2 Grasping Mugs with a Robot Arm

Overview. In this example, we consider a robot arm performing the task of grasping a mug (Fig. 11.6). There are three mugs on the table and the arm needs to lift one of them off the table. We use the PyBullet simulator [53] for training the policy and failure predictor, and then test these on a hardware setup with a Franka Panda 7-DOF arm equipped with a wrist-mounted Intel RealSense D435 camera (Fig. 11.6). Failure is defined as either (1) the arm failing to lift the mug from the table or (2) the gripper contacting the mug with excessive force before grasping (e.g., if the bottom of the gripper finger hits the rim of the mug). In simulation, we check the force readings at the end-effector joint of the arm, and

set the threshold to be 20N for (2). We use the same threshold on the physical arm.

Policy and Environment Distributions. We consider a multi-view grasping setting: with a fixed horizon of five steps, the robot starts at a fixed pose and gradually moves towards the mug. The gripper always closes at the last step to grasp the mug. The policy takes a 100×150 pixels RGB-D image from the camera and outputs the desired pose relative to the current pose, $(\Delta_x, \Delta_y, \Delta_z, \Delta_\psi)$, in 3D translation and yaw orientation. The relative pose is normalized between $[-0.02, 0.02]$ cm for Δ_x and Δ_y , $[-0.05, -0.03]$ cm for Δ_z , and $[-15, 15]^\circ$ for Δ_ψ . To simulate more realistic camera images, we add pixel-wise Gaussian noise to the depth image, and randomly change the color values for 5% of the pixels in the RGB image. In order to evaluate the failure predictor on policies with different task success rates, we choose three different policies saved at different epochs during training. Shown in Table 11.3, the policies (‘High’, ‘Medium’, and ‘Low’ settings) achieve 11.8%, 22.1%, and 46.6% failure rates respectively on test environments in simulation.

In order to create different environments for the robot, we obtained 50 mugs of diverse geometries from the ShapeNet dataset [224]. We then randomly scale them to different diameters between $[8, 12]$ cm. For each environment, we sample three different mugs and their initial locations on the table (while ensuring that the mugs do not intersect). We randomly assign a RGB value as the color of the mug.

Failure Predictor and Training. Similar to the drone example, the predictor is parameterized with a DNN that takes the RGB-D image from the camera and outputs the probability of success and failure with a (two-class) Softmax layer at the end. If the predicted probability of failure is higher than 0.5, the predictor outputs $\hat{y} = 1$. Since the gripper does not reach the mug in the first two out of five steps of the rollout, we only train the failure predictor using data from the last three steps of each rollout. We use 5000 training environments to train the prior distribution $\mathcal{D}_{\mathcal{F},0}$ over failure predictors and an additional 5000 training environments to optimize the PAC-Bayes upper bound and provide a posterior $\mathcal{D}_{\mathcal{F}}$.

Simulation Results. Generalization bounds and test prediction failure rates of the grasping example are shown in Table 11.3. Across all three settings, we achieve tight guarantees on failure prediction compared to the true expected failure rate of the policies.

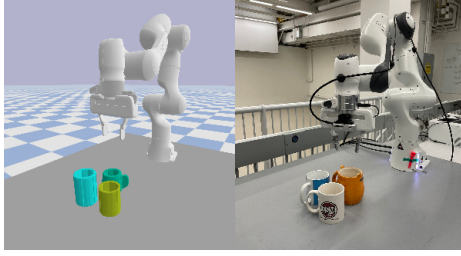


Figure 11.6: (a) Simulation environment in PyBullet simulator (virtual wrist-mounted camera not shown). (b) Real environment with an arm and a wrist-mounted camera for grasping task.

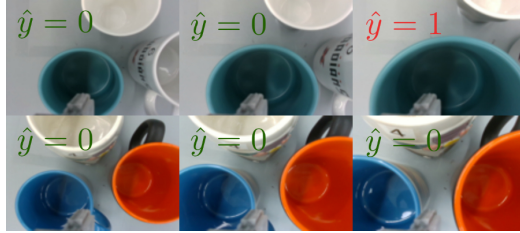


Figure 11.7: Camera observation (depth not shown) at the last three steps in a trial in the grasping task on the hardware platform. (Top) A true-positive trial, where the predictor outputs failure at the last step, and then the right finger of the gripper hits the mug rim (not shown); (bottom) a true-negative trial.

When testing on additional 5000 environments in simulation (‘Sim’), the expected prediction rate validates the bound in all settings. We also plot different levels of bounds on FNR and FPR by varying the importance of false negatives relative to that of false positive in the ‘Medium’ setting (see Fig. 11.8 in Appendix 11.7.3). Similar to Fig. 11.5 in the drone example, we achieve strong bounds on conditional misclassification errors and the empirical FNR and FPR match the bounds.

Table 11.3: Results of Failure Prediction in Grasping Task

Setting	High	Medium	Low
True Expected Failure (Sim)	0.124	0.223	0.485
Misclassification Bound	0.087	0.130	0.142
True Expected Misclassification (Sim)	0.058	0.101	0.118
True Expected Misclassification (Real)	0.067	0.133	0.133

Hardware Results. We also test the failure predictor on the hardware setup (Fig. 11.6). We collect 18 real mugs of diverse geometries and visuals and split them into 6 sets. For each set, we perform 5 experiments with random initial mug poses and randomly sampled

failure predictor from the posterior distribution. In order to check if the prediction is correct, we continue the trial even when the predictor detects failure. Fig. 11.7 shows the RGB images from the camera and failure predictions at the last three steps of two trials. A video with representative trials is available at <https://youtu.be/z4UwQzTjhqo>.

The results of 30 trials for each setting are shown in Table 11.3 (Real). In the ‘High’ setting, $p_{1\cap 1} = \frac{4}{30}$, $p_{0\cap 0} = \frac{24}{30}$, $p_{1\cap 0} = \frac{2}{30}$, $p_{0\cap 1} = 0$; in the ‘Medium’ setting $p_{1\cap 1} = \frac{6}{30}$, $p_{0\cap 0} = \frac{20}{30}$, $p_{1\cap 0} = \frac{3}{30}$, $p_{0\cap 1} = \frac{1}{30}$; and in the ‘Low’ setting $p_{1\cap 1} = \frac{11}{30}$, $p_{0\cap 0} = \frac{15}{30}$, $p_{1\cap 0} = \frac{3}{30}$, $p_{0\cap 1} = \frac{1}{30}$. In both ‘High’ and ‘Low’ settings, the empirical results validate the corresponding misclassification bounds. In the ‘Medium’ setting, the empirical rate is slightly higher than the bound. This could be due to bias from the limited number of real experiment trials and disparities between camera observations in simulation and in real. Nonetheless, the hardware results overall demonstrates the tightness of the theoretical guarantees on failure prediction.

11.6 Conclusion

In this work we propose an approach for training a failure predictor with rigorous guarantees on error rates by leveraging PAC-Bayes generalization theory. We develop novel class-conditional bounds that allow us to trade-off the relative rates of false-positive and false-negative errors. We present an algorithmic approach that parameterizes the predictor using a neural network and minimizes the theoretical error bounds. Through extensive experiments in vision-based drone navigation and robot grasping tasks in both simulation and on real hardware, we demonstrate strong guarantees on failure prediction rates.

Future work. Our approach assumes that environments that the failure predictor will be deployed in are drawn from the same underlying (but unknown) distribution from which training environments are generated. Extending our approach to perform failure prediction in *out-of-distribution* environments (e.g., using *distributionally robust* bounds) would be of significant interest. In addition, the results we present in this paper are for relatively short-horizon tasks; we are interested to test our approach on longer-horizon problems (e.g., using transformers or long short-term memory models). We also note that in the settings we test,

the black-box policies have as high as 87.5% success rate. However, a limitation of our approach, and of statistical approaches in general, is developing meaningful guarantees in settings where the policy has a significantly higher (e.g., 99.9%) success rate. Scaling to such settings where failures are extremely rare may require a large number of training samples without further assumptions about the policy or training data; this is a good direction for future work. Finally, the use of more sophisticated network architectures (e.g., those based on attention models) could have an impact on the quality of the bounds and empirical performance. In particular, attention models could learn to focus on particular portions of the robot’s observations that are most predictive of failures.

11.7 Appendix

11.7.1 Class-Conditional Bound Proof

Proof of Theorem 3.

Lemma 11.1. (*Bernstein Inequality for Bernoulli Variables*)

For any $\delta \in (0, 1)$, $p_0 \in (0, 1)$ and $p_1 = 1 - p_0$, with probability greater than $1 - \delta$ over samples S of size N of i.i.d. Bernoulli random variables, the following inequalities hold for $\underline{p}_0, \underline{p}_1$ and ratios $\underline{K}_i$:

$$\begin{aligned} \underline{K}_i &= \frac{3}{5} \left[\sqrt{\frac{N \underline{p}_i}{2 \log \frac{2}{\delta}}} \right], \\ p_i &\geq \hat{p}_i - \frac{1}{\underline{K}_i + 1} \hat{p}_i. \end{aligned} \tag{11.14}$$

Proof. For the empirical mean of Bernoulli random variables, the Bernstein inequality allows us to show directly that

$$\mathbb{P} \left[\hat{p}_i - p_i \geq \frac{10}{3} \sqrt{\frac{\sigma^2 \log \frac{2}{\delta}}{N}} \right] \leq \frac{\delta}{2}.$$

Defining $K_{\delta N} = \frac{100 \log \frac{2}{\delta}}{9}$, the above term inside the brackets simplifies to

$$p_i^2(1 + K_{\delta N}) - (2\hat{p}_i + K_{\delta N})p_i + \hat{p}_i^2 \leq 0.$$

This is a scalar quadratic equation in p_i ; taking the minimal solution gives $\underline{p}_i$. Now, analyze the multiplicative bound

$$\frac{10}{3} \sqrt{\frac{\sigma^2 \log \frac{2}{\delta}}{N}} \leq \frac{p_i}{K_i}.$$

This bound implies

$$\begin{aligned} K_i &\leq \frac{3}{5} \sqrt{\frac{N p_i}{2 \log \frac{2}{\delta}}}, \\ N &\geq \frac{50 K_i^2 \log \frac{2}{\delta}}{9 p_i}. \end{aligned}$$

To be practically feasible, we must be able to bound p_i away from 0. This is equivalent to achieving $K_i > 1$. Of course, observing a few instances (or even a single instance) of class i is sufficient to ensure this condition is met for a Bernoulli variable. Assuming this holds, we have that

$$\begin{aligned} \hat{p}_i - p_i &\leq \frac{p_i}{K_i}, \\ \implies \hat{p}_i &\leq p_i \left(1 + \frac{1}{K_i}\right), \\ \implies p_i &\geq \frac{K_i}{K_i + 1} \hat{p}_i, \\ &= \hat{p}_i - \frac{1}{K_i + 1} \hat{p}_i \triangleq \underline{p}_i. \end{aligned}$$

□

Using the error functions, we can rearrange to show that the structure is maintained. Specifically,

$$\begin{aligned} \hat{C}(r_f(E, \pi), S) &= \sum_{i=0}^1 \frac{\lambda_i}{\hat{p}_i} \mathbb{1}[f(E) \neq y], \\ C_\lambda &\triangleq \frac{\lambda_0}{\underline{p}_0} + \frac{\lambda_1}{\underline{p}_1}, \\ \tilde{C}(r_f(E, \pi)) &= \frac{1}{C_\lambda} \sum_{i=0}^1 \frac{\lambda_i}{\underline{p}_i} \mathbb{1}[f(E) \neq y], \\ &= \sum_{i=0}^1 \lambda'_i \mathbb{1}[f(E) \neq y]. \end{aligned}$$

Because $\tilde{C}$ is a valid error function, we can apply the PAC-Bayes framework. Defining for notational shorthand

$$R_{PAC} \triangleq C_\lambda \sqrt{\frac{D_{\text{KL}}(\mathcal{D}_{\mathcal{F}} \parallel \mathcal{D}_{\mathcal{F},0}) + \log(\frac{2\sqrt{N}}{\delta})}{2N}},$$

we see that under standard PAC-Bayes, we can achieve the following bound:

$$\mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [\tilde{C}(r_f(E, \pi))] \leq \frac{1}{N} \sum_{E \in S}^N \tilde{C}(r_f(E, \pi)) + R_{PAC}.$$

This implies then that we can bound $\hat{C}$ as follows:

$$\begin{aligned} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [\hat{C}(r_f(E, \pi), S)] &\leq \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [C_\lambda \tilde{C}(r_f(E, \pi))], \\ &\leq C_\lambda \frac{1}{N} \sum_{i=1}^N \tilde{C}(r_f(E, \pi)) + R_{PAC}, \\ &\leq \frac{1}{N} \sum_{i=1}^N \hat{C}(r_f(E, \pi), S) (1 + \frac{1}{\underline{K}_{min}}) + R_{PAC}. \end{aligned}$$

The first and last inequalities follow from Lemma 11.1 and the fact that $\underline{p}_i < \hat{p}_i$. Specifically, the latter fact implies that $\hat{C} \leq C_\lambda \tilde{C}(\cdot)$ (in words: $C_\lambda \tilde{C}$ is an over-approximation), while the first fact implies that the over-approximation ratio is less than $1 + 1/K$ (i.e. is relatively tight). Defining $\underline{p} = \min_i \underline{p}_i$ and $\underline{K} = \min_i \underline{K}_i$, and noting that the empirical mean error must be less than $1/2$ by definition of λ (that is, there exists a policy that achieves $\min\{\lambda, 1 - \lambda\}$ cost by simply always guessing the appropriate class), we have that (conservatively):

$$\begin{aligned} \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [\hat{C}(r_f(E, \pi), S)] &\leq \mathbb{E}_{E \sim \mathcal{D}_{\mathcal{E}}} \mathbb{E}_{f \sim \mathcal{D}_{\mathcal{F}}} [C_\lambda \tilde{C}(r_f(E, \pi))], \\ &\leq \frac{1}{N} \sum_{i=1}^N \hat{C}(r_f(E, \pi), S) (1 + \frac{1}{\underline{K}}) + R_{PAC}, \\ &\leq \frac{1}{N} \sum_{i=1}^N \hat{C}(r_f(E, \pi), S) + \frac{1}{2\underline{K}} + R_{PAC}, \\ &= \frac{1}{N} \sum_{i=1}^N \hat{C}(r_f(E, \pi), S) + \frac{5}{3} \sqrt{\frac{(1 - \underline{p}) \log \frac{2}{\delta}}{N\underline{p}}} + R_{PAC}. \end{aligned}$$

This matches the result of (11.13) stated in Theorem 11.3, and completes the proof. $\square$

11.7.2 Toy Example Calculations

Let $w_i = x_i + z_i$. Then for each training sample, the true class label is defined by $y_i = \mathbb{1}[w_i \geq c]$. The optimal classifier in expectation is $\hat{y}_i^* = f^*(x_i) = \mathbb{1}[x_i \geq c]$. It can be shown that the probability density for $W = X + Z$ is

$$p(w) = 0.5 - 0.25|w|, \quad w \in [-2, 2]. \quad (11.15)$$

Further, the probability of a new example being in classes 0 and 1 as a function of the cutoff $c \in [-2, 0]$ is

$$\begin{aligned} p_0(c) &= \frac{(c+2)^2}{8}, \\ p_1(c) &= 1 - p_0(c). \end{aligned} \quad (11.16)$$

By symmetry, for $c \in [0, 2]$ the same probabilities hold but for the opposite quantities. We note that the optimal policy, based only on the observable x_i , is invariant for $|c| \geq 1$. As such, we will generally restrict our attention to $c \in [-1, 1]$.

Cap notation will again note the intersection of two events, which will be in the order {predicted class, true class}. I.e. $p_{1 \cap 0}$ denotes the probability that the predictor chooses class 1, but that the example is truly in class 0.

We now calculate $p_{1 \cap 0}$ and $p_{0 \cap 1}$ as a function of $c \in [-1, 0]$ (we will argue from symmetry that the results extend naturally to $c \in [0, 1]$, and argue by the optimal policy invariance for large-magnitude c that the extension is trivial to $|c| > 1$). For clarity, because $X \sim \mathcal{U}[-1, 1]$,

$d\mu_x = (1/2)dx$; this substitution will be very common throughout the following analysis:

$$\begin{aligned}
p_{1 \cap 0} &= p(\text{predict } 1 \cap \text{truly } 0), \\
&= p((x_i \geq c) \cap (w_i < c)), \\
&= \int_c^{c+1} \frac{1 - (x - c)}{2} d\mu_x, \\
&= \frac{1}{4} \int_c^{c+1} (1 + c - x) dx, \\
&= \frac{1}{4} (1 + c)x - \frac{x^2}{2} \Big|_c^{c+1}, \\
&= \frac{1}{4} [(1 + c) - (\frac{c^2 + 2c + 1 - c^2}{2})], \\
&= \frac{1}{8}.
\end{aligned}$$

We have that for $c \in [-1, 0]$:

$$\begin{aligned}
p_{1|0}p_0 &= p_{1 \cap 0}, \\
\implies p_{1|0}(c) &= \frac{1/8}{(c+2)^2/8}, \\
&= \frac{1}{(c+2)^2}, \\
\implies p_{1|0}(-1) &= 1, \\
p_{1|0}(0) &= \frac{1}{4}.
\end{aligned}$$

Similarly, again for $c \in [-1, 0]$:

$$\begin{aligned}
p_{0 \cap 1} &= p(\text{predict } 0 \cap \text{truly } 1), \\
&= p((x_i < c) \cap (w_i > c)), \\
&= \int_{-1}^c (1/2) - (1/2)(c - x) d\mu_x, \\
&= \frac{1}{4} \int_{-1}^c (1 - c + x) dx, \\
&= \frac{1}{4} [(1 - c)x + \frac{x^2}{2}] \Big|_{-1}^c, \\
&= \frac{1}{4} [(1 - c)(c + 1) + (c^2/2 - 1/2)], \\
&= \frac{1}{8} [1 - c^2].
\end{aligned}$$

Which leads to the expression:

$$\begin{aligned}
p_{0|1}p_1 &= p_{0\cap 1}, \\
\implies p_{0|1}(c) &= \frac{\frac{1}{8}(1-c^2)}{\frac{1}{8}(8-(c+2)^2)}, \\
&= \frac{(1-c^2)}{(8-(c+2)^2)}, \\
\implies p_{0|1}(-1) &= \frac{0}{7} = 0, \\
p_{0|1}(0) &= \frac{1}{4}.
\end{aligned}$$

The total error is:

$$\begin{aligned}
p_{err} &= p_{0\cap 1} + p_{1\cap 0}, \\
&= \frac{1}{8} - \frac{c^2}{8} + \frac{1}{8}, \\
&= \frac{1}{4} - \frac{1}{8}c^2.
\end{aligned}$$

Finally, we calculate the derivative of the curve $(p_{1|0}(c), p_{0|1}(c))$ parameterized by c :

$$\begin{aligned}
\frac{d(p_{1|0})}{dc} &= \frac{-2}{(c+2)^3}, \\
\frac{d(p_{0|1})}{dc} &= \frac{(8-(c+2)^2)(-2c) - (1-c^2)(-2)(c+2)}{(8-(c+2)^2)^2}, \\
&= \frac{4c^2 - 6c + 4}{(8-(c+2)^2)^2}, \\
\implies \frac{d(p_{0|1})}{d(p_{1|0})}(c) &= \frac{\frac{4c^2-6c+4}{(8-(c+2)^2)^2}}{\frac{-2}{(c+2)^3}}, \\
&= \frac{-(c+2)^3(2c^2-3c+2)}{(8-(c+2)^2)^2}, \\
\implies \frac{d(p_{0|1})}{d(p_{1|0})}(-1) &= -\frac{1}{7}, \\
\frac{d(p_{0|1})}{d(p_{1|0})}(0) &= -1.
\end{aligned}$$

Algorithm 11.1 Failure Predictor Training

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over policies

Input: Rollout function r , learning rate γ

Input: Training dataset $S = \{E_i\}_{i=1}^N$

Output: Optimized ψ^*

for $i = \{1, 2, \dots, N\}$ **do**

$\{(o_{i,j}, y_{i,j})\}_{j=1}^T \leftarrow r(E_i, \pi)$

end for

while not converged **do**

Sample $w \sim \mathcal{N}_\psi$

for $i = \{1, 2, \dots, N\}$ **do**

for $j = \{1, 2, \dots, T\}$ **do**

$\hat{y}_{i,j} = f_w(\{o_{i,l}\}_{l=1}^j)$

end for

end for

$B \leftarrow \frac{1}{N} \sum_{i=1}^N C_s(\{(\hat{y}_{i,j}, y_{i,j})\}_{j=1}^T)$

$+ \sqrt{\frac{D_{\text{KL}}(\mathcal{N}_\psi \| \mathcal{N}_{\psi_0}) + \ln \frac{2\sqrt{N}}{\delta}}{2N}}$

$\psi \leftarrow \psi - \gamma \nabla_\psi B$

end while

11.7.3 Additional Implementation Details

In Sec. 11.5, the failure predictors are parameterized with deep neural networks (DNNs) and trained using stochastic gradient descent. We assume access to an additional rollout function $r : \mathcal{E} \times \Pi \rightarrow \mathcal{O}^T \times \mathcal{Y}^T$ which maps a given policy and environment to a series of observations that the failure predictor will use and true labels y about whether the policy has failed or not, for $t \in \{1, \dots, T\}$. We let the prior and posterior distributions over failure predictors be multivariate Gaussian functions and write them as $\mathcal{D}_{\mathcal{F},0} = \mathcal{N}(\mu_0, \text{diag}(s_0))$, and $\mathcal{D}_{\mathcal{F}} = \mathcal{N}(\mu, \text{diag}(s))$. Define $\psi = (\mu, \log s)$ and let $\mathcal{N}_\psi := \mathcal{N}(\mu, \text{diag}(s))$. We can then sample a single set of network weights w from the distribution over weights $\mathcal{N}_\psi$ and let the associated failure predictor be f_w . The upper bound presented in (11.8) has a potentially intractable expectation taken over $f \sim \mathcal{D}_{\mathcal{F}}$. We use the same technique presented in [63] to minimize the upper bound by sampling unbiased estimates of the expectation over $f \sim \mathcal{D}_{\mathcal{F}}$. Additionally, the error $C(r_f(E, \pi))$ we present as part of the upper bound in (11.8) is discontinuous in general, and may be difficult to minimize directly. As such, we use a surrogate error function C_s in training. We let C_s take in a series of failure predictions from $f \sim \mathcal{D}_{\mathcal{F}}$ and the true labels at each point in the rollout to assign an error for that series of

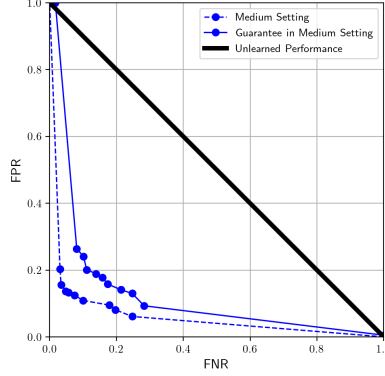


Figure 11.8: The dashed lines show the resulting failure predictor’s FNR and FPR when the importance of a false negative is varied when training in the ‘Medium’ setting in the grasping task. The solid lines and markers show the associated PAC-Bayes guarantee on the FNR and FPR using the class-condition guarantees introduction in Section 11.4.4.

predictions. We use a surrogate error similar to the cross entropy loss:

$$C_s(\{\hat{y}_j, y_j\}_{j=1}^T) := \frac{1}{T} \sum_{j=1}^T [\omega(y_{\min(j+k, T)}) \log(\hat{y}_j) + (1 - y_{\min(j+k, T)}) \log(1 - \hat{y}_j)], \quad (11.17)$$

where ω represents a weight on the importance of a false negative as compared to a false positive and k represents the number of time steps ahead that the failure predictor must detect a failure. This weight ω is analogous to λ used in (11.13) and provides a way to scale the importance of false negatives. We use this error function in training for our experiments and different values of ω and k . The resulting approach is presented in Algorithm 11.1. After training is complete, we compute the misclassification, FNR, and FPR guarantees using a sample convergence bound as in [63].

Chapter 12

Task-Relevant Failure Detection for Trajectory Predictors in Autonomous Vehicles

ALEC FARID,* SUSHANT VEER,* BORIS IVANOVIC, KAREN LEUNG, AND MARCO PAVONE

In modern autonomy stacks, prediction modules are paramount to planning motions in the presence of other mobile agents. However, failures in prediction modules can mislead the downstream planner into making unsafe decisions. Indeed, the high uncertainty inherent to the task of trajectory forecasting ensures that such mispredictions occur frequently. Motivated by the need to improve safety of autonomous vehicles without compromising on their performance, we develop a probabilistic run-time monitor that detects when a “harmful” prediction failure occurs, i.e., a *task-relevant* failure detector. We achieve this by propagating trajectory prediction errors to the planning cost to reason about their impact on the AV. Furthermore, our detector comes equipped with performance measures on the false-positive and the false-negative rate and allows for data-free calibration. In our experiments we compared our detector with various others and found that our detector has the highest area under the receiver operator characteristic curve.

12.1 Introduction

In general, autonomous vehicle (AV) stacks forecast the trajectories that non-ego agents—such as other vehicles, pedestrians, bicyclists, etc.—are likely to take. The accuracy of these predictions is crucial for the motion planner to ensure that the AV does not find itself in an unsafe state. Unfortunately, trajectory forecasting is an endeavor wrought with a high degree of uncertainty, a large portion of which, arguably, is aleatoric, arising from the stochasticity of human behavior. The situation is further exacerbated by the epistemic uncertainty that arises from unavoidable learning errors when training trajectory-prediction networks from finite driving logs. Therefore, it should come as no surprise that the trajectory-prediction networks frequently mispredict, leading to misguided motion plans. How then, in the face of such uncertainty, do we ensure safety of the AV without compromising its functionality? We approach this challenge by developing a *task-relevant* failure detector for the prediction module that only triggers if the estimate of the realized cost at a given time-instant is significantly worse than the cost predictions based on trajectory forecasting.

We require our detector to embody the following desiderata: First, the detector should be *task relevant* to reduce the number of unnecessary interventions that hurt the AV’s functionality, i.e., the detector should only trigger if the failure is harmful to the AV’s safety. To better illustrate task-relevance, consider the scenarios in Fig. 12.1. If the non-ego vehicle is predicted to stay in its lane, but it turns right, into the AV’s lane (Fig. 12.1(a)), the cost for the planner would increase and therefore, the task-relevant detector should trigger. However, if the car turns left, away from the AV’s lane (Fig. 12.1(b)), the cost for the planner would decrease, hence, the detector should not trigger. Note that in both the scenarios in Fig. 12.1, the predictions are incorrect, but they are task-relevant only in Fig. 12.1(a). Second, the detector should be an independent module, i.e., it should not modify the prediction module that it monitors. This enables easier integration of the detector with existing AV stacks and also facilitates the prediction module to specialize in its task without having to forego performance in favor of detectability of failures. Finally, the detector should be accompanied with a *metric to gauge its performance quality* to facilitate interpretable and trustworthy decision making. Most prior work satisfies only a subset of

these desiderata [123, 129, 70, 83]. In this paper we propose an approach that satisfies all these three desiderata by reasoning about the effect of erroneous predictions further downstream at the planner module of the AV stack.

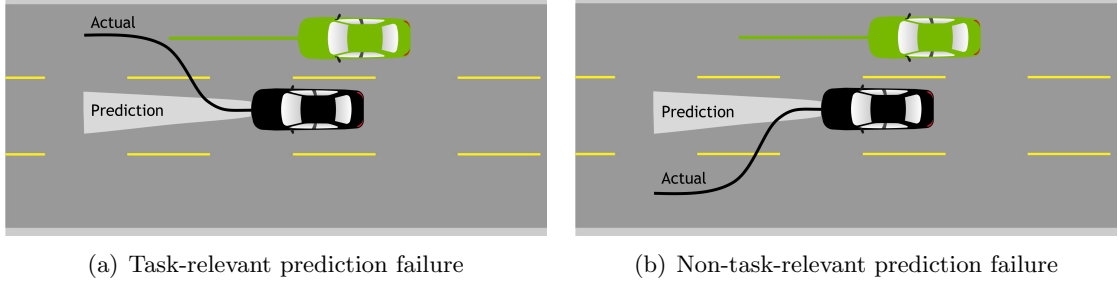


Figure 12.1: Illustration of task-relevant failure detection. The ego-vehicle and its motion plan is in green, the non-ego vehicle and the trajectory it takes is in black while the predicted trajectory distribution is in light-grey.

Statement of Contributions. The main contributions of this paper are as follows: (i) We formalize a *notion of anomaly* that underpins the design of a task-relevant prediction failure. (ii) We develop a *fast and efficient task-relevant failure detector* for the prediction module used in AV stacks which is simultaneously accompanied by *guarantees on the false-positive and false-negative detection rates* for improved interpretability. (iii) Owing to the interpretability of our approach, we can perform *data-free calibration* for our detector. Furthermore, we show in our experiments that our data-free calibrated detector’s performance is not very far from the optimally calibrated detector using the Receiver Operator Characteristic (ROC) curve. (iv) We demonstrate the efficacy of our approach on nuScenes [38] and nuPlan teaser [39] datasets and provide comparisons with other detection methods in the literature. We find that our detector has the highest Area Under the ROC (AUROC) curve.

12.2 Related Work

AV Safety. Since autonomous vehicles require interactions with other road users and are safety-critical by nature, there has been a lot of effort, from regulators, industry, and researchers alike, in ensuring safe AV operations [1, 153, 190, 13, 110, 77, 133, 230, 216] (see [56] for a review). A common approach is to compute “inevitable” collision sets (ICS), e.g., via

Hamilton-Jacobi reachability computation [146], with selected assumptions on other agents’ behaviors, and perform *shielding* where the AV will flag a situation as unsafe whenever it is close to entering the ICS and execute an appropriate evasive action (e.g., [123, 100, 101]). A primary challenge is selecting *reasonable* behavior assumptions for ICS computation to balance tractability, interpretability, and compatibility with real-world driving interactions [22, 202]. Most of these approaches assume that other agents act in an adversarial manner leading to overly conservative ICS. Instead, in this work, we build a run-time monitor that identifies only those prediction failures which are safety-critical in nature. Furthermore, the general detection approach we present could also be used as a run-time task-relevant safety monitor.

Anomaly Detection. Anomaly detection or change detection in signal processing literature has been an active area of work for decades (see [24] for a review). The goal is to identify events which deviate significantly from a reference input. Such methods have been applied in supervised learning settings to higher-dimensional inputs such as images [92, 121, 91] (see also [175] for a review); however, these methods are not task-relevant in nature. Other methods perform detection by estimating agent [27, 219] or input [191] atypicality. Recent techniques also allow for detection using the reward in reinforcement learning settings using streams of sensor inputs [83, 184, 185, 40, 222]. However, these methods do not provide guarantees on the false-positive and false-negative detection rates. Some methods that can provide guarantees come with caveats such as specific architectures [129, 194], only providing guarantees in retrospect (i.e. after multiple anomalies have occurred) [70], or on test data that is statistically similar to the training data [68]. We develop an *online* anomaly detector which provides guarantees for both false positive rate and false negative rate. Unlike many of the works described above, our detector does not require modifying the existing module on which it operates.

Planning-aware Prediction. The vast majority of behavior prediction works focus solely on the task of prediction (improving forecasting accuracy, incorporating additional sources of scene context, accelerating runtime, etc), in isolation from other components of the autonomy stack [174]. In recent years, however, there has been increasing interest in better integrating prediction with the rest of the robotic autonomy stack. Examples include

works that more tightly integrate prediction with object classification [103], tracking [218, 217], and planning [229, 128, 66, 226, 102, 50, 179]. While these approaches focus on better *architectural* integrations across the stack, a few recent works have also proposed to integrate modules *metrically*. For instance, [104, 143] present metrics that evaluate perception and prediction modules in a planning-aware fashion, e.g., by heavily weighting detection or prediction errors that could cause a collision. Our work is similar in that it can identify anomalous predictions which would detrimentally affect ego-vehicle planning, however, it does so online with guaranteed false-positive and false-negative detection rates.

12.3 Problem Definition

We express the dynamics of the AV and the scene around it as a Partially-Observable Markov Decision Process (POMDP) defined as a tuple $(\mathcal{X}, \mathcal{O}, \mathcal{U}, f)$ where $\mathcal{X}$ is the state space which is composed of the state of the ego agent x^e , non-ego agents x^{ne} , and other variables such as the environment map x^{m} , $\mathcal{O}$ is the space of observations that the ego agent receives from its sensors, $\mathcal{U}$ is the space of the control inputs for the ego agent, and $f(x_t|x_{t-1}, u_{t-1})$ is the stochastic state-transition function.

Let $c_t : \mathcal{X} \times \mathcal{U} \rightarrow [0, \infty)$ be a cost function at a time instant t . Our framework is agnostic to the choice of the cost function; we only assume that the cost function is formulated such that higher values imply worse / less safe outcomes for the AV. As an example cost function is the closest distance of an AV to other non-ego agents and obstacles in the map. Given the predicted state $\hat{x}_{t+\tau}$ and the control input $u_{t+\tau}$ at τ time steps in the future, the predicted cost is $\hat{c}_{t+\tau} := c_{t+\tau}(\hat{x}_{t+\tau}, u_{t+\tau})$.

With a slight abuse of notation, let $x_t \in \mathcal{X}$ be an *estimate* of the state (instead of the true state) at time t provided by the perception module by processing the sensor observation o_t . The prediction module takes in a history of state estimates $x_{0:t}$ and provides a distribution $\psi(\hat{x}_{t+1:t+T}^{\text{ne}}|x_{0:t})$ on the predicted future state trajectories $\hat{x}_{t+1:t+T}^{\text{ne}}$ of non-ego agents over the horizon T . The distribution produced by the prediction module induces a sequence of distributions $\{\phi_{t+\tau}(\hat{c}_{t+\tau}|x_{0:t})\}_{\tau=1}^T$ on the predicted cost $\hat{c}_{t+\tau}$ for each time-step in the prediction horizon T . The observed cost $c_{t+\tau}^* := c_{t+\tau}(x_{t+\tau}, u_{t+\tau})$ at time-step $t + \tau$ is the

cost estimated or “observed” by the AV given state-estimates of the ego and non-ego agents $x_{t+\tau}$ and the control input $u_{t+\tau}$. We define the notion of anomaly that will underpin our approach to prediction failure detection:

Definition 12.1 (p -quantile anomaly). *For any time-step $t + \tau$, let $\phi_{t+\tau}(\hat{c}_{t+\tau}|x_{0:t})$ be the distribution on the predicted cost as described above. If the observed cost $c_{t+\tau}^*$ lies in the top p -quantile of $\phi_{t+\tau}(\hat{c}_{t+\tau}|x_{0:t})$, then the prediction module suffers from a p -quantile anomaly.*

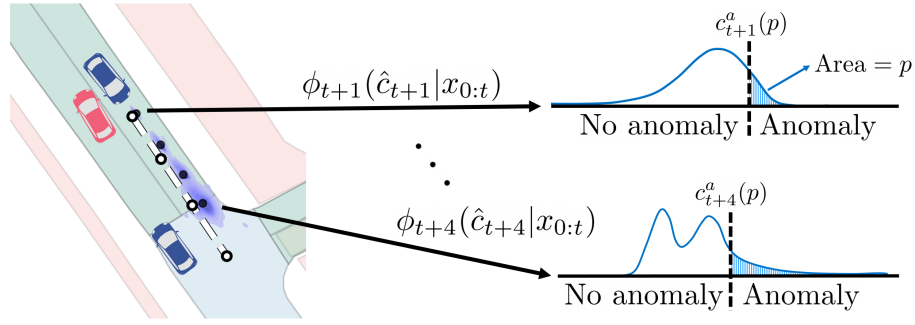


Figure 12.2: Illustration of the concept of p -quantile anomalies. The ego vehicle is red, non-ego vehicles are blue, ground truth trajectories are white, and trajectory predictions are the purple contour plots. At each time step, the cost function induces a distribution $\phi_{t+\tau}$ on predicted costs depicted by the blue plots on the right. If the observed cost $c_{t+\tau}^*$ is within the top- p quantile of $\phi_{t+\tau}$, it is a p -quantile anomaly.

The observed cost $c_{t+\tau}^*$ is, theoretically speaking, not necessarily drawn from the predicted cost distribution $\phi_{t+\tau}$ since the observed cost might depend on unmodeled/unobservable factors. However, under nominal circumstances (i.e., few or no task-relevant prediction errors and no major deviations by the AV from the open-loop motion plan), it is reasonable to expect the observed cost to be similar to the sampled predicted costs. This motivates the definition of anomaly as per Def. 12.1 and the design of a prediction failure detector based on the concept of p -quantile anomalies (illustrated in Fig. 12.2). Importantly, the notion of p -quantile anomaly, being a cost-based quantity, intrinsically captures the notion of task-relevance. Note that if the predictor predicts cost-degrading behavior correctly, the observed cost would not lie in top p -quantile as this is in alignment with the predictions.

12.4 p -Quantile Anomaly Detector

In this section, we present an algorithm for detecting p -quantile anomalies along with guaranteed false-positive and false-negative detection rates afforded by our algorithm. Let $c_{t+\tau}^a(p)$ be the cost which separates the top p proportion of $\phi_{t+\tau}(\hat{c}_{t+\tau}|x_{0:t})$ from the bottom $1 - p$. If $c_{t+\tau}^* \geq c_{t+\tau}^a(p)$, then we have a p -quantile anomaly. The lack of an analytical expression¹ for ϕ prevents us from exactly computing $c_{t+\tau}^a(p)$, hence, we use a sampling-based approach for detecting if the observed cost lies within the p -quantile of the distribution on the predicted cost; see Fig. 12.2 for an illustration of the p -quantile anomaly within the trajectory prediction framework.

Suppose at time $t + \tau$ we sample M predicted costs $S_{t+\tau} := \{\hat{c}_{t+\tau}^1, \dots, \hat{c}_{t+\tau}^M\}$ independent and identically distributed (i.i.d.) from $\phi_{t+\tau}(\hat{c}_{t+\tau}|x_{0:t})$ and arrange them in an increasing order (i.e., $\hat{c}_{t+\tau}^1 \leq \hat{c}_{t+\tau}^2 \leq \dots \leq \hat{c}_{t+\tau}^M$). Our detector D returns a positive detection if the observed cost is greater than at least $M - n$ predicted costs in S , where $n \in \{0, \dots, M - 1\}$:

$$D(n, S) := c_{t+\tau}^* \geq \hat{c}_{t+\tau}^{M-n} . \quad (12.1)$$

Intuitively, by checking where the observed cost ranks against the sampled predicted costs, our detector estimates the quantile of ϕ in which the observed cost lies. In the rest of this section we present our detection algorithm, provide bounds on its false-positive and false-negative detection rates, and use these bounds to analytically calibrate the detector by choosing appropriate M and n .

12.4.1 Detection Algorithm

The detector, Algorithm 12.1, is executed at each planning cycle. We sample M trajectories for each non-ego agent from the trajectory-prediction network ψ . Inference from prediction networks is usually a bottleneck in real-time control; however, note that for most AV stacks, we do not need to sample these trajectories specially for detection, as they would be drawn regardless for use in planning. Hence, by reusing these samples our detector does

¹Even if the distribution on the predicted trajectories ψ takes the form of a Gaussian mixture model [177], the nonlinear cost function may prevent the predicted cost distribution ϕ from acquiring an analytical form.

not generate significant computational overhead. At each planning cycle, our detector receives M predicted costs for each non-ego agent at each prediction time-step (see line 1 in Algorithm 12.1). For each subsequent time-step in the prediction horizon, we monitor D in (12.1). If the detection criteria is met, the detector returns **True** along with the identifier of the non-ego agent responsible for the detection. In Sec. 12.5.1, we show that this detector runs in less than 0.1 milliseconds (ms) which is sufficiently fast for real-time monitoring.

Algorithm 12.1 p -Quantile Anomaly Detector (QAD)

```

1: Input: For each non-ego agent, a sequence of sets  $S_{t+\tau} = \{\hat{c}_{t+\tau}^1, \dots, \hat{c}_{t+\tau}^M\}$  for each
    $\tau \in \{1, \dots, T\}$ 
2: Input: Cost functions  $c_{t+\tau}$  for each  $\tau \in \{1, \dots, T\}$ 
3: for wall-clock time  $w$  in  $t+1 : t+T$  do
4:   for each non-ego agent do
5:      $c_w^* \leftarrow c_w(x_w, u_w)$ 
6:     if  $c_w^* \geq \hat{c}_w^{M-n}$  then
7:       return (True, non-ego agent Id)
8:     end if
9:   end for
10: end for

```

12.4.2 Bounds on False-positive and False-negative Rates

Let us first define the false-positive rate (FPR) and the false-negative rate (FNR) for D .

Definition 12.2 (False-positive Rate). *FPR of the detector is defined as the probability of drawing $S_{t+\tau}$ such that a p -quantile anomaly detection occurs when the anomaly does not occur, i.e.,*

$$FPR := \mathbb{P}_{S_{t+\tau} \sim \phi_{t+\tau}^M} [D(n, S_{t+\tau}) \wedge c_{t+\tau}^* < c_{t+\tau}^a(p)] = \mathbb{P}_{S_{t+\tau} \sim \phi_{t+\tau}^M} [c_{t+\tau}^* \geq \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* < c_{t+\tau}^a(p)] . \quad (12.2)$$

Definition 12.3 (False-negative Rate). *FNR of the detector is defined as the probability of drawing $S_{t+\tau}$ such that a p -quantile anomaly detection does not occur when the anomaly does occur, i.e.,*

$$FNR := \mathbb{P}_{S_{t+\tau} \sim \phi_{t+\tau}^M} [\neg D(n, S_{t+\tau}) \wedge c_{t+\tau}^* \geq c_{t+\tau}^a(p)] = \mathbb{P}_{S_{t+\tau} \sim \phi_{t+\tau}^M} [c_{t+\tau}^* < \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* \geq c_{t+\tau}^a(p)] . \quad (12.3)$$

Ideally, for safety-critical systems, such as AVs, we desire a low FNR to ensure that we

do not miss any unsafe scenarios. This can be trivially achieved by having a detector that triggers all the time (high FPR), but clearly that is not desirable for the AV’s performance. Hence, a good detector must also exhibit a low FPR. Motivated by this, we provide bounds on the FPR and FNR for our detector.

Theorem 12.1 (FPR and FNR Bound). *Let the detector D for the occurrence of a p -quantile anomaly be defined as in (12.1). Then, the FPR (Def. 12.2) and the FNR (Def. 12.3) for the detector satisfies:*

$$FPR \leq \sum_{i=0}^n \binom{M}{i} p^i (1-p)^{M-i} =: \overline{FPR} , \quad FNR \leq \sum_{i=n+1}^M \binom{M}{i} p^i (1-p)^{M-i} =: \overline{FNR} .$$

The proof of this theorem is presented in App. 12.7.1. Thm. 12.1 ensures that if the detector identifies an anomaly (i.e., D holds true), then with probability at least $1 - \overline{FPR}$, the observed cost does lie in the top- p quantile of $\phi_{t+\tau}$, while if our detector does not detect an anomaly (i.e., D is not true), then with probability at least $1 - \overline{FNR}$, the observed cost does not lie in the top- p quantile of $\phi_{t+\tau}$.

Our detector satisfies all the three desiderata laid out in Sec. 12.1. First, our detector encodes the notion of *task-relevant failure detection* since it only triggers when the observed cost is high compared to the predicted costs, in the sense of the p -quantile anomaly (Def. 12.1). Second, the detector is *independent of the trajectory prediction module* that it monitors. Finally, we provide bounds on FPR and FNR of our detector in Thm. 12.1 that serve as a metric on the detector’s performance. These bounds facilitate greater interpretability and promote easier detector calibration.

12.4.3 Data-free Detector Calibration

The analytical upper bounds for the FPR and the FNR in Thm. 12.1 allows data-free calibration of D by letting us choose M and n so as allow a user to specify FPR and FNR values. However, we can only provide a strong bound on either the FPR or the FNR, but not both. This follows from the observation that the upper bounds in Thm. 12.1 satisfy $\overline{FPR} + \overline{FNR} = 1$. It is worth clarifying that this is *not* a fundamental limitation of our detector, but that of our analysis. We show in Sec. 12.5 that our detector, in fact, achieves

low FPR and low FNR simultaneously.

12.5 Experimental Results

In this section, we benchmark the performance of our detector on nuScenes [38] and nuPlan teaser [39] datasets against various other methods. We study the detector in two applications: (i) run-time monitoring of task-relevant prediction network failures and (ii) filtering interesting scenarios from unlabelled expert driving logs. Through these experiments we demonstrate the ability of our detector to provide low FPR and low FNR for detecting task-relevant mispredictions. All experiments were conducted on a desktop computer with an Intel i9-10980XE CPU (18 cores) and 64 GB of RAM. Our code is available at: <https://github.com/NVlabs/pred-fail-detector>.

Trajectory prediction module. While our method is agnostic to the particular prediction method used, in our experiments we employ Trajectron++ [177], a state-of-the-art multi-agent trajectory forecasting model, to predict the motion of non-ego agents. It is a graph-structured recurrent neural network that predicts an agent’s future position distribution given its past trajectory history and the past trajectories of its neighboring agents. At its core, Trajectron++ uses a conditional variational autoencoder (CVAE) to model the potential for multiple future trajectories.

Detection and comparison methods. We compare our detector— p -quantile anomaly detection (QAD)—against four other approaches: (i) *likelihood detection* baseline which triggers when the likelihood of the achieved states of an agent in the predicted GMM is below a threshold; (ii) *uniform and partial cost degradation* tests [83] which uses p -values to detect shifts in the cost distribution; (iii) *time-to-collision (TTC)* detector which triggers when the TTC drops below a threshold; and (iv) detection based on *Hamilton-Jacobi (HJ) reachability* analysis [123] which triggers if a collision is possible assuming an adversarial behavior by the non-ego agent. See Sec. 12.7.2 for more information about the comparison methods.

12.5.1 Task-relevant Detection

Overview. We study the use of our detector and other baseline detectors mentioned above as run-time monitors for task-relevant prediction failure. The experiments are conducted in the recently released nuPlan teaser dataset [39] which allows the inclusion of reactive agents in the driving logs.

Planner. We assume a behavior generator provides us with a reference trajectory to follow. Our planner is then tasked with following this trajectory while avoiding collision with other agents on the road, ensuring ego comfort and satisfying the ego’s dynamical constraints. The planner achieves this by generating a motion primitive tree [180] and choosing the trajectory that minimizes the weighted sum of a series of cost functions; further details on the planner are supplied in App. 12.7.2. We use time-to-collision cost c_{ttc} (computed assuming that agents continue moving at a constant speed in their current heading), momentum-shaped distance to other agents cost c_{d2a} , distance to goal cost c_{d2g} , distance to the reference trajectory cost c_{d2r} , speed limit cost $c_{velocity}$, ego comfort cost $c_{comfort}$, and reversing cost $c_{reverse}$ which penalizes the car for going against the heading direction of the lane:

$$c_{t+\tau} = w_1 c_{ttc} + w_2 c_{d2a} + w_3 c_{d2g} + w_4 c_{d2r} + w_5 c_{velocity} + w_6 c_{comfort} + w_7 c_{reverse} . \quad (12.4)$$

The exact expressions for the cost terms and the weights $w_1, \dots, w_7$ used in the experiments are provided in App. 12.7.2. The motion planner uses trajectory forecasts of other non-ego agents drawn from Trajectron++ to compute the momentum-shaped distance and the time-to-collision cost. The cost functions take about 1 ± 0.5 ms to evaluate. Note that these cost evaluations for planning are reused by our anomaly detector in Algorithm 12.1, thereby minimizing any excessive computational overhead by our detector. The anomaly detection computation takes less than 0.1 ms.

Dataset and Labeling. We test on 260 nuPlan [39] driving scenarios which comprises of a total of 2568 prediction horizons. We hand-label each prediction horizon for task-relevant prediction errors by visually inspecting plots of true trajectories of agents along with their predicted trajectories. We emphasize that the labeling is done merely by

visual inspection and does not use the planning costs or prediction likelihoods. This is to ensure that the labeling procedure is independent of the detection methods to promote fair comparisons; see App. 12.7.2 for more details on the labeling procedure.

Remark 12.1 (Imbalance in Dataset). *Despite the use of reactive agents along with our custom planner in nuPlan, there is a strong bias in the dataset towards benign scenarios. Only 3.5% of this dataset is labeled as positive. Due to the imbalance, there is a bias toward detectors to exhibit lower FPR and higher FNR, since false-negative errors are amplified while the false-positive errors are mitigated.*

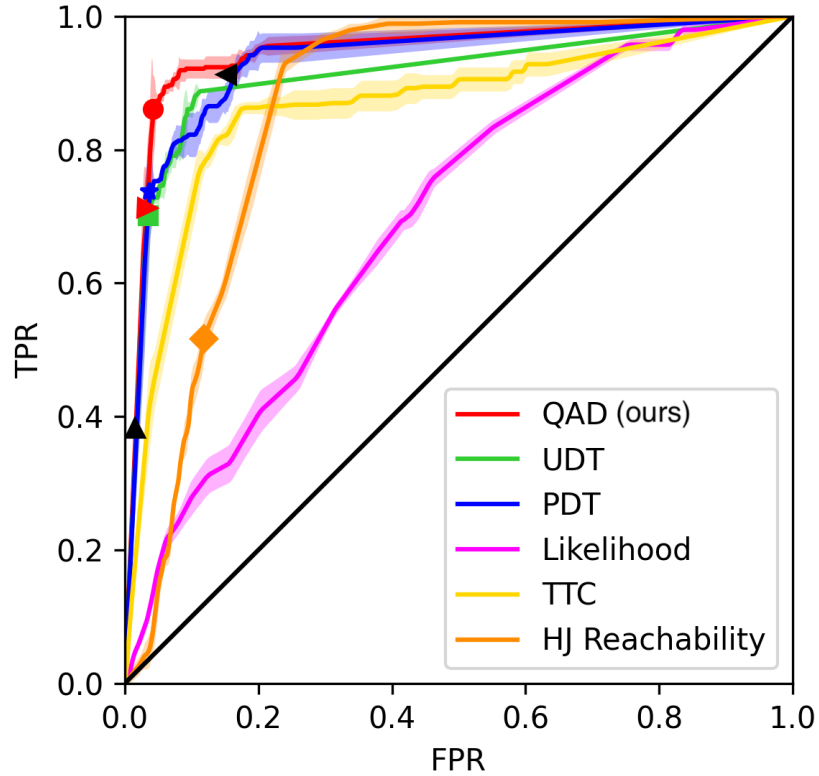


Figure 12.3: ROC curve for detectors. We report the mean of 5 trials, the shaded region is 2 times the standard deviation.

Results. We plot the Receiver Operator Characteristic (ROC) curve in Fig. 12.3 for each detector by varying: the detection rank $M - n$ in (12.1) for our detector, the p -value for UDT and PDT, the likelihood threshold for the likelihood-based detector, the TTC threshold for TTC detector, and the sub-level set of the value function for the HJ reachability based detector. The ROC curve for our detector is the closest to an ideal

Table 12.1: AUROC and best point from ROC ($\uparrow$ AUROC)

Detection Method	AUROC	FPR %	FNR %
QAD (ours)	0.946$\pm$0.002	8.4$\pm$0.4	7.9 $\pm$ 0.7
PDT [83]	0.932 $\pm$ 0.005	18.5 $\pm$ 0.2	5.8$\pm$1.1
UDT [83]	0.912 $\pm$ 0.002	10.6 $\pm$ 0.3	11.5 $\pm$ 0.4
TTC	0.864 $\pm$ 0.004	17.3 $\pm$ 0.5	13.7 $\pm$ 0.4
HJ Reachability [123]	0.862 $\pm$ 0.003	23.7 $\pm$ 0.3	7.2 $\pm$ 0.6
Likelihood	0.693 $\pm$ 0.006	45.9 $\pm$ 0.4	24.5 $\pm$ 0.4

Table 12.2: Data-free calibration ($\uparrow$ distance from $y = x$)

Detection Method		FPR %	FNR %	Dist.
QAD (FNR) (ours)	●	4.2 $\pm$ 0.3	13.9 $\pm$ 1.1	0.58$\pm$0.01
FNR Ensemble (ours)	◀	15.2 $\pm$ 0.3	8.7$\pm$1.2	0.54 $\pm$ 0.01
PDT (FPR) [83]	★	3.7 $\pm$ 0.3	26.3 $\pm$ 0.6	0.50 $\pm$ 0.01
QAD (FPR) (ours)	▶	3.4 $\pm$ 0.3	28.8 $\pm$ 0.9	0.48 $\pm$ 0.01
UDT (FPR) [83]	■	3.5 $\pm$ 0.3	29.9 $\pm$ 0.9	0.47 $\pm$ 0.01
HJ Reachability [123]	◆	11.8 $\pm$ 0.2	48.6 $\pm$ 1.7	0.28 $\pm$ 0.01
FPR Ensemble (ours)	▲	1.5$\pm$0.3	61.6 $\pm$ 1.8	0.26 $\pm$ 0.01

detector and has the highest Area Under the ROC (AUROC) curve. We obtain the best calibration for each detector by choosing the point that is farthest from the $y = x$ line in Fig. 12.3 and report the empirical FPR and FNR for them in Table 12.1. Our detector has the lowest FPR and the third-lowest FNR. PDT has a slightly better FNR than our method, but the FPR is much higher, while for all other methods both FPR and FNR are higher than our method. As one would expect, the naive likelihood baseline does not perform very well while the HJ reachability detector is overly conservative exhibiting a lot of false-positives.

We also compare the performance of our detector with data-free calibration using Thm. 12.1 with the data-free calibration of UDT, PDT, and HJ reachability. For our detector, we consider an anomaly to have occurred when the true cost lies in the top-5% of the predicted cost distribution (see Def. 12.1); we chose the 5% quantile because it is a commonly used threshold for detection in statistics. We include two versions of our detector in this study: (i) calibrated to bound the FPR below 5% and (ii) calibrated to bound the FNR below 5% using Thm. 12.1; as discussed in Sec. 12.4.3, we cannot simultaneously obtain a strong bound on both FPR and FNR. We use a p -value of 0.05 for UDT and PDT,

and a 0-sublevel set of the value function for HJ reachability. Additionally, we construct an FPR ensemble detector, which only detects positive if *all* of QAD (FPR), UDT, PDT, and HJ reachability are positive, and an FNR ensemble detector, which detects positive if *any* of QAD (FNR), UDT, PDT, and HJ reachability are positive. For the ensemble detectors, we use the data-free calibration for each method described above.

These detectors are labeled using markers in the ROC curves in Fig. 12.3. Their empirical FPR and FNR is reported in Table 12.2 and we sort the table by distance from the line $y = x$. We observe that our FPR bounded detector satisfies the 5% bound provided by Thm. 12.1; however, our FNR bounded detector violates the bound. This disagreement between the theory and the empirical performance for FNR occurs because of the mismatch between our anomaly criteria of top-5% quantile and what the human data labeller considers to be an anomaly. A higher empirical FNR suggests that the anomalous quantile should be expanded beyond 5%; case in point, the optimal p -quantile anomaly detector obtained using the ROC curve in Table 12.1 with 7% FNR corresponds to approximately the top-35% quantile as an anomaly. Furthermore, the imbalance in the dataset also contributes to this problem, as discussed in Rem. 12.1. More data as well as a more balanced dataset can perhaps remedy this mismatch and lower the FNR; we will explore this in future work. Despite this mismatch, we see in Table 12.2 that our FNR detector performs the best out of all detectors in terms of distance to the $y = x$ line in the ROC plot. The FNR ensemble and PDT also perform well, favoring false-negatives and false-positive respectively. Since false-negatives can be safety critical, a designer may choose our FNR ensemble detector at the cost of more false-positives. Ultimately, our FNR detector is not very far off from the optimal detector obtained from the ROC curve, i.e., with *no data at all* we are able to calibrate our detector using Thm. 12.1 to perform very well.

12.5.2 Filtering expert driving logs

Overview. Given driving data logs, we consider the task of filtering scenarios with anomalies that may be of interest for robustifying the prediction network or for diagnostic evaluations. We use the nuScenes and nuPlan datasets as the driving logs. For the likelihood detector we choose the threshold 0.05, and for TTC we choose 1 second, while for all other

detectors, we use the data-free calibration specified in Sec. 12.5.1.

Dataset. The dataset consists of 375 nuPlan and 152 nuScenes scenarios with a total of 3890 prediction horizons, in all of which the ego-vehicle is driven by a human driver and there are no collisions.

Cost functions. Since the nuScenes and nuPlan driving logs are collected by a human driver, we do not have access to the true planning “cost” used by the driver. Instead, we use a weighted sum of cost terms contributed by the momentum-shaped distance and the time-to-collision for each non-ego agent as a proxy cost: $c_{t+\tau} = w_1 c_{\text{ttc}} + w_2 c_{\text{d2a}}$. The exact expressions of c_{ttc} and c_{d2a} are provided in App. 12.7.2. Our results suggest that this cost function is well-aligned with human intuition as it allows us to filter out interesting scenarios while ignoring the uninteresting ones.

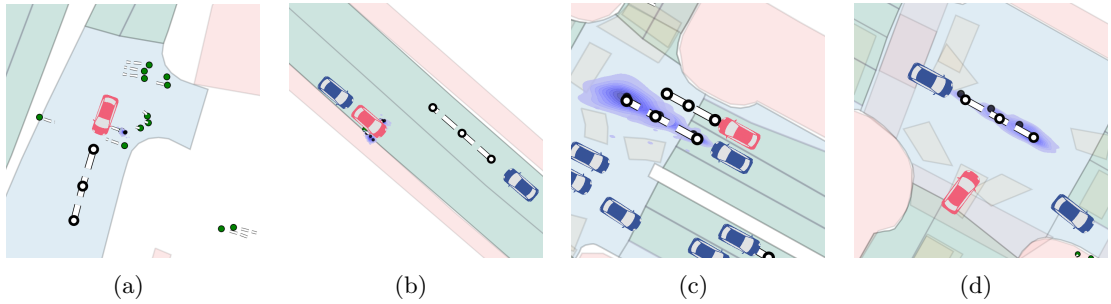


Figure 12.4: Selected scenarios from nuScenes. The ego vehicle is red, non-ego vehicles are blue, pedestrians are green circles, ground truth trajectories are white, and Trajectron++ predictions are the purple contour plots.

Table 12.3: Detections from expert driving logs

Detection Method	Percentage Detected
UDT [83]	5.4 ± 0.1
QAD (FPR) (ours)	5.4 ± 0.1
PDT [83]	5.8 ± 0.1
QAD (FNR) (ours)	6.4 ± 0.1
HJ Reachability [123]	8.8 ± 0.0
TTC	17.4 ± 0.0
Likelihood	58.5 ± 0.0

Results. We compare our detector to all baseline methods described earlier. See Table 12.3 for a comparison of the methods and Fig. 12.4 for representative examples of positive detections. We observe that the likelihood detection and the TTC-based detection both have many more positives than the other methods, followed by HJ reachability. Other

methods have a similar number of positives. Ultimately, no driving scenario from this dataset is unsafe, so we inspect the positive detections from each method to find interesting scenarios. All methods detect a handful of interesting scenarios, such as Fig 12.4(b) where pedestrians are right beside the ego (possibly entering or exiting). We also see additional interesting scenarios detected by our method but not HJ reachability such as in Fig. 12.4(a) where a close-by pedestrian is predicted to stand still, but moves quickly towards the ego at a crosswalk. Qualitatively, it seems that our detector, UDT, and PDT all do a good job of identifying interesting scenarios such as those shown in Fig. 12.4(a) and (b). We see that HJ reachability, and likelihood detection methods more frequently labels driving scenarios where the predictions are accurate and there is nothing inherently unsafe as anomalous such as in Fig. 12.4(c) and (d).

12.6 Limitations, Future Work, and Conclusion

Limitations. As discussed in Sec. 12.4.3, we cannot provide a strong FPR and FNR guarantee simultaneously for our detector. This does not seem to be a fundamental limitation of the detector itself because our empirical results in Table 12.1 suggest that we can achieve low FPR and low FNR. As part of our future work, we intend to explore providing stronger guarantees on both FPR and FNR simultaneously. A second limitation of our approach is identifying what quantile of the cost distribution corresponds to an actual anomaly. As we saw in Sec. 12.5.1, the mismatch in the quantile can result in a disagreement with the theoretical bounds. From a practical perspective, this is not a major limitation as we demonstrated that our detector performs well despite making a completely uninformed quantile choice for anomaly detection. Furthermore, we can mitigate this problem by using a small dataset to identify an appropriate p -quantile to be considered an anomaly.

Future Work. As part of our future work, we hope to use this detector for adapting planning frequencies. If the planner re-plans only when necessary, it would free up some of the compute budget for other modules in the AV stack and lower the energy consumption. Since our detector directly monitors the cost of the planner, it would serve as a good indicator of when the motion plan is not evolving as expected and therefore, requires re-planning.

As a proof of concept, we present some motivating results in App. 12.7.2. An additional direction for future work is to require multiple p -quantile anomalies before declaring a detection instead of a single one used in this work. Another exciting direction for this work is providing causal explanations for the detections; e.g., we anticipated the agent to continue straight, but it is switching into our lane. Such explanations can actively be leveraged to plan reactions to prediction failures.

Conclusion. We presented a task-relevant detector for mispredictions by the trajectory forecasting networks used in AVs. Our detector builds a distribution of the predicted future costs and identifies an anomaly when the true cost of the planner lies in the top- p quantile of that distribution. We provided FPR and FNR guarantees for our detector that facilitate data-free calibration. In experiments, we demonstrated that our detector out-performs all other baseline approaches we compared against and is compute-efficient. Additionally, we showed that our data-free calibrated detector’s performance is close to the optimal detector chosen from the ROC curve. We used our detector for run-time misprediction monitoring and for filtering prediction failures from unlabeled driving logs.

12.7 Appendix

12.7.1 Proof of Theorem 12.1

Proof. We will first establish the bound on the FPR. Note that

$$c_{t+\tau}^* \geq \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* < c_{t+\tau}^a(p) \implies \hat{c}_{t+\tau}^{M-n} < c_{t+\tau}^a(p) . \quad (12.5)$$

Hence,

$$\mathbb{P}_{S \sim \phi^M} [c_{t+\tau}^* \geq \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* < c_{t+\tau}^a(p)] \leq \mathbb{P}_{S \sim \phi^M} [\hat{c}_{t+\tau}^{M-n} < c_{t+\tau}^a(p)] . \quad (12.6)$$

Since $\hat{c}$ ’s are ordered by their value in S , the upper bound in (12.6) is essentially the probability of sampling $S \sim \phi_{t+\tau}(\hat{c}|o_t)^M$ such that at most n elements of S lie above $c_{t+\tau}^a(p)$, i.e., lie in the p -quantile tail of ϕ . Whether a sample of ϕ lies in the top p -quantile tail or

the remainder of the $1 - p$ portion of the distribution is Bernoulli distributed. Therefore,

$$\mathbb{P}_{S \sim \phi^M}[\hat{c}_{t+\tau}^{M-n} < c_{t+\tau}^a(p)] = \sum_{i=0}^n \binom{M}{i} p^i (1-p)^{M-i} . \quad (12.7)$$

Using (12.7) in (12.6) followed by (12.2) gives the upper bound on FPR.

Analogously, we can bound the FNR. We have the following:

$$c_{t+\tau}^* < \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* \geq c_{t+\tau}^a(p) \implies \hat{c}_{t+\tau}^{M-n} > c_{t+\tau}^a(p) \implies \hat{c}_{t+\tau}^{M-n} \geq c_{t+\tau}^a(p) . \quad (12.8)$$

Hence,

$$\mathbb{P}_{S \sim \phi^M}[c_{t+\tau}^* < \hat{c}_{t+\tau}^{M-n} \wedge c_{t+\tau}^* \geq c_{t+\tau}^a(p)] \leq \mathbb{P}_{S \sim \phi^M}[\hat{c}_{t+\tau}^{M-n} \geq c_{t+\tau}^a(p)] . \quad (12.9)$$

As before, we use the fact that drawing samples from ϕ such that they lie in the p or the $1 - p$ portions of the distribution is Bernoulli distributed to write

$$\mathbb{P}_{S \sim \phi^M}[\hat{c}_{t+\tau}^{M-n} \geq c_{t+\tau}^a(p)] = \sum_{i=n+1}^M \binom{M}{i} p^i (1-p)^{M-i} . \quad (12.10)$$

Using (12.10) in (12.9) followed by (12.3) gives the upper bound on FNR, completing the proof. $\square$

12.7.2 Additional Experimental Details

Comparison methods

We compare our detector against four other approaches: (i) likelihood detection, (ii) uniform and partial degradation tests (UDT and PDT) [83], (iii) a time-to-collision (TTC) based detector, and (iv) detection based on Hamilton-Jacobi (HJ) reachability analysis [123].

Likelihood detection. We label the likelihood of the estimated state $x_{t+\tau}$ as the value in the PDF of the distribution over the predicted non-ego agent states $\psi(\hat{x}_{t+1:t+T}^{\text{ne}} | x_{0:t})$ provided by the prediction module. As such, we directly represent the likelihood of the estimated non-ego positions given the predicted positions. For anomaly detection, we only check agents within 10 m of the ego since this is the region in which trajectory prediction

is most accurate, and the non-ego agents are most relevant.

UDT and PDT. These degradation tests use the predicted costs for each agent as a set of reference signals. We use the same cost function as our method. UDT compares a weighted mean of the reference signals to the online signal in order to determine when reward degradation is significant and is optimal when the reference signals come from a multivariate normal distribution. PDT detects deterioration using a subset of the time steps of the reference signal and tends to perform better in practice when the normality assumption does not hold, see [83] for more information on these tests. After predicted costs are computed, we use the false alarm rate control method provided in [83] to choose a threshold for detection based on the desired p -value and the reference (i.e. predicted) costs. If the achieved cost is above the threshold, it is labeled as an anomaly.

TTC detection. For TTC calculations, we assume vehicles are a 1m radius circle and pedestrians are a 0.2m radius circle. We propagate the agents forward in time to determine if there would be a collision at some point in the future if the agents maintained their velocity along the current heading.

HJ reachability. We follow the method for HJ reachability analysis from [123]. We assume the ego and other vehicles follow a four-state bicycle model with the following dynamics

$$[\dot{x}, \dot{y}, \dot{\theta}, \dot{v}] = [v \cos(\theta), v \sin(\theta), \tan(d)/L, a] \quad (12.11)$$

Where d is the control steering angle, a is the control acceleration, and L is the length of the vehicle. We assume that the vehicles' acceleration lies within $[-2, 1] \text{ ms}^{-2}$ and the steering angle lies within $[-10, 10]$ degrees. Pedestrians are modeled as agents which can move in any direction with acceleration in $[-0.5, 0.5] \text{ ms}^{-2}$. We assume a 1m radius circle for vehicles and a 0.2m radius circle for pedestrians and set the initial value function such that any collision should result in a negative value. We compute the value function offline to improve operation speed. Online, we check the value for each agent with the ego using a lookup table. If the lowest value is below a threshold, we label the scene as anomalous.

Planning

We follow a similar procedure as in [180] to make a motion plan for the ego. First, we discretize the control space so that we can generate a tree of motion primitive trajectories by integrating along all possible combinations of discrete control actions at each time step over the planning horizon. In particular, we use $[-2, 0, 1] \text{ ms}^{-2}$ for the acceleration control and $[-0.3, -0.1, 0, 0.1, 0.3] \text{ rad s}^{-1}$ for the radial velocity control as the discretization. To allow for time to run cost computations, we set the first control action of each primitive to be the most recent control action as in [180]. As such, there are $15^{(T-1)}$ possible motion primitives where T is the planning horizon. We compute the predicted cost for each motion primitive using predictions of all other agents in the scene and select the primitive which has the lowest predicted cost over the planning horizon. We parallelize the predicted cost computations to improve run-time. A cost computation for a single primitive takes about $1 \pm 0.5\text{ms}$ and for a planning horizon of 2 seconds (4 control inputs), there are 3375 motion primitives (15^3) which need cost computations. On the 18 core (36 thread) computer we use for simulations, with parallelization, the best motion primitive can be reliably computed in less than 0.25s, which allows for real-time planning.

Cost functions

Let $x_e, v_e, a_e, j_e, \theta_e, \dot{\theta}_e, \ddot{\theta}_e$ be the position, velocity, acceleration, jerk, heading, rotational velocity, and rotational acceleration of the ego, respectively. Similarly, $x_a, v_a, a_a, j_a, \theta_a, \dot{\theta}_a, \ddot{\theta}_a$ represent the analogous states for the non-ego agents. Note that when we compute predictions of the cost, we use the predicted non-ego agent states instead of the achieved agent states in the cost computation; otherwise, the cost computations are identical for the predicted and the observed costs. We add subscript $\parallel$ or $\perp$ to denote decomposition parallel to or perpendicular to the *ego's heading* (e.g. ego's longitudinal velocity $v_{e,\parallel}$ and agent's lateral acceleration with respect to the ego's heading $a_{a,\perp}$). We define x_{goal} as the position of the end of the reference trajectory (i.e., the goal), and x_r, θ_r are points and headings on the reference trajectory. Let ϵ 's represent scaling factors and let ttc represent the time-to-collision function; ttc outputs the time until a collision between the ego and another agent

(assuming constant velocity at their current heading direction) and modeling vehicles as a 1 m radius circle and pedestrians as a 0.2 m radius circle. We now express the cost functions we use as follows:

$$c_{\text{ttc}} = 1 - \max_{\text{non-ego agent}} \min \left(\frac{\text{ttc}(x_e, x_a, v_e, v_a)}{\epsilon_{\text{ttc}}}, 1 \right) , \quad (12.12)$$

$$c_{\text{d2a}} = \max_{\text{non-ego agent}} \left[e^{-0.5\epsilon_{\text{rbf}}((x_{a,\parallel} - x_{e,\parallel})^2(v_{a,\parallel} - v_{e,\parallel})^2 + (x_{a,\perp} - x_{e,\perp})^2(v_{a,\perp} - v_{e,\perp})^2)} \right] , \quad (12.13)$$

$$c_{\text{d2g}} = \|x_{\text{goal}} - x_e\| / \|x_{\text{goal}} - x_{0,e}\| \text{ where } x_{0,e} = x_e \text{ at } t = 0 , \quad (12.14)$$

$$c_{\text{d2r}} = \frac{1}{4} \|x_r^* - x_e\|^4 + \frac{1}{2} (\theta_r^* - \theta_e)^2 \text{ where } x_r^*, \theta_r^* = \underset{(x_r, \theta_r) \in \text{reference trajectory}}{\text{argmin}} \|x_r - x_e\| \quad (12.15)$$

$$c_{\text{velocity}} = \max(\|v_e - \epsilon_{v,r}\| - \epsilon_r, 0)^2 / \epsilon_{\text{limit}}^2 , \quad (12.16)$$

$$c_{\text{comfort}} = \frac{1}{6} \sum \max \left(\left[\frac{|a_{e,\parallel}|}{\epsilon_{a,\parallel}} - 1, \frac{|a_{e,\perp}|}{\epsilon_{a,\perp}} - 1, \frac{|j_{e,\parallel}|}{\epsilon_{j,\parallel}} - 1, \frac{\|j_e\|}{\epsilon_j} - 1, \frac{|\dot{\theta}_e|}{\epsilon_{\dot{\theta}}} - 1, \frac{|\ddot{\theta}_e|}{\epsilon_{\ddot{\theta}}} - 1 \right], 0 \right) , \quad (12.17)$$

$$c_{\text{reverse}} = 1[v_{e,\parallel} < 0] . \quad (12.18)$$

In our experiments, we use $\epsilon_{\text{ttc}} = 3$, $\epsilon_{\text{rbf}} = 0.5$ when the agent is a vehicle, and $\epsilon_{\text{rbf}} = 1$ when the agent is a pedestrian. The reference velocity $\epsilon_{v,r} = 0.8\|x_{\text{goal}} - x_{0,e}\| / (\text{time of reference trajectory})$, and $\epsilon_r = \max(0.1\epsilon_{v,r}, 1)$, $\epsilon_{\text{limit}} = \max(30 - \epsilon_{v,r}, 10)$. The comfort tuning parameters are based on the built-in comfort metric from nuPlan [39], $(\epsilon_{a,\parallel}, \epsilon_{a,\perp}, \epsilon_{j,\parallel}, \epsilon_j, \epsilon_{\dot{\theta}}, \epsilon_{\ddot{\theta}}) = (2.4, 4.89, 4.13, 8.37, 0.95, 1.93)$ and the max in (12.17) refers to element-wise maximum. Along with the tuning parameters, the weights are set as $[w_1, \dots, w_7] = [1, 10, 1, 1, 1, 0.5, 10]$.

Hand labeling scenarios

See Fig. 12.5 for examples of positive and negative hand labels. Fig. 12.5(a) is an example of a negative label since agents are not close to the ego and predictions are relatively accurate. Fig. 12.5(b) is an example of a positive label. The circled agents are predicted to slow down or stop before reaching the ego. Instead, these agents get very close to the ego. Lastly, Fig. 12.5(c) is an example of a negative label with task-irrelevant prediction errors. Non-ego

agents which are circled are predicted to turn into the ego’s lane, but they do not.

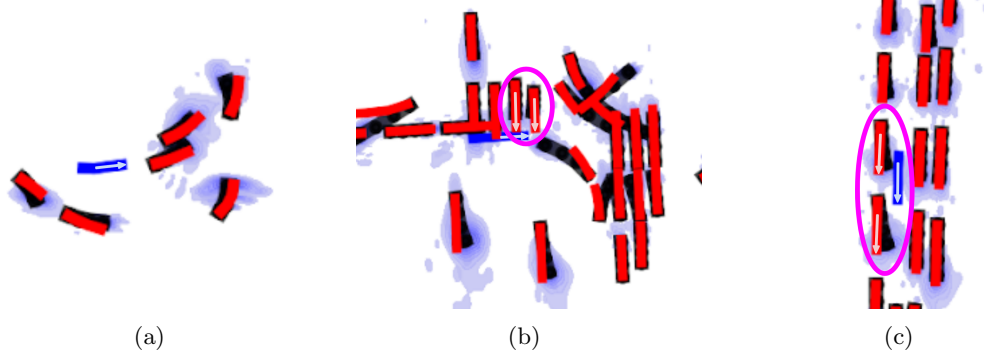


Figure 12.5: Illustration of hand labeling through selected scenarios from nuPlan. The ego vehicle trajectory is in blue, non-ego vehicle trajectories are in red, arrows indicate direction of travel, Trajectron++ predictions are the purple contour, and the black line is the most likely trajectory given the predictions. Important agents are circled. (a) Example of a negative label. (b) Example of a positive label. (c) Example of a negative label with task-irrelevant prediction errors.

Additional experimental results: Representative success and failure cases

See Fig. 12.6 for representative examples of true negatives, true positives, false negatives, and false positives. The images provide qualitative information about where the failure predictor succeeds and fails. The true negatives in Fig. 12.6(a) show standard driving scenarios and the ability of our detector to ignore task-irrelevant prediction errors. The true positives in Fig. 12.6(b) are scenarios where other agents are predicted to maintain their speed but speed up or slow down more quickly. The false negatives in Fig. 12.6(c) are cases where predictions have are incorrect but have larger variance. The images are labeled as positive by the hand-labeling, but the higher variance is enough to prevent a p -quantile anomaly detection from occurring. The false positives in Fig. 12.6(d) are cases which look like normal driving scenarios, but anomalies are detected due to the very close agents and small variance in predictions.

Additional experimental results: Adaptive re-planning rate

In order to assess if our detector can feasibly be used for an adaptive re-planning rate, we set up another experiment. Initially, the ego is given a long horizon plan (10 seconds,

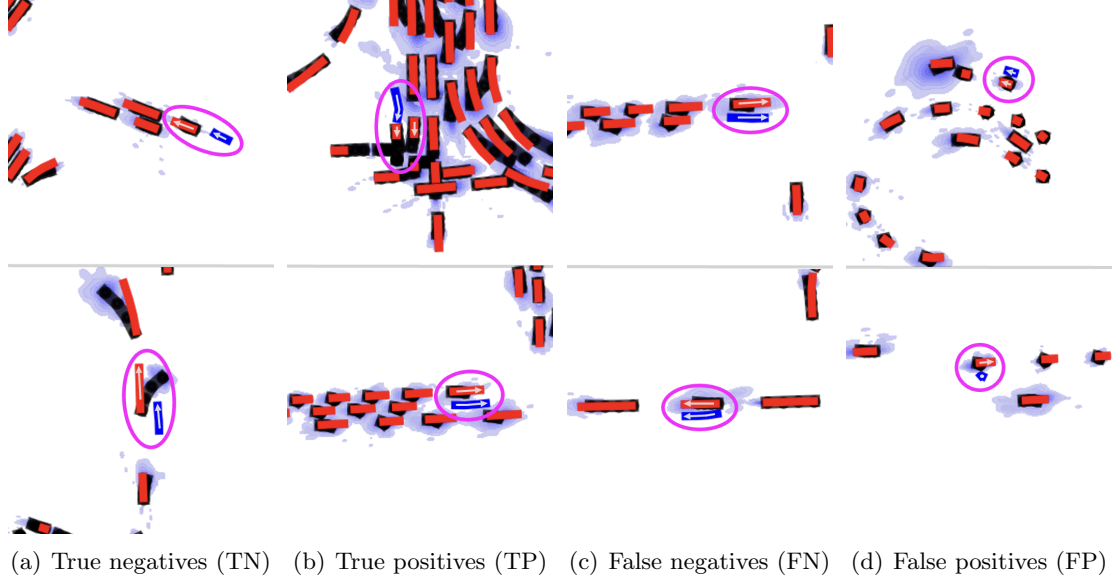


Figure 12.6: Illustration of representative (a) true negatives (TN), (b) true positives (TP), (c) false negatives (FN), and (d) false positives (FP) from the nuPlan dataset. The ego vehicle trajectory is in blue, non-ego vehicle trajectories are in red, arrows indicate direction of travel, Trajectron++ predictions are the purple contour, and the black line is the most likely trajectory given the predictions. Important agents are circled.

20 time steps) from the reference trajectory. We run our quantile anomaly detector with quantile $p = 0.25$ and re-plan once an anomaly is detected. The result is a detector which determines when the predictions are incorrect enough such that re-planning may be required. The dataset is made of 651 scenarios which come from 4 classes of scenario types: (i) ego at pick-up/drop-off (PUDO), (ii) ego following vehicle, (iii) ego stopping at traffic light, and (iv) nearby dense vehicle traffic. See Fig. 12.7 for a cumulative distribution function (CDF) of detections for each scenario type. We also present the mean detection time for each of the scene types in Table 12.4. We see that the ego following vehicle scene type has the lowest time between re-plans while the ego stopping at traffic light and ego at pick up/drop off both scene types have, on average, longer time between re-planning. The nearby dense traffic scenarios tend to have many more vehicles and the ego following vehicle has many more lane-changes and a large diversity of agent speeds. This results in a larger number of task-relevant prediction failures and we therefore see faster re-planning rate for these scenarios. In contrast, we observe that the ego at PUDO and the ego stopping at traffic light scenarios have many fewer non-ego agents around as compared with the other scenario

types and therefore have fewer task-relevant prediction failures. The long average time duration between re-plans in Table 12.4 suggest the viability of our approach as an effective way to adapt the re-planning rate.

Table 12.4: Average time between re-plans

Scene type:	Ego at PUDO	Following vehicle	Stopping at light	Nearby traffic
Mean re-plan time	5.18 s	3.55 s	5.23 s	4.49 s

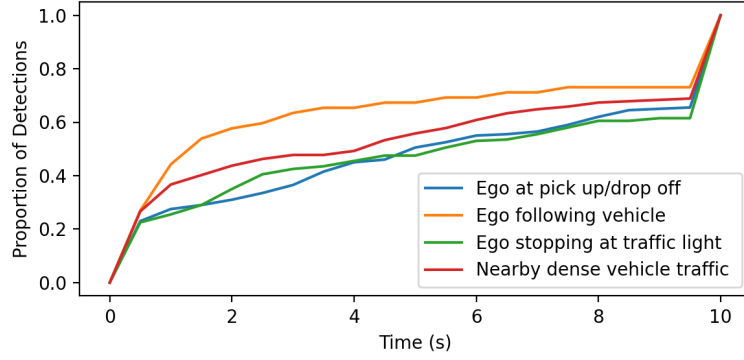


Figure 12.7: Cumulative distribution function for the length of time between re-planning for various types of scenarios. For ego following vehicle scenes (orange line), there is more frequent re-planning. For the ego stopping at traffic light (green line) and ego at pick up/drop off (blue line) scene types, re-planning is less frequent.

Chapter 13

Generalization Bounds for Meta-Learning via PAC-Bayes and Uniform Stability

ALEC FARID AND ANIRUDHA MAJUMDAR

We are motivated by the problem of providing strong generalization guarantees in the context of meta-learning. Existing generalization bounds are either challenging to evaluate or provide vacuous guarantees in even relatively simple settings. We derive a probably approximately correct (PAC) bound for gradient-based meta-learning using two different generalization frameworks in order to deal with the qualitatively different challenges of generalization at the “base” and “meta” levels. We employ bounds for uniformly stable algorithms at the base level and bounds from the PAC-Bayes framework at the meta level. The result of this approach is a novel PAC bound that is tighter when the base learner adapts quickly, which is precisely the goal of meta-learning. We show that our bound provides a tighter guarantee than other bounds on a toy non-convex problem on the unit sphere and a text-based classification example. We also present a practical regularization scheme motivated by the bound in settings where the bound is loose and demonstrate improved performance over baseline techniques.

13.1 Introduction

A major challenge with current machine learning systems is the need to acquire large amounts of training data in order to learn a new task. Over the past few decades, meta-learning [181, 201] has emerged as a promising avenue for addressing this challenge. Meta-learning relies on the intuition that a new task often bears significant similarity to previous tasks; hence, a learner can learn to perform a new task very quickly by exploiting data from previously-encountered related tasks. The meta-learning problem formulation thus assumes access to datasets from a variety of tasks during meta-training. The goal of the meta learner is then to learn inductive biases from these tasks in order to train a base learner to achieve few-shot generalization on a new task.

Over the past few years, there has been tremendous progress in practical algorithms for meta-learning (see, e.g., [178, 164, 72, 99]). Techniques such as model-agnostic meta-learning (MAML) [72] have demonstrated the ability to perform few-shot learning in a variety of supervised learning and reinforcement learning domains. However, our theoretical understanding of these techniques lags significantly behind successes on the empirical front. In particular, the problem of deriving *generalization bounds* for meta-learning techniques remains an outstanding challenge. Current methods for obtaining generalization guarantees for meta-learning [16, 109, 225] either (i) produce bounds that are extremely challenging to compute or (ii) produce vacuous or near-vacuous bounds in even highly simplified settings (see Section 13.5 for numerical examples). Indeed, we note that existing work on generalization theory for meta-learning techniques do not explicitly report numerical values for generalization bounds. This is in contrast to the state of generalization theory in the supervised learning setting, where recent techniques demonstrate the ability to obtain non-vacuous generalization guarantees on benchmark problems (e.g. visual classification problems [63, 227, 162]).

The generalization challenge in meta-learning is similar to, but distinct from, the supervised learning case. In particular, any generalization bound for meta-learning must account for *two levels* of generalization. First, one must account for generalization at the base level, i.e., the ability of the base learner to perform well on new data from a given task. This

is particularly important in the few-shot learning setting. Second, one must account for generalization at the meta level, i.e., the ability of the meta learner to generalize to new tasks not encountered during meta-training. Moreover, the generalization performance at the two levels is coupled since the meta learner is responsible for learning inductive biases that the base learner can exploit for future tasks.

The key technical insight of this work is to bound the generalization error at the two levels (base and meta) using two *different* generalization theory frameworks that each are particularly well-suited for addressing the specific challenges of generalization. At the base level, we utilize the fact that a learning algorithm that exhibits uniform stability [35, 36] also generalizes well in expectation (see Section 13.4.1 for a formal statement). Intuitively, uniform stability quantifies the sensitivity of the output of a learning algorithm to changes in the training dataset. As demonstrated by [88], limiting the number of training epochs of a gradient-based learning algorithm leads to uniform stability. In other words, a gradient-based algorithm that *learns quickly* is stable. Since the goal of meta-learning is *precisely* to train the base learner to learn quickly, we posit that generalization bounds based on stability are particularly well-suited to bounding the generalization error at the base level. At the meta level, we employ a generalization bound based on *Probably Approximately Correct (PAC)-Bayes* theory. Originally developed two decades ago [142, 116], there has been a recent resurgence of interest in PAC-Bayes due to its ability to provide strong generalization guarantees for neural networks [63, 20, 162]. Intuitively, the challenge of generalization at the meta level (i.e., generalizing to new tasks) is similar to the challenge of generalizing to new data in the standard supervised learning setting. In both cases, one must prevent over-fitting to the particular tasks/data that have been seen during meta-training/training. Thus, the strong empirical performance of PAC-Bayes theory in supervised learning problems makes it a promising candidate for bounding the generalization error at the meta level.

Contributions. The primary contributions of this work are the following. First, we leverage the insights above in order to develop a novel generalization bound for gradient-based meta-learning using uniform stability and PAC-Bayes theory (Theorem 13.3). Second, we develop a regularization scheme for MAML [72] that explicitly minimizes the de-

rived bound (Algorithm 13.1). We refer to the resulting approach as *PAC-BUS* since it combines PAC-Bayes and Uniform Stability to derive generalization guarantees for meta-learning. Third, we demonstrate our approach on two meta-learning problems: (i) a toy non-convex classification problem on the unit-ball (Section 13.5.1), and (ii) the *Mini-Wiki* benchmark introduced in [109] (Section 13.5.2). Even in these relatively small-scale settings, we demonstrate that recently-developed generalization frameworks for meta-learning provide either near-vacuous or loose bounds, while PAC-BUS provides significantly stronger bounds. Fourth, we demonstrate our approach in larger-scale settings where it remains challenging to obtain non-vacuous bounds (for our approach as well as others). Here, we propose a practical regularization scheme which re-weights the terms in the rigorously-derived PAC-BUS upper bound ($PAC-BUS(H)$; Algorithm 13.3 in the appendix). Recent work [225] introduces a challenging variant of the *Omniglot* benchmark [113] which highlights and tackles challenges with *memorization* in meta-learning. We show that PAC-BUS(H) is able to prevent memorization on this variant (Section 13.5.3).

13.2 Problem formulation

Samples, tasks, and datasets. Formally, consider the setting where we have an unknown meta distribution P_t over tasks (roughly, “tasks” correspond to different, but potentially related, learning problems). A sampled task $t \sim P_t$ induces an (unknown) distribution $P_{z|t}$ over sample space $\mathcal{Z}$. We assume that all sampling is independent and identically distributed (i.i.d.). Note that the sample space $\mathcal{Z}$ is shared between tasks, but the distribution $P_{z|t}$ may be different. We then sample within-task samples $z \sim P_{z|t}$ and within-task datasets $S = \{z_1, z_2, \dots, z_m\} \sim P_{z|t}^m$. We assume that each sample z has a single corresponding label $o(z)$, where the function o is an oracle which outputs the correct label of z . At meta-training time, we assume access to l datasets, which we call $\mathbf{S} = \{S_1, S_2, \dots, S_l\}$. Each dataset S_i in $\mathbf{S}$ is drawn by first selecting a task t_i from P_t , and then drawing $S_i \sim P_{z|t_i}^m$.

Hypotheses and losses. Let h denote a hypothesis and $L(h, z)$ be the loss incurred by hypothesis h on sample z . The loss is computed by comparing $h(z)$ with the true label $o(z)$. For simplicity, we assume that there is no noise on the labels; we can thus assume that all loss functions have access to the label oracle function o and thus the loss

depends only on hypothesis h and sample z . We note that this assumption is not required for our analysis and is made for the ease of exposition. Overloading the notation, we let $L(h, P_{z|t}) := \mathbb{E}_{z \sim P_{z|t}} L(h, z)$ and $\widehat{L}(h, S) := \frac{1}{|S|} \sum_{i=1}^{|S|} L(h, z_i)$.

Meta-learning. As with model-agnostic meta-learning (MAML) [72], we let meta parameters $\theta \in \mathbb{R}^{n_\theta}$ correspond to an initialization of the base learner’s hypothesis. Let h_θ be the θ -initialized hypothesis. Generally, the initialization θ is learned from the multiple datasets we have access to at meta-training time. In this work, we will learn a *distribution* P_θ over initializations so that we can use bounds from the PAC-Bayes framework. At test time, a new task $t \sim P_t$ is sampled and we are provided with a new dataset $S \sim P_{z|t}^m$. The base learner uses an algorithm A (e.g., gradient descent), the dataset S , and the initialization $\theta \sim P_\theta$ in order to fine-tune the hypothesis and perform well on future samples drawn from $P_{z|t}$. We denote the base learner’s updated hypothesis by $h_{A(\theta, S)}$. More formally, our goal is to learn a distribution P_θ with the following objective:

$$\min_{P_\theta} \mathcal{L}(P_\theta, P_t) := \min_{P_\theta} \mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}). \quad (13.1)$$

We are particularly interested in the the few-shot learning case, where the number of samples which the base learner can use to adapt is small. A common technique to improve test performance in the few-shot learning case is to allow for validation data at meta-training time. Thus, in addition to a generalization guarantee on meta-learning without validation data, we will derive a bound when allowing for the use of validation data $S_{\text{va}} \sim P_{z|t}^n$ during meta-training.

13.3 Related work

Meta-learning. Meta-learning is a well-studied technique for exploiting similarities between learning tasks [181, 201]. Often used to reduce the need for large amounts of training data, a number of approaches for meta-learning have been explored over decades [26, 34, 42, 94, 210, 178, 164, 99]. Recently, methods based on model-agnostic meta-learning (MAML) [72] have demonstrated strong performance across different application domains and benchmarks such as *Omniglot* [113] and *Mini-ImageNet* [212]. These methods operate

by optimizing a set of initial parameters that can be quickly fine-tuned via gradient descent on a new task. The approaches mentioned above typically do not provide any generalization guarantees, and none of them compute explicit numerical bounds on generalization performance. Our approach has the structure of gradient-based meta-learning while providing guarantees on generalization.

Generalization bounds for supervised learning. Multiple frameworks have been developed for providing generalization guarantees in the classical supervised learning setting. Early breakthroughs include Vapnik-Chervonenkis (VC) theory [206, 17], Rademacher complexity [188], and the minimum description length principle [31, 169, 114]. More recent frameworks include algorithmic stability bounds [35, 45, 88, 171, 3] and PAC-Bayes theory [192, 142, 186]. The connection between stability and learnability has been established in [189, 211, 88], and suggests that algorithmic stability bounds are a strong choice of generalization framework. PAC-Bayes theory in particular provides some of the tightest known generalization bounds for classical supervised learning approaches such as support vector machines [186, 116, 78, 171, 14, 156]. Since its development, researchers have continued to tighten [116, 141, 162] and generalize the framework [43, 44, 170]. Exciting recent results [63, 149, 150, 23, 20, 162] have demonstrated the promise of PAC-Bayes to provide strong generalization bounds for neural networks on supervised learning problems (see [105] for a recent review of generalization bounds for neural networks). It is also possible to combine frameworks such as PAC-Bayes and uniform stability to derive bounds for supervised learning [130]. We will use these two frameworks to bound generalization in the two levels of meta-learning. In contrast to the standard supervised learning setting, generalization bounds for meta-learning are less common and remain loose.

Generalization bounds for meta-learning. As described in Section 13.1, meta-learning bounds must account for two “levels” of generalization (base level and meta level). The approach presented in [140] utilizes algorithmic stability bounds at both levels. However, this requires both meta and base learners to be uniformly stable. This is a strong requirement that is challenging to ensure at the meta level. Another recent method, known as follow-the-meta-regularized-leader (FMRL) [109], provides guarantees for a regularized meta-learning version of the follow-the-leader (FTL) method for online learning, see e.g.

[90]. The generalization bounds provided are derived from the application of online-to-batch techniques [10, 58]. A regret bound for meta-learning using an aggregation technique at the meta-level and an algorithm with a uniform generalization bound at the base level is provided in [10]. The techniques mentioned do not present an algorithm which makes use of validation data (in contrast to our approach). Using validation data (i.e., held-out data) is a common technique for improving performance in meta-learning and is particularly important for the few-shot learning case.

Another method for deriving a generalization bound on meta-learning is to use PAC-Bayes bounds at both the base and meta levels [160, 161]. In [16], generalization bounds based on such a framework are provided along with practical optimization techniques. However, the method requires one to maintain distributions over distributions of initializations, which can result in large computation times during training and makes it extremely challenging to numerically compute the bound. Moreover, the approach also does not allow one to incorporate validation data to improve the bound. Recent work has made progress on some of these challenges. In [173], the computational efficiency of training is improved but the challenges associated with numerically computing the generalization bound or incorporating validation data are not addressed. State-of-the-art work tightens the two-level PAC-Bayes guarantee, addresses computation times for training and evaluation of the bound, and allows for validation data [225]. However, all of the two-level PAC-Bayes bounds require a separate PAC-Bayes bound for each task, and thus a potentially loose union bound.

We present a framework which, to our knowledge, is the first to combine algorithmic stability and PAC-Bayes bounds (at the base- and meta- levels respectively) in order to derive a meta-learning algorithm with associated generalization guarantees. As outlined in Section 13.1, we believe that the algorithmic stability and PAC-Bayes frameworks are particularly well-suited to tackling the specific challenges of generalization at the different levels. We also highlight that *none* of the approaches mentioned above report numerical values for generalization bounds, even for relatively simple problems. Here, we empirically demonstrate that prior approaches tend to provide either near-vacuous or loose bounds even in relatively small-scale settings while our proposed method provides significantly stronger bounds.

13.4 Generalization bound on meta-learning

We use two different frameworks for the two levels of generalization required in a meta-learning bound. We utilize the PAC-Bayes framework to bound the expected training loss on future tasks, and uniform stability bounds to argue that if we have a low training loss when using a uniformly stable algorithm, then we achieve a low test loss. The following section will introduce these frameworks independently. We then present the overall meta-learning bound and associated algorithm to find a distribution over initialization parameters (i.e., meta parameters) that minimizes the upper bound.

13.4.1 Preliminaries: two generalization frameworks

Uniform stability

Let $S = \{z_1, z_2, \dots, z_m\} \in \mathcal{Z}^m$ be a set of m elements of $\mathcal{Z}$. Let $S^i = \{z_1, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_m\}$ be identical to dataset S except that the i^{th} sample z_i is replaced by some $z'_i \in \mathcal{Z}$. Note that our analysis can be extended to allow for losses bounded by some finite M , but we work with losses bounded within $[0, 1]$ for the sake of simplicity. With these precursors, we define an analogous notion of *uniform stability* to [88, Definition 2.1] for deterministic algorithms A and distributions P_θ over initializations.¹

Definition 13.1 (Uniform Stability). *A deterministic algorithm A has $\beta > 0$ uniform stability with respect to loss L if $\forall z \in \mathcal{Z}, \forall S \in \mathcal{Z}^m, \forall i \in \{1, \dots, m\}$, and all distributions P_θ over initializations, the following holds:*

$$\mathbb{E}_{\theta \sim P_\theta} |L(h_{A(\theta, S)}, z) - L(h_{A(\theta, S^i)}, z)| \leq \beta. \quad (13.2)$$

We define β_{US} as the minimal such β .

In this work, we will bound β_{US} as a function of the algorithm, form of the loss, and number of samples that the algorithm uses (See Appendix 13.7.4 for further details on the bounds on β_{US} for our setup). We then establish a relationship between uniform stability

¹We use deterministic algorithms to avoid excess computation when calculating the provided meta-learning upper bounds. See Appendix 13.7.4 for further details.

and generalization in expectation. The following is adapted from [88, Theorem 2.2] for the notion of uniform stability presented in Definition 13.1:

Theorem 13.1 (Algorithmic Stability Generalization in Expectation). *Fix a task $t \sim P_t$. The following inequality holds for hypothesis $h_{A(\theta, S)}$ learned using β_{US} uniformly stable algorithm A with respect to loss L :*

$$\mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}. \quad (13.3)$$

Proof. The proof is similar to the one presented for [88, Theorem 2.2] and is presented in Appendix 13.7.1. $\square$

PAC-Bayes theory

For the meta-level bound, we make use of the PAC-Bayes generalization bound introduced in [142]. Note that other PAC-Bayes bounds such as the quadratic variant [172] and PAC-Bayes- λ variant [200] may be used and substituted in the following analysis. We first present a general version of the PAC-Bayes bound and then specialize it to our meta-learning setting in Section 13.4.2. Let $f(\theta, s)$ be an arbitrary loss function which only depends on parameters θ and the sample s which has been drawn from an arbitrary distribution P_s . The following bound is a tightened version of the bound presented in [142] for when $l \geq 8$.

Theorem 13.2 (PAC-Bayes Generalization Bound [139]). *For any data-independent prior distribution $P_{\theta,0}$ over θ , some loss function f where $0 \leq f(\theta, s) \leq 1, \forall s, \forall \theta$, $l \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of $\{s_1, s_2, \dots, s_l\} \sim P_s^l$, the following holds simultaneously for all distributions P_θ over θ :*

$$\mathbb{E}_{s \sim P_s} \mathbb{E}_{\theta \sim P_\theta} f(\theta, s) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} f(\theta, s_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l), \quad (13.4)$$

where the PAC-Bayes “regularizer” term is defined as follows

$$R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l) := \sqrt{\frac{D_{\text{KL}}(P_\theta \| P_{\theta,0}) + \ln \frac{2\sqrt{l}}{\delta}}{2l}}, \quad (13.5)$$

and D_{KL} is the Kullback-Leibler (KL) divergence.

13.4.2 Meta-learning bound

In order to obtain a generalization guarantee for meta-learning, we utilize the two frameworks above. We first specialize the PAC-Bayes bound in Theorem 13.2 to bound the expected training loss on future tasks. We then utilize Theorem 13.1 to demonstrate that if we have a low expected training loss when using a uniformly stable algorithm, then we achieve a low expected test loss. These two steps allow us to combine the generalization frameworks above to derive an upper bound on (13.1) which can be computed with known quantities. With the following assumption, the resulting generalization bound is presented in Theorem 13.3.

Assumption 13.1 (Bounded loss.). *The loss function L is bounded: $0 \leq L(h, z) \leq 1$ for any h in the hypothesis space for the given problem, and any z in the sample space.*

Theorem 13.3 (Meta-Learning Generalization Guarantee). *For hypotheses $h_{A(\theta, S)}$ learned with β_{US} uniformly stable algorithm A , data-independent prior $P_{\theta, 0}$ over initializations θ , loss L which satisfies Assumption 13.1, $l \geq 8$, and $\delta \in (0, 1)$, with probability at least $1 - \delta$ over a sampling of the meta-training dataset $\mathcal{S} \sim P_S^l$, the following holds simultaneously for all distributions P_θ over θ :*

$$\mathcal{L}(P_\theta, P_t) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \beta_{\text{US}}. \quad (13.6)$$

Proof. The proof can be split into three steps:

Step 1.

Let P_S in Theorem 13.2 be the marginal distribution P_S over datasets of size m (see Appendix 13.7.2 for details) and note that sampling $S \sim P_S$ is equivalent to first sampling $t \sim P_t$ and then sampling $S \sim P_{z|t}^m$. Additionally let $f(\theta, S) := \widehat{L}(h_{A(\theta, S)}, S)$ where $A(\theta, S)$ is any deterministic algorithm. Plugging in these definitions into Inequality (13.4) results in the following inequality which holds under the same assumptions as Theorem 13.2, and

with probability at least $1 - \delta$ over the sampling of $\mathbf{S} \sim P_S^l$:

$$\begin{aligned} \mathbb{E}_{S \sim P_S} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) &= \mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) \\ &\leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l). \end{aligned} \quad (13.7)$$

Step 2.

Now assume that algorithm A is β_{US} uniformly stable. For a fixed task $t \sim P_t$ we have the following by Theorem 13.1:

$$\mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}.$$

Take the expectation over $t \sim P_t$. We then have:

$$\mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}, \quad (13.8)$$

since $\mathbb{E}_{t \sim P_t} \beta_{\text{US}} = \beta_{\text{US}}$. This establishes a bound on the true expected loss for a new task after running algorithm A on a training dataset corresponding to the new task.

Step 3.

Note that (13.7) provides an upper bound on the first term of the RHS of (13.8) when algorithm A is β_{US} uniformly stable. Thus we have the following by plugging (13.7) in the RHS of (13.8):

Under the same assumptions as both Theorems 13.1 and 13.2, and with probability at least $1 - \delta$ over the sampling of $\mathbf{S} \sim P_S^l$:

$$\mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_i) + R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l) + \beta_{\text{US}},$$

completing the proof. $\square$

Theorem 13.3 is presented for any distributions P_θ and $P_{\theta,0}$ over initializations. However, in practice we will use multivariate Gaussian distributions for both. The specialization of

Theorem 13.3 to Gaussian distributions is provided in Appendix 13.7.3. Next, we allow for validation data $S_{\text{va}} \sim P_{z|t}^n$ at meta-training time so that the bound is more suited to the few-shot learning case. We compute the upper bound using the evaluation data $S_{\text{ev}} = \{S, S_{\text{va}}\}$ sampled from the marginal distribution $P_{S_{\text{ev}}}$ over datasets of size $m + n$. However, we still only require m samples at meta-test time; see Appendix 13.7.3 for the derivation. Note that the training data S is often excluded from the data used to update the meta-learner. However, this is necessary for our approach to obtain a guarantee on few-shot learning performance. The result is a guarantee with high probability over a sampling of $\mathbf{S}_{\text{ev}} \sim P_{S_{\text{ev}}}^l$:

$$\mathcal{L}(P_\theta, P_t) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_{\text{ev}, i}) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \frac{m\beta_{\text{US}}}{m+n}. \quad (13.9)$$

13.4.3 PAC-BUS algorithm

Recall that we aim to find a distribution P_θ over initializations that minimizes $\mathcal{L}(P_\theta, P_t)$ as stated in Equation (13.1). We cannot minimize $\mathcal{L}(P_\theta, P_t)$ directly due to the expectations taken over unknown distributions P_t and $P_{z|t}$ for sampled task t , but we may indirectly minimize it by minimizing the upper bounds in Inequalities (13.6) or (13.9).

Computing the upper bound requires evaluating an expectation taken over $\theta \sim P_\theta$. In general, this is intractable. However, we aim to minimize this upper bound to provide the tightest guarantee possible. Similar to the method in [63], we use an unbiased estimator of $\mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, \cdot)$. Let P_θ be a multivariate Gaussian distribution over initializations θ with mean μ and covariance $\text{diag}(s)$; thus $P_\theta = \mathcal{N}(\mu, \text{diag}(s))$ and $P_{\theta, 0} = \mathcal{N}(\mu_0, \text{diag}(s_0))$. Further, let $\psi := (\mu, \log(s))$, and use the shorthand $\mathcal{N}_{\psi_0}$ for the prior and $\mathcal{N}_\psi$ for the posterior distribution over initializations. We use the following estimator of $\mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, \cdot)$:

$$L(h_{A(\theta, S)}, \cdot), \quad \theta \sim \mathcal{N}_\psi. \quad (13.10)$$

We present the resulting training technique in Algorithm 13.1. This algorithm can be used to learn a distribution over initializations that minimizes the upper bound presented in Theorem 13.3 and its specializations. This is presented for the case when A is β_{US} uniformly stable for some β_{US} . For gradient-based algorithms, the learning rate α often

Algorithm 13.1 PAC-BUS: meta-learning via PAC-Bayes and Uniform Stability

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over initializations
Input: β_{US} uniformly stable Algorithm A
Input: Meta-training dataset $\mathbf{S}$, learning rate γ
Initialize: $\psi \leftarrow \psi_0$
Output: Optimized ψ^*
 $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l) := \frac{1}{l} \sum_{i=1}^l \widehat{L}(h_{\theta'_i}, S_i) + R_{\text{PAC-B}}(\mathcal{N}_{\psi}, \mathcal{N}_{\psi_0}, \delta, l) + \beta_{\text{US}}$
while not converged **do**
 Sample $\theta \sim \mathcal{N}_{\psi}$
 for $i = 1$ **to** l **do**
 $\theta'_i \leftarrow A(\theta, S_i)$
 end for
 $\psi \leftarrow \psi - \gamma \nabla_{\psi} B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l)$
end while

appears directly in the bound for β_{US} [88]. Thus it is potentially beneficial to update α as well. We present Algorithm 13.1 without learning the learning rate. To meta-learn the learning rate, we can augment ψ_0 to include a parameterization of a prior distribution over learning rates and update it using the same gradient step presented in 13.1 for ψ .

Determining the gradient of $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l)$ with respect to ψ requires computing the Hessian of the loss function if algorithm $A(\theta, S)$ uses a gradient update to compute θ'_i . First order approximations often perform similarly to the second-order meta-learning techniques [81, 72, 152], and can be used to speed up the training. Additionally, Algorithm 13.1 can be modified to use mini-batches of tasks instead of all tasks in the meta update to improve training times; we present an algorithm which uses mini-batches of tasks in Appendix 13.7.5.

In practice, we are interested in algorithms such as stochastic gradient descent (SGD) and gradient descent (GD) for the base learner. We can obtain bounds on the uniform stability constant β_{US} when using gradient methods with the results from [88]. See Appendix 13.7.4 for details on the β_{US} bounds we use in this work. With a bound on β_{US} , we can calculate all the terms in $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_l)$ and use Algorithm 13.1 to minimize the meta-learning upper bound. When evaluating the upper bound, we use the sample convergence bound [115, 63] to upper bound the expectation taken over $\theta \sim P_{\theta}$. See Appendix 13.7.6 for details.

Table 13.1: We present the generalization bounds (for $\delta = 0.01$) provided by each method if applicable, and use the sample convergence bound [115] for MR-MAML, and PAC-BUS, but not MLAP-M.² Note that for these methods, we specifically minimize their respective meta-learning bounds. We also report the meta-test loss (the softmax activated cross-entropy loss – CEL_s) for all methods. We present the mean and standard deviation after 5 trials. We highlight that our approach provides the strongest generalization guarantee.

Classification on Ball	MAML [72]	MLAP-M [16]	MR-MAML [225]	PAC-BUS (ours)
Bound ↓	None	1.0538 ± 0.0012^2	0.3422 ± 0.0006	0.2213 ± 0.0012
Test Loss ↓	0.1701 ± 0.0070	0.1645 ± 0.0045	0.1584 ± 0.0012	0.1657 ± 0.0014

13.5 Examples

We demonstrate our approach on three examples below. All examples we provide are few-shot meta-learning problems. To adapt at the base level, m examples from each class are given for an “ m -shot” learning problem. If applicable, n samples can be given as validation data for each task during the meta-training step. In the first two examples, our primary goal is to demonstrate the tightness of our generalization bounds compared to other meta-learning bounds. We also present empirical test performance on held-out data; however, we emphasize that the focus of our work is to obtain improved generalization guarantees (and not necessarily to improve empirical test performance). In the third example, we present an algorithm that is motivated by our theoretical framework and demonstrate its ability to improve empirical performance on a challenging task. All the code required to run the following examples is available at <https://github.com/irom-lab/PAC-BUS>.

13.5.1 Example: classification on the unit ball

We evaluate the tightness of the generalization bound in Equation (13.9) on a toy two-class classification problem where the sample space $\mathcal{Z}$ is the unit ball $B^2(0, 1)$ in two dimensions with radius 1 and centered at the origin. Data points for each task are sampled from $P_{z|t}$, where a task corresponds to a particular concept which labels the data as (+) if within $B^2(c_t, r_t)$ and (−) otherwise. Center c_t is sampled uniformly from the $y \geq 0$ semi-ball $B_{y \geq 0}^2(0, 0.4)$ of radius 0.4. The radius r_t is then sampled uniformly from $[0.1, 1 - \|c_t\|]$. Notably, the decision boundary between classes is nonlinear. Thus, generalization bounds which rely on convex losses (such as [109]) will have difficulty with providing guarantees for

networks that perform well. We choose the softmax-activated cross-entropy loss, CEL_s , as the loss function. Before running Algorithm 13.1, we address a few technical challenges that arise from Assumption 13.1 as well as computing c_L and c_S . We address these in Appendix 13.7.5.²

We then apply Algorithm 1 using the few-shot learning bound in Inequality (13.9). We present the guarantee on the meta-test loss associated with each training method in Table 13.1. In addition, we present the average meta-test loss after training with 10 samples. We compare our bounds and empirical performance with the meta-learning by adjusting priors (MLAP) technique [16] and the meta-regularized MAML (MR-MAML) technique [225]. All methods are given held-out data to learn a prior before minimizing their respective upper bounds (see Appendix 13.7.11 for further details on the prior training step). Additionally, since all bounds require the loss to be within $[0, 1]$, networks N are constrained such that the Frobenius norm of the output is bounded by r , i.e., $\|N(z)\|_F \leq r$. We compare the aforementioned methods’ meta-test loss to MAML with weights constrained in the same manner (note that MAML does not provide a guarantee). Upper bounds which use the PAC-Bayes framework are computed with many evaluations from the posterior distribution. This allows us to apply the sample convergence bound [115] (as in Equation (13.35) for our bound) unless otherwise noted.

We find that PAC-BUS provides a significantly stronger guarantee compared with the other methods. Note that the guarantee provided by MLAP-M [16] is vacuous because the meta-test loss is bounded between 0 and 1, while the guarantee is above 1.

13.5.2 Example: Mini-Wiki

Next, we present results on the *Mini-Wiki* benchmark introduced in [109]. This is derived from the Wiki3029 dataset presented in [21]. The dataset is comprised of 4-class, m -shot learning tasks with sample space $\mathcal{Z} = \{z \in \mathbb{R}^d \mid \|z\|_2 = 1\}$. Sentences from various Wikipedia articles are passed through the continuous-bag-of-words GloVe embedding [159] into dimension $d = 50$ to generate samples. For this learning task, we use a k -class version

²Due to high computation times associated with estimating the MLAP upper bound, this value is not computed with the sample convergence bound as the other upper bounds are. Thus, the value presented does not carry a guarantee, but would be similar if computed with the sample convergence bound. The value is shown to give a qualitative sense of the guarantee.

of CEL_s and logistic regression. Since this example is convex, we can use GD and bound β_{US} with Theorem 13.4 in the appendix [88]. We keep the loss bounded by constraining the network $\|N(z)\|_F \leq r$ and scale the loss as in the previous example. The tightness of the bounds on c_L and c_S affected the upper bound in Inequality (13.9) more than in the previous example, so we bound them as tightly as possible. See Appendix 13.7.9 for the calculations.

We apply Algorithm 13.1 using the bound which allows for validation data, Inequality (13.9), to learn on 4-way *Mini-Wiki* $m = \{1, 3, 5\}$ -shot. The results are presented in Table 13.2. We compare our results with the FMRL variant which provides a guarantee [109], follow-the-last-iterate (FLI)-Batch, and with MR-MAML [225]. FLI-Batch does not require bounded losses explicitly, but requires that the parameters of the network lie within a ball of radius r . For the logistic regression used in the example, this is equivalent to $\|N(z)\|_F \leq r$. Thus, we scale the loss and use the same r for each method to provide a fair comparison. We also show the results of training with MAML constrained in the same way for reference. Each method is given the same amount of held-out data for training a prior (see Appendix 13.7.11 for further details on training the prior).

As in the previous example, PAC-BUS provides a significantly tighter guarantee than the other methods (Table 13.2). We see similar empirical meta-test loss for MAML [72], MR-MAML [225], and PAC-BUS with slightly higher loss for FLI-Batch [109]. In addition, we computed the meta-test accuracy as the percentage of correctly classified sentences. See Table 13.4 in Section 13.7.11 for these results along with other experimental details.

13.5.3 Example: memorizable Omniglot

We have demonstrated the ability of our approach to provide strong generalization guarantees for meta-learning in the settings above. We now consider a more complex setting where we are unable to obtain strong guarantees. In this example, we employ a learning heuristic based on the PAC-BUS upper bound, $\text{PAC-BUS}(H)$; see Appendix 13.7.5 for the details and the Algorithm. We relax Assumption 13.1 and no longer constrain the network as in previous sections. Instead, we maintain and update estimates of the Lipschitz and smoothness constants of the network, using [195], and incorporate them into the uniform stability

Table 13.2: We compare the generalization bounds (for $\delta = 0.01$) provided by each method where applicable and use the sample convergence bound for MR-MAML and PAC-BUS. Since we specifically minimize these methods’ upper bounds, we can fairly compare the relative tightness of each bound. We also report the meta-test loss (CEL_s) for each method for exposition. We report the mean and standard deviation after 5 trials. We highlight that our approach provides the strongest guarantee.

4-Way <i>Mini-Wiki</i>	1-shot ↓	3-shot ↓	5-shot ↓
FLI-Batch Bound [109]	0.6638 ± 0.0011	0.6366 ± 0.0006	0.6343 ± 0.0014
MR-MAML Bound [225]	0.7400 ± 0.0003	0.7312 ± 0.0003	0.7283 ± 0.0005
PAC-BUS Bound (ours)	0.4999 ± 0.0003	0.5058 ± 0.0002	0.5101 ± 0.0002
MAML [72]	0.3916 ± 0.0009	0.3868 ± 0.0005	0.3883 ± 0.0005
FLI-Batch [109]	0.4091 ± 0.0008	0.4078 ± 0.0005	0.4097 ± 0.0012
MR-MAML [225]	0.3922 ± 0.0009	0.3869 ± 0.0003	0.3884 ± 0.0005
PAC-BUS (ours)	0.3922 ± 0.0009	0.3878 ± 0.0003	0.3895 ± 0.0005

regularizer term, β_{US} . We then scale each regularizer term (i.e., $R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l)$ and β_{US}) by hyper-parameters λ_1 and λ_2 respectively. Analogous to the technique described in [225], we aim to incorporate the form of the theoretically-derived regularizer into the loss, without requiring it to be as restrictive during learning. The result is a regularizer that punishes large deviation from the prior $P_{\theta,0}$ and too much adaptation at the base-learning level.

We test our method on *Omniglot* [113] for 20-way, $m = \{1, 5\}$ -shot classification in the non-mutually exclusive (NME) case [225]. In [225], the problem of memorization in meta-learning is explored and demonstrated with non-mutually exclusive learning problems. *NME Omniglot* corresponds to randomization of class labels for a task at test time only. This worsens the performance of any network that memorized class labels; see [225] for more details.³ We compare our method to an analogous heuristic presented in [225], which also has a $D_{\text{KL}}(P_\theta \| P_{\theta,0})$ term in the loss. Thus, this heuristic (referred to as MR-MAML(W) [225]) regularizes the change in weights of the network. Additionally, we compare to the heuristic described in [109] (FLI-Online) which performs better in practice than the FLI-Batch method. We do not provide data for training a prior in this case since we do not aim to compute a bound in this example. We use standard MAML as a reference. See Table 13.3 for the results.

We see that MAML [72] and FLI-Online [109] do not prevent memorization on *NME*

Table 13.3: We present the meta-test accuracy as a percentage on non-mutually-exclusive *Omniglot* [225]. In contrast to the previous examples, here we aim to achieve the best empirical performance for each method. In particular, this task compares each methods’ ability to prevent memorization. We report the mean and standard deviation after 5 trials.

20-WAY <i>Omniglot</i>	NME 1-SHOT $\uparrow$	NME 5-SHOT $\uparrow$
MAML [72]	23.4 ± 2.2	75.1 ± 4.8
FLI-ONLINE [109]	22.4 ± 0.5	39.1 ± 0.5
MR-MAML(W) [225]	84.2 ± 2.2	94.3 ± 0.3
PAC-BUS(H) (OURS)	87.9 ± 0.5	95.0 ± 0.9

Omniglot [225]. This is especially apparent in the 1-shot learning case, where their performance suffers significantly due to this memorization. Both MR-MAML(W) [225] and PAC-BUS(H) prevent memorization, with PAC-BUS(H) outperforming MR-MAML(W). Note that PAC-BUS(H) outperforms MR-MAML(W) by a wider margin in the 1-shot case as compared with the 5-shot case. We believe this is due to the effectiveness of the uniform stability regularizer at the base level. MR-MAML(W) suffers more in the 1-shot case because over-adaptation is more likely with fewer within-task examples.³

13.6 Conclusion and discussion

We presented a novel generalization bound for gradient-based meta-learning: PAC-BUS. We use different generalization frameworks for tackling the distinct challenges of generalization at the two levels of meta-learning. In particular, we employ uniform stability bounds and PAC-Bayes bounds at the base- and meta-learning levels respectively. On a toy non-convex problem and the *Mini-Wiki* meta-learning task [109], we provide significantly tighter generalization guarantees as compared to state-of-the-art meta-learning bounds while maintaining comparable empirical performance. To our knowledge, this work presents the first numerically-evaluated generalization guarantees associated with a proposed meta-learning bound. On memorizable *Omniglot* [113, 225], we show that a heuristic based on the PAC-BUS bound prevents memorization of class labels in contrast to MAML [72], and better performance than meta-regularized MAML [225]. We believe our framework is well suited

³We use a slightly different task setup as the one in [225]; see Appendix 13.7.11 for the details of our setup.

to the few-shot learning problems for which we present empirical results, but our framework is potentially applicable to a broad range of different settings (e.g., reinforcement learning).

We note a few challenges with our method as motivation for future work. Our bound is vacuous on larger scale learning problems such as *Omniglot*. This is partially caused by a larger KL-divergence term in the PAC-Bayes bound when using deep convolutional networks (due to the increased dimensionality of the weight vector). In addition, we do not have a theoretical analysis on the convergence properties of the algorithms presented, so we must experimentally determine the number of samples required for tight bounds. In the results of Section 13.5.1 and 13.5.2, despite an improved bound over other methods, our method does not necessarily improve empirical test performance. We emphasize that our focus in this work was on deriving stronger generalization guarantees rather than improving empirical performance. However, obtaining approaches that provide both stronger guarantees and empirical performance is an important direction for future work.

Future work can also explore ways in which to incorporate tighter PAC-Bayes bounds or those with less restrictive assumptions. One interesting avenue is to extend PAC-BUS by using a PAC-Bayes bound for unbounded loss functions for the meta-generalization step (e.g. as presented in [86]). Another promising direction is to incorporate regularization on the weights of the network directly (e.g., L_2 regularization or gradient clipping) to create networks with smaller Lipschitz and smoothness constants. Additionally, it would be interesting to explore learning of the base-learner’s algorithm while maintaining uniform stability. For example, one could parameterize a set of uniformly stable algorithms and learn a posterior distribution over the parameters.

Broader impact. The approach we present in this work aims to strengthen performance guarantees for gradient-based meta-learning. We believe that strong generalization guarantees in meta-learning, especially in the few-shot learning case, could lead to broader application of machine learning in real-world applications. One such example is for medical diagnosis, where abundant training data for certain diseases may be difficult to obtain. Another example on which poor performance is not an option is any safety critical robotic system, such as ones which involve human interaction.

Meta-learning methods typically require a lot of data and training time, and ours is not

an exception. In our case, it took multiple weeks of computation time on Amazon Web Services (AWS) instances to train and compute all networks and results we present in this paper. This creates challenges with accessibility and energy usage.

13.7 Appendix

13.7.1 Proof of Theorem 13.1

Theorem 13.1 (Algorithmic Stability Generalization in Expectation). *Fix a task $t \in P_t$. The following inequality holds for hypothesis $h_{A(\theta, S)}$ learned using β_{US} uniformly stable algorithm A with respect to loss L :*

$$\mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) + \beta_{\text{US}}. \quad (13.11)$$

Proof. Let $S = \{z_1, z_2, \dots, z_m\} \sim P_{z|t}^m$ and $S' = \{z'_1, z'_2, \dots, z'_m\} \sim P_{z|t}^m$ be two independent random samples and let $S^i = \{z_1, \dots, z_{i-1}, z'_i, z_{i+1}, \dots, z_m\}$ be identical to S except with the i^{th} sample replaced with z'_i . Fix a distribution P_θ over initializations. Consider the following

$$\mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S)}, S) = \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \left[\frac{1}{m} \sum_{i=1}^m L(h_{A(\theta, S)}, z_i) \right] \quad (13.12)$$

$$= \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{S' \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \left[\frac{1}{m} \sum_{i=1}^m L(h_{A(\theta, S_i)}, z_i) \right] \quad (13.13)$$

$$= \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{S' \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \left[\frac{1}{m} \sum_{i=1}^m L(h_{A(\theta, S)}, z'_i) \right] + \delta \quad (13.14)$$

$$= \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) + \delta \quad (13.15)$$

where

$$\delta = \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{S' \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} \left[\frac{1}{m} \sum_{i=1}^m L(h_{A(\theta, S_i)}, z'_i) - \sum_{i=1}^m L(h_{A(\theta, S)}, z'_i) \right]. \quad (13.16)$$

We bound δ with the supremum over datasets S and S' differing by a single sample

$$\delta \leq \sup_{S, S', z} \mathbb{E}_{\theta \sim P_\theta} [L(h_{A(\theta, S)}, z) - L(h_{A(\theta, S')}, z)] \leq \beta_{\text{US}} \quad (13.17)$$

by Definition 13.1. □

13.7.2 Definition of Marginal Distribution P_S

In this section we formally define the marginal distribution P_S which we make use of in the proof of Theorem 13.3. This is the distribution over datasets one obtains by first sampling a task t from P_t , and then sampling a dataset S from $P_{z|t}^m$. Consider the following equations (for simplicity, we use summations instead of integrals to compute expectations; $p(t)$ represents the probability of sampling t and $p(S|t)$ is the probability of sampling S given t):

$$\mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) = \sum_t p(t) \sum_S p(S|t) \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) \quad (13.18)$$

$$= \sum_{t, S} p(t) p(S|t) \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) \quad (13.19)$$

$$= \sum_{t, S} p(S, t) \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) \quad (13.20)$$

$$= \sum_S \left(\mathbb{E}_{\theta \sim P_\theta} f(\theta, S) \underbrace{\sum_t p(S, t)}_{=p(S)} \right). \quad (13.21)$$

Here $p(S)$ corresponds to the marginal distribution over datasets S . Note that the last line above holds because $\mathbb{E}_{\theta \sim P_\theta} f(\theta, S)$ does not depend on t .

Definition 13.2 (Marginal Distribution Over Datasets S). *Let $P_S := p(S)$ from above.*

Thus we have

$$\mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) = \sum_S p(S) \mathbb{E}_{\theta \sim P_\theta} f(\theta, S) = \mathbb{E}_{S \sim P_S} \mathbb{E}_{\theta \sim P_\theta} f(\theta, S). \quad (13.22)$$

13.7.3 Specializing the Bound

Meta-Learning Bound for Gaussian Distributions

In practice, the distribution P_θ over initializations will be a multivariate Gaussian distribution. Thus, in this section, we present a specialization of the bound for Gaussian distributions. Let P_θ have mean μ and covariance Σ ; thus $P_\theta = \mathcal{N}(\mu, \Sigma)$ and analogously $P_{\theta,0} = \mathcal{N}(\mu_0, \Sigma_0)$. We can then apply the analytical form for the KL-divergence between two multivariate Gaussian distributions to the bound presented in Theorem 13.3. The result is the following bound holding under the same assumptions as Theorem 13.3:

$$\begin{aligned} \mathcal{L}(P_\theta, P_t) &\leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S_i)}, S_i) + \beta_{\text{US}} \\ &\quad + \sqrt{\frac{(\mu - \mu_0)\Sigma_0^{-1}(\mu - \mu_0) + \ln \frac{|\Sigma_0|}{|\Sigma|} + \text{tr}(\Sigma_0^{-1}\Sigma) - n_{\text{dim}} + 2 \ln \frac{2\sqrt{l}}{\delta}}{4l}}, \end{aligned} \quad (13.23)$$

where n_{dim} is the number of dimensions of the Gaussian distribution. We implement the above bound in code instead of the non-specialized form of the KL divergence to speed up computations and simplify gradient computations.

Few-Shot Learning Bound with Validation Data

In this section, we will assume that, in addition to the training data $S \sim P_{z|t}^m$, we have access to validation data $S_{\text{va}} \sim P_{z|t}^n$ at meta-training time. We will show that a meta-learning generalization bound can still be obtained in this case. Notably, this will not require validation data at meta-testing time.

We begin by bounding the expected loss on evaluation data $S_{\text{ev}} = \{S, S_{\text{va}}\}$ after training on S . Note that for other meta-learning techniques, the training data S is often excluded from the data used to update the meta-learner. Including it here helps relate the loss on $P_{z|t}$ to the loss on S_{ev} after adaptation with S (see derivation below), and is necessary to achieve a guarantee on performance for the few-shot learning case. From Inequality (13.4), we set the arbitrary distribution P_s to the marginal distribution $P_{S_{\text{ev}}}$ over datasets of size $m+n$ and $f(\theta, s) := \hat{L}(h_{A(\theta, S)}, S_{\text{ev}})$. Note that with this marginal distribution, we have an

equivalence of sampling given by

$$\mathbb{E}_{S_{\text{ev}} \sim P_{S_{\text{ev}}}} [\cdot] = \mathbb{E}_{t \sim P_t} \mathbb{E}_{S_{\text{ev}} \sim P_{z|t}^{m+n}} [\cdot]. \quad (13.24)$$

The following inequality holds with high probability over a sampling of $\mathbf{S}_{\text{ev}} = \{S_{\text{ev},1}, S_{\text{ev},2}, \dots, S_{\text{ev},l}\} \sim P_{S_{\text{ev}}}^l$:

$$\begin{aligned} & \mathbb{E}_{S_{\text{ev}} \sim P_{S_{\text{ev}}}} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S_{\text{ev}}) = \\ & \mathbb{E}_{t \sim P_t} \mathbb{E}_{S_{\text{ev}} \sim P_{z|t}^{m+n}} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S_{\text{ev}}) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S_i)}, S_{\text{ev},i}) + R_{\text{PAC-B}}(P_{\theta}, P_{\theta,0}, \delta, l). \end{aligned} \quad (13.25)$$

In the next steps, we aim to isolate for a $\widehat{L}(h_{A(\theta,S)}, S)$ term so that we may still combine with Inequality (13.8) as we did in Section 13.4.2. We decompose the LHS of Inequality (13.25),

$$\frac{1}{m+n} \mathbb{E}_{t \sim P_t} \left[m \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S) + n \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{S_{\text{va}} \sim P_{z|t}^n} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S_{\text{va}}) \right]. \quad (13.26)$$

Since the validation data S_{va} is sampled independently from S , the expected training loss on the validation data is the true expected loss over sample space $P_{z|t}$,

$$\mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{S_{\text{va}} \sim P_{z|t}^n} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S_{\text{va}}) = \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_{\theta}} L(h_{A(\theta,S)}, P_{z|t}). \quad (13.27)$$

We plug Equality (13.27) into Equation (13.26), and then the decomposition in Equation (13.26) into Inequality (13.25). We can then isolate for the $\widehat{L}(h_{A(\theta,S)}, S)$ term,

$$\begin{aligned} \mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S)}, S) & \leq \frac{m+n}{m} \left[\frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_{\theta}} \widehat{L}(h_{A(\theta,S_i)}, S_{\text{ev},i}) + R_{\text{PAC-B}}(P_{\theta}, P_{\theta,0}, \delta, l) \right] \\ & \quad - \frac{n}{m} \mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P_{z|t}^m} \mathbb{E}_{\theta \sim P_{\theta}} L(h_{A(\theta,S)}, P_{z|t}), \end{aligned} \quad (13.28)$$

and plug into the LHS of Equation (13.8). By simplifying, we find that

$$\mathbb{E}_{t \sim P_t} \mathbb{E}_{S \sim P^m} \mathbb{E}_{z|t} \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, P_{z|t}) \leq \frac{1}{l} \sum_{i=1}^l \mathbb{E}_{\theta \sim P_\theta} \widehat{L}(h_{A(\theta, S_i)}, S_{\text{ev}, i}) + R_{\text{PAC-B}}(P_\theta, P_{\theta, 0}, \delta, l) + \frac{m\beta_{\text{US}}}{m+n}. \quad (13.29)$$

This resulting bound is very similar to the one in Inequality (13.6). We compute the loss term in the upper bound with evaluation data and as a result, the size of the uniform stability regularization term is reduced.

13.7.4 Bounds on the Uniform Stability Constant

In this section, we present bounds from [88] on the uniform stability constant β_{US} which are applicable to our settings. We first formalize the definitions of Lipschitz continuous (“Lipschitz” with constant c_L) and Lipschitz smoothness (“smooth” with constant c_S).

Definition 13.3 (c_L -Lipschitz). *Function f is c_L -Lipschitz if $\forall \theta, \theta' \in \mathbb{R}^{n_\theta}, \forall z \in \mathcal{Z}$ the following holds:*

$$|f(\theta, z) - f(\theta', z)| \leq c_L \|\theta - \theta'\|. \quad (13.30)$$

Definition 13.4 (c_S -smooth). *Function f is c_S -smooth if $\forall \theta, \theta' \in \mathbb{R}^{n_\theta}, \forall z \in \mathcal{Z}$ the following holds:*

$$\|\nabla f(\theta, z) - \nabla f(\theta', z)\| \leq c_S \|\theta - \theta'\|. \quad (13.31)$$

Using a convex loss and stochastic gradient descent (SGD) allows us to directly bound the uniform stability constant β_{US} [88]:

Theorem 13.4 (Convex Loss SGD is Uniformly Stable [88]). *Assume that convex loss function L is c_S -smooth and c_L -Lipschitz $\forall z \in \mathcal{Z}$. Suppose we run SGD on S with step size $\alpha \leq \frac{2}{c_S}$ for T steps. Then SGD satisfies β_{US} uniform stability with*

$$\beta_{\text{US}} \leq \frac{2c_L^2}{m} T \alpha. \quad (13.32)$$

Note that the bounds on β_{US} presented in [88] guarantee β_{US} uniform stability in expectation for a randomized algorithm A . However, for deterministic algorithms, this reduces to

β_{US} uniform stability. Using the uniform stability in expectation definition introduces another expectation (over a draw of algorithm A) into the upper bound of the meta-learning generalization guarantee in Inequality (13.6). So as to not increase the computation required to estimate the upper bound, we let A be deterministic. This is achieved either by fixing the order of the samples on which we perform gradient updates for SGD, or by using gradient descent (GD). Additionally, in the convex case, T steps of GD satisfies the same bound on β_{US} as T steps of SGD; see Appendix 13.7.8 for the proof. For non-convex losses, a bound on β_{US} is still achieved when algorithm A is SGD [88]:

Theorem 13.5 (Non-Convex Loss SGD is Uniformly Stable [88]). *Let non-convex loss L be c_S -smooth and c_L -Lipschitz $\forall z \in P_{z|t}$ and satisfy Assumption 13.1. Suppose we run T steps of SGD with monotonically non-increasing step size $\alpha_t \leq \frac{c}{t}$. Then SGD satisfies β_{US} uniform stability with*

$$\beta_{\text{US}} \leq \frac{1 + \frac{1}{c_S c}}{n - 1} (2c_L^2 c)^{\frac{1}{c_S c + 1}} T^{\frac{c_S c}{c_S c + 1}} \quad (13.33)$$

Note that this bound does not hold when GD is used.

13.7.5 Algorithms

Before running the algorithms presented in this paper, we must deal with a few technical challenges that arise from our method’s assumptions and terms which need to be computed. In this paragraph, we discuss the approach we take to deal with these challenges. For arbitrary networks, the softmax-activated cross entropy loss (CEL_s) is not bounded and would not satisfy Assumption 13.1. We thus constrain the network parameters to lie within a ball and scale the loss function such that all samples $z \in \mathcal{Z}$ achieve a loss within $[0, 1]$; see Appendix 13.7.7 for details. However, the PAC-BUS framework works with distributions P_θ over initializations. One option is to let P_θ be a projected multivariate Gaussian distribution. This prevents the network’s output from becoming arbitrarily large. However, the upper bound in Inequality (13.9) requires the KL-divergence between the prior and posterior distribution over initializations. This is difficult to calculate for projected multivariate Gaussian distributions and would require much more computation during gradient steps. Since the KL-divergence between projected Gaussians is less than that between Gaussians

Algorithm 13.2 PAC-BUS using Mini-Batches of Tasks

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over initializations
Input: β_{US} uniformly stable Algorithm A
Input: Meta-training dataset $\mathbf{S} = \{S_1, S_2, \dots, S_l\}$, learning rate γ
Initialize: $\psi \leftarrow \psi_0$
Output: Optimized ψ^*
 $B(\psi, \theta'_1, \theta'_2 \dots, \theta'_k) := \frac{1}{l} \sum_{i=1}^k \widehat{L}(h_{\theta'_i}, S_i) + R_{\text{PAC-B}}(\mathcal{N}_\psi, \mathcal{N}_{\psi_0}, \delta, l) + \beta_{\text{US}}$
while not converged **do**
 Sample $\theta \sim \mathcal{N}_\psi$
 for $i = 1$ **to** k **do**
 $j \sim \text{Uniform}\{1, 2, \dots, l\}$
 $\theta'_i \leftarrow A(\theta, S_j)$
 end for
 $\psi \leftarrow \psi - \gamma \nabla_\psi B(\psi, \theta'_1, \theta'_2 \dots, \theta'_k)$
end while

(due to the data processing inequality [54]), we can loosen the upper bound in (13.6) and (13.9) by computing the upper bound using the non-projected distributions (but using the projected Gaussians for the algorithm). After sampling a base learner's initialization, we re-scale the network such that its parameters lie within a ball of radius r . We also re-scale the base learner's parameters after each gradient step to guarantee that the loss stays within $[0, 1]$. Projection after gradient steps is not standard SGD, but we show that it maintains the same bound on β_{US} ; see Section 13.7.8 for details of the proof. Thus, we let algorithm A be SGD with projections after each update and use Theorem 13.5 to bound β_{US} for non-convex losses [88]. Additionally, we can upper bound the Lipschitz c_L and smoothness c_S constants for the network using the methods presented in [195]. After working through these technicalities, we can compute all terms in the upper bound.

PAC-BUS using Mini-Batches of Tasks

We present the PAC-BUS algorithm modified for mini-batches of tasks to improve training times. For batches of size k , the algorithm is presented in 13.2.

PAC-BUS(H)

In addition to providing algorithms which minimize the upper bound in Inequalities (13.6) and (13.9), we are also interested in a regularization scheme which re-weights the regularizer

Algorithm 13.3 PAC-BUS(H): Meta-learning heuristic based on PAC-BUS upper bound

Input: Fixed prior distribution $\mathcal{N}_{\psi_0}$ over initializations

Input: Meta-training dataset $\mathbf{S}$, learning rates α and γ

Input: Scale factors λ_1, λ_2 for regularization terms

Initialize: $\psi \leftarrow \psi_0$

Output: Optimized ψ^*

$B(\psi, c_L, c_S, \theta'_1, \theta'_2, \dots, \theta'_l) := \frac{1}{l} \sum_{i=1}^l \widehat{L}(h_{\theta'_i}, S_i) + \lambda_1 R_{\text{PAC-B}}(\mathcal{N}_\psi, \mathcal{N}_{\psi_0}, \delta, l) + \lambda_2 \beta_{\text{US}}(c_L, c_S)$

Estimate c_L and c_S using $\mathcal{N}_{\psi_0}$

while not converged **do**

 Sample $\theta \sim \mathcal{N}_\psi$

for $i = 1$ **to** l **do**

$\theta'_i \leftarrow \theta - \alpha \nabla_\theta \widehat{L}(h_\theta, S_i)$

end for

$\psi \leftarrow \psi - \gamma \nabla_\psi B(\psi, c_L, c_S, \theta'_1, \theta'_2, \dots, \theta'_l)$

 Estimate c_L and c_S using $\mathcal{N}_\psi$

end while

terms in these bounds. For larger scale and complex settings, it is challenging to provide a non-vacuous guarantee on performance, but weighting regularizer terms has been shown to be an effective training technique [225]. We calculate β_{US} with a one-gradient-step version of Theorem 13.5. This Theorem requires the algorithm A to be SGD, but we let A be a single step of GD to improve training times. We also relax Assumption 13.1 Since the β_{US} depends on both c_L and c_S , we update estimates of them after each iteration by sampling multiple $\theta \sim P_\theta$, bound the c_L and c_S for those sets of parameters using Section 4 of [195], and then choose the maximum to compute β_{US} . This is in contrast to limiting the network parameters directly by bounding the output of the loss. Instead, the β_{US} term in the upper bound and the scale factor will determine how much to restrict the network parameters. The resulting method is presented in Algorithm 13.3. In order to provide strong performance in practice, we tune λ_1 and λ_2 .

13.7.6 Sample Convergence Bound

After training is complete, we aim to compute the upper bound. However, this requires evaluating an expectation $\theta \sim P_\theta$, which may be intractable. Providing a valid PAC guarantee without needing to evaluate the expectation taken over $\theta \sim P_\theta$ requires the use of the sample convergence bound [115]. We have the following guarantee with probability $1 - \delta'$

over a random draw of $\{\theta_1, \theta_2, \dots, \theta_N\} \sim P_\theta^N$ for any dataset S [115],

$$D_{\text{KL}}\left(\sum_{j=1}^N L(h_{A(\theta_j, S)}, \cdot) \left\| \mathbb{E}_{\theta \sim P_\theta} L(h_{A(\theta, S)}, \cdot) \right.\right) \leq \frac{\log(\frac{2}{\delta'})}{N}. \quad (13.34)$$

We can invert this KL-style bound (i.e. a bound of the form $D_{\text{KL}}(p\|q^*) \leq c$) by solving the optimization problem, $q^* \leq D_{\text{KL}}^{-1}(q\|c) := \sup\{q \in [0, 1] : D_{\text{KL}}(p\|q) \leq c\}$, as described in [63]. After the inversion is performed on Inequality (13.34), we use a union bound to combine the result with Inequality (13.6) and retain a guarantee with probability $1 - \delta - \delta'$ as in [63],

$$\mathcal{L}(P_\theta, P_t) \leq \frac{1}{l} \sum_{i=1}^l D_{\text{KL}}^{-1}\left(\sum_{j=1}^N L(h_{A(\theta_j, S_i)}, S_i) \left\| \frac{\log(\frac{2}{\delta'})}{N} \right.\right) + R_{\text{PAC-B}}(P_\theta, P_{\theta,0}, \delta, l) + \beta_{\text{US}}. \quad (13.35)$$

An analogous bound is achieved when combined with Inequality (13.9). Thus, after training, we evaluate Inequality (13.35) to provide the guarantee. Note that use of the sample convergence bound is a loosening step. However, in our experiments, the upper bound in Inequality (13.35) is less than 5% looser than unbiased estimates of Inequality (13.6). This can be reduced further at the expense of computation time (if we utilize a larger number of samples in the concentration inequality).

13.7.7 Constraining Parameters and Scaling Losses

In order to maintain a guarantee, the PAC-Bayes upper bound in Theorem 13.2 requires a loss function bounded between 0 and 1. However, the losses we use are not bounded in general. Let N_θ be an arbitrary network parameterized by θ and $N_\theta(z)$ be the output of the network given sample $z \in \mathcal{Z}$. Consider arbitrary loss f , which maps the network's output to a real number. If $\|N_\theta(z)\| \leq r, \forall \theta \in \mathbb{R}^{n_\theta}, \forall z \in \mathcal{Z}$, then we can perform a linear scaling of f to map it onto the interval $[0, 1]$. We define the minimum and maximum value

achievable by loss function f as follows

$$M_f := \max_{z \in \mathcal{Z}, \theta \in \mathbb{R}^{n_\theta}, \|N_\theta(z)\| \leq r} f(\theta, z) \quad (13.36)$$

$$m_f := \min_{z \in \mathcal{Z}, \theta \in \mathbb{R}^{n_\theta}, \|N_\theta(z)\| \leq r} f(\theta, z). \quad (13.37)$$

Now we can define a scaled function

$$f_S(\theta, z) := \frac{f(\theta, z) - m_f}{M_f - m_f} \quad (13.38)$$

such that $f_S(\theta, z) \in [0, 1]$. Note that the Lipschitz and smoothness constants of f_S are also scaled by $\frac{1}{M_f - m_f}$. When we choose loss CEL_s , the k -class cross entropy loss with softmax activation, we have

$$M_{\text{CEL}_s} := \log \left(\frac{e^{-r} + (k-1)}{e^{-r}} \right), \quad m_{\text{CEL}_s} := \log \left(\frac{e^r + (k-1)}{e^r} \right). \quad (13.39)$$

However, we must restrict the parameters in such a way that satisfies $\|N_\theta(z)\| \leq r$. For arbitrary networks structures, this is not straightforward, so we only analyze the case we use in this paper. Consider an L -layer network with ELU activation. Let parameters θ contain weights $\mathbf{W}_1, \dots, \mathbf{W}_L$, and biases $b_1, \dots, b_L$, and assume bounded input $\|z\| \leq r_z, \forall z \in \mathcal{Z}$.

$$\|N_\theta(z)\| = \|\text{ELU}(\mathbf{W}_L \text{ELU}(\mathbf{W}_{L-1}(\dots) + b_{L-1}) + b_L)\| \quad (13.40)$$

$$\leq \|\mathbf{W}_L\|_F (\|\mathbf{W}_{L-1}\|_F (\dots) + \|b_{L-1}\|) + \|b_L\| \leq r \quad (13.41)$$

We can satisfy $\|N_\theta(z)\| \leq r$ by restricting

$$\|\theta\|^2 = \sum_{i=1}^L \|\mathbf{W}_i\|_F^2 + \sum_{i=1}^L \|b_i\|^2 \leq \left(\frac{r}{\max(1, r_z)} \right)^2. \quad (13.42)$$

Equation (13.42) implies Equation (13.41) by applying the inequality of arithmetic and geometric means. Thus, we ensure $\|\theta\| \leq r / \max(1, r_z)$ by projecting the network parameters onto the ball of radius $\min(r, \frac{r}{r_z})$ after each gradient update.

13.7.8 Uniform Stability Considerations

Uniform Stability for Gradient Descent

In this section, we will prove that T steps of GD has the same uniform stability constant as T steps of SGD in the convex case. This will allow us to use GD when attempting to minimize a convex loss, Section 13.5.2. Let the gradient update rule G be given by $G(\theta, z) = \theta - \alpha \nabla_{\theta} f(\theta, z)$ for *convex* loss function f , initialization $\theta \in \mathbb{R}^{n_{\theta}}$, sample $z \in \mathcal{Z}$, and positive learning rate α . We define two key properties for gradient updates: expansiveness and boundedness [88].

Definition 13.5 (c_E -expansive, Definition 2.3 in [88]). *Update rule G is c_E -expansive if $\forall \theta, \theta' \in \mathbb{R}^{n_{\theta}}, \forall z \in \mathcal{Z}$ the following holds:*

$$\|G(\theta, z) - G(\theta', z)\| \leq c_E \|\theta - \theta'\|. \quad (13.43)$$

Definition 13.6 (c_B -bounded, Definition 2.4 in [88]). *Update rule G is c_B -bounded if $\forall \theta \in \mathbb{R}^{n_{\theta}}, \forall z \in \mathcal{Z}$ the following holds:*

$$\|\theta - G(\theta, z)\| \leq c_B. \quad (13.44)$$

Now, consider dataset $S \in \mathcal{Z}^m$ and define $\bar{f}(\theta, S) := \frac{1}{m} \sum_{i=1}^m f(\theta, z_i)$. We also define $\bar{G}(\theta, S) := \theta - \alpha \nabla_{\theta} \bar{f}(\theta, S) = \sum_{i=1}^m G(\theta, z_i)$. Assume that $G(\theta, z)$, is c_E -expansive and c_B -bounded $\forall z \in \mathcal{Z}$. We then bound the expansiveness of $\bar{G}(\theta, S)$,

$$\|\bar{G}(\theta, S) - \bar{G}(\theta', S)\| \leq \frac{1}{m} \sum_{i=1}^m \|G(\theta, z_i) - G(\theta', z_i)\| \leq \frac{1}{m} \sum_{i=1}^m c_E \|\theta - \theta'\| = c_E \|\theta - \theta'\|. \quad (13.45)$$

To compute the boundedness, consider

$$\|\theta - \bar{G}(\theta, S)\| \leq \frac{1}{m} \sum_{i=1}^m \|\theta - G(\theta, z_i)\| \leq \frac{1}{m} \sum_{i=1}^m c_B = c_B. \quad (13.46)$$

For a single gradient step on sample z , we see the same bounds on c_E and c_B when performing a single GD step on dataset S . Thus, if Lemmas 2.5, 3.3, and 3.7 in [88] are true

for gradient updates G , they are also true for gradient updates $\bar{G}$. We can then run through the proof of Theorem 3.8 in [88] to show that it holds for T steps of GD if it holds for T steps of SGD.

Let $S \sim P_S$ be a dataset of size m and S' be an identical dataset with one element changed. We run T steps of GD updates, $\bar{G}$, on each of S and S' . This results in parameters $\theta_1, \dots, \theta_T$ and $\theta'_1, \dots, \theta'_T$ respectively. Fix learning rate $\alpha \leq \frac{2}{c_S}$ and consider

$$\mathbb{E}_{S, S'} \|\theta_{t+1} - \theta'_{t+1}\| = \mathbb{E}_{S, S'} \|\bar{G}(\theta_t, S) - \bar{G}(\theta'_t, S')\| \quad (13.47)$$

$$\leq \frac{1}{m} \sum_{j=1, i \neq j}^m \mathbb{E}_{S, S'} \|G(\theta_t, z_j) - G(\theta'_t, z_j)\| + \frac{1}{m} \mathbb{E}_{S, S'} \|G(\theta_t, z_i) - G(\theta'_t, z_i)\| \quad (13.48)$$

$$\leq \frac{m-1}{m} \mathbb{E}_{S, S'} \|\theta_t - \theta'_t\| + \frac{1}{m} \mathbb{E}_{S, S'} \|\theta_t - \theta'_t\| + \frac{2\alpha c_L}{m} = \mathbb{E}_{S, S'} \|\theta_t - \theta'_t\| + \frac{2\alpha c_L}{m} \quad (13.49)$$

The steps above follow from Lemmas 2.5, 3.3, and 3.7 in [88] and the linearity of expectation. The rest of the proof follows naturally and results in a uniformly stable constant $\beta_{\text{US}} \leq \frac{2c_L^2}{m} T\alpha$ for T steps of GD. Thus, we have the following result.

Corollary 13.1. *Assume that loss convex function f is c_S -smooth and c_L Lipschitz $\forall z \in \mathcal{Z}$. Suppose T steps of SGD on S satisfies β_{US} uniform stability. This implies that T steps of GD on S satisfies β_{US} uniform stability.*

Uniform Stability Under Projections

Projecting parameters onto a ball after gradient updates does not constitute standard SGD nor GD, so we analyze the stability constant after T steps of $G_P(\theta, z) = \text{Proj}[\theta - \alpha \nabla_{\theta} f(\theta, z)]$. Assume $\|z\| \leq r_z, \forall z \in \mathcal{Z}$. The function Proj scales parameters to satisfy $\|\theta\| \leq \max(r, \frac{r}{r_z})$ if it is not already satisfied. See Appendix 13.7.7 for an explanation of this restriction.

As in Appendix 13.7.8, we compute bounds on the expansiveness and boundedness of G_P . Suppose θ is a vector containing all weights of an L -layer network. Network hyperparameters such as learning rate and activation parameters do not need to be projected, so

they will not be included. Assume that θ, θ' already satisfy $\|N_\theta(z)\| \leq r, \forall z \in \mathcal{Z}$. Consider

$$\|G_P(\theta, z) - G_P(\theta', z)\| = \|\text{Proj}(G(\theta, z)) - \text{Proj}(G(\theta', z))\| \leq \|G(\theta, z) - G(\theta', z)\| \leq c_E \|\theta - \theta'\|. \quad (13.50)$$

Note that any required scaling is equivalent to orthogonal projection of the parameters onto a euclidean norm ball of radius r in R^d , where d is the number of parameters in the network. Thus, the first inequality follows from the fact that orthogonal projections onto closed convex sets satisfy the contractive property [182]. Next, consider

$$\|\theta - G_P(\theta, z)\| = \|\text{Proj}(\theta) - \text{Proj}(G_P(\theta, z))\| \leq \|\theta - G(\theta, z)\| \leq c_B. \quad (13.51)$$

The equality follows from the assumption that θ already satisfies the norm constraint. As above, the first inequality follows from the fact that the Proj function satisfies the contractive property [182].

With these bounds, gradient update G_P satisfies Lemmas 2.5, 3.3, and 3.7 from [88] if G does. Note that an analogous procedure can be used to show that scaling after a GD update, $\bar{G}_P$, also satisfies these Lemmas. When function f or $\bar{f}$ is convex, the proof of Theorem 3.8 in [88] applies, and shows that using gradient updates G_P or $\bar{G}_P$ achieve the same bound on the uniform stability constant β_{US} . Thus, when f is convex, we may use G_P or $\bar{G}_P$ to compute updates and maintain the guarantee presented in Theorem 13.1. Suppose now that f is not convex. Using Lemmas 2.5, 3.3, 3.7, and 3.11 from [88], the proof of Theorem 3.12 in [88] follows naturally to achieve a bound on SGD using projected gradient updates G_P when f is not convex.

13.7.9 Lipschitz and Smoothness Constant Calculation

Recall Definitions 13.3 and 13.4 for a function which is c_L -Lipschitz and c_S -smooth from Appendix 13.7.4. We define the softmax activation function.

Definition 13.7 (Softmax Function). $s : \mathbb{R}^k \rightarrow \mathbb{P}^k$

$$s(u)_i = \frac{e^{u_i}}{\sum_{j=1}^k e^{u_j}}, \quad \forall i. \quad (13.52)$$

Where every element in $\mathbb{P}^k$ is a probability distribution in k dimensions (i.e. if $v \in \mathbb{P}^k$, then $\sum_{i=1}^k v_i = 1$ and $v_i \geq 0 \forall i$). Since the stability constant β_{US} depends directly on the Lipschitz constant of the loss function, and β_{US} appears in the regularizer of the final bound, we will be as tight as possible when bounding the Lipschitz constant to keep the generalization as tight as possible. Section 6.2 of [215] describes an approach for bounding the Lipschitz constant for the 2-class, sigmoid activated, cross entropy loss. We are interested in the k -class case with softmax activation, and also aim to bound the smoothness constant. We begin with a similar analysis to the one described in [215].

Given unit-length column vector $z \in \mathbb{R}^d$ and row vector $y \in \mathbb{P}^k$, with weight matrix $\mathbf{W} \in \mathbb{R}^{d \times k}$ (representing a single-layer network), the loss function is given by:

$$\text{CEL}_s(\mathbf{W}) = - \sum_{i=1}^k y_i \log(s(z^T \mathbf{W})_i). \quad (13.53)$$

Note that while y is any probability distribution, in practice, y will be an indicator vector, describing the correct label with a 1 in the index of the correct class and 0 elsewhere. However, the analysis that follows does not depend on this assumption.

We will take the Hessian of this loss to determine convexity and the Lipschitz constant. However, since the weights are given by a matrix, the Hessian would be a 4-tensor. To simplify the analysis, we will define

$$\mathbf{w} = \begin{bmatrix} \mathbf{W}_{:,1} \\ \mathbf{W}_{:,2} \\ \vdots \\ \mathbf{W}_{:,k} \end{bmatrix}. \quad (13.54)$$

Where $\mathbf{W}_{:,i}$ is the i^{th} column of $\mathbf{W}$ such that $\mathbf{w} \in \mathbb{R}^{dk}$. We also let

$$\mathbf{z}(i)^T = \begin{bmatrix} \bar{0} & \dots & \bar{0} & z^T & \bar{0} & \dots & \bar{0} \end{bmatrix} \quad (13.55)$$

such that z is placed in the i^{th} group of d elements and $\bar{0}$ is a row vector of d zeros. Vector $\mathbf{z}(i) \in \mathbb{R}^{dk}$ since there are k groups. With these definitions, we write the softmax activated

network defined by $\mathbf{W}$ with input z :

$$s(z^T \mathbf{W})_i = \frac{e^{\mathbf{z}(i)^T \mathbf{w}}}{\sum_{j=1}^k e^{\mathbf{z}(j)^T \mathbf{w}}}. \quad (13.56)$$

We can simplify this by plugging in for the definition of s :

$$\text{CEL}_s(\mathbf{w}) := \text{CEL}_s(\mathbf{W}) = - \sum_{i=1}^k y_i \left[\mathbf{z}(i)^T \mathbf{w} - \log \left(\sum_{j=1}^k e^{\mathbf{z}(j)^T \mathbf{w}} \right) \right] \quad (13.57)$$

$$= - \sum_{i=1}^k y_i \mathbf{z}(i)^T \mathbf{w} + \log \left(\sum_{i=1}^k e^{\mathbf{z}(i)^T \mathbf{w}} \right). \quad (13.58)$$

These are equivalent because $\sum_{i=1}^k y_i = 1$. For readability, we let $p_i := s(z^T \mathbf{W})_i$. With these preliminaries the Hessian will be a 2-tensor and the $\nabla_{\mathbf{w}}^3$ term will be a 3-tensor. We compute the gradient and Hessian and $\nabla_{\mathbf{w}}^3$ term:

$$\nabla_{\mathbf{w}} \text{CEL}_s(\mathbf{w}) = - \sum_{i=1}^k y_i \mathbf{z}(i) + \sum_{i=1}^k \mathbf{z}(i) p_i \quad (13.59)$$

$$\nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w}) = \sum_{i=1}^k \mathbf{z}(i) \mathbf{z}(i)^T p_i - \left(\sum_{i=1}^k \mathbf{z}(i) p_i \right) \left(\sum_{j=1}^k \mathbf{z}(j)^T p_j \right). \quad (13.60)$$

We write $\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})$ termwise to simplify notation:

$$\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w}) = \begin{cases} (p_i - 3p_i^2 + 2p_i^3) z \otimes z^T \otimes z^\perp & i = j = l \\ (-p_i p_l + 2p_i^2 p_l) z \otimes z^T \otimes z^\perp & i = j \neq l \\ (-p_j p_i + 2p_j^2 p_i) z \otimes z^T \otimes z^\perp & j = l \neq i \\ (-p_l p_j + 2p_l^2 p_j) z \otimes z^T \otimes z^\perp & l = i \neq j \\ (2p_i p_j p_l) z \otimes z^T \otimes z^\perp & i \neq j \neq l \end{cases} \quad (13.61)$$

Where $\otimes$ is the tensor product and $z \otimes z^T \otimes z^\perp \in \mathbb{R}^{d \times d \times d}$ is a 3-tensor with the abuse of notation: $z \in \mathbb{R}^{d \times 1 \times 1}$, $z^T \in \mathbb{R}^{1 \times d \times 1}$, and $z^\perp \in \mathbb{R}^{1 \times 1 \times d}$. Thus $\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w}) \in \mathbb{R}^{dk \times dk \times dk}$.

For twice-differentiable functions, the Lipschitz constant is given by the greatest eigenvalue of the Hessian. Correspondingly, the smoothness constant is given by the greatest

eigenvalue of the $\nabla_{\mathbf{w}}^3$ term for thrice-differentiable functions. Thus, we aim to bound the largest value that the Rayleigh quotient can take for any unit-length vector x . For the Hessian:

$$x^T \nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w}) x \leq |x^T \nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w}) x| = \|x^T \nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w}) x\|_F \quad (13.62)$$

$$\leq \|x\|^2 \|\nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w})\|_F = \|\nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w})\|_F \quad (13.63)$$

$$= \sqrt{\sum_{i=1}^k \|zz^T\|_F (p_i - p_i^2)^2 + \sum_{i=1}^k \sum_{j=1, j \neq i}^k \|zz^T\|_F (p_i p_j)^2} \quad (13.64)$$

$$= \sqrt{\sum_{i=1}^k (p_i - p_i^2)^2 + \sum_{i=1}^k \sum_{j=1, j \neq i}^k (p_i p_j)^2}. \quad (13.65)$$

The Frobenius norm is maximized when $p_i = \frac{1}{k}$ for $k > 1$:

$$\|\nabla_{\mathbf{w}}^2 \text{CEL}_s(\mathbf{w})\|_F \leq \sqrt{k \left(\frac{1}{k} - \frac{1}{k^2} \right)^2 + k(k-1) \left(\frac{1}{k^2} \right)^2} \quad (13.66)$$

$$= \frac{\sqrt{k-1}}{k}. \quad (13.67)$$

Thus, for $\text{CEL}_s(\mathbf{w})$, the Lipschitz constant, $c_L \leq \frac{\sqrt{k-1}}{k}$ when $k > 1$. We can also show that the Rayleigh quotient is lower bounded by 0 by following analogous steps in [215] (these steps are omitted from this appendix), and thus $\text{CEL}_s(\mathbf{w})$ is convex. Next, we examine the Rayleigh quotient of the $\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})$. Analogous to the procedure for the Hessian, we make use of a 3-tensor analog of the Frobenius norm: $\|M\|_{3,F} := \sqrt{\sum_{i=1}^k \sum_{j=1}^k \sum_{l=1}^k M(i, j, l)^2}$. Thus we have the following inequality

$$x^T \otimes [x^\perp \otimes \nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})] \otimes x \leq \|\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})\|_{3,F}. \quad (13.68)$$

Since $\|z \otimes z^T \otimes z^\perp\|_{3,F} = 1$, we can write this as

$$\|\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})\|_{3,F} \leq \sqrt{\begin{aligned} & \sum_{i=1}^k (p_i - 3p_i^2 + 2p_i^3)^2 + \sum_{i=1}^k \sum_{j=1, j \neq i}^k (-p_i p_l + 2p_i^2 p_l)^2 \\ & + \sum_{j=1}^k \sum_{l=1, l \neq j}^k (-p_j p_i + 2p_j^2 p_i)^2 + \sum_{l=1}^k \sum_{i=1, i \neq l}^k (-p_l p_j + 2p_l^2 p_j)^2 \\ & + \sum_{i=1}^k \sum_{j=1, j \neq i}^k \sum_{l=1, l \neq j}^k (2p_i p_j p_l)^2. \end{aligned}} \quad (13.69)$$

This is maximized when $p_i = \frac{1}{k}$ for $k > 2$, which was verified with the symbolic integrator Mathematica [221]. Simplifying results in:

$$\|\nabla_{\mathbf{w}}^3 \text{CEL}_s(\mathbf{w})\|_{3,F} \leq \sqrt{\frac{(k-1)(k-2)}{k^3}}. \quad (13.70)$$

Thus for $\text{CEL}_s(\mathbf{w})$, the smoothness constant, $c_S \leq \sqrt{\frac{(k-1)(k-2)}{k^3}}$ when $k > 2$. When $k = 2$, $p_1, p_2 = \frac{1}{2} \pm \frac{\sqrt{3}}{6}$ and $c_S \leq \sqrt{\frac{2}{27}}$.

13.7.10 Study on Base-learning Learning Rate and Number of Update Steps

In this section we present additional results on the performance of the algorithms with different iterations and learning rates using the same example setup as in Section 13.5.1. Note that we have not used the sample convergence bound (see Appendix 13.7.6) and present results for a single sample $\theta \sim P_\theta$. The true values of the upper bounds for MLAP-M [16], MR-MAML [225], and PAC-BUS (our method) are unlikely to change by more than 5% as the sample complexity bound does not loosen the guarantee very much. We present these results to provide a qualitative sense of the guarantees and their trends for varying base-learning learning rates and number of update steps.

Below we present test losses for MAML [72] (as a baseline), MLAP-M [16], MR-MAML [225], and PAC-BUS for base-learning rates (lr_b) of 0.01 to 10 using $\{1, 3, 10\}$ adaptation steps (AS). We also present the computed bounds for MLAP-M [16], MR-MAML [225], and PAC-BUS for the same set of hyper-parameters.

MAML Test Loss, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.184±0.007	0.184±0.008	0.168±0.006	0.152±0.004	0.120±0.001	0.114±0.001	0.133±0.007
Adaptation steps = 3	0.177±0.006	0.179±0.002	0.149±0.002	0.126±0.001	0.115±0.001	0.106±0.001	0.123±0.004
Adaptation steps = 10	0.179±0.004	0.155±0.002	0.128±0.001	0.124±0.001	0.113±0.001	0.104±0.002	0.129±0.008
MLAP-M Test Loss, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.181±0.010	0.175±0.014	0.150±0.006	0.129±0.009	0.083±0.001	0.065±0.003	0.220±0.044
Adaptation steps = 3	0.178±0.006	0.159±0.007	0.102±0.005	0.081±0.003	0.064±0.001	0.050±0.004	0.379±0.021
Adaptation steps = 10	0.161±0.005	0.115±0.002	0.078±0.004	0.063±0.002	0.050±0.001	0.045±0.002	0.919±0.036
MR-MAML Test Loss, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.171±0.003	0.169±0.003	0.163±0.003	0.146±0.002	0.127±0.001	0.128±0.000	0.178±0.008
Adaptation steps = 3	0.170±0.002	0.166±0.001	0.146±0.002	0.128±0.001	0.123±0.001	0.118±0.001	0.163±0.022
Adaptation steps = 10	0.165±0.002	0.152±0.002	0.129±0.001	0.126±0.001	0.118±0.001	0.115±0.001	0.139±0.009
PAC-BUS Test Loss, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.176±0.002	0.171±0.003	0.160±0.001	0.145±0.002	0.127±0.001	0.129±0.002	0.164±0.019
Adaptation steps = 3	0.170±0.002	0.165±0.002	0.145±0.001	0.129±0.002	0.123±0.001	0.120±0.001	0.144±0.014
Adaptation steps = 10	0.163±0.001	0.150±0.001	0.130±0.001	0.126±0.002	0.119±0.002	0.115±0.002	0.130±0.004
MLAP-M Bound, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	1.003±0.000	1.015±0.001	1.223±0.020	1.946±0.043	3.113±0.154	5.435±0.220	21.874±0.420
Adaptation steps = 3	1.008±0.000	1.087±0.027	1.864±0.062	3.072±0.157	4.147±0.095	6.760±0.233	28.356±1.826
Adaptation steps = 10	1.050±0.006	1.535±0.044	2.574±0.064	4.009±0.107	5.98±0.057	10.119±0.087	47.971±1.346
MR-MAML Bound, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.344±0.002	0.343±0.002	0.335±0.002	0.320±0.001	0.300±0.000	0.303±0.001	0.351±0.006
Adaptation steps = 3	0.344±0.002	0.340±0.002	0.320±0.002	0.302±0.002	0.296±0.001	0.292±0.001	0.335±0.018
Adaptation steps = 10	0.339±0.001	0.324±0.002	0.303±0.000	4.752±0.808	5.330±0.187	6.316±0.639	9.134±1.448
PAC-BUS Bound, $lr_b =$	0.01	0.03	0.1	0.3	1	3	10
Adaptation steps = 1	0.216±0.002	0.216±0.002	0.204±0.002	0.188±0.002	0.169±0.000	0.171±0.001	0.207±0.021
Adaptation steps = 3	0.252±0.001	0.247±0.002	0.228±0.002	0.211±0.001	0.204±0.001	0.200±0.002	0.228±0.017
Adaptation steps = 10	0.383±0.002	0.372±0.001	0.350±0.001	1.160±0.093	1.288±0.056	1.650±0.055	2.221±0.256

These results show the dependence that the PAC-BUS upper bound (specifically the uniform stability regularizer term β_{US}) has on the learning rate and number of base-learning update steps whereas the bound for MR-MAML does not suffer with increasing base-learning steps or learning rate. However, once the learning rate and number of adaptation steps are too large, all bounds worsen significantly. The tightest guarantee obtained using PAC-BUS is significantly stronger than those for any tuning of MR-MAML and MLAP-M. We bold the tightest guarantee achieved in the tables above to highlight this.

13.7.11 Additional Experimental Details

In this section, we report information about the data used, the procedure for prior, train, and test splits, as well as other experimental details. Code capable of reproducing the results in this paper is publicly available at <https://github.com/irom-lab/PAC-BUS>. All results provided in this paper were computed on an Amazon Web Services (AWS) p2 in-

stances. Tuning and intermediate results were computed on a desktop computer with a 12-core Intel i7-8700k CPU and an NVIDIA Titan Xp GPU. In addition, we made use of several existing software assets: SciKit-learn [158] (BSD license), PyTorch [157] (BSD license), CVXPY [60, 6] (Apache License, Version 2.0), MOSEK [147] (software was used with a personal academic license, see <https://www.mosek.com/products/license-agreement> for more details), learn2learn [18] (MIT License), and h5py [52] (Python license, see <https://docs.h5py.org/en/stable/licenses.html> for more details).

Circle Class

We randomly sample points from the unit ball $B^2(0, 1)$ and classify them as (+) or (−) according to whether or not the points are outside the ball $B^2(c_t, r_t)$. For the tasks which are used to train a prior, we sample c_t from $[0.1, 0.5]$ and r_t from $[0.1, 1 - \|c_t\|]$. For the meta-training and meta-testing tasks, we sample c_t from $[0.1, 0.4]$ and r_t from $[0.1, 1 - \|c_t\|]$.

For all methods, we train the prior on 500 tasks, train the network on 10000 tasks, and test on 1000 tasks. We report the meta-test loss and a guarantee on the loss if applicable. A single task is a 2-class 10-sample (i.e. there are 10 samples given in total for training, not 10 samples from each class) learning problem. The evaluation dataset S_{ev} consists of a dataset S of 10 base-learner training samples and a dataset S_{va} of 250 validation samples. For PAC-BUS, we searched for the meta-learning rate in $[1\text{e-}4, 1]$, the base-learning rate in $[0.01, 10]$, and the number of base-learning update steps in $[1, 10]$. The resulting parameters for the 10-shot learning problems are: meta-learning rate $1\text{e-}3$, base-learning rate 0.05, and 1 base-learning update step. Note that in this example and the *Mini-wiki* example, we select the number of base-learning steps such that the upper bound is minimized. A lower loss may have been achievable with more base-learning update steps, but we aim to produce the tightest bound possible. Training for each method took less than 1 hour on the AWS p2 instance and computing the sample convergence upper bound took approximately 3 days when applicable.

Mini-wiki

In Table 13.4, we present additional results – the percentage of correctly classified sentences on test tasks (after the base learner’s adaptation step). Note that we present these results with the same posterior as was used to generate the results in Table 13.2.

Table 13.4: Meta-test accuracy as a percentage for MAML, FLI-Batch, MR-MAML, and PAC-BUS. We report the mean and standard deviation after 5 trials.

4-WAY <i>Mini-Wiki</i>	1-SHOT $\uparrow$	3-SHOT $\uparrow$	5-SHOT $\uparrow$
MAML [72]	60.2 ± 0.9	68.3 ± 0.7	71.9 ± 0.6
FLI-BATCH [109]	46.0 ± 5.9	48.7 ± 4.9	54.5 ± 2.4
MR-MAML [225]	59.9 ± 0.8	68.4 ± 0.7	71.8 ± 0.7
PAC-BUS (OURS)	59.9 ± 0.8	68.1 ± 0.7	71.2 ± 0.7

We use the *Mini-wiki* dataset from [109], which consists of 813 classes each with at least 1000 example sentences from that class’s corresponding Wikipedia article. The dataset was derived from the Wiki3029 dataset presented in [21], which was created from a public domain (CC0 license) Wikipedia dump. Although the Wikipedia dump is open source, it is possible that content which is copyrighted was used since the datasets are large and it is difficult to moderate all content on the website. In addition, it is possible that the dataset has some offensive content such as derogatory terms or curse words. However, since these are in the context Wikipedia articles, the authors trust that the original article was not written maliciously, but for the purposes of education. We use the first 62 classes of *Mini-wiki* for training the prior, the next 625 for the meta-training, and the last 126 for meta-testing. Before creating learning tasks, we remove all sentences with fewer than 120 characters.

For all methods, we train the prior on 100 tasks, train the network on 1000 tasks, and test on 200 tasks. We report the meta-test score, the meta-test loss, and a guarantee on the loss if applicable. A single task is a 4-class $\{1, 3, 5\}$ -shot learning problem. The evaluation dataset S_{ev} consists of a dataset S of $\{1, 3, 5\}$ base-learner training samples and a dataset S_{va} of $\{250, 250, 250\}$ validation samples respectively. For PAC-BUS, we search for the meta-learning rate in $[0.01, 1]$ the base-learning rate in $[1e-3, 100]$, and the number of base-learning update steps in $[1, 50]$. The resulting parameters for the $\{1, 3, 5\}$ -shot

learning problems are: meta-learning rate $\{0.1, 0.1, 0.1\}$, base-learning rate $\{2.5, 5, 5\}$, and $\{2, 4, 5\}$ base-learning update steps respectively. Training for each method took less than 1 hour on the AWS p2 instance and computing the sample convergence upper bound took approximately 2 days when applicable.

Omniglot

We use the *Omniglot* dataset from [113], which consists of 1623 characters each with 20 examples. The dataset was collected using Amazon’s Mechanical Turk (AMT) and is available on GitHub with an MIT license. This dataset was collected voluntarily by AMT workers. Since the dataset is small enough, it can be checked visually for personally-identifiable information. We use the first 1200 characters for meta-training and the remaining 423 for meta-testing. The image resolution is reduced to 28×28 . In the non-mutually exclusive setting, the 1200 training characters are randomly partitioned into 20 equal-sized groups which are assigned a fixed class label from 1 to 20. Note that this is distinct from the method described in [225] where the data is partitioned into 60 disjoint sets. Both experimental setups cause memorization, but the setup used in [225] causes more severe memorization than ours. This is why our implementation of MAML performs better than the results for MAML reported in [225]. However, our implementation of MR-MAML(W) method performs similarly to what is reported in [225].

For all methods, we trained on 100000 batches of 16 tasks and report the meta-test score on 8000 test tasks. We also used 5 base-learning update steps for all methods. A single task is a 20-way $\{1, 5\}$ -shot learning problem. The evaluation dataset S_{ev} consists of a dataset S of $\{1, 5\}$ base-learner training samples and a dataset S_{va} of $\{4, 5\}$ validation samples respectively. For PAC-BUS(H), we searched for the regularization scales λ_1 and λ_2 in $[1\text{e-}7, 1]$ and $[1\text{e-}4, 1\text{e}4]$ respectively. Additionally, the meta-learning rate was selected from $[5\text{e-}4, 0.1]$, and the base-learning rate was selected from $[0.01, 10]$. The resulting parameters for the $\{1, 5\}$ -shot learning problems are: $\lambda_1 = \{1\text{e-}3, 1\text{e-}4\}$, $\lambda_2 = \{10, 10\}$, meta-learning rate $\{1\text{e-}3, 1\text{e-}3\}$, and base-learning rate $\{0.5, 0.5\}$ respectively. Training for each method took approximately 3 days on the AWS p2 instance.

Chapter 14

Towards a Framework for Comparing the Complexity of Robotic Tasks

MICHELLE HO,^{*} ALEC FARID,^{*} AND ANIRUDHA MAJUMDAR

We are motivated by the problem of comparing the *complexity* of one robotic task relative to another. To this end, we define a notion of *reduction* that formalizes the following intuition: Task 1 reduces to Task 2 if we can efficiently transform *any* policy that solves Task 2 into a policy that solves Task 1. We further define a quantitative measure of the *relative complexity* between any two tasks for a given robot. We prove useful properties of our notion of reduction (e.g., reflexivity, transitivity, and antisymmetry) and relative complexity measure (e.g., nonnegativity and monotonicity). In addition, we propose practical algorithms for estimating the relative complexity measure. We illustrate our framework for comparing robotic tasks using (i) examples where one can analytically establish reductions, and (ii) reinforcement learning examples where the proposed algorithm can estimate the relative complexity between tasks.

14.1 Introduction

Consider the following pairs of robotic tasks: (i) an autonomous truck driving on one side of the road vs. the other side, (ii) an autonomous car driving in one city vs. another, and (iii) a cartpole balancing the pole upright vs. downward. Intuitively, the two tasks in (i) are “as hard as each other”, while one task in (iii) (i.e., balancing upright) is more challenging than the other (i.e., balancing downward). The tasks in (ii) may be challenging in different ways and thus may not admit a straightforward ordering. How can we *formally* compare the *complexity* of each pair of tasks?

Despite decades of algorithmic advancements in robotics, we currently lack precise mathematical foundations for answering such questions. In contrast, *computational complexity theory* [19] provides a unifying framework and set of tools for establishing and comparing the difficulty of *all* computational problems. It also guides practitioners by setting expectations for the kinds of algorithms that will solve a given problem. For example, a problem that is as hard as 3-SAT will not admit a polynomial-time solution (unless $P = NP$); thus, a practitioner is motivated to find *approximation algorithms* for such a problem. We currently lack such a unifying theory for understanding the complexity of robotic tasks.

Statement of Contributions. Motivated by this challenge, we take a step towards developing a precise framework for establishing the *relative complexity* of robotic tasks. To this end, we make five specific contributions.

- **Reductions between tasks.** Our key insight is to define a notion of *reduction* [19, Ch. 2] between robotic tasks (Definition 14.4). This definition formalizes the following intuition: Task 1 reduces to Task 2 if we can transform *any* policy that solves Task 2 into a policy that solves Task 1. The reduction is “easy” if the transformation is computationally efficient (e.g., polynomial-time). Crucially, this notion of reduction captures the relative complexity of tasks in terms of the complexity of *online* task execution (i.e., how challenging the two tasks are from the perspective of the robot rather than the robot’s software designer).
- **Quantifying relative complexity.** We propose a *quantitative measure* (that outputs values in the range $[0, 1]$) for comparing the complexity of one robotic task relative

to another (Definition 14.7). One can think of this measure as a “smoothed” version of our notion of reduction, where this notion of relative complexity can be defined for arbitrary tasks (in contrast to the definition of reduction, which captures a strict and binary notion of relative complexity).

- **Properties of reduction and relative complexity.** We prove basic properties of our notion of reduction between tasks (e.g., reflexivity, transitivity, and antisymmetry in Proposition 14.1) and the relative complexity measure (e.g., non-negativity and monotonicity in Proposition 14.5).
- **Algorithm.** We propose a practical algorithm based on *adversarial training* for estimating the relative complexity measure for robotic tasks in reinforcement learning contexts (Algorithm 14.1).
- **Examples.** We demonstrate our framework using (i) illustrative examples where one can analytically establish reductions (Sec. 14.5), and (ii) numerical examples based on reinforcement learning problems where we apply our proposed algorithm for estimating relative complexity (Sec. 14.8).

14.2 Related Work

Complexity and robotics. The study of complexity theoretic questions in robotics has a long history. Early work [165, 41] established the PSPACE-completeness of the general motion planning problem. Other results include PSPACE-hardness [98, 97, 106, 55, 197] and NP-hardness [148, 32, 33, 89, 87] of various planning problems. Complexity results for systems with nontrivial (e.g., nonlinear or uncertain) dynamics have also been explored in control theory [30, 8]. The problem we consider in this paper is fundamentally different from the ones above. Specifically, the model of computation in the problems mentioned above assumes that the Turing machine is provided with a complete encoding of the problem on its tape at the very outset (e.g., rational numbers encoding the linear inequalities that define polytopic obstacles in the environment); the algorithm’s task is then to perform a given computation on this fixed encoding. In contrast, we are interested in the computational

resources required by a robot *as it is performing a given task*. In this model, the robot’s embodiment is of crucial importance; instead of a fixed encoding of the input, the robot’s sensors provide information incrementally and interactively based on its control actions and the environment.

Reductions for comparing robots and sensors. The pioneering work in [61] proposed a formalism for measuring the intrinsic complexity of robotic tasks in terms of *information invariants* and a notion of reduction for comparing different robots. The work presented in [154] has a similar goal, but employs the notion of *information spaces* [117, Chapter 11] in order to handle tasks where sensors only provide partial state information. Notions of reductions for comparing the power of different sensors have also been developed [118, 65]. While our work is inspired by the frameworks above and also uses the idea of reductions, our focus is distinct. In particular, our goal is to compare different robotic *tasks* (instead of different robots). In addition, we propose a measure of *relative complexity* between two tasks that goes beyond the strict notion of reduction and allows one to quantify how complex one task is with respect to another in terms of online computational resources required by the robot.

Reductions and task complexity in learning. Our definition of reduction between tasks formally captures the intuition that Task 2 is at least as complex as Task 1 if any solution (in the form of a control policy) for Task 2 can be transformed into a solution for Task 1. The idea of transforming solutions for one task into solutions for another has also been exploited for tackling problems in machine learning. In particular, [47, 126] propose approaches for composing previously-learned “modules” in order to speed up the process of learning on a new problem. In contrast to our work, these approaches do not seek to compare the difficulty of tasks (their goal is to obtain practical benefits in terms of sample efficiency and speed of learning). There has also been work on defining (asymmetric) notions of distance between supervised learning tasks [4, 5, 205] (e.g., using techniques from Kolmogorov complexity theory [125]). These approaches are motivated by problems in *transfer learning* and attempt to capture how quickly solutions from one learning problem can be fine-tuned for a new learning problem. In contrast to these measures, we seek to capture the relative complexity between tasks in terms of the complexity of computations

that must be executed *online* (instead of the complexity of offline fine-tuning). In addition, we are motivated by robotic tasks in contrast to supervised learning problems.

14.3 Problem Formulation

Robots, Environments, and Rewards. Let $p(s_t|s_{t-1}, a_{t-1})$ describe a robot’s dynamics, where $s_t \in \mathcal{S}$ represents the combined state of the robot and its environment at time-step t , and $a_t \in \mathcal{A}$ represents the action. Let p_0 denote the initial state distribution. We denote the robot’s sensor mapping as $\sigma(o_t|s_t)$. We consider robotic tasks that are prescribed using reward functions; let $\sum_t r(s_t, a_t) \in \mathbb{R}$ denote the cumulative reward over a given (finite or infinite) time horizon.

Policies. Let $\pi : \mathcal{O} \rightarrow \mathcal{A}$ denote a policy for some observation space $\mathcal{O}$ and action space $\mathcal{A}$. We will denote the set of all policies (i.e., all mappings from $\mathcal{O}$ to $\mathcal{A}$) by Π . One can extend the formulation we present to policies with memory by augmenting the observation space to keep track of memory states (potentially in a similar way to [176, 193]). We will define the reward achieved by a policy as:

$$R(\pi) := \min \left(\mathbb{E}_{p_0, p, \sigma} \sum_t r(s_t, \pi(o_t)), R^* \right), \quad (14.1)$$

where R^* is a chosen success threshold and forms an upper bound on $R(\pi)$.

Tasks. A *task* is formally defined as a partially observable Markov decision process (POMDP) using the tuple $\tau := (\mathcal{S}, \mathcal{A}, \mathcal{O}, p, \sigma, r, p_0, R^*)$ which describes the state space, action space, observation space, dynamics, sensor, reward function, the distribution over initial states, and the threshold for success. For example, a task τ could correspond to a cartpole swinging up and balancing (with random initial conditions) or a drone navigating through random obstacle environments (where the randomness over obstacle placements is defined using p_0 ; recall that the state s_t encapsulates the combined state of the robot and its environment). We will index tasks using ξ and let $\mathcal{T} = \{\tau_\xi\}_\xi$ denote a set of tasks. A particular task in $\mathcal{T}$ is thus denoted as $\tau_\xi := (\mathcal{S}_\xi, \mathcal{A}_\xi, \mathcal{O}_\xi, p_\xi, \sigma_\xi, r_\xi, p_{0,\xi}, R_\xi^*)$. We will say that a policy π_ξ is *admissible* on task τ_ξ if $R_\xi(\pi_\xi) = R_\xi^*$. We use shorthand π_ξ^* to denote an

admissible policy on task τ_ξ . Let $\Pi_\xi^* \subseteq \Pi_\xi$ be the set of all admissible policies on task τ_ξ .

Task reduction and relative complexity. Our goal is to introduce notions which meaningfully and quantitatively compare the complexity of robotic tasks. First, we aim to develop a *binary relation* (denoted by “ $\preceq$ ”) between two tasks which will provide a notion of reduction, i.e., if task τ_1 reduces to task τ_2 ($\tau_1 \preceq \tau_2$), then task τ_2 is at least as complex as task τ_1 . Second, we aim to formulate a measure $\mathcal{T} \times \mathcal{T} \rightarrow [0, 1]$ that compares the relative complexity of one task with respect to another. The goal is to not only establish if one task is more complex than another, but the *degree* to which it is more complex.

14.4 Task Reduction

We propose a definition of *reduction* for robotic tasks in order to formalize what it means for one task to be as hard as another. The idea of reductions comes from the theory of computational complexity; we review basic definitions in Sec. 14.4.1 before describing reductions between robotic tasks in Sec. 14.4.2.

14.4.1 Background

We begin with the definition of a *Turing* reduction between two computational problems. Let A and B be *decision problems*, i.e., problems where each instance has a yes/no answer (e.g., 3-SAT). Let O_B be an oracle for decision problem B . This oracle is a blackbox which outputs the solution to any instance $b \in B$.

Definition 14.1 (Turing Reduction [196]). *Decision problem A reduces to B (written $A \preceq_T B$) if we can compute the solution to all instances of A using an oracle O_B for B .*

Intuitively, A reduces to B if one can use an oracle for B as a sub-routine for solving instances of A , and thus B is at least as complex as A . Note that this definition does not include reference to the complexity of the resulting function for solving A . We are specifically interested in *efficient* reductions. A particular notion of efficiency is formalized by *polynomial-time* reductions, as defined below.

Definition 14.2 (Cook Reduction [196]). *Decision problem A is polynomial-time reducible*

to B (written $A \preceq_T^P B$) if $A \preceq_T B$ and the solution to any instance of A makes only a polynomial number of calls to O_B .

Importantly, if O_B is efficient (i.e., runs in polynomial time) and we have $A \preceq_T^P B$, then the resulting function for solving A is also efficient. We aim to define a notion of reduction for robotic tasks that is inspired by the notions above for computational problems. In particular, we will leverage the core concept of using an oracle for one decision problem in order to solve another.

14.4.2 Task Reduction: Definition and Properties

In this section, we propose a notion of reduction between two robotic tasks and demonstrate that this defines a partial ordering on a given space $\mathcal{T}$ of tasks. Our notion of reduction captures the following intuition: task τ_1 reduces to task τ_2 if we can utilize a policy (“oracle”) for τ_2 in order to solve τ_1 . In order to formalize this, we first introduce *encoders* and *decoders*.

Definition 14.3 (Encoder and Decoder). *Let $\mathcal{T}$ be a space of tasks and let $\tau_1, \tau_2 \in \mathcal{T}$ be two tasks (as defined in Sec. 14.3). Let $\mathcal{O}_1, \mathcal{O}_2$ and $\mathcal{A}_1, \mathcal{A}_2$ be the corresponding observation and action spaces. Let $H_{1,2}$ denote a space of functions from $\mathcal{O}_1$ to $\mathcal{O}_2$, and let $G_{2,1}$ denote a space of functions from $\mathcal{A}_2$ to $\mathcal{A}_1$. We will refer to a function $h \in H_{1,2}$ as an encoder and a function $g \in G_{2,1}$ as a decoder.*

Intuitively, a task τ_1 reduces to τ_2 if we can utilize *any* admissible policy for τ_2 and transform it via an encoder and decoder into a policy that solves τ_1 ; see Fig. 14.1 for a visual representation of this transformation. Importantly, none of the elements which define a task ($\tau := (\mathcal{S}, \mathcal{A}, \mathcal{O}, p, \sigma, r, p_0, R^*)$) need to be shared between the two tasks. We formalize the notion of *task reduction* below.

Definition 14.4 (Task Reduction). *Task τ_1 reduces to task τ_2 (written $\tau_1 \preceq \tau_2$) if for all admissible policies $\pi_2^* \in \Pi_2^*$, there exists an encoder $h \in H_{1,2}$ and a decoder $g \in G_{2,1}$ such that:*

$$g \circ \pi_2^* \circ h \in \Pi_1^* \quad (14.2)$$

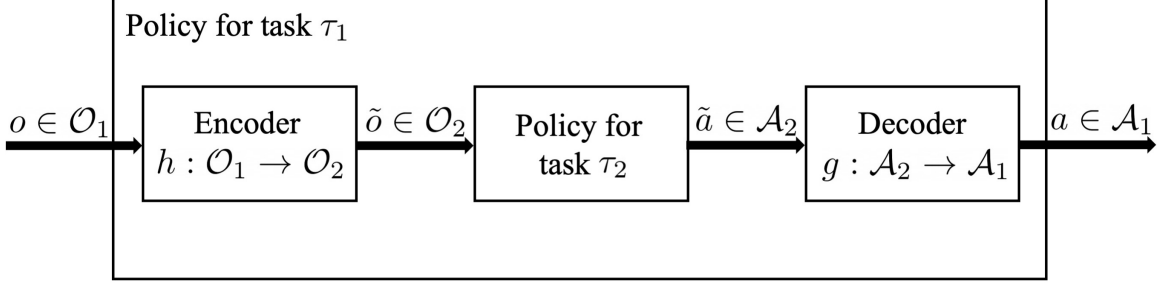


Figure 14.1: The transformation of a policy for one task to another using *encoders* and *decoders*. An observation $o \in \mathcal{O}_1$ from task τ_1 is encoded to an observation $\tilde{o} \in \mathcal{O}_2$ for τ_2 . The policy for τ_2 outputs an action $\tilde{a} \in \mathcal{A}_2$ which is decoded to an action $a \in \mathcal{A}_1$ for task τ_1 . Together, the encoder, policy for τ_2 , and decoder are a policy for τ_1 .

This definition is illustrated in Fig. 14.1. This notion of reduction captures the relative complexity of tasks in terms of the complexity of online task execution. Thus, we compare the complexity of solutions rather than the complexity of computing solutions. It is important to note that the definition of reduction calls for the ability to transform *any* admissible policy $\pi_2^* \in \Pi_2^*$ into an admissible policy for τ_1 . We also note that task reductions are conditioned on the selection of $H_{1,2}$ and $G_{2,1}$. Intuitively, this corresponds to the “complexity” of the reduction. For example, if $H_{1,2}$ and $G_{2,1}$ only include functions which can be evaluated in polynomial time, then the reduction $\tau_1 \preceq \tau_2$ is *efficient* (analogous to Cook reductions). Additionally, note that a robot executing the policy $g \circ \pi_2^* \circ h \in \Pi_1^*$ for task τ_1 over a time horizon of T requires T evaluations of π_2^* . Thus, if $H_{1,2}$ and $G_{2,1}$ only include efficiently-computable functions and π_2^* is efficiently computable, then the *online* execution of τ_1 is efficient. One can specify different classes of functions for $H_{1,2}$ and $G_{2,1}$ in order to capture different notions of efficiency (e.g., neural networks with bounded size). Next, we define *equivalence* between two tasks.

Definition 14.5 (Task Equivalence). *Tasks τ_1 and τ_2 are equivalent ($\tau_1 \equiv \tau_2$) if $\tau_1 \preceq \tau_2$ and $\tau_2 \preceq \tau_1$.*

Note that task reduction and equivalence are both binary relations (over $\mathcal{T} \times \mathcal{T}$). As we show below, these relations satisfy properties that are intuitively desirable for any definition of reduction (or equivalence) between robotic tasks. In order to state these properties, we first introduce a notion of function closure.

Definition 14.6 (Closure under Composition on $\mathcal{T}$). *Let $g_1 \in G_{2,1}$, $g_2 \in G_{3,2}$, $h_1 \in H_{1,2}$, and $h_2 \in H_{2,3}$. We say that $G_{3,1}$ and $H_{1,3}$ are closed under composition if for any $\tau_1, \tau_2, \tau_3 \in \mathcal{T}$, the following are true:*

$$g_1 \circ g_2 \in G_{3,1}, \quad \forall g_1 \in G_{2,1} \quad \forall g_2 \in G_{3,2}, \quad (14.3)$$

$$h_2 \circ h_1 \in H_{1,3}, \quad \forall h_1 \in H_{1,2} \quad \forall h_2 \in H_{2,3}. \quad (14.4)$$

When $G_{\zeta,\xi}$ and $H_{\xi,\zeta}$ are the same for all tuples of tasks $(\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, this definition simplifies to closure under function composition for $G_{\zeta,\xi}$ and $H_{\xi,\zeta}$.

Proposition 14.1 (Task Reduction is a Non-Strict Partial Ordering Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $G_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task reductions satisfy the following properties and thus define a non-strict partial ordering relation.*

Property 14.1.a. *Reflexivity:* $\tau_1 \preceq \tau_1$.

Property 14.1.b. *Antisymmetry:* $\tau_1 \prec \tau_2 \implies \neg(\tau_2 \preceq \tau_1)$, where $\tau_1 \prec \tau_2$ is defined as $(\tau_1 \preceq \tau_2) \wedge \neg(\tau_1 \equiv \tau_2)$.

Property 14.1.c. *Transitivity:* $(\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \implies \tau_1 \preceq \tau_3$.

Proof. A complete proof of these properties is provided in Appendix 14.10.1.

Additionally, note that *strict* task reduction $\tau_1 \prec \tau_2$ is a strict partial ordering relation. We provide a complete proof of this in Appendix 14.10.1. These properties formalize a series of intuitions about task reductions. Task reduction consists of transforming an admissible policy from one task to another. In the case of reflexivity, intuition suggest that if the tasks are the same, no transformation will be required for the policy to work. Thus, G and H must include the identity to allow for this. Antisymmetry establishes a clear sense of “dominance” between tasks in terms of complexity — if a task τ_1 strictly reduces to another task τ_2 , then τ_2 should not reduce to τ_1 . Building on antisymmetry, we also require transitivity to make a clear chain of increasingly complex tasks. A series of tasks which

reduce to each other cyclically (i.e. $\tau_1 \preceq \tau_2 \wedge \tau_2 \preceq \tau_3 \wedge \tau_3 \preceq \tau_1$) should only be possible if all the tasks are equivalent. The following proposition proves useful properties of task equivalence.

Proposition 14.2 (Task Equivalence is an Equivalence Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $G_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task equivalence satisfies the following properties and thus defines an equivalence relation.*

Property 14.2.a. *Reflexivity:* $\tau_1 \equiv \tau_1$.

Property 14.2.b. *Symmetry:* $\tau_1 \equiv \tau_2 \implies \tau_2 \equiv \tau_1$.

Property 14.2.c. *Transitivity:* $\tau_1 \equiv \tau_2 \wedge \tau_2 \equiv \tau_3 \implies \tau_1 \equiv \tau_3$.

Proof. A complete proof of these properties is provided in Appendix 14.10.1.

14.5 Examples of Reductions

In this section, we provide two concrete examples of task reductions (Definition 14.4) and equivalence (Definition 14.5). We present an efficient reduction between navigation tasks with differing goal locations in an environment which has 90 degree rotational symmetry. We then show reduction between a set of tasks (i.e., an *equivalence* class of tasks) for grasping with varying camera viewpoints.

14.5.1 Navigation to a Goal with Map Rotations

Consider a robot which must navigate to a goal location and avoid obstacles. Suppose that the robot lives on an $[-n, n] \times [-n, n] \in \mathbb{R}^2$ world and can move a distance d in a cardinal direction $\{N, E, S, W\}$ by taking action $(d, \{0, 1, 2, 3\})$, where 0, 1, 2, 3 correspond to N, E, S, W. Additionally, for all tasks in this setting, assume that m circular obstacles are randomly placed in the environment (while always allowing for a path to the goal for the robot). Suppose that the robot's observation at time t corresponds to a complete map of the grid-world which consists of a list of locations including obstacle locations, the goal location, and the robot location. Let $o_E^i = (x^i, y^i)$ correspond to the location of the i^{th}

observation in o_E . The robot is initialized in a random position and its goal is to navigate to the goal location. The robot receives a reward of 1 if it successfully navigates to the goal and a reward of 0 otherwise.

Let task τ_N (“north”) have goal location at $(0, n)$ and τ_E (“east”) have goal location $(n, 0)$. An admissible policy π_N^* on τ_N will always successfully navigate to the goal location $(0, n)$ (regardless of the initial state of the robot and the locations of the obstacles).

Proposition 14.3 ($\tau_E \preceq \tau_N$). *Let $G_{N,E}$ contain functions that can perform addition modulo 4 and $H_{E,N}$ contain functions which can be evaluated in linear time (in the size of the observation). Then $\tau_E \preceq \tau_N$.*

Proof. For any observation o_E from task τ_E , let $h_{EN}(o_E^i) := (y^i, -x^i) \forall i$. For any action $a_N = (d, i)$ from π_N^* , let $g_{NE}(d, i) := (d, i + 1 \bmod 4)$. Then we have that $g_{NE} \circ \pi_N^* \circ h_{EN} \in \Pi_E^*, \forall \pi_N^* \in \Pi_N^*$. $\square$

Intuitively, h_{EN} rotates observations from τ_E so that they look exactly like a corresponding observation from τ_N . Then, any admissible policy π_N^* must find an admissible action (i.e., one which ultimately results in solving the task) based on the rotated observation. The decoder g_{NE} then transforms the output action to the corresponding one required for task τ_E . With analogous constructions, we can show that $\tau_N \equiv \tau_E \equiv \tau_S \equiv \tau_E$.

14.5.2 Grasping Objects with Differing Camera Viewpoints

Consider a robotic arm which must grasp one of a set of known objects using an RGB-D image. A randomly-selected object is placed with a random pose on a table (which is at height $z = 0$). A task $\tau \in \mathcal{T}$ will correspond to this grasping challenge when the RGB-D camera is placed at a particular viewpoint. Assume that the camera in each task is always pointed at the center of the table (i.e., $(0, 0, 0)$). Additionally, we will require that for each of the possible objects, the identity of the particular object and its pose are uniquely determinable from the camera. Further assume that at the beginning of the task, the robot arm does not occlude the camera’s view of the object. We treat this as a single time-step task; the robot selects a grasp pose based on the camera observation and then controls M motors in order to grasp the object. A successful grasp results in a reward of 1 and 0

otherwise. Let $\tau_{(\theta,\phi)}$ correspond to the grasping task with the camera located at spherical coordinates $(1, \theta, \phi)$.

Proposition 14.4 ($\tau_{(\theta_1,\phi_1)} \preceq \tau_{(\theta_2,\phi_2)}$). *For any (θ_1, ϕ_1) and (θ_2, ϕ_2) , task $\tau_{(\theta_1,\phi_1)}$ reduces to task $\tau_{(\theta_2,\phi_2)}$ if (θ_1, ϕ_1) and (θ_2, ϕ_2) are selected such that the cameras can view the entire table from either viewpoint.*

Proof. Let g be the identity. Assume that the camera placed at (θ_1, ϕ_1) can view the entire table. Let h take as input a RGB-D image from viewpoint $(1, \theta_1, \phi_1)$ and output the same environmental setup from the perspective of a camera placed at $(1, \theta_2, \phi_2)$. Then $g \circ \pi_{(\theta_2,\phi_2)}^* \circ h \in \Pi_{(\theta_1,\phi_1)}^*$. $\square$

Note that the function h requires a model of the known objects and potentially a simulation of the environment in order to generate the image from a differing perspective. As such, the reduction with this encoder may not be efficient. A direct consequence of this proposition is that $\tau_{(\theta_1,\phi_1)} \equiv \tau_{(\theta_2,\phi_2)}$ if (θ_1, ϕ_1) and (θ_2, ϕ_2) are selected such that the cameras can view the entire table. Let Θ be the set of all (θ, ϕ) values which reduce to each other using the reduction provided in the proposition. The corresponding set of tasks $\mathcal{T}_\Theta \subseteq \mathcal{T}$ defines a class of tasks which are all equivalently complex.

14.6 Relative Complexity

In general, it may be difficult to establish a reduction between two completely distinct tasks such as grasping an object and avoiding obstacles; indeed an arbitrary pair of tasks is unlikely to satisfy the notion of reduction introduced in Sec. 14.4.2. Additionally, our definition of reduction is a binary one and does not capture *how* complex a particular task is relative to another. Motivated by these observations, we propose a definition of *relative complexity* that captures the degree to which one task is more complex than another. This quantity can be thought of as a “continuous” or “smoothed” version of task reductions (in a precise sense, which we elucidate below). Note that we will omit the subscripts on G and H when it is clear which tasks they transform between.

Definition 14.7 (Relative Complexity). *The relative complexity of task τ_1 with respect to task τ_2 is*

$$C_{\tau_1/\tau_2} := \sup_{\pi_2^* \in \Pi_2^*} \inf_{h \in H, g \in G} \left[1 - \frac{R_1(g \circ \pi_2^* \circ h)}{R_1^*} \right], \quad (14.5)$$

where we assume that rewards are nonnegative. We use the notation $C_{\tau_1/\tau_2}(H, G)$ when we want to highlight dependence on H and G .

Intuitively, if $\tau_1 \preceq \tau_2$, then this implies that $C_{\tau_1/\tau_2} = 0$ (as we show formally below). If $C_{\tau_1/\tau_2} > 0$, then an admissible policy $\pi_2^* \in \Pi_2^*$ may not be transformed into an admissible policy for τ_1 using encoders and decoders in H and G . A key advantage of this definition is that we can compare *any* two tasks and quantify the relative complexity of one with respect to another. As with our previous definitions, we prove a set of useful properties below. Importantly, Properties 14.5.c and 14.5.d will establish a link between the relative complexity and task reduction.

Proposition 14.5 (Properties of the Relative Complexity). *Relative Complexity satisfies the following properties:*

Property 14.5.a. *Nonnegativity and boundedness:* $C_{\tau_1/\tau_2} \in [0, 1]$.

Property 14.5.b. *Monotonicity with respect to H and G :* If $H \subseteq H'$ and $G \subseteq G'$, then $C_{\tau_1/\tau_2}(H', G') \leq C_{\tau_1/\tau_2}(H, G)$.

Assume that the supremum and infimum in Definition 14.7 are attained by functions in Π_2^, H, G . Then:*

Property 14.5.c. *Equivalence between reduction and 0 relative complexity:* $C_{\tau_1/\tau_2} = 0 \iff \tau_1 \preceq \tau_2$.

Property 14.5.d. *Equivalence between no reduction and positive relative complexity:* $C_{\tau_1/\tau_2} \in (0, 1] \iff \neg(\tau_1 \preceq \tau_2)$.

Proof. A complete proof of these properties is provided in Appendix 14.10.1.

Algorithm 14.1 Approximating Relative Complexity

```
1: Input: Learning rates  $\lambda_1, \lambda_2$ , adversarial tuning parameter  $\alpha$ 
2: Input: Function spaces  $H, G$ 
3: Input: Loss functions  $L_1, L_2$  for  $\tau_1, \tau_2$  respectively
4: Output: Approximate relative complexity  $\tilde{C}_{\tau_1/\tau_2} \approx C_{\tau_1/\tau_2}$ 
5: while  $\neg(\text{converged} \wedge R_2(\pi_2) = R_2^*)$  do
6:   Step 1:  $\pi_2$  update
7:    $c_1 \leftarrow L_2(\pi_2) - \alpha L_1(g \circ \pi_2 \circ h)$ 
8:    $\pi_2 \leftarrow \pi_2 - \lambda_1 \nabla_{\pi_2} c_1$ 
9:   Step 2: encoder/decoder update
10:   $c_2 \leftarrow L_1(g \circ \pi_2 \circ h)$ 
11:   $[h, g] \leftarrow [h, g] - \lambda_2 \nabla_{[h, g]} c_2$ 
12: end while
13:  $\tilde{C}_{\tau_1/\tau_2} \leftarrow \left[ 1 - \frac{R_1(g \circ \pi_2^* \circ h)}{R_1^*} \right]$ 
```

14.7 Algorithmic Approach

We can frame the optimization problem in (14.5) as a two-player zero-sum game: an adversary chooses an admissible policy $\pi_2^* \in \Pi_2^*$ which maximizes the relative complexity, and then the player chooses $g \in G$ and $h \in H$ such that the relative complexity is minimized. It may not be possible in general to find an optimal strategy to this game. However, we can still compute a meaningful estimate via approximate methods.

There are multiple ways to approximate optimal strategies for zero-sum games such as the one presented in (14.5); see e.g., [76]. One approach is to use *best-response dynamics*, where the players update their strategies in rounds based on the *best response* to the opponent's choice. We parameterize the policies, encoders, and decoders with neural networks. Thus, we aim to develop a method which uses gradient steps to approximate the solution to (14.5). A technique which exploits best-response dynamics is the update rule for training generative adversarial networks (GANs) [80], where gradient steps update the players' strategies based on batches of data. The algorithm we present has a similar structure.

The resulting approach has two steps and is presented in Algorithm 14.1. The first step is to update the policy π_2 to maximize the relative complexity. However, π_2 must also eventually succeed on task τ_2 . Thus, the update for π_2 has two terms: one to train π_2 to succeed on τ_2 and one to train $g \circ \pi_2 \circ h$ to fail on τ_1 . We scale the latter with the “adversarial tuning parameter” α which allows weighting of the terms relative to each other. The second

step is to update the encoder and decoder to minimize the relative complexity. We present Algorithm 14.1 with general loss functions L_1, L_2 on tasks τ_1, τ_2 to allow for specialization to reinforcement learning techniques (e.g., Q-learning). An example for L_1 and L_2 based on best-response dynamics is $L_2(\pi_2) := -R_2(\pi_2)$ and $L_1(g \circ \pi_2 \circ h) := 1 - R_1(g \circ \pi_2 \circ h)/R_1^*$ since these directly capture the objectives of each player.

In practice, when α is too low, the policy π_2 is not adversarial enough and may not prevent $g \circ \pi_2 \circ h$ from succeeding on task τ_1 . When α is too large, the policy π_2 may not ever succeed on task τ_2 . As such we increase α as much as possible while ensuring π_2 is admissible at the end of training in order to provide an estimate of (14.5). After training is complete, we have π_2^*, g , and h which we use to compute the approximate relative complexity $\tilde{C} = 1 - R_1(g \circ \pi_2^* \circ h)/R_1^*$.

14.8 Examples

We implement Algorithm 14.1 on two reinforcement learning examples using Q-learning and Soft Actor-Critic (SAC). On the OpenAI Gym [37] Cartpole, we compare the complexity of balancing the pole upright relative to balancing it downwards. Using the Mujoco [204] 2D walker, we compare the complexity of walking at various speeds. We demonstrate that our estimates of relative complexity correspond to intuitive notions of complexity for these tasks.

14.8.1 Cartpole Balancing Task

Overview. We use the OpenAI Gym [37] Cartpole environment to define two tasks: balancing the friction-less cart’s pole against gravity $\tau_\uparrow$ (at the unstable equilibrium) and balancing the cart’s pole with gravity $\tau_\downarrow$ (at the stable equilibrium). Equivalently, one can think of $\tau_\uparrow$ and $\tau_\downarrow$ being specified by the direction in which gravity acts ($-y$ and $+y$). The initial state of the system is randomized close to the equilibrium for each task. The policy receives the system’s state vector as input and can apply forces on the cart using three actions: {no force, push left, push right}. A task runs for 200 time steps and the policy receives a reward of 1 for each time step that the pole stays balanced (within 24° of the

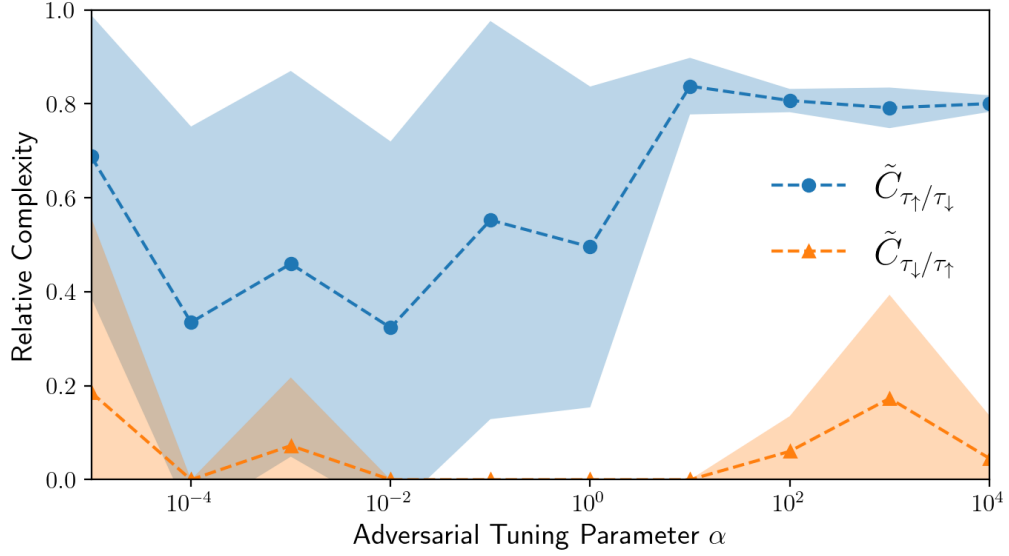


Figure 14.2: The approximate relative complexity of task $\tau_{\uparrow}$ with respect to $\tau_{\downarrow}$ (circles) and $\tau_{\downarrow}$ with respect to $\tau_{\uparrow}$ (triangles) for varied adversarial tuning parameter α . For large enough α (≥ 10), the relative complexity of $\tau_{\uparrow}$ with respect to $\tau_{\downarrow}$ is consistently close to 1. In contrast, the relative complexity of $\tau_{\downarrow}$ with respect to $\tau_{\uparrow}$ is consistently close to 0 regardless of the choice of α . The plots suggest that $\tau_{\uparrow}$ is *more complex* than $\tau_{\downarrow}$. We plot the mean and standard deviation (shaded region) across 5 seeds for each α .

equilibrium). Since we only switch the direction of gravity, the states which achieve a reward of 1 are consistent between tasks. A reward of 0 is given if the pole falls beyond 24° of the equilibrium or the cart strays too far from the start position; the trial is then stopped. A policy for either task is admissible if it successfully balances the pole for the entire 200 time-step trial, i.e., $R_{\downarrow}^* = R_{\uparrow}^* = 200$.

Policy, Encoder, Decoder, and Training. The policy, encoder, and decoder all consist of few-layer neural networks. We apply Algorithm 14.1 using Q-learning to approximate the relative complexities $C_{\tau_{\uparrow}/\tau_{\downarrow}}$ and $C_{\tau_{\downarrow}/\tau_{\uparrow}}$. Note that we can directly apply Q-learning to Algorithm 14.1 by letting the loss functions L_1 and L_2 correspond to a Q-learning loss (see Appendix 14.10.2 for further details). In our experiments, we vary the adversarial tuning parameter to examine its effect and also vary the number of layers in encoder h and decoder g to approximate the relative complexity given varying complexity of H and G . The result of training is an admissible policy π^* , encoder g , and decoder h ; these allow us to compute the estimated relative complexity $\tilde{C}$.

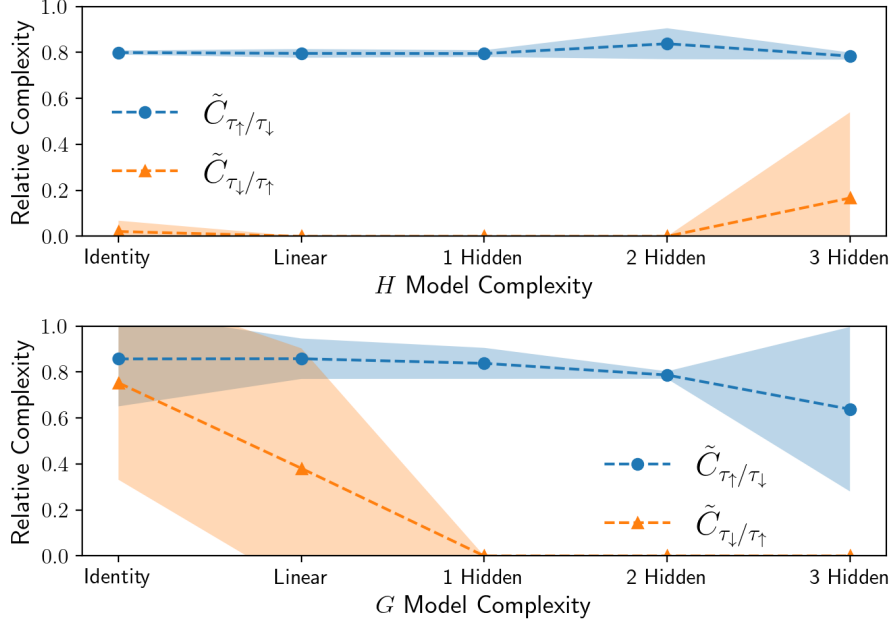


Figure 14.3: The approximate relative complexity for varied H and G complexity. (Top) There is little or no change in the relative complexity for varied H . (Bottom) We see significant decrease in $C_{\tau_{\downarrow}/\tau_{\uparrow}}$ and a slight decrease in $C_{\tau_{\uparrow}/\tau_{\downarrow}}$ for increasing G complexity. This suggest that reduction $\tau_{\downarrow} \preceq \tau_{\uparrow}$ is not possible when G includes only linear neural networks. We plot the mean of 5 trials for each α value and the standard deviation is shaded.

Results: Sensitivity to Adversarial Tuning Parameter. We compute the approximate relative complexity for a wide range of the adversarial tuning parameter α : $[10^{-5}, 10^4]$, and plot the results in Fig. 14.2. We choose g to be a neural network with a single hidden layer and h to have two hidden layers. When approximating the relative complexity of task $\tau_{\uparrow}$ with respect to $\tau_{\downarrow}$ we see that when α is large enough, the value settles at approximately 0.8. This suggests that $\neg(\tau_{\uparrow} \preceq \tau_{\downarrow})$ by Property 14.5.d. Additionally, note that a decrease in α corresponds to decreased weight on minimizing $R_{\uparrow}(g \circ \pi_{\downarrow} \circ h)$. As such, when α is too small, $\pi_{\downarrow}$ cannot reliably ensure that $g \circ \pi_{\downarrow} \circ h$ fails on $\tau_{\uparrow}$. In the case of approximating the relative complexity of task $\tau_{\downarrow}$ with respect to $\tau_{\uparrow}$, we see that for all values of α , a low relative complexity is achieved. The plot suggests that $\tau_{\downarrow} \preceq \tau_{\uparrow}$ by Property 14.5.c. This suggests that the task of balancing at a stable equilibrium $\tau_{\downarrow}$ is less complex than the task of balancing at an unstable equilibrium $\tau_{\uparrow}$, which is consistent with our intuition.

Results: Sensitivity to H and G Model Complexity. We also approximate the

relative complexity for different H and G with varying model complexity (achieved by varying the number of hidden layers). In all experiments, the neural network architecture for the (inner) policy π is kept consistent. When varying the complexity of H , we choose g to be a neural network with a single hidden layer; when varying G , we choose h to be a neural network with two hidden layers. The results are plotted in Fig. 14.3. We see that there is no significant change in the approximate relative complexity when H increases in complexity. However, when the complexity of G is increased, we see a clear decrease in $\tilde{C}_{\tau_{\downarrow}/\tau_{\uparrow}}$ indicating that (i) $\neg(\tau_{\downarrow} \preceq \tau_{\uparrow})$ when G only contains the identity or a single linear layer, and (ii) $\tau_{\downarrow} \preceq \tau_{\uparrow}$ for more complex G . We see a slight decrease in $\tilde{C}_{\tau_{\uparrow}/\tau_{\downarrow}}$ for increasing complexity of G . By Property 14.5.b, the relative complexity is monotonic with respect to H and G ; specifically, increasing the complexity of H and G should result in a monotonic decrease in the relative complexity. This is consistent with the results in Fig. 14.3.

14.8.2 Mujoco 2D Walker at varied speeds

Overview. Using the Mujoco [204] Walker2D-v2 environment in OpenAI Gym [37], we create a set of tasks τ_v with the goal of walking at a particular speed v (from 0.6 m/s to 1.4 m/s). Thus, the maximum reward on τ_v is achieved when the robot travels at v . The policy receives a 17-dimensional observation vector (corresponding to joint angles and velocities) and outputs a 6-dimensional action vector (corresponding to joint torques) to control the 2D walking robot. The task runs for at most 1000 time steps. The reward at time t is $1 - |v_t - v|$, where v_t is the speed at the current time step. Note that to help with training, we also provide a reward for staying upright and a reward which penalizes large policy outputs to help with training. When evaluating $\tilde{C}$, we lower-bound the reward by 0 to ensure that the reward is non-negative. If the robot falls over, a reward of 0 is given and the trial stops. Since the challenge of maintaining a particular speed may differ between tasks, the threshold for success may vary. Thus, to find R_v^* , we first train an individual policy to travel at speed v . We then let $R_v^* = 0.95 \times (\text{individual policy reward on } \tau_v)$.

Policy, Encoder, Decoder and Training. As in Sec. 14.8.1, the policy, encoder, and decoder all consist of neural networks with 3 hidden layers. However, since the action space is continuous and 6-dimensional, we use soft actor-critic (SAC) [85]. This requires a

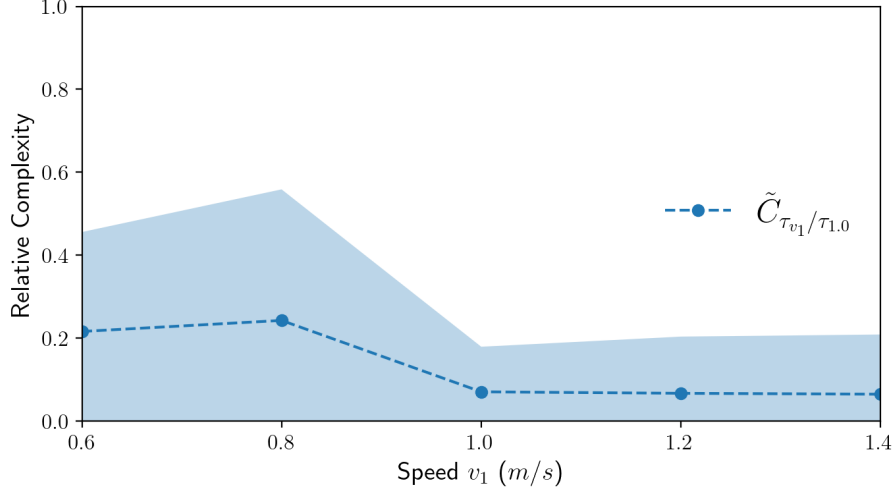


Figure 14.4: Reduction from the task of walking at speed v_1 (τ_{v_1}) to walking at 1.0 m/s ($\tau_{1.0}$). The estimated relative complexity $\tilde{C}_{\tau_{v_1}/\tau_{1.0}}$ may be larger when $v_1 < 1.0$ m/s as compared with $v_1 \geq 1.0$ m/s. Thus, it is possible that a policy for walking at 1.0 m/s cannot directly be transformed to a policy for walking at 0.6 or 0.8 m/s. We plot the mean over 15 seeds for each v_1 ; the standard deviation is shaded.

modification to Algorithm 14.1 for training critics for τ_{v_1} and τ_{v_2} (see Appendix 14.10.2 for the algorithm and additional experimental details). The result of training is an admissible policy π_2^* and the encoder/decoder h/g ; these are used to calculate the approximate relative complexity $\tilde{C}$.

Results: Relative Complexity for Increasing Speed. We compute the approximate relative complexity of the task of walking at speeds $v_1 \in [0.6, 1.4]$ m/s with respect to the task of walking at 1.0 m/s ($\tau_{1.0}$). We sweep through a wide range of the adversarial tuning parameter $\alpha = [10^{-6}, 10]$, and for each $\tilde{C}$ computation, choose the largest α which results in an admissible policy on task $\tau_{1.0}$. The results are presented in Fig. 14.4. The choice of H and G is kept consistent for all relative complexity computations. We see that $\tilde{C}_{\tau_{v_1}/\tau_{1.0}}$ may be larger when $v_1 < 1.0$ m/s as compared with $v_1 \geq 1.0$ m/s. Importantly, these results are conditioned on the choice of H , and G . The plot suggests that there exists an admissible policy for walking at 1.0 m/s which cannot be directly transformed to walk at 0.6 or 0.8 m/s but no such policy exists which prevents a transformation to a policy for walking faster.

14.9 Conclusion

We have presented a framework for comparing the complexity of robotic tasks. In order to achieve this, we defined a notion of reduction between two tasks that captures the ability of a robot to solve one task given a policy for another. We also presented a measure of relative complexity that quantifies how complex one task is relative to another. Our theoretical results establish basic properties satisfied by these notions and also establish the relationship between reductions and relative complexity. We presented a practical algorithm for estimating the relative complexity between tasks and demonstrated this using reinforcement learning tasks. Our results demonstrate consistency with intuitive notions of hardness for these tasks and empirical correspondence to theoretical properties.

The work presented here opens up a number of exciting directions for future research. We begin by discussing some of the limitations of this framework as motivation. One such limitation is the slow convergence of the computation for approximating the relative complexity (which required longer to converge than learning a policy for either individual task). We also need admissible policies for both τ_1 and τ_2 to compute the relative complexity. Additionally, the results we present are conditioned on the choice of encoder H and decoder G spaces. The value and interpretability of the relative complexity measure thus depend on how easily one can interpret the expressiveness of H and G .

On the theoretical front, it would be interesting to devise more general notions of reductions than the one presented here, e.g., allowing the policy from one task to be used as an oracle to solve another task in a more general manner (instead of the specific encoder-policy-decoder architecture we utilize). Another research direction is to use our notion of reduction to establish equivalences between broad classes of tasks (similar to complexity classes P and NP in computational complexity). It may also be of practical interest to define relative complexity in terms of different metrics such as data efficiency or difficulty of finding a policy instead of the notion of complexity we use, which is based on the complexity of online execution of policies.

On the algorithmic front, it would be of practical interest to develop different techniques for estimating the relative complexity. One could potentially use other algorithms (beyond

best-response dynamics) for approximately solving games [76]. Finally, a particularly exciting direction would be to draw inspiration from *cryptography* and turn statements about the complexity of certain tasks into statements about *robustness* for a robot. This could potentially be achieved by establishing the hardness of an *adversary's* task of foiling our robot.

14.10 Appendix

14.10.1 Proof of Properties

Proposition 14.1. (Task Reduction is a Non-Strict Partial Ordering Relation).

Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $G_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task reductions satisfy the following properties and thus define a non-strict partial ordering relation.

Property 14.1.a. Reflexivity: $\tau_1 \preceq \tau_1$.

Property 14.1.b. Antisymmetry: $\tau_1 \prec \tau_2 \implies \neg(\tau_2 \preceq \tau_1)$, where $\tau_1 \prec \tau_2$ is defined as $(\tau_1 \preceq \tau_2) \wedge \neg(\tau_1 \equiv \tau_2)$.

Property 14.1.c. Transitivity: $(\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \implies \tau_1 \preceq \tau_3$.

Proof. Property 14.1.a: $\tau_1 \preceq \tau_1 \implies \exists g \in G_{1,1}, h \in H_{1,1}$ such that

$$g \circ \pi_1^* \circ h \in \Pi_1^*. \quad (14.6)$$

If g and h are the identity function, then $g \circ \pi_1^* \circ h = \pi_1^* \forall \pi_1^* \in \Pi_1^*$. Thus, $\tau_1 \preceq \tau_1$ when $G_{1,1}$ and $H_{1,1}$ include their respective identity functions.

Property 14.1.b: Suppose $\tau_1 \prec \tau_2$ and thus $(\tau_1 \preceq \tau_2) \wedge \neg(\tau_1 \equiv \tau_2)$. Note that $\neg(\tau_1 \equiv \tau_2) \implies \neg((\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)) \implies \neg(\tau_1 \preceq \tau_2) \vee \neg(\tau_2 \preceq \tau_1)$. We assumed $(\tau_1 \preceq \tau_2)$, so we must have that $\neg(\tau_2 \preceq \tau_1)$.

Property 14.1.c: Suppose $\tau_1 \preceq \tau_2$ and $\tau_2 \preceq \tau_3$. By Definition 14.4, $\exists g_1 \in G_{2,1}$, $g_2 \in G_{3,2}$, $h_1 \in H_{1,2}$, and $h_2 \in H_{2,3}$ such that $g_1 \circ \pi_2^* \circ h_1 \in \Pi_1^* \forall \pi_2^* \in \Pi_2^*$ and $g_2 \circ \pi_3^* \circ h_2 \in$

$\Pi_2^* \vee \pi_3^* \in \Pi_3^*$. Consider

$$\underbrace{g_1 \circ g_2 \circ \overbrace{\pi_3^*}^{\in \Pi_2^*} \circ h_2 \circ h_1}_{\in \Pi_1^*} \quad (14.7)$$

for all $\pi_3^* \in \Pi_3^*$. Let $g_3 := g_1 \circ g_2$ and $h_3 := h_2 \circ h_1$ so that $g_3 \circ \pi_3^* \circ h_3 \in \Pi_1^*$ for all $\pi_3^* \in \Pi_3^*$. If $G_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$, then $g_3 \in G_{3,1}$ and $h_3 \in H_{1,3}$ and $\tau_1 \preceq \tau_3$. Thus, task reductions are transitive if $G_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. $\square$

Proposition 14.6 (Strict Task Reduction is a Strict Partial Ordering Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $G_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, strict task reductions satisfy the following properties and thus define a strict partial ordering relation.*

Property 14.6.a. *Irreflexivity:* $\neg(\tau_1 \prec \tau_1)$.

Property 14.6.b. *Asymmetry:* $\tau_1 \prec \tau_2 \implies \neg(\tau_2 \prec \tau_1)$.

Property 14.6.c. *Transitivity:* $\tau_1 \prec \tau_2 \wedge \tau_2 \prec \tau_3 \implies \tau_1 \prec \tau_3$.

Proof. **Property 14.6.a:** Suppose $\tau_1 \prec \tau_1 \implies \tau_1 \preceq \tau_1 \wedge \neg(\tau_1 \equiv \tau_1)$. $\tau_1 \equiv \tau_1$ by Property 14.2.b. $\implies \neg(\tau_1 \prec \tau_1)$ when $H_{1,1}$ and $G_{1,1}$ include their respective identity functions.

Property 14.6.b: $\tau_1 \prec \tau_2 \implies \neg(\tau_2 \preceq \tau_1)$ by Property 14.1.b, since $\tau_1 \prec \tau_2 \implies \neg(\tau_1 \equiv \tau_2)$. $\neg(\tau_2 \preceq \tau_1) \iff \neg(\tau_2 \preceq \tau_1) \vee \tau_2 \equiv \tau_1 \implies \neg(\tau_2 \preceq \tau_1 \wedge \neg(\tau_2 \equiv \tau_1)) \implies \neg(\tau_2 \prec \tau_1)$.

Property 14.6.c: $\tau_1 \prec \tau_2 \wedge \tau_2 \prec \tau_3 \implies \tau_1 \preceq \tau_2 \wedge \tau_2 \preceq \tau_3 \wedge \neg(\tau_1 \equiv \tau_2) \wedge \neg(\tau_2 \equiv \tau_3) \implies \tau_1 \preceq \tau_3 \wedge \neg(\tau_1 \equiv \tau_3)$ by Properties 14.1.c and 14.2.c. $\implies \tau_1 \prec \tau_3$ when $H_{1,3}$ and $G_{3,1}$ are closed under composition on $\mathcal{T}$. $\square$

Proposition 14.2. (Task Equivalence is an Equivalence Relation). *Suppose that $\forall (\tau_\xi, \tau_\zeta) \in \mathcal{T}^2$, $H_{\xi,\zeta}$ and $G_{\zeta,\xi}$ include the identity and are closed under composition on $\mathcal{T}$. Then, task equivalence satisfies the following properties and thus defines an equivalence relation.*

Property 14.2.a. *Reflexivity:* $\tau_1 \equiv \tau_1$.

Property 14.2.b. *Symmetry:* $\tau_1 \equiv \tau_2 \implies \tau_2 \equiv \tau_1$.

Property 14.2.c. *Transitivity:* $\tau_1 \equiv \tau_2 \wedge \tau_2 \equiv \tau_3 \implies \tau_1 \equiv \tau_3$.

Proof. **Property 14.2.a:** $\tau_1 \equiv \tau_1 \implies \tau_1 \preceq \tau_1$ by Property 14.1.a when $G_{1,1}$ and $H_{1,1}$ include the identity. Thus, task equivalence is reflexive if $G_{1,1}$ and $H_{1,1}$ include the identity.

Property 14.2.b: $\tau_1 \equiv \tau_2 \implies (\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)$ by Definition 14.5 $\implies (\tau_2 \preceq \tau_1) \wedge (\tau_1 \preceq \tau_2) \implies \tau_2 \equiv \tau_1$.

Property 14.2.c: $(\tau_1 \equiv \tau_2) \wedge (\tau_2 \equiv \tau_3) \implies (\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \wedge (\tau_3 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1)$ by Definition 14.5. $(\tau_3 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_1) \implies (\tau_3 \preceq \tau_1)$ by Property 14.1.c when $G_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. Similarly, $(\tau_1 \preceq \tau_2) \wedge (\tau_2 \preceq \tau_3) \implies (\tau_1 \preceq \tau_3)$. Thus $(\tau_1 \preceq \tau_3) \wedge (\tau_3 \preceq \tau_1) \implies \tau_1 \equiv \tau_3$. Thus task equivalence is transitive if $G_{3,1}$ and $H_{1,3}$ are closed under composition on $\mathcal{T}$. $\square$

Proposition 14.5. (Properties of the Relative Complexity). *Relative Complexity satisfies the following properties:*

Property 14.5.a. *Nonnegativity and boundedness:* $C_{\tau_1/\tau_2} \in [0, 1]$.

Property 14.5.b. *Monotonicity with respect to H and G :* If $H \subseteq H'$ and $G \subseteq G'$, then $C_{\tau_1/\tau_2}(H', G') \preceq C_{\tau_1/\tau_2}(H, G)$.

Assume that the supremum and infimum in Definition 14.7 are attained by functions in Π_2^, H, G . Then:*

Property 14.5.c. *Equivalence between reduction and 0 relative complexity:* $C_{\tau_1/\tau_2} = 0 \iff \tau_1 \preceq \tau_2$.

Property 14.5.d. *Equivalence between no reduction and positive relative complexity:* $C_{\tau_1/\tau_2} \in (0, 1] \iff \neg(\tau_1 \preceq \tau_2)$.

Proof. **Property 14.5.a:** $R_1(g \circ \pi_2^* \circ h) \in [0, R_1^*]$. Therefore, $R_1(g \circ \pi_2^* \circ h)/R_1^* \in [0, 1] \implies C_{\tau_1/\tau_2} \in [0, 1]$ for any H, G .

Property 14.5.b: Consider H, H' such that $H \subseteq H'$ and G, G' such that $G \subseteq G'$. For any function f , the following is true $\forall \pi_2^*$:

$$\inf_{h \in H', g \in G'} f(h, g, \pi_2^*) \leq \inf_{h \in H, g \in G} f(h, g, \pi_2^*). \quad (14.8)$$

This implies the following:

$$\sup_{\pi_2 \in \Pi_2^*} \inf_{h \in H', g \in G'} \left[1 - \frac{R_1(g \circ \pi_2^* \circ h)}{R_1^*} \right] \leq \sup_{\pi_2 \in \Pi_2^*} \inf_{h \in H, g \in G} \left[1 - \frac{R_1(g \circ \pi_2^* \circ h)}{R_1^*} \right]. \quad (14.9)$$

Property 14.5.c: Assume $C_{\tau_1/\tau_2} = 0$ for some H and $G \iff$ for any $\pi_2^* \in \Pi_2^* \exists g \in G$ and $h \in H$ such that $R_1(g \circ \pi_2^* \circ h) = R_1^*$. $R_1(\pi_1) = R_1^* \iff \pi_1 \in \Pi_1^*$. Thus, for all $\pi_2^* \in \Pi_2^* \exists g \in G$ and $h \in H$ such that $g \circ \pi_2^* \circ h \in \Pi_1^* \iff \tau_1 \preceq \tau_2$.

Property 14.5.d: The contrapositive of Property 14.5.c is $\neg(\tau_1 \preceq \tau_2) \iff C_{\tau_1/\tau_2} \neq 0$. By Property 14.5.a, the complexity measure is $C_{\tau_1/\tau_2} \in [0, 1]$, therefore, $C_{\tau_1/\tau_2} \in (0, 1] \iff C_{\tau_1/\tau_2} \neq 0$. Thus $C_{\tau_1/\tau_2} \in (0, 1] \iff \neg(\tau_1 \preceq \tau_2)$. $\square$

Algorithm 14.2 Approximating Relative Complexity using SAC

```

1: Input: Learning rates  $\lambda_1, \lambda_2$ , adversarial tuning parameter  $\alpha$ 
2: Input: Function spaces  $H, G$ 
3: Input: Q-functions  $Q, Q$  loss functions for  $\tau_1, \tau_2$  respectively
4: Output: Approximate relative complexity  $\tilde{C}_{\tau_1/\tau_2} \approx C_{\tau_1/\tau_2}$ 
5: while  $\neg(\text{converged} \wedge R_2(\pi_2) = R_2^*)$  do
6:   Step 0: critic update
7:   Update critic  $Q_2^{\pi_2}$ 
8:   Update critic  $Q_1^{g \circ \pi_2 \circ h}$ 
9:   Step 1:  $\pi_2$  update
10:   $\pi_2 \leftarrow \pi_2 + \lambda_1 \nabla_{\pi_2} [Q_2^{\pi_2} - \alpha Q_1^{g \circ \pi_2 \circ h}]$ 
11:  Step 2: encoder/decoder update
12:   $[h, g] \leftarrow [h, g] + \lambda_2 \nabla_{[h, g]} [Q_1^{g \circ \pi_2 \circ h}]$ 
13: end while
14:  $\tilde{C}_{\tau_1/\tau_2} \leftarrow \left[ 1 - \frac{R_1(g \circ \pi_2^* \circ h)}{R_1^*} \right]$ 

```

14.10.2 Additional Experimental Details

Approximating Relative Complexity using Q-learning. We apply Q-learning to Algorithm 14.1 by letting the loss functions L_1 and L_2 correspond to a Q-learning loss:

$L_\xi(\pi_\xi) = -\frac{1}{B} \sum_{b=1}^B [Q^{\pi_\xi}(s_b, a_b) \log p(a_b)]$, where $p(a_b)$ corresponds to the probability of an action for policy π_ξ (which may be a transformation of another policy such as $\pi_\xi = g \circ \pi_\zeta \circ h$), $Q^{\pi_\xi}(s_b, a_b)$ are the Q-values, and B is the batch size. We run Algorithm 14.1 for 1000 iterations and use a batch size B of 1000 transitions.

Approximating Relative Complexity using SAC. We modify Algorithm 14.1 to use SAC for approximating the relative complexity. Let $Q_2^{\pi_2}$ be a critic of π_2 on task τ_2 and $Q_1^{g \circ \pi_2 \circ h}$ be a critic of $g \circ \pi_2 \circ h$ on task τ_1 . We add an additional step to the algorithm for updating the critics on task τ_1 and τ_2 . The critics are then used in the updates for the policy π_2 and the encoder/decoder. The resulting method is presented in Algorithm 14.2. We run Algorithm 14.2 for 50,000 iterations and use a batch size of 200 transitions.

Bibliography

- [1] Ieee standard for assumptions in safety-related models for automated driving systems. *IEEE Std 2846-2022*, pages 1–59, 2022.
- [2] M. Abdar, F. Pourpanah, S. Hussain, D. Rezazadegan, L. Liu, M. Ghavamzadeh, P. W. Fieguth, X. Cao, A. Khosravi, U. R. Acharya, V. Makarenkov, and S. Nahavandi. A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, 76:243–297, 2021.
- [3] K. Abou-Moustafa and C. Szepesvári. An Exponential Efron-Stein Inequality for L_q Stable Learning Rules. *Proceedings of the 30th International Conference on Algorithmic Learning Theory*, 2019.
- [4] A. Achille, G. Mbeng, and S. Soatto. Dynamics and reachability of learning tasks. *arXiv:1810.02440*, 2019.
- [5] A. Achille, G. Paolini, G. Mbeng, and S. Soatto. The information complexity of learning tasks, their structure and their distance. *Information and Inference: A Journal of the IMA*, 10(1):51–72, 2021.
- [6] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd. A rewriting system for convex optimization problems. *Journal of Control and Decision*, 5(1):42–60, 2018.
- [7] P. Agrawal, A. V. Nair, P. Abbeel, J. Malik, and S. Levine. Learning to poke by poking: Experiential learning of intuitive physics. *Advances in Neural Information Processing Systems*, pages 5074–5082, 2016.

- [8] A. A. Ahmadi, A. Majumdar, and R. Tedrake. Complexity of ten decision problems in continuous time dynamical systems. *Proceedings of the American Control Conference (ACC)*, pages 6376–6381, 2013.
- [9] A. K. Akametalu, J. F. Fisac, J. H. Gillula, S. Kaynama, M. N. Zeilinger, and C. J. Tomlin. Reachability-based safe learning with gaussian processes. *IEEE Conference on Decision and Control*, pages 1424–1431, 2014.
- [10] P. Alquier, T. T. Mai, and M. Pontil. Regret Bounds for Lifelong Learning. *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, 2017.
- [11] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. *Proceedings of AAAI Conference on Artificial Intelligence*, 2018.
- [12] D. Althoff, M. Althoff, and S. Scherer. Online safety verification of trajectories for unmanned flight with offline computed robust invariant sets. *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3470–3477, 2015.
- [13] M. Althoff and J. M. Dolan. Online verification of automated road vehicles using reachability analysis. *IEEE Transactions on Robotics*, 30(4):903–918, 2014.
- [14] A. Ambroladze, E. Parrado-hernández, and J. Shawe-taylor. Tighter PAC-Bayes Bounds. *Advances in Neural Information Processing Systems 19*, 2007.
- [15] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2016.
- [16] R. Amit and R. Meir. Meta-Learning by Adjusting Priors Based on Extended PAC-Bayes Theory. *Proceedings of the International Conference on Machine Learning*, 2018.
- [17] M. Anthony and P. L. Bartlett. *Neural Network Learning: Theoretical Foundations*. Cambridge University Press, 1999.

- [18] S. Arnold, P. Mahajan, D. Datta, I. Bunner, and K. S. Zarkias. learn2learn: A library for meta-learning research. *arxiv preprint. preprint arXiv:2008.12284*, 2020.
- [19] S. Arora and B. Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, New York, NY, 2009.
- [20] S. Arora, R. Ge, B. Neyshabur, and Y. Zhang. Stronger Generalization Bounds for Deep Nets via a Compression Approach. *arXiv preprint arXiv:1802.05296*, 2018.
- [21] S. Arora, H. Khandeparkar, M. Khodak, O. Plevrakis, and N. Saunshi. A Theoretical Analysis of Contrastive Unsupervised Representation Learning. *Proceedings of the International Conference on Machine Learning*, 2019.
- [22] A. Bajcsy, K. Leung, E. Schmerling, and M. Pavone. Towards the unification and data-driven synthesis of autonomous vehicle safety concepts. *arXiv preprint arXiv:2107.14412*, 2021.
- [23] P. L. Bartlett, D. J. Foster, and M. J. Telgarsky. Spectrally-Normalized Margin Bounds for Neural Networks. *Advances in Neural Information Processing Systems*, pages 6240–6249, 2017.
- [24] M. Basseville. Detecting changes in signals and systems – a survey. *Automatica*, 24:309–326, 1988.
- [25] S. Bates, E. Candès, L. Lei, Y. Romano, and M. Sesia. Testing for Outliers with Conformal p-values. *arXiv preprint arXiv:2104.08279*, 2021.
- [26] S. Bengio, Y. Bengio, J. Cloutier, and J. Gecsei. On the Optimization of a Synaptic Learning Rule. *Proceedings of the Conference on Optimality in Artificial and Biological Neural Networks*, pages 6–8, 1992.
- [27] A. Bera, S. Kim, and D. Manocha. Realtime anomaly detection using trajectory-level crowd behavior learning. *IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1289–1296, 2016.

- [28] J. Bitterwolf, A. Meinke, and M. Hein. Certifiably adversarially robust detection of out-of-distribution data. *Advances in Neural Information Processing Systems*, 2020.
- [29] L. Blackmore, H. Li, and B. Williams. A probabilistic approach to optimal robust path planning with obstacles. *Proceedings of the American Control Conference (ACC)*, 2006.
- [30] V. Blondel and J. Tsitsiklis. A survey of computational complexity results in systems and control. *Automatica*, 36(9):1249–1274, 2000.
- [31] A. Blumer, A. Ehrenfeucht, D. Haussler, and M. K. Warmuth. Occam’s Razor. *Information Processing Letters*, 24(6):377–380, 1987.
- [32] R. Borie, C. Tovey, and S. Koenig. Algorithms and complexity results for pursuit-evasion problems. *International Joint Conference on Artificial Intelligence (IJCAI)*, 9:59–66, 2009.
- [33] R. Borie, C. Tovey, and S. Koenig. Algorithms and complexity results for graph-based pursuit evasion. *Autonomous Robots*, 31(4):317–332, 2011.
- [34] L. Bottou and V. Vapnik. Local Learning Algorithms. *Neural Computation*, 4:888–900, 1992.
- [35] O. Bousquet and A. Elisseeff. Stability and Generalization. *Journal of Machine Learning Research*, 2:499–526, 2002.
- [36] O. Bousquet, Y. Klochkov, and N. Zhivotovskiy. Sharper Bounds for Uniformly Stable Algorithms. *Proceedings of the Conference on Learning Theory*, 2020.
- [37] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba. Openai gym. *arXiv:1606.01540*, 2016.
- [38] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A multimodal dataset for autonomous driving. *Conference on Computer Vision and Pattern Recognition*, 2020.

- [39] H. Caesar, J. Kabzan, K. S. Tan, W. K. Fong, E. Wolff, A. Lang, L. Fletcher, O. Beijbom, and S. Omari. nuPlan: A closed-loop ml-based planning benchmark for autonomous vehicles. *Conference on Computer Vision and Pattern Recognition ADP3 workshop*, 2021.
- [40] F. Cai and X. Koutsoukos. Real-time out-of-distribution detection in learning-enabled cyber-physical systems. *Proceedings of the International Conference on Cyber-Physical Systems*, 2020.
- [41] J. Canny. *The Complexity of Robot Motion Planning*. MIT press, Cambridge, MA, 1988.
- [42] R. Caruana. Multitask Learning. *Machine Learning*, 28:41–75, 1997.
- [43] O. Catoni. *Statistical Learning Theory and Stochastic Optimization*. École d’Été de Probabilités de Saint-Flour 2001. Springer, 2004.
- [44] O. Catoni. *PAC-Bayesian Supervised Classification: The Thermodynamics of Statistical Learning*, volume 56 of *Lecture notes - Monograph Series*. Institute of Mathematical Statistics, 2007.
- [45] A. Celisse and B. Guedj. Stability Revisited: New Generalisation Bounds for the Leave-one-Out. *arXiv preprint arXiv:1608.06412*, 2016.
- [46] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, et al. Shapenet: An information-rich 3D model repository. *arXiv preprint arXiv:1512.03012*, 2015.
- [47] M. B. Chang, A. Gupta, S. Levine, and T. L. Griffiths. Automatically composing representation transformations as a means for generalization. *Proceedings of the International Conference on Learning Representations*, 2019.
- [48] A. Charnes and W. W. Cooper. Chance-constrained programming. *Management Science*, 6(1):73–79, 1959.

- [49] J. Chen, Y. Li, X. Wu, Y. Liang, and S. Jha. Robust out-of-distribution detection for neural networks. *Proceedings of the AAAI Workshop on Adversarial Machine Learning and Beyond*, 2022.
- [50] Y. Chen, U. Rosolia, C. Fan, A. D. Ames, and R. Murray. Reactive motion planning with probabilistic safety guarantees. *Conference on Robot Learning*, 2020.
- [51] Y. S. Chow and H. Teicher. Conditional expectation, conditional independence, introduction to martingales. In *Probability Theory: Independence, Interchangeability, Martingales*, Springer Texts in Statistics, pages 210–269. Springer, New York, NY, 1997.
- [52] A. Collette. *Python and HDF5*. O’Reilly, 2013.
- [53] E. Coumans and Y. Bai. Pybullet, a python module for physics simulation for games, robotics and machine learning. <http://pybullet.org>, 2016–2021.
- [54] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. John Wiley & Sons, 2012.
- [55] J. Culberson. Sokoban is PSPACE-complete. Technical Report TR 97-02, University of Alberta, Edmonton, Alberta, Canada, 1997.
- [56] J. Dahl, G. Rodrigues de Campos, C. Olsson, and J. Fredriksson. Collision avoidance: A literature review on threat-assessment techniques. *IEEE Transactions on Intelligent Vehicles*, 4(1):101–113, 2019.
- [57] G. Dalal, K. Dvijotham, M. Vecerik, T. Hester, C. Paduraru, and Y. Tassa. Safe exploration in continuous action spaces. *arXiv:1801.08757 [cs.AI]*, 2018.
- [58] G. Denevi, D. Stamos, C. Ciliberto, and M. Pontil. Online-Within-Online Meta-Learning. *Advances in Neural Information Processing Systems 32*, 2019.
- [59] T. DeVries and G. W. Taylor. Learning confidence for out-of-distribution detection in neural networks. *arXiv:1802.04865 [stat.ML]*, 2018.

- [60] S. Diamond and S. Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.
- [61] B. R. Donald. On information invariants in robotics. *Artificial Intelligence*, 72(1):217–304, 1995.
- [62] M. D. Donsker and S. R. S. Varadhan. Asymptotic evaluation of certain Markov process expectations for large time. *Communications on Pure and Applied Mathematics*, 28:1—47, 1975.
- [63] G. K. Dziugaite and D. M. Roy. Computing Nonvacuous Generalization Bounds for Deep (Stochastic) Neural Networks with Many More Parameters than Training Data. *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, 2017.
- [64] G. K. Dziugaite and D. M. Roy. Data-dependent PAC-Bayes priors via differential privacy. *Advances in Neural Information Processing Systems*, pages 8430–8441, 2018.
- [65] M. Erdmann. Understanding action and sensing by designing action-based sensors. *The International Journal of Robotics Research (IJRR)*, 14(5):483–509, 1995.
- [66] H. Fan, F. Zhu, C. Liu, L. Zhang, L. Zhuang, D. Li, W. Zhu, J. Hu, H. Li, and Q. Kong. Baidu apollo em motion planner. *arXiv preprint arxiv:1807.08048*, 2018.
- [67] A. Farid and A. Majumdar. Generalization bounds for meta-learning via PAC-Bayes and uniform stability. *Advances in Neural Information Processing Systems*, 34:2173–2186, 2021.
- [68] A. Farid, D. Snyder, A. Z. Ren, and A. Majumdar. Failure prediction with statistical guarantees for vision-based robot control. *Proceedings of Robotics: Science and Systems*, 2022.
- [69] A. Farid, S. Veer, B. Ivanovic, K. Leung, and M. Pavone. Task-relevant failure detection for trajectory predictors in autonomous vehicles. *Proceedings of the Conference on Robot Learning*, 2022.

- [70] A. Farid, S. Veer, and A. Majumdar. Task-driven out-of-distribution detection with statistical guarantees for robot learning. *Proceedings of the Conference on Robot Learning*, 2021.
- [71] A. Farid, S. Veer, D. Pachisia, and A. Majumdar. Task-driven detection of distribution shifts with statistical guarantees for robot learning. *arXiv preprint arXiv:2106.13703*, 2021.
- [72] C. Finn, P. Abbeel, and S. Levine. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. *Proceedings of the International Conference on Machine Learning*, 2017.
- [73] R. Foygel Barber, E. J. Candes, A. Ramdas, and R. J. Tibshirani. The limits of distribution-free conditional predictive inference. *Information and Inference: A Journal of the IMA*, 10(2):455–482, June 2021.
- [74] T. Fraichard. A short paper about motion safety. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1140–1145, 2007.
- [75] B. A. Francis. *A Course in H-Infinity Control Theory*. Springer-Verlag, 1987.
- [76] D. Fudenberg, F. Drew, and D. K. Levine. *The theory of learning in games*, volume 2. MIT press, Cambridge, MA, 1998.
- [77] General Motors. Self-driving safety report, 2018. Available at <https://www.gm.com/content/dam/company/docs/us/en/gmcom/gmsafetyreport.pdf>.
- [78] P. Germain, A. Lacasse, F. Laviolette, and M. Marchand. PAC-Bayesian Learning of Linear Classifiers. *Proceedings of the International Conference on Machine Learning*, pages 353–360, 2009.
- [79] M. Gil, F. Alajaji, and T. Linder. Rényi divergence measures for commonly used univariate continuous distributions. *Information Sciences*, 249:124–131, 2013.

- [80] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.
- [81] I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and Harnessing Adversarial Examples. *Proceedings of the International Conference on Learning Representations*, 2015.
- [82] R. M. Gray. *Entropy and Information Theory*. Springer Science & Business Media, 2nd edition, 2011.
- [83] I. Greenberg and S. Mannor. Detecting rewards deterioration in episodic reinforcement learning. *International Conference on Machine Learning*, 139:3842–3853, 2021.
- [84] S. Gupta, J. Davidson, S. Levine, R. Sukthankar, and J. Malik. Cognitive mapping and planning for visual navigation. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2616–2625, 2017.
- [85] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *Proceedings of the International Conference on Machine Learning*, pages 1861–1870, 2018.
- [86] M. Haddouche, B. Guedj, O. Rivasplata, and J. Shawe-Taylor. PAC-Bayes Unleashed: Generalisation Bounds With Unbounded Losses. *arXiv preprint arXiv:2006.07279*, 2020.
- [87] S. Han, N. Stiffler, A. Krontiris, K. Bekris, and J. Yu. High-quality tabletop rearrangement with overhand grasps: Hardness results and fast methods. *Proceedings of Robotics: Science and Systems (RSS)*, 2017.
- [88] M. Hardt, B. Recht, and Y. Singer. Train Faster, Generalize Better: Stability of Stochastic Gradient Descent. *Proceedings of the International Conference on Machine Learning*, 2016.
- [89] K. Hauser. The minimum constraint removal problem with three robotics applications. *The International Journal of Robotics Research (IJRR)*, 33(1):5–17, 2014.

- [90] E. Hazan. Introduction to Online Convex Optimization. *Foundations and Trends in Optimization*, 2(3-4):157–325, 2016.
- [91] D. Hendrycks, S. Basart, M. Mazeika, M. Mostajabi, J. Steinhardt, and D. Song. Scaling out-of-distribution detection for real-world settings. *arXiv preprint arXiv:1911.11132*, 2020.
- [92] D. Hendrycks and K. Gimpel. A baseline for detecting misclassified and out-of-distribution examples in neural networks. *Proceedings of the International Conference on Learning Representations*, 2017.
- [93] D. Hendrycks, M. Mazeika, and T. G. Dietterich. Deep anomaly detection with outlier exposure. *Proceedings of the International Conference on Learning Representations*, 2019.
- [94] T. Heskes. Solving a Huge Number of Similar Tasks: a Combination of Multi-Task Learning and a Hierarchical Bayesian Approach. *Proceedings of the International Conference on Machine Learning*, 1998.
- [95] M. Ho, A. Farid, and A. Majumdar. Towards a framework for comparing the complexity of robotic tasks. *Proceedings of the Workshop on the Algorithmic Foundations of Robotics*, 2022.
- [96] R. V. Hogg, J. McKean, and A. T. Craig. *Introduction to Mathematical Statistics*. Pearson Education, 2005.
- [97] J. Hopcroft, D. Joseph, and S. Whitesides. Movement problems for 2-dimensional linkages. *SIAM Journal on Computing (SICOMP)*, 13(3):610–629, 1984.
- [98] J. Hopcroft, J. Schwartz, and M. Sharir. On the complexity of motion planning for multiple independent objects; PSPACE-hardness of the Warehouseman’s Problem. *The International Journal of Robotics Research (IJRR)*, 3(4):76–88, 1984.
- [99] T. Hospedales, A. Antoniou, P. Micaelli, and A. Storkey. Meta-Learning in Neural Networks: A Survey. *arXiv preprint arXiv:2004.05439*, 2020.

- [100] K.-C. Hsu, A. Z. Ren, D. P. Nguyen, A. Majumdar, and J. F. Fisac. Sim-to-lab-to-real: Safe reinforcement learning with shielding and generalization guarantees. *arXiv:2201.08355 [cs.RO]*, 2022.
- [101] H. Hu, K. Nakamura, and J. F. Fisac. SHARP: Shielding-aware robust planning for safe and efficient human-robot interaction. *IEEE Robotics and Automation Letters*, 7(2):5591–5598, 2022.
- [102] B. Ivanovic, A. Elhafi, G. Rosman, A. Gaidon, and M. Pavone. MATS: An interpretable trajectory forecasting representation for planning and control. *Conf. on Robot Learning*, 2020.
- [103] B. Ivanovic, K.-H. Lee, P. Tokmakov, B. Wulfe, R. McAllister, A. Gaidon, and M. Pavone. Heterogeneous-agent trajectory forecasting incorporating class uncertainty. *arXiv preprint arXiv:2104.12446*, 2021.
- [104] B. Ivanovic and M. Pavone. Injecting planning-awareness into prediction and detection evaluation. *IEEE Intelligent Vehicles Symposium*, 2022.
- [105] Y. Jiang, B. Neyshabur, H. Mobahi, D. Krishnan, and S. Bengio. Fantastic Generalization Measures and Where to Find Them. *Proceedings of the International Conference on Learning Representations*, 2020.
- [106] D. Joseph and W. H. Plantings. On the complexity of reachability and motion planning questions. *Proceedings of the Symposium on Computational Geometry*, pages 62–66, 1985.
- [107] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1-2):99–134, 1998.
- [108] G. Kahn, A. Villafior, V. Pong, P. Abbeel, and S. Levine. Uncertainty-aware reinforcement learning for collision avoidance. *arXiv:2104.00543 [cs.LG]*, 2017.
- [109] M. Khodak, M.-F. Balcan, and A. Talwalkar. Provable Guarantees for Gradient-Based Meta-Learning. *Proceedings of the International Conference on Machine Learning*, 2019.

- [110] R. Kianfar, P. Falcone, and J. Fredriksson. Safety verification of automated driving systems. *IEEE Intelligent Transportation Systems Magazine*, 5(4):73–86, 2013.
- [111] A. Kumar, Z. Fu, D. Pathak, and J. Malik. RMA: Rapid motor adaptation for legged robots. *arXiv preprint arXiv:2107.04034*, 2021.
- [112] A. Kumar, Z. Li, J. Zeng, D. Pathak, K. Sreenath, and J. Malik. Adapting rapid motor adaptation for bipedal robots. *arXiv preprint arXiv:2205.15299*, 2022.
- [113] B. Lake, R. Salakhutdinov, J. Gross, and J. Tenenbaum. One Shot Learning of Simple Visual Concepts. *Cognitive Science*, 33, 2011.
- [114] J. Langford. Tutorial on Practical Prediction Theory for Classification. *Journal of Machine Learning Research*, 6(Mar):273–306, 2005.
- [115] J. Langford and R. Caruana. (Not) Bounding the True Error. *Advances in Neural Information Processing Systems 14*, 2002.
- [116] J. Langford and J. Shawe-Taylor. PAC-Bayes & margins. *Advances in Neural Information Processing Systems 15*, 2003.
- [117] S. M. LaValle. *Planning Algorithms*. Cambridge University Press, Cambridge, MA, 2006.
- [118] S. M. LaValle. *Sensing and Filtering: A Fresh Perspective Based on Preimages and Information Spaces*. Publishers Inc., Hanover, MA, 2012.
- [119] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47), 2020.
- [120] K. Lee, H. Lee, K. Lee, and J. Shin. Training confidence-calibrated classifiers for detecting out-of-distribution samples. *Proceedings of the International Conference on Learning Representations*, 2018.
- [121] K. Lee, K. Lee, H. Lee, and J. Shin. A simple unified framework for detecting out-of-distribution samples and adversarial attacks. *Advances in Neural Information Processing Systems*, 2018.

- [122] I. Lenz, H. Lee, and A. Saxena. Deep learning for detecting robotic grasps. *The International Journal of Robotics Research*, 34(4-5):705–724, 2015.
- [123] K. Leung, E. Schmerling, M. Zhang, M. Chen, J. Talbot, J. C. Gerdes, and M. Pavone. On infusing reachability-based safety assurance within planning frameworks for human-robot vehicle interactions. *Int. Journal of Robotics Research*, 39:1326–1345, 2020.
- [124] S. Levine, C. Finn, T. Darrell, and P. Abbeel. End-to-end training of deep visuomotor policies. *The Journal of Machine Learning Research*, 17(1):1334–1373, 2016.
- [125] M. Li, P. Vitányi, et al. *An introduction to Kolmogorov complexity and its applications*, volume 3. Springer, New York, NY, 2008.
- [126] Y. Li, Y. Wu, H. Xu, X. Wang, and Y. Wu. Solving compositional reinforcement learning problems via task reduction. *Proceedings of the International Conference on Learning Representations*, 2021.
- [127] S. Liang, Y. Li, and R. Srikant. Enhancing the reliability of out-of-distribution image detection in neural networks. *Proceedings of the International Conference on Learning Representations*, 2018.
- [128] C. Liu, S. Lee, S. Varnhagen, and H. E. Tseng. Path planning for autonomous vehicles using model predictive control. *IEEE Intelligent Vehicles Symposium*, 2017.
- [129] S. Liu, R. Garrepalli, T. G. Dietterich, A. Fern, and D. Hendrycks. Open category detection with pac guarantees. *Proceedings of the International Conference on Machine Learning*, 2018.
- [130] B. London. A PAC-Bayesian Analysis of Randomized Learning with Application to Stochastic Gradient Descent. *Advances in Neural Information Processing Systems 30*, 2017.
- [131] R. Luo, S. Zhao, J. Kuck, B. Ivanovic, S. Savarese, E. Schmerling, and M. Pavone. Sample-efficient safety assurances using conformal prediction. *arXiv:2109.14082 [cs.RO]*, 2021.

- [132] B. Lütjens, M. Everett, and J. P. How. Safe reinforcement learning with model uncertainty estimates. *IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [133] Lyft. Self-driving safety report, 2020. Available at http://autonomous.lyft.com/wp-content/uploads/2020/06/Safety_Report_2020.pdf.
- [134] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. A. Ojea, and K. Goldberg. Dex-Net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics. *arXiv preprint arXiv:1703.09312*, 2017.
- [135] A. Majumdar, A. Farid, and A. Sonar. PAC-Bayes control: learning policies that provably generalize to novel environments. *The International Journal of Robotics Research*, 40(2-3):574–593, 2021.
- [136] A. Majumdar and M. Goldstein. PAC-Bayes control: Synthesizing controllers that provably generalize to novel environments. *Proceedings of the Conference on Robot Learning*, 2018.
- [137] A. Majumdar and M. Pavone. How should a robot assess risk? towards an axiomatic theory of risk in robotics. *Robotics Research*, pages 75–84, 2020.
- [138] A. Majumdar and R. Tedrake. Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research (IJRR)*, 36(8):947–982, July 2017.
- [139] A. Maurer. A Note on the PAC Bayesian Theorem. *arXiv preprint arXiv:0411099*, 2004.
- [140] A. Maurer. Algorithmic Stability and Meta-Learning. *Journal of Machine Learning Research*, 6:967–994, 2005.
- [141] D. McAllester. A PAC-Bayesian Tutorial with A Dropout Bound. *arXiv preprint arXiv:1307.2118*, 2013.

- [142] D. A. McAllester. PAC-Bayesian Model Averaging. *Proceedings of the Conference on Learning Theory*, 1999.
- [143] R. McAllister, B. Wulfe, J. Mercat, L. Ellis, S. Levine, and A. Gaidon. Control-aware prediction objectives for autonomous driving. *Proc. IEEE Conf. on Robotics and Automation*, 2022.
- [144] A. Meinke, J. Bitterwolf, and M. Hein. Provably Robust Detection of Out-of-distribution Data (almost) for free. *arXiv preprint arXiv:2106.04260*, 2021.
- [145] A. Meinke and M. Hein. Towards Neural Networks That Provably Know When They Don’t Know. *Proceedings of the International Conference on Learning Representations*, 2020.
- [146] I. M. Mitchell, A. M. Bayen, and C. J. Tomlin. A time-dependent hamilton-jacobi formulation of reachable sets for continuous dynamic games. *IEEE Transactions on automatic control*, 50(7):947–957, 2005.
- [147] MOSEK ApS. Mosek fusion api for python 9.0.105, 2019.
- [148] R. Murrieta-Cid, R. Monroy, S. Hutchinson, and J.-P. Laumond. A Complexity result for the pursuit-evasion game of maintaining visibility of a moving evader. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2657–2664, 2008.
- [149] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro. A PAC-Bayesian Approach to Spectrally-Normalized Margin Bounds for Neural Networks. *preprint arXiv:1707.09564*, 2017.
- [150] B. Neyshabur, S. Bhojanapalli, D. McAllester, and N. Srebro. Exploring Generalization in Deep Learning. *Advances in Neural Information Processing Systems 30*, pages 5949–5958, 2017.
- [151] X. Nguyen, M. J. Wainwright, and M. I. Jordan. Estimating divergence functionals and the likelihood ratio by convex risk minimization. *IEEE Transactions On Information Theory*, 56(11):5847–5861, 2010.

- [152] A. Nichol, J. Achiam, and J. Schulman. On First-Order Meta-Learning Algorithms. *arXiv preprint arXiv:1803.02999*, 2018.
- [153] D. Nistér, H. L. Lee, J. Ng, and Y. Wang. An introduction to the safety force field. <https://www.nvidia.com/content/dam/en-zz/Solutions/self-driving-cars/safety-force-field/the-safety-force-field.pdf>, 2019.
- [154] J. M. O’Kane and S. M. LaValle. Comparing the Power of Robots. *The International Journal of Robotics Research (IJRR)*, 27(1):5–23, 2008.
- [155] M. Ono, M. Pavone, Y. Kuwata, and J. Balaram. Chance-constrained dynamic programming with application to risk-aware robotic space exploration. *Autonomous Robots*, 39(4):555–571, 2015.
- [156] E. Parrado-Hernández, A. Ambroladze, J. Shawe-Taylor, and S. Sun. PAC-Bayes Bounds with Data Dependent Priors. *Journal of Machine Learning Research*, 13:3507–3531, 2012.
- [157] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems 32*, pages 8024–8035, 2019.
- [158] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [159] J. Pennington, R. Socher, and C. D. Manning. GloVe: Global Vectors for Word Representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014.
- [160] A. Pentina and C. H. Lampert. A PAC-Bayesian Bound for Lifelong Learning. *Proceedings of the International Conference on Machine Learning*, 2014.

- [161] A. Pentina and C. H. Lampert. Lifelong Learning with Non-i.i.d. Tasks. *Advances in Neural Information Processing Systems 28*, 2015.
- [162] M. Pérez-Ortiz, O. Risvaplata, J. Shawe-Taylor, and C. Szepesvári. Tighter risk certificates for neural networks. *Journal of Machine Learning Research*, 22(227):1–40, 2021.
- [163] J. Peters, D. Janzing, and B. Schölkopf. *Elements of Causal Inference: Foundations and Learning Algorithms*. The MIT Press, 2017.
- [164] S. Ravi and H. Larochelle. Optimization as a Model for Few-Shot Learning. *Proceedings of the International Conference on Learning Representations*, 2017.
- [165] J. Reif. Complexity of the mover’s problem and generalizations. *Symposium on Foundations of Computer Science*, pages 421–427, 1979.
- [166] A. Z. Ren, S. Veer, and A. Majumdar. Generalization Guarantees for Imitation Learning. *Proceedings of the Conference on Robot Learning*, 2020.
- [167] C. Richter and N. Roy. Safe visual navigation via deep learning and novelty detection. *Proceedings of Robotics: Science and Systems*, 2017.
- [168] C. Richter, W. Vega-Brown, and N. Roy. Bayesian learning for safe high-speed navigation in unknown environments. *Proceedings of the International Symposium on Robotics Research (ISRR)*, 2015.
- [169] J. Rissanen. *Stochastic Complexity in Statistical Inquiry*. World Scientific, 1989.
- [170] O. Rivasplata, I. Kuzborskij, C. Szepesvari, and J. Shawe-Taylor. PAC-Bayes Analysis Beyond the Usual Bounds. *Advances in Neural Information Processing Systems 33*, 2020.
- [171] O. Rivasplata, E. Parrado-Hernandez, J. Shawe-Taylor, S. Sun, and C. Szepesvari. PAC-Bayes Bounds for Stable Algorithms with Instance-Dependent Priors. *Advances in Neural Information Processing Systems 31*, 2018.

- [172] O. Rivasplata, V. M. Tankasali, and C. Szepesvari. PAC-Bayes with Backprop. *arXiv preprint arXiv:1908.07380*, 2019.
- [173] J. Rothfuss, V. Fortuin, M. Josifoski, and A. Krause. PACOH: Bayes-optimal meta-learning with pac-guarantees. *Proceedings of the International Conference on Machine Learning*, 139:9116–9126, 2021.
- [174] A. Rudenko, L. Palmieri, M. Herman, K. M. Kitani, D. M. Gavrilu, and K. O. Arras. Human motion trajectory prediction: A survey. *Int. Journal of Robotics Research*, 39(8):895–935, 2020.
- [175] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller. A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE*, 109(5):756–795, 2021.
- [176] F. Z. Saberifar, S. Ghasemlou, D. A. Shell, and J. M. O’Kane. Toward a language-theoretic foundation for planning and filtering. *The International Journal of Robotics Research (IJRR)*, 38(2-3):236–259, 2019.
- [177] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. *European Conference on Computer Vision*, pages 683–700, 2020.
- [178] A. Santoro, S. Bartunov, M. Botvinick, D. Wierstra, and T. Lillicrap. Meta-Learning with Memory-Augmented Neural Networks. *Proceedings of the International Conference on Machine Learning*, 2016.
- [179] S. Schaefer, K. Leung, B. Ivanovic, and M. Pavone. Leveraging neural network gradients within trajectory optimization for proactive human-robot interactions. *Proc. IEEE Conf. on Robotics and Automation*, 2021.
- [180] E. Schmerling, K. Leung, W. Vollprecht, and M. Pavone. Multimodal probabilistic model-based planning for human-robot interaction. *Proc. IEEE Conf. on Robotics and Automation*, 2018.

- [181] J. Schmidhuber. Evolutionary Principles in Self-Referential Learning. On Learning how to Learn: The Meta-Meta-Meta...-Hook. Diploma thesis, Technische Universität München, Germany, 1987.
- [182] R. Schneider. *Convex Bodies: The Brunn–Minkowski Theory*. Encyclopedia of Mathematics and its Applications. Cambridge University Press, 2 edition, 2013.
- [183] T. Schouwenaars, J. How, and E. Feron. Receding horizon path planning with implicit safety guarantees. *Proceedings of the American Control Conference (ACC)*, 6:5576–5581, 2004.
- [184] A. Sedlmeier, T. Gabor, T. Phan, L. Belzner, and C. Linnhoff-Popien. Uncertainty-based out-of-distribution classification in deep reinforcement learning. *arXiv:2001.00496 [cs.LG]*, 2019.
- [185] A. Sedlmeier, R. Müller, S. Illium, and C. Linnhoff-Popien. Policy entropy for out-of-distribution classification. *Artificial Neural Networks and Machine Learning – ICANN*, 2020.
- [186] M. Seeger. PAC-Bayesian Generalisation Error Bounds for Gaussian Process Classification. *Journal of Machine Learning Research*, 3(Oct):233–269, 2002.
- [187] G. Shafer and V. Vovk. A tutorial on conformal prediction. *Journal of Machine Learning Research*, 9(12):371–421, 2008.
- [188] S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.
- [189] S. Shalev-Shwartz, O. Shamir, N. Srebro, and K. Sridharan. Learnability, Stability and Uniform Convergence. *Journal of Machine Learning Research*, 11:2635–2670, 2010.
- [190] S. Shalev-Shwartz, S. Shammah, and A. Shashua. On a formal model of safe and scalable self-driving cars. *arXiv preprint 1708.06374*, 2017.

- [191] A. Sharma, N. Azizan, and M. Pavone. Sketching curvature for efficient out-of-distribution detection for deep neural networks. *Proc. Conf. on Uncertainty in Artificial Intelligence*, 2021.
- [192] J. Shawe-Taylor and R. C. Williamson. A PAC Analysis of a Bayesian Estimator. *Proceedings of the 10th Conference on Computational Learning Theory*, 1997.
- [193] D. A. Shell and J. M. O’Kane. Reality as a simulation of reality: robot illusions, fundamental limits, and a physical demonstration. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 10327–10334, 2020.
- [194] M. A. Siddiqui, A. Fern, T. G. Dietterich, and S. Das. Finite sample complexity of rare pattern anomaly detection. *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, 2016.
- [195] A. Sinha, H. Namkoong, R. Volpi, and J. Duchi. Certifying Some Distributional Robustness with Principled Adversarial Training. *arXiv preprint arXiv:1710.10571*, 2020.
- [196] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, Boston, MA, 2013.
- [197] K. Solovey and D. Halperin. On the hardness of unlabeled multi-robot motion planning. *The International Journal of Robotics Research (IJRR)*, 35(14):1750–1759, 2016.
- [198] N. Sünderhauf, O. Brock, W. Scheirer, R. Hadsell, D. Fox, J. Leitner, B. Upcroft, P. Abbeel, W. Burgard, M. Milford, and P. Corke. The limits and potentials of deep learning for robotics. *The International Journal of Robotics Research (IJRR)*, 37(4-5):405–420, 2018.
- [199] B. Thananjeyan, A. Balakrishna, S. Nair, M. Luo, K. Srinivasan, M. Hwang, J. E. Gonzalez, J. Ibarz, C. Finn, and K. Goldberg. Recovery rl: Safe reinforcement learning with learned recovery zones. *IEEE Robotics and Automation Letters*, 6(3):4915–4922, 2021.

- [200] N. Thiemann, C. Igel, O. Wintenberger, and Y. Seldin. A Strongly Quasiconvex PAC-Bayesian Bound. *Machine Learning Research*, 76:1–26, 2017.
- [201] S. Thrun and L. Pratt. *Learning to Learn*. Springer Science & Business Media, 1998.
- [202] R. Tian, L. Sun, A. Bajcsy, M. Tomizuka, and A. D. Dragan. Safety assurances for human-robot interaction via confidence-aware game-theoretic human models. *Proc. IEEE Conf. on Robotics and Automation*, 2022.
- [203] J. Tobin, W. Zaremba, and P. Abbeel. Domain randomization and generative models for robotic grasping. *arXiv preprint arXiv:1710.06425*, 2017.
- [204] E. Todorov, T. Erez, and Y. Tassa. Mujoco: A physics engine for model-based control. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.
- [205] A. T. Tran, C. V. Nguyen, and T. Hassner. Transferability and hardness of supervised classification tasks. *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1395–1405, 2019.
- [206] V. N. Vapnik and A. Y. Chervonenkis. On the Uniform Convergence of Relative Frequencies of Events to Their Probabilities. *Dokl. Akad. Nauk*, 181(4), 1968.
- [207] S. Veer, A. E. Gurgun, A. Farid, and A. Majumdar. Probably approximately correct vision-based planning using motion primitives. *arXiv preprint arXiv:2002.12852*, 2022.
- [208] S. Veer and A. Majumdar. Probably approximately correct vision-based planning using motion primitives. *Proceedings of the Conference on Robot Learning*, 2020.
- [209] P. Viallard, P. Germain, A. Habrard, and E. Morvant. A General Framework for the Derandomization of PAC-Bayesian Bounds. *arXiv preprint arXiv:2102.08649*, 2021.
- [210] R. Vilalta and Y. Drissi. A Perspective View and Survey of Meta-Learning. *Artificial Intelligence Review*, 18:77–95, 2002.

- [211] S. Villa, L. Rosasco, and T. Poggio. On Learnability, Complexity and Stability. In *Empirical Inference*, pages 59–69. Springer, 2013.
- [212] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra. Matching Networks for One Shot Learning. *Advances in Neural Information Processing Systems* 29, 2016.
- [213] M. P. Vitus and C. J. Tomlin. Closed-loop belief space planning for linear, Gaussian systems. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 2152–2159, 2011.
- [214] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer US, 2005.
- [215] J. Watt, R. Borhani, and A. K. Katsaggelos. *Machine Learning Refined: Foundations, Algorithms, and Applications*. Cambridge University Press, 2016.
- [216] Waymo. Safety report, 2021. Available at <https://waymo.com/safety/safety-report>. Retrieved on July 4, 2021.
- [217] X. Weng, B. Ivanovic, and M. Pavone. MTP: Multi-hypothesis tracking and prediction for reduced error propagation. *IEEE Intelligent Vehicles Symposium*, 2022.
- [218] X. Weng, Y. Yuan, and K. Kitani. PTP: Parallelized tracking and prediction with graph neural networks and diversity sampling. *IEEE Robotics and Automation Letters*, 6(3):4640–4647, 2021.
- [219] J. Wiederer, A. Bouazizi, M. Troina, U. Kressel, and V. Belagiannis. Anomaly detection in multi-agent trajectories for automated driving. *Proceedings of the Conference on Robot Learning*, 164:1223–1233, 2022.
- [220] D. Wierstra, T. Schaul, T. Glasmachers, Y. Sun, J. Peters, and J. Schmidhuber. Natural evolution strategies. *The Journal of Machine Learning Research*, 15(27):949–980, 2014.
- [221] Wolfram Research, Inc. Mathematica, Version 12.0, 2019. Champaign, IL.

- [222] T. Wu and J. Ortiz. RLAD: time series anomaly detection through reinforcement learning and active learning. *arXiv:2104.00543 [cs.LG]*, 2021.
- [223] Y. Wu, W. Yan, T. Kurutach, L. Pinto, and P. Abbeel. Learning to manipulate deformable objects without demonstrations. *Proceedings of Robotics: Science and Systems (RSS)*, 2020.
- [224] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3d shapenets: A deep representation for volumetric shapes. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1912–1920, 2015.
- [225] M. Yin, G. Tucker, M. Zhou, S. Levine, and C. Finn. Meta-Learning without Memorization. *Proceedings of the International Conference on Learning Representations*, 2020.
- [226] W. Zeng, W. Luo, S. Suo, A. Sadat, B. Yang, S. Casas, and R. Urtasun. End-to-end interpretable neural motion planner. *IEEE Conf. on Computer Vision and Pattern Recognition*, 2019.
- [227] W. Zhou, V. Veitch, M. Austern, R. Adams, and P. Orbanz. Nonvacuous Generalization Bounds at the ImageNet Scale: A PAC-Bayesian Compression Approach. *Proceedings of the International Conference on Learning Representations*, 2019.
- [228] Y. Zhu, R. Mottaghi, E. Kolve, J. J. Lim, A. Gupta, L. Fei-Fei, and A. Farhadi. Target-driven visual navigation in indoor scenes using deep reinforcement learning. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 3357–3364, 2017.
- [229] J. Ziegler, P. Bender, T. Dang, and C. Stiller. Trajectory planning for bertha - a local, continuous method. *IEEE Intelligent Vehicles Symposium*, 2014.
- [230] Zoox. Safety report volume 2.0, 2021. Available at <https://zoox.com/safety/>.